



DesignWare SDRAM and Static Memory Controller Databook

DW_memctl

Copyright Notice and Proprietary Information

Copyright © 2012 Synopsys, Inc. All rights reserved. This software and documentation contain confidential and proprietary information that is the property of Synopsys, Inc. The software and documentation are furnished under a license agreement and may be used or copied only in accordance with the terms of the license agreement. No part of the software and documentation may be reproduced, transmitted, or translated, in any form or by any means, electronic, mechanical, manual, optical, or otherwise, without prior written permission of Synopsys, Inc., or as expressly provided by the license agreement.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Registered Trademarks (®)

Synopsys, AEON, AMPS, ARC, Astro, Behavior Extracting Synthesis Technology, Cadabra, CATS, Certify, CHIPit, CoMET, Confirma, CODE V, Design Compiler, DesignSphere, DesignWare, Eclipse, Formality, Galaxy Custom Designer, Global Synthesis, HAPS, HapsTrak, HDL Analyst, HSIM, HSPICE, Identify, Leda, LightTools, MAST, MaVeric, METeor, ModelTools, NanoSim, NOVeA, OpenVera, ORA, PathMill, Physical Compiler, PrimeTime, SCOPE, SiVL, SNUG, SolvNet, Sonic Focus, STAR Memory System, SVP Café, Syndicated, Synplicity, the Synplicity logo, Synplify, Synplify Pro, Synthesis Constraints Optimization Environment, TetraMAX, UMRBus, VCS, Vera, and YieldExplorer are registered trademarks of Synopsys, Inc.

Trademarks (™)

AFGen, Apollo, ASAP, Astro-Rail, Astro-Xtalk, Aurora, AvanWaves, BEST, Columbia, Columbia-CE, Cosmos, CosmosLE, CosmosScope, CRITIC, CustomExplorer, CustomSim, Custom WaveView, DC Expert, DC Professional, DC Ultra, Design Analyzer, Design Vision, DesignerHDL, DesignPower, DFTMAX, Direct Silicon Access, Discovery, Encore, EPIC, Galaxy, HANEX, HDL Compiler, Hercules, Hierarchical Optimization Technology, High-performance ASIC Prototyping System, HSIMplus, i-Virtual Stepper, IC Compiler, IICE, in-Sync, iN-Tandem, Intelli, Jupiter, Jupiter-DP, JupiterXT, JupiterXT-ASIC, Liberty, Libra-Passport, Library Compiler, Macro-PLUS, Magellan, Mars, Mars-Rail, Mars-Xtalk, Milkyway, ModelSource, Module Compiler, MultiPoint, ORAengineering, Physical Analyst, Planet, Planet-PL, Platform Architect, Polaris, Power Compiler, Processor Designer, Raphael, RippledMixer, Saturn, Scirocco, Scirocco-i, SiWare, SPW, Star-RCXT, Star-SimXT, StarRC, Symphony Model System Compiler, System Designer, System Studio, Taurus, TotalRecall, TSUPREM-4, VCSi, VHDL Compiler, VMC, and Worksheet Buffer are trademarks of Synopsys, Inc.

Service Marks (SM)

MAP-in and TAP-in are service marks of Synopsys, Inc.

Third Party Trademark Acknowledgements

SystemC is a trademark of the Open SystemC Initiative and is used under license.

ARM and AMBA are registered trademarks of ARM Limited.

Saber is a registered trademark of SabreMark Limited Partnership and is used under license.

Entrust is a registered trademark of Entrust Inc. in the United States and in certain other countries. In Canada, Entrust is a trademark or registered trademark of Entrust Technologies Limited. Used by Entrust.net Inc. under license.

All other product or company names may be trademarks of their respective owners.

Synopsys, Inc.
700 E. Middlefield Road
Mountain View, CA 94043

www.synopsys.com

Contents

Preface	9
Chapter 1	
Product Overview	13
1.1 DesignWare System Overview	13
1.2 Configuration Flow	15
1.3 General Product Description	16
1.3.1 DW_memctl Block Diagram	17
1.4 Features	18
1.4.1 AMBA AHB Interface Features	18
1.4.2 SDRAM Interface Features	18
1.4.3 Static Memory Interface Features	19
1.5 Standards Compliance	19
1.6 Verification Environment Overview	20
1.7 Licenses	20
1.8 Where To Go From Here	20
Chapter 2	
Building and Verifying a Component or Subsystem	21
2.1 Setting up Your Environment	21
2.2 Overview of the coreConsultant Configuration and Integration Process	22
2.2.1 coreConsultant Usage	22
2.2.2 Configuring the DW_memctl within coreConsultant	23
2.2.3 Creating Gate-Level Netlists within coreConsultant	24
2.2.4 Verifying the DW_memctl within coreConsultant	24
2.2.5 Running Leda on Generated Code with coreConsultant	24
2.3 Overview of the coreAssembler Configuration and Integration Process	25
2.3.1 coreAssembler Usage	25
2.3.2 Configuring the DW_memctl within a Subsystem	28
2.3.3 Creating Gate-Level Netlists within coreAssembler	28
2.3.4 Verifying the DW_memctl within coreAssembler	28
2.3.5 Running Leda on Generated Code with coreAssembler	29
2.4 Database Files	29
2.4.1 Design/HDL Files	29
2.4.2 Synthesis Files	30
2.4.3 Verification Reference Files	30
Chapter 3	
Architecture	33

3.1 Automation of DW_memctl Flow	34
3.2 Block Diagram	36
3.3 AMBA Host Interface Unit (HIU)	37
3.3.1 Address FIFO	39
3.3.2 Write Data FIFO	40
3.3.3 Control Register Access	42
3.4 Memory Interface Unit (MIU)	46
3.5 Bus Sharing and the EBI	47
 Chapter 4	
DW_memctl as SDRAM Controller Functional Description	49
4.1 SDR-SDRAM and Mobile SDR-SDRAM Controller	49
4.1.1 Power-On Initialization	49
4.1.2 Read/Write Operations	51
4.1.3 Set-Mode and Extended-Mode Register	62
4.1.4 Auto-Refresh	63
4.1.5 Self-Refresh	65
4.1.6 Power-Down	66
4.2 DDR-SDRAM and Mobile DDR-SDRAM Controller	67
4.2.1 DDR-SDRAM Power-On Initialization	67
4.2.2 Mobile DDR-SDRAM Power-On Initialization	69
4.2.3 Read/Write Operations	71
4.2.4 DDR-SDRAM Write Data Path	82
4.2.5 Set-Mode and Extended-Mode Register	83
4.2.6 Auto-Refresh	83
4.2.7 Self-Refresh	86
4.2.8 Power-Down	87
4.2.9 Deep Power-Down (Mobile DDR-SDRAM Only)	88
4.3 Address Multiplexing	89
4.3.1 Use of Higher AHB Address for Bank Address Decoding	91
4.4 Chip Select Decoding	91
4.4.1 Scenario 1	92
4.4.2 Scenario 2	94
4.5 Interfacing DW_memctl with SDRAM SIMM/DIMM Modules	96
4.6 Interfacing DW_memctl to Single Bank DIMMS	98
4.7 Interfacing DW_memctl to Two-Bank DIMMS	99
 Chapter 5	
DW_memctl as Static Memory Controller Functional Description	101
5.1 Configuring DW_memctl for Static Memory	101
5.1.1 Static Memory Chip Selection	102
5.1.2 FLASH Reset/Power-Down Control	102
5.1.3 FLASH Memory Write Protection	103
5.1.4 FLASH Memory Status Information	103
5.1.5 Static Memory Timing Diagrams	103
5.2 Address Multiplexing	108
5.3 Interfacing with Asynchronous SRAMs	109
5.3.1 Specifying the SRAM Timing Parameters	109
5.3.2 I/O Interface Between SRAM and DW_memctl	109

5.3.3 Using CellularRAM in Asynchronous Burst Mode	110
5.4 Interfacing with Asynchronous FLASH Memories	110
5.4.1 Specifying FLASH Memory Timing Parameters	110
5.4.2 I/O Interface Between FLASH Memory and DW_memctl	111
5.5 Interfacing to Synchronous SRAMs	112
5.5.1 Interfacing to an On-Chip SSRAM	112
5.5.2 Interfacing to an Off-Chip SSRAM	114
5.6 Interfacing with Synchronous FLASH Memories	115
5.6.1 I/O Interface Between Synchronous FLASH Memory and DW_memctl	116
5.6.2 Timing Diagrams of Synchronous Reads	118
5.6.3 Specifying Synchronous FLASH Memory Timing Parameters	119
5.7 Interfacing with Non-Memory Devices with Ready Pin	121
5.7.1 Timing Diagrams of Read/Write Accesses	121
5.7.2 Aborted Pre-Fetched Reads or Write Address Setup (with Ready Signal)	122
5.7.3 Specifying the Timing Parameters	123
Chapter 6	
Parameters	125
6.1 Parameter Descriptions	125
6.1.1 Configurable Parameters	126
Chapter 7	
Signals	145
7.1 DW_memctl Interface Diagrams	145
7.2 DW_memctl Signal Descriptions	152
Chapter 8	
Registers	167
8.1 Register Memory Map	168
8.2 Register and Field Descriptions	169
8.2.1 Configuration Register	169
8.2.2 SDRAM Timing Registers	172
8.2.3 SDRAM Control Registers	175
8.2.4 Address Decoder Registers	181
8.2.5 Chip Select Registers (SCSLR _n)	181
8.2.6 Address Aliasing and Remapping	181
8.2.7 Address Register Details	182
8.2.8 Aliasing Registers (CSALIAS _n)	185
8.2.9 Remap Registers	187
8.2.10 Component Identification Registers	194
8.2.11 Component Configuration Registers	195
Chapter 9	
Programming the DW_memctl	201
9.1 Software Drivers	201
9.2 DW_memctl Configuration	201
9.2.1 DW_memctl Register Programming Sequence	202
Chapter 10	
Verification	205

10.1 Verification Environment Overview	206
10.1.1 DW_memctl Testbench	206
10.1.2 Testbench Structure	208
10.1.3 DW_memctl	208
10.1.4 vera_shell	209
10.1.5 Clock Generation	210
10.1.6 Address Decoder	210
10.1.7 Memory Banks	211
10.1.8 Waveform Output	211
10.2 Simulation Command Files	212
10.3 Simulation Output Files	212
10.3.1 Using Log Files	213
10.4 Simulation Scripts	213
10.4.1 run.scr	214
10.5 Verilog PLI Integration	215
10.6 Verifying Default DW_memctl Configuration	216
10.6.1 Verifying Non-Default DW_memctl Configurations	216
10.7 DW_memctl Core VERA Files	216
10.8 Installing DWMMs Manually	218
10.8.1 Downloading TSP	218
10.8.2 Downloading models	218
10.8.3 Installing DWMM software (TSP)	218
10.8.4 Installing Models	218
10.8.5 Using DesignWare Memory Models with VCS	219
Chapter 11	
Integrating DW_memctl into Your Design	221
11.1 Instantiating DW_memctl	221
11.2 DW_memctl Integration	223
11.2.1 Bi-directional Data Buses	224
11.2.2 System Reset	225
11.2.3 Data Connections	226
11.2.4 Typical Systems	230
11.2.5 Register Insertion	234
11.2.6 Design Considerations	241
11.2.7 Different System Clock Topologies	243
11.2.8 Typical Interfaces	244
Appendix A	
Application Notes	261
A.1 Frequently Asked Questions	261
A.1.1 General	261
A.1.2 SDRAM Controller	268
A.1.3 Static Memory Controller	268
A.1.4 Configuration	269
A.2 Frequently Asked Debug Questions	270
A.2.1 Testbench/Verification	271
A.3 On-Chip SDRAM BIST Solution	276
A.3.1 Sample SDRAM BIST Wrapper Code	277

A.4 Synthesizing DW_memctl Using dc_script283

 A.4.1 Sample Synthesis Scripts284

A.5 External Bus Interface293

 A.5.1 EBI Module Functionality293

 A.5.2 EBI Interface Signals and Parameters294

 A.5.3 Interfacing DW_memctl with EBI297

 A.5.4 Verilog RTL for EBI Logic302

A.6 DW_memctl in a Multi-Layer AMBA System306

Appendix B

Glossary309

Index313

Preface

This manual provides information about the DW_memctl (DW_memctl). This component conforms to the *AMBA Specification, Revision 2.0* from ARM.

The information in this manual includes a functional description, signal and parameter descriptions, and a memory map. Also provided are an overview of the component testbench, a description of the tests that are run to verify the coreKit, and synthesis information for the coreKit.

Databook Organization

The chapters of this databook are organized as follows:

- Chapter 1, “[Product Overview](#)” provides a system overview, a component block diagram, basic features, and an overview of the verification environment.
- Chapter 2, “[Building and Verifying a Component or Subsystem](#)” introduces you to using the DW_memctl within the coreAssembler and coreConsultant tools.
- Chapter 3, “[Architecture](#)” describes the functional operation of the DW_memctl.
- Chapter 4, “[DW_memctl as SDRAM Controller Functional Description](#)” identifies the configurable parameters supported by the DW_memctl.
- Chapter 5, “[DW_memctl as Static Memory Controller Functional Description](#)” provides a list and description of the DW_memctl signals.
- Chapter 6, “[Parameters](#)” identifies the configurable parameters supported by the DW_memctl.
- Chapter 7, “[Signals](#)” provides a list and description of the DW_memctl signals.
- Chapter 8, “[Registers](#)” describes the programmable registers of the DW_memctl.
- Chapter 9, “[Programming the DW_memctl](#)” provides information needed to program the configured DW_memctl.
- Chapter 10, “[Verification](#)” provides information on verifying the configured DW_memctl.
- Chapter 11, “[Integrating DW_memctl into Your Design](#)” includes information you need to integrate the configured DW_memctl into your design.
- Appendix A, “[Application Notes](#)” provides a comprehensive set of application notes for DW_memctl.
- Appendix B, “[Glossary](#)” provides a glossary of general terms.

Related Documentation

- [DW_memctl Driver Kit User Guide](#) – Contains information on the Driver Kit for the DW_memctl; requires source code license (DWC-APB-Periph-Source)
- [Using DesignWare Library IP in coreAssembler](#) – Contains information on getting started with using DesignWare SIP components for AMBA 2 and AMBA 3 AXI components within coreTools
- [coreAssembler User Guide](#) – Contains information on using coreAssembler
- [coreConsultant User Guide](#) – Contains information on using coreConsultant

Document Revision History

This section tracks the significant documentation changes that occur from release-to-release and during a release.

Table 1-1 Databook Revision History

Version	Databook Date	Description
2.79c	Mar 2012	Version change for decoupling of DW_memctl from dw_iip_amba image.
2.79b	Oct 2011	Version change for 2011.10a release.
2.79a	Jun 2011	Version change for 2011.06a release; updated system diagram in Figure 1-1; enhanced “Related Documents” section in Preface.
2.78b	Apr 2011	Removed code for SYNC_FLASH_OPCODE, SFCONR, SFCTLR, and SFTMGR in example in Programming chapter; added material for component type, version, and parameter registers.
2.78a	Sep 2010	Corrected names of include files and vcs command used for simulation; corrected d_dqs signal to s_dqs in register insertion illustrations; corrected sm_oe_n signal waveform in write access diagram; corrected links for DesignWare Memory Models; specified that DW_memctl does not support 2-bank SDRAM.
2.75b	Dec 2009	Updated databook to new template for consistency with other IIP/VIP/PHY databooks; corrected clock conditions in which s_we_n signal is synchronous.
2.75b	May 2008	Removed references to QuickStarts, as they are no longer supported.
2.75b	Jun 2008	Version change for 2008.06a release.

Table 1-1 Databook Revision History

Version	Databook Date	Description
2.75a	March 27, 2008	Added: ■ Section “Using CellularRAM in Asynchronous Burst Mode” on page 110 ■ New section “Aborted Pre-Fetched Reads or Write Address Setup (with Ready Signal)” on page 122 ■ The following note in several places in the databook:  Note If you configure the DW_memctl as both a Static memory controller and an SDRAM controller, then the Static memory data width must be less than or equal to the SDRAM data width. ■ Synchronous to: <clock> field to Chapter 7, “Signals” on page 145 ■ Updated: ■ The <i>DesignWare DW_memctl User Manual and DesignWare DW_memctl Application Notes</i> have been merged into this databook. From this release forward, the information included in those manuals will be maintained in this databook. ■ Extended Mode Register to clarify how it is automatically updated during initialization (see the note in EXN_MODE_REG.dll register bit field) ■ Web page addresses on a number of broken links ■ Section name “Installing DWMMs Manually” on page 218
2.74b	June 14, 2007	Version changed due to upgrade of supported tools.
2.74a	April 16, 2007	Version that went out in General AMBA release 2007.04a.

Web Resources

- DesignWare IP product information: <http://www.designware.com>
- Your custom DesignWare IP page: <http://www.mydesignware.com>
- Documentation through SolvNet: <http://solvnet.synopsys.com> (Synopsys password required)
- Synopsys Common Licensing (SCL): <http://www.synopsys.com/keys>

Customer Support

To obtain support for your product:

- First, prepare the following debug information, if applicable:
 - For environment setup problems or failures with configuration, simulation, or synthesis that occur within coreConsultant or coreAssembler, use the following menu entry:
File > Build Debug Tar-file

Check all the boxes in the dialog box that apply to your issue. This menu entry gathers all the Synopsys product data needed to begin debugging an issue and writes it to the file `<core tool startup directory>/debug.tar.gz`.

- For simulation issues outside of coreConsultant or coreAssembler:
 - Create a waveforms file (such as VPD or VCD)
 - Identify the hierarchy path to the DesignWare instance
 - Identify the timestamp of any signals or locations in the waveforms that are not understood
- Then, contact Support Center, with a description of your question and supplying the above information, using one of the following methods:
 - *For fastest response*, use the SolvNet website. If you fill in your information as explained below, your issue is automatically routed to a support engineer who is experienced with your product. The **Sub Product** entry is critical for correct routing.

Go to <http://solvnet.synopsys.com/EnterACall> and click on the link to enter a call. Provide the requested information, including:

- **Product:** DesignWare Library IP
- **Sub Product:** Memory
- **Tool Version:** *product version number*
- **Problem Type:**
- **Priority:**
- **Title:** DW_memctl
- **Description:** For simulation issues, include the timestamp of any signals or locations in waveforms that are not understood

After creating the case, attach any debug files you created in the previous step.

- Or, send an e-mail message to support_center@synopsys.com (your email will be queued and then, on a first-come, first-served basis, manually routed to the correct support engineer):
 - Include the Product name, Sub Product name, and Tool Version number in your e-mail (as identified above) so it can be routed correctly.
 - For simulation issues, include the timestamp of any signals or locations in waveforms that are not understood
 - Attach any debug files you created in the previous step.
- Or, telephone your local support center:
 - North America:
Call 1-800-245-8005 from 7 AM to 5:30 PM Pacific time, Monday through Friday.
 - All other countries:
<http://www.synopsys.com/Support/GlobalSupportCenters>

1

Product Overview

The DesignWare DW_memctl is a memory controller that you can configure and synthesize in order to control Synchronous DRAMs – SDR-SDRAM, Mobile SDR-SDRAM, Mobile DDR-SDRAM, DDR-SDRAM – as well as Static memories – SRAMs and FLASHes.

**Note**

Unless otherwise noted in this manual, SDR-SDRAM, Mobile SDR-SDRAM (also known as BAT-RAM or low-power SDRAM), Mobile DDR-SDRAM (also known as LPDDR), and DDR-SDRAM are collectively referred to as an SDRAM. Similarly, “Static memory” refers to SRAMs and NOR-FLASH memories, unless otherwise noted; NAND-FLASH memories are not supported.

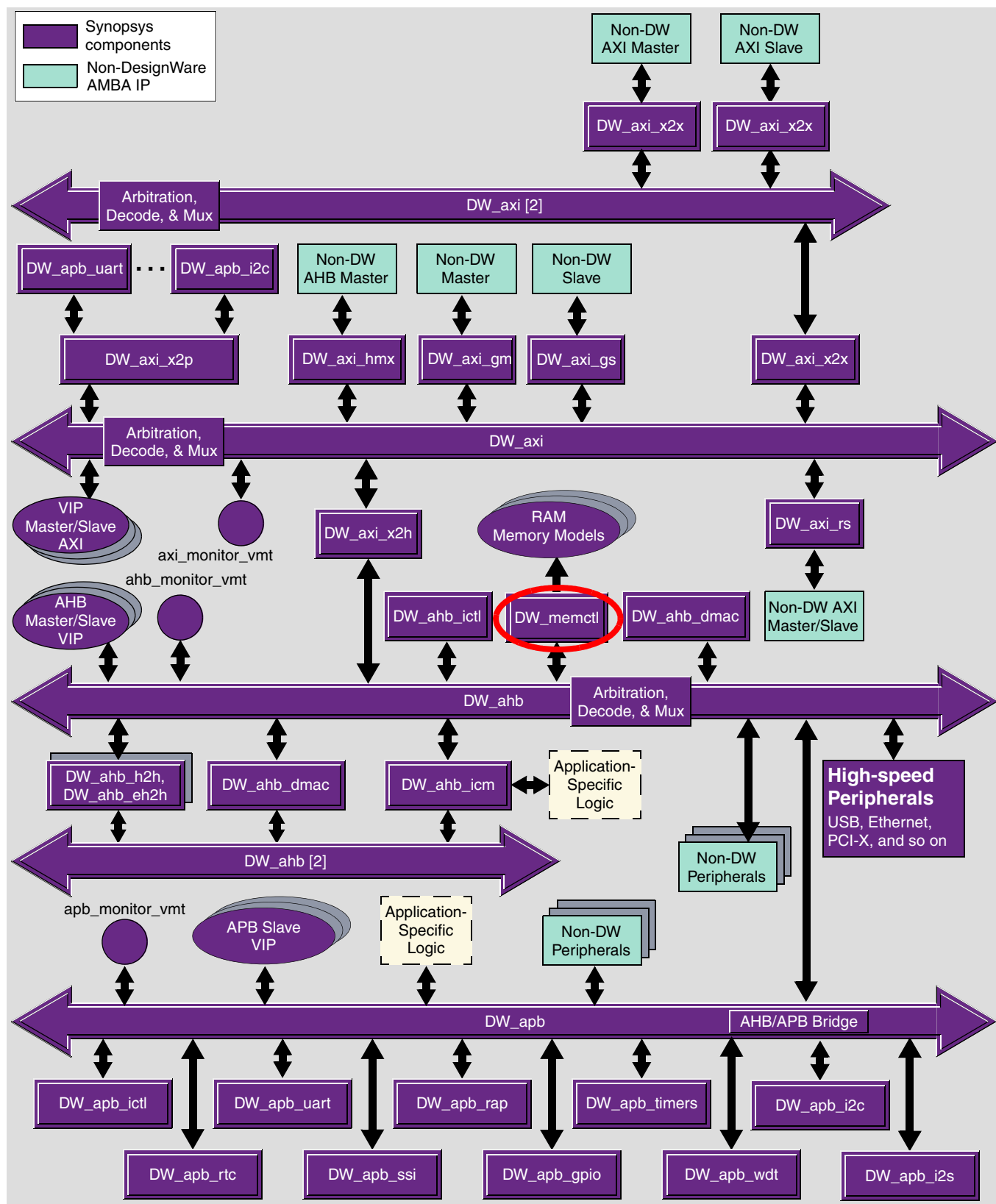
1.1 DesignWare System Overview

The Synopsys DesignWare Synthesizable Components environment is a parameterizable bus system containing AMBA version 2.0-compliant AHB (Advanced High-performance Bus) and APB (Advanced Peripheral Bus) components, and AMBA version 3.0-compliant AXI (Advanced eXtensible Interface) components.

Figure 1-1 illustrates one example of this environment, including the AXI bus, the AHB bus, and the APB bus. Included in this subsystem are synthesizable IP for AXI/AHB/APB peripherals, bus bridges, and an AXI interconnect and AHB bus fabric. Also included are verification IP for AXI/AHB/APB master/slave models and bus monitors. In order to display the databook for a DW_* component, click on the corresponding component object in the illustration.

**Attention**

Links resolve only if you are viewing this databook from your \$DESIGNWARE_HOME tree, and to only those components that are installed in the tree.

Figure 1-1 Example of DW_memctl in a Complete System

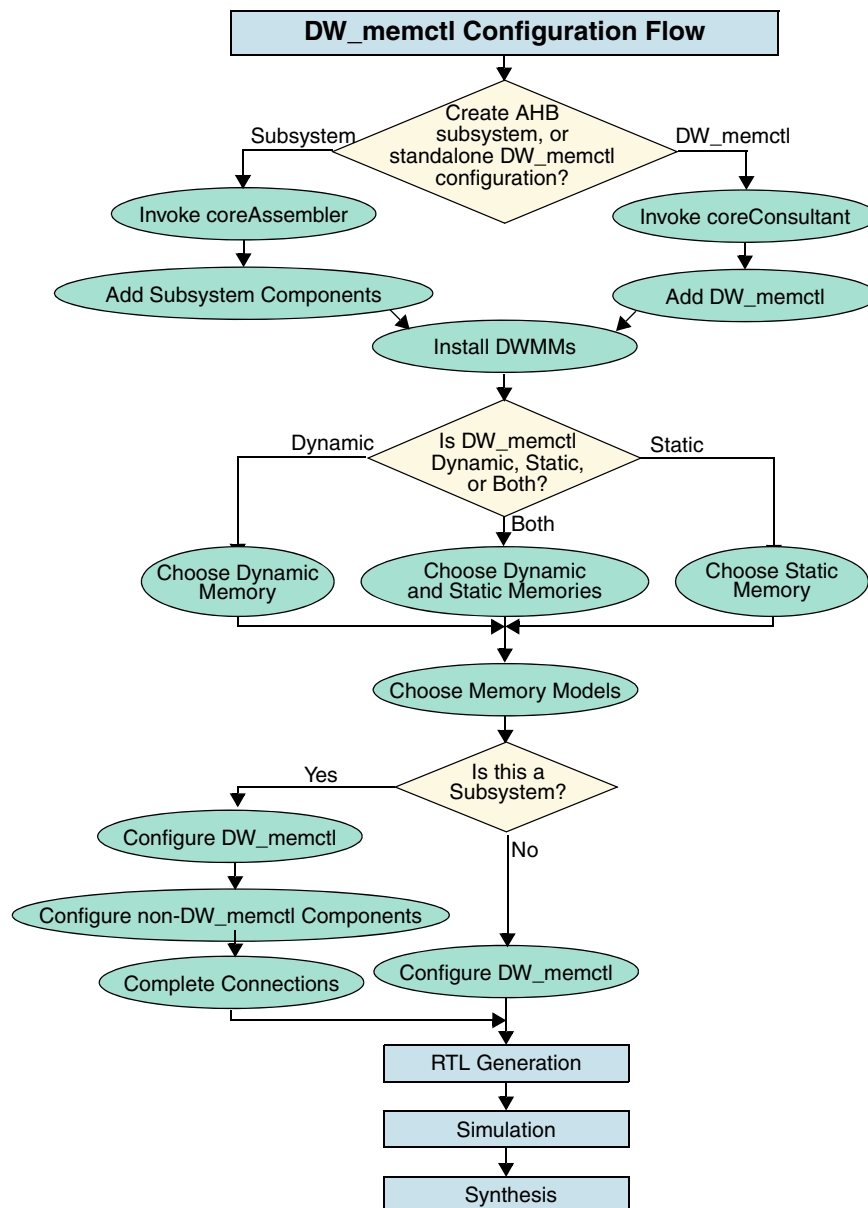
You can connect, configure, synthesize, and verify the DW_memctl within a DesignWare subsystem using coreAssembler, documentation for which is available on the web in the [coreAssembler User Guide](#).

If you want to configure, synthesize, and verify a single component such as the DW_memctl component, you might prefer to use coreConsultant, documentation for which is available in the [coreConsultant User Guide](#).

1.2 Configuration Flow

Figure 1-2 illustrates the steps in the DW_memctl configuration flow. You can click on the hyperlinks to take you directly to the steps in which you are interested.

Figure 1-2 DW_memctl Configuration Flow



1.3 General Product Description

To provide a complete solution for implementing a memory control interface in an ASIC design, the DW_memctl includes:

- DW_memctl – A memory controller that you can configure, synthesize, and use to control synchronous DRAMs (SDRAMs) and Static memories; also provides limited support for synchronous SRAM and FLASH interface.

The DW_memctl, illustrated in [Figure 1-3](#) on page 17, consists of two primary components:

- AMBA Host Interface Unit (HIU) – Generates memory read/write requests or register read/write requests to the MIU block
 - Memory Interface Unit (MIU) – Generates address, data, and control signals corresponding to read/write transfers for a memory device
- DW_memctl Verification Environment – A verification environment for the DW_memctl that includes an automated test suite and testbench. The DW_memctl also includes AHB BFM on the AMBA side, as well as DWMMs.



Attention

Although the DW_memctl supports as many DesignWare Memory Models (DWMMs) as possible through automation, due to continuous development on DWMMs and inconsistent vendor specifications, there can be interoperability issues that result in simulation failures. You should check the automatically resolved timing configuration parameters and manually overwrite them if necessary. Also there can be issues resolving the ports on automated testbench generation; you can correct this by analyzing the `sim/testbench/mem_banks_auto.v` model instantiation file in your workspace.

For information on how to download the DW_memctl, refer to the [DesignWare SDRAM and Static Memory Controller Installation Guide](#).

- VERA¹ Compile Capability – The DW_memctl verification environment is based on encrypted source code VERA BFMs. The top-level testbench is a Verilog wrapper that connects the DW_memctl to the VERA shell component of the testbench.

DesignWare products include several licenses, one of which is DesignWare-VERA. When using the DW_memctl, you must install VERA, but you do not need a VERA license to do so. Only when you verify the DW_memctl in the VERA-based testbench do you need the VERA license and tree.

For information on how to download VERA, refer to the [DesignWare SDRAM and Static Memory Controller Installation Guide](#).

- Synopsys' coreAssembler – A software tool that guides you through configuration, simulation, and synthesis of the DW_memctl. With coreAssembler, you do not have to manually edit HDL source files or write/edit synthesis scripts. coreAssembler automatically generates a user-defined

1. VERA automates critical portions of the verification process, reducing the process of verifying complex designs; a restricted VERA compile license is included with the DesignWare library. The VERA system is based on VERA HVL, a language that allows functional verification. You do not need to know VERA in order to use it in the DW_memctl verification environment.

configuration of the DW_memctl, based on your design requirements, and automatically drives the synthesis process to create a gate-level implementation that is optimized for your DW_memctl configuration and the cell set characteristics of your technology library.

The DW_memctl is specifically designed for use with coreAssembler, which provides you with automatic configuration, test suite operation, and synthesis for the DW_memctl.

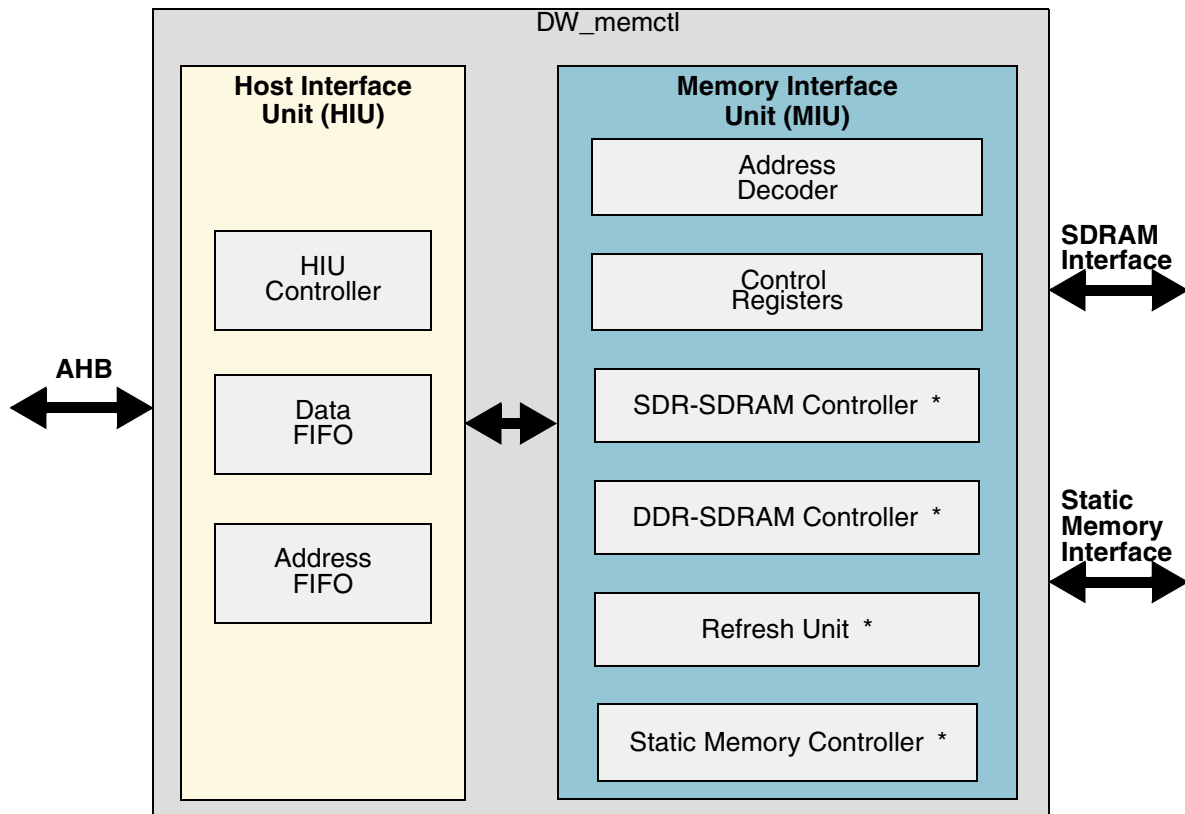
The Synopsys DW_memctl conforms to the [AMBA Specification, Revision 2.0](#) from ARM.

1.3.1 DW_memctl Block Diagram

As shown in [Figure 1-3](#), the DW_memctl consists of the following main functional blocks:

- AMBA Host Interface Unit (HIU) – Interface between the DW_memctl and the AMBA Advanced High-performance Bus (AHB).
- Memory Interface Unit (MIU) – Interface for both SDRAM and Static memories; it generates appropriate address, data, and control signals corresponding to memory read/write transfers.

Figure 1-3 DW_memctl Block Diagram



Note: * Conditional Instantiations

1.4 Features

The following are features of the DW_memctl.

1.4.1 AMBA AHB Interface Features

The following are features of the AMBA AHB interface to the DW_memctl:

- AMBA AHB bus-compatible
- Supports pin-based little- or big-endian modes of operation
- Supports all AMBA fixed-length bursts (SINGLE, WRAP4, INCR4, WRAP8, INCR8, WRAP16, and INCR16), as well as undefined-length bursts (INCR)



Attention

DW_memctl is not optimized for partial burst transfers on AHB – All partial bursts are internally converted to single transfers, and burst access protocol to a memory—especially DRAM—is not exploited, which incurs performance loss. An example of this is when a burst transfer with HSIZE of 16 bits is performed on a 32-bit wide AHB configuration.

- Supports AHB data widths of 32, 64, or 128 bits
- Supports AHB address width of 32 bits
- Supports narrow access on wide AMBA bus
- Supports busy and early terminations on AHB transactions
- Does not generate split, retry, or error responses on the AMBA bus
- Two-clock-cycle latency from AMBA bus hsel_mem assertion to issue of memory command, depending on optional registering of memory control and data signals
- Supports separate or shared memory address and/or data buses between SDRAM and Static memories
- Supports optional interface for multiplexing memory address and data pins with address and data pins of other external interface blocks

1.4.2 SDRAM Interface Features

The following are features of the SDRAM interface to the DW_memctl:

- Glueless connection to all JEDEC-compliant SDRAM
- Supports up to 16 SDRAM address bits
- SDR-SDRAM and Mobile SDR-SDRAM memory data width that can be configured to 16, 32, 64, or 128, with 1:1 or 1:2 ratios with AHB data width; does *not* include 2-bank mobile SDRAM. In the case of DDR-SDRAM and Mobile DDR-SDRAM, memory data width can be configured to 8, 16, 32, or 64, with 1:2 or 1:4 ratios with the AHB data width.

- Programmable row and column address bit widths up to:
 - 15-bit column address
 - 16-bit row address
 - 4-bit bank address
- Supports 2K to 64K rows, 256 to 32K columns, and 2 to 16 banks
- Supports up to 8 chip selects, with a maximum of 4 GB of address space per chip select
- Compile-time and programmable configuration parameters
- SDRAM timing parameters – tRAS, tRCD, tRP, tWR, tWTR, tRCAR, tXSR, and tRC, – can be programmed to values supported by different SDRAM vendors
- Supports auto refresh with programmable refresh intervals
- Supports self-refresh
- Supports SDRAM power-down mode
- Programmable immediate precharge or delayed precharge modes
- Supports 1 to 16 (programmable) open banks for performance; pages can be non-contiguous – Least Recently Used (LRU) algorithm used during page miss replacements
- Supports SDRAM module serial presence detect

1.4.3 Static Memory Interface Features

The following are features of the Static Memory interface to the DW_memctl:

- Supports asynchronous SRAMs and page-mode FLASHes
- Supports up to three sets of timing registers
- Configurable address width of up to 32 bits
- Memory data width can be configured to 8, 16, 32, 64, or 128; can be 1:1 to 1:16 ratio with the AHB data width
- Supports external “READY” handshake pin to interface non-SRAM-type device
- Limited synchronous SRAM and FLASH interface support
- Synchronous SRAM and FLASH frequency could be 1, 1/2, 1/3, 1/4, and so on of the AHB frequency

1.5 Standards Compliance

The DW_memctl component conforms to the [AMBA Specification, Revision 2.0](#) from ARM. Readers are assumed to be familiar with this specification.

1.6 Verification Environment Overview

The DW_memctl includes an extensive verification environment, which sets up and invokes your selected simulation tool to execute tests that verify the functionality of the configured component. You can then analyze the results of the simulation.

[“Verification”](#) on page 205 discusses the specific procedures for verifying the DW_memctl.

1.7 Licenses

Before you begin using the DW_memctl, you must have a valid license. For more information, refer to [“Licenses”](#) in the *the DesignWare SDRAM and Static Memory Controller Installation Guide*.

1.8 Where To Go From Here

At this point, you may want to get started working with the DW_memctl component within a subsystem or by itself. Synopsys provides several tools within its coreTools suite of products for the purposes of configuration, synthesis, and verification of single or multiple synthesizable IP components – coreConsultant and coreAssembler. For information on the different coreTools, refer to [Guide to coreTools Documentation](#).

For more information about configuring, synthesizing, and verifying just your DW_memctl component, refer to [“Overview of the coreConsultant Configuration and Integration Process”](#) on page 22.

For more information about implementing your DW_memctl component within a DesignWare subsystem using coreAssembler, refer to [“Overview of the coreAssembler Configuration and Integration Process”](#) on page 25.

2

Building and Verifying a Component or Subsystem

DesignWare Synthesizable IP (SIP) components for AMBA 2 and AMBA 3 AXI are packaged using Synopsys coreTools, which enable the user to configure, synthesize, and run simulations on a single SIP title, or to build a configured AMBA subsystem. You do this by generating a workspace view using one of the following coreTools applications:

- **coreConsultant** – Used for configuration, RTL generation, synthesis, and execution of packaged verification for a single SIP title. The [coreConsultant User Guide](#) provides complete information on using coreConsultant.
- **coreAssembler** – Used for building and configuration of a subsystem that connects multiple SIP titles, RTL generation, synthesis, and creation of a template subsystem testbench. The [coreAssembler User Guide](#) provides complete information on using coreAssembler.

A workspace is your working version of a DesignWare SIP component or subsystem. In fact, you can create several workspaces to experiment with different design alternatives.

**Hint**

If you are unfamiliar with coreTools—which is comprised of the coreAssembler, coreConsultant, and coreBuilder tools—you can go to [Using DesignWare Library IP in coreAssembler](#) to “get started” learning how to work with DesignWare SIP components.

2.1 Setting up Your Environment

The DW_memctl is included in a release of DesignWare SIP components. It is assumed that you have already downloaded and installed the release. If you have not, you can download and install the latest versions of required tools using the [DesignWare SDRAM and Static Memory Controller Installation Guide](#).

You also need to set up your environment correctly using specific environment variables, such as DESIGNWARE_HOME, VERA_HOME, PATH, and SYNOPSYS. If you are not familiar with these requirements and the necessary licenses, refer to “[Setting up Your Environment](#)” in the *DesignWare SDRAM and Static Memory Controller Installation Guide*.

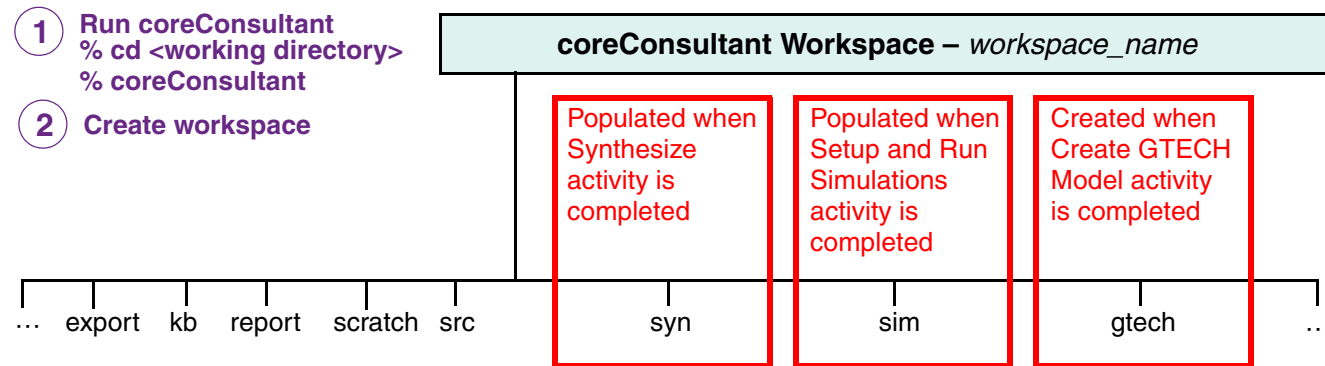
2.2 Overview of the coreConsultant Configuration and Integration Process

Once you have correctly downloaded and installed a release of DesignWare SIP components and then set up your environment, you can begin work on the DW_memctl using coreConsultant.

2.2.1 coreConsultant Usage

Figure 2-1 illustrates some general directories and files in a coreConsultant workspace.

Figure 2-1 coreConsultant Usage Flow



3 Use coreConsultant to create, synthesize, and verify your component

Table 2-1 provides a description of the implementation workspace directory and subdirectories.

Table 2-1 coreConsultant Implementation Workspace Directory Contents

Directory/Subdirectory	Description
auxiliary	Scripts and text files used by coreConsultant. Generated upon first creating workspace.
doc	Contains local copies of component-specific databooks. Generated upon first creating workspace.
export	Contains files used to integrate results from the completed source configuration and synthesis activities into your design (outside coreConsultant). Generated upon first creating workspace; populated during Specify Configuration activity.
gtech	Contains synthesis scripts and output netlists from gtech generation; also used for RTL simulation of encrypted source code. Generated during Generate GTECH Model activity.
kb	Contains knowledge base information used by coreConsultant. These are binary files containing the state of the design. Generated upon first creating workspace; populated and updated throughout activities.

Table 2-1 coreConsultant Implementation Workspace Directory Contents (Continued)

Directory/Subdirectory	Description
leda	Contains Leda configuration files for the component. Generated upon first creating workspace; updated during Run Leda Coding Checker activity.
pkg	Contains RTL preprocessor scripts. Generated during Specify Configuration activity.
report	Contains all of the reports created by coreConsultant during build, configuration, test and synthesis phases. An index.html file in this directory links to many of these generated reports. Generated upon first creating workspace; populated and updated throughout activities.
scratch	Contains temp files used during the coreConsultant processes. Generated upon first creating workspace; populated and updated throughout activities.
sim	Contains test stimulus and output files. Generated upon first creating workspace; updated during Setup and Run Simulations activity.
src	Includes the top-level RTL file, <i>design_name.v</i> . If you have a source license, this will contain plain-text RTL; if you only have a designware license, this will contain encrypted RTL. Generated upon first creating workspace; populated during Specify Configuration activity.
syn	Contains synthesis files for the component. Generated upon first creating workspace; updated during Synthesis activity and Formal Verification activity.
tcl	Contains synthesis intent scripts. Generated upon first creating workspace.

For details on some key files created during coreConsultant activities, refer to [“Database Files”](#) on page 29.

For information on using coreConsultant, refer to the [coreConsultant User Guide](#).

2.2.2 Configuring the DW_memctl within coreConsultant

The [“Parameters”](#) chapter on [page 125](#) describes the DW_memctl hardware configuration parameters that you configure using the coreConsultant GUI.

The [“Creating the RTL View of a Core”](#) chapter in the [coreConsultant User Guide](#) discusses how to specify a configuration for an individual component like the DW_memctl.

2.2.3 Creating Gate-Level Netlists within coreConsultant

The “Creating the Gate-Level Netlist for a Core” chapter in the *coreConsultant User Guide* discusses how to create a translation of the RTL view into a technology-specific netlist for an individual component like the DW_memctl.

2.2.4 Verifying the DW_memctl within coreConsultant

The “Verification” chapter on [page 205](#) provides an overview of the testbench available for DW_memctl verification using the coreConsultant GUI.

The “Verifying Your Implementation” chapter in the *coreConsultant User Guide* discusses how to simulate an individual component like the DW_memctl.

2.2.5 Running Leda on Generated Code with coreConsultant

When you select **Verify Component > Run Leda Coding Checker** from the Activity List, the corresponding Activity View appears. In this Activity View you select rules configuration file and define Leda command line switches.

Once you have correctly downloaded and installed a release of DesignWare SIP components and then set up your environment, you can begin work on your DesignWare subsystem with coreAssembler.

Figure 2-2 illustrates some general directories and files in a coreAssembler workspace.

1 Run coreAssembler
`% cd <working directory>`
`% coreAssembler`

2 Create workspace

coreAssembler Workspace – workspace_name

Directory containing subsystem-specific directories and files

export kb report scratch src syn

components

i_ahb ... i_ahb_h2h ... i_apb ... i_apb_ssi ... i_axi ... i_axi_x2x

Directory containing component-specific directories and files

... export kb report scratch src syn sim gtech ...

... export kb report scratch src syn sim gtech ...

... export kb report scratch src syn sim gtech ...

...

25

Table 2-2 provides a description of the implementation workspace directory and subdirectories.

Table 2-2 coreAssembler Implementation Workspace Directory Contents

Directory/Subdirectory	Description
components	Contains a directory for each IP component instance connected in the subsystem. Generated and populated with separate component directories upon first adding components; populated and updated throughout activities.
i_component/auxiliary	Scripts and text files used by coreAssembler. Generated during Add Subsystem Components activity.
i_component/doc	Contains local copies of component-specific databooks. Generated during Add Subsystem Components activity.
i_component/export	Contains files used to integrate results from the completed source configuration and synthesis activities into your design (outside coreAssembler). Generated during Add Subsystem Components activity; populated during Configure Components activity.
i_component/gtech	Contains synthesis scripts and output netlists from gtech generation; also used for RTL simulation of encrypted source code. Generated during Create Component GTECH Simulation Model activity.
i_component/kb	Contains knowledge base information used by coreAssembler. These are binary files containing the state of the design. Generated during Add Subsystem Components activity; populated and updated throughout activities.
i_component/leda	Contains Leda configuration files for the component. Generated during Add Subsystem Components activity; populated during Run Leda Coding Checker (for /i_component/) activity.
i_component/pkg	Contains RTL preprocessor scripts. Generated during Configure Components activity.
i_component/report	Contains all of the reports created by coreAssembler during build, configuration, test and synthesis phases. An index.html file in this directory links to many of these generated reports. Generated during Add Subsystem Components activity; populated and updated throughout activities.
i_component/scratch	Contains temp files used during the coreAssembler processes. Generated during Add Subsystem Components activity; populated and updated throughout activities.
i_component/sim	Contains test stimulus and output files. Generated during Add Subsystem Components activity; updated during Setup and Run Simulations (for /i_component/) activity.

Table 2-2 coreAssembler Implementation Workspace Directory Contents (Continued)

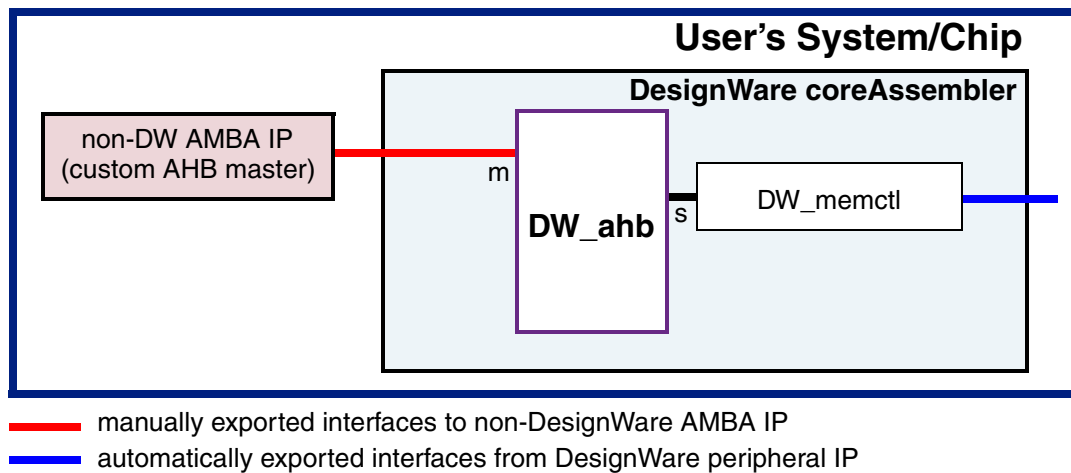
Directory/Subdirectory	Description
<code>i_component/src</code>	Includes the top-level RTL file, <i>design_name.v</i> . If you have a source license, this will contain plain-text RTL; if you only have a designware license, this will contain encrypted RTL. Generated during Add Subsystem Components activity; populated during Specify Configuration activity.
<code>i_component/syn</code>	Contains synthesis files for the component. Generated during Add Subsystem Components activity; updated during Synthesis activity.
<code>i_component/tcl</code>	Contains synthesis intent scripts. Generated during Add Subsystem Components activity.
<code>export</code>	Contains subsystem files used to integrate the results from the completed source configuration and synthesis activities into your design (outside coreAssembler). Generated upon first creating workspace; populated starting with Memory Map Specification activity.
<code>kb</code>	Contains subsystem knowledge base information used by coreAssembler. These are binary files containing the state of the design. Generated upon first creating workspace; populated and updated throughout activities.
<code>report</code>	Contains subsystem reports created by coreAssembler during build, configuration, test and synthesis phases. An <code>index.html</code> file in this directory links to many of these generated reports. Generated upon first creating workspace; populated and updated throughout activities.
<code>scratch</code>	Contains subsystem temp files used during the coreAssembler processes. Generated upon first creating workspace; populated and updated throughout activities.
<code>src</code>	Includes the RTL related to the subsystem. If you have a source license, this will contain plain-text RTL; if you only have a designware license, this will contain encrypted RTL. Generated upon first creating workspace; populated starting with Generate Subsystem RTL activity.
<code>syn</code>	Contains synthesis files for the subsystem. Generated upon first creating workspace; updated during Synthesize activity and Formal Verification activity.

For details on some key files created during coreAssembler activities, refer to [“Database Files”](#) on page 29.

For information on using coreAssembler, refer to the [coreAssembler User Guide](#). For information on getting started with using DesignWare SIP components for AMBA 2 and AMBA 3 AXI components within coreTools, refer to [Using DesignWare Library IP in coreAssembler](#).

Figure 2-3 illustrates the DW_memctl in a simple subsystem.

Figure 2-3 DW_memctl in Simple Subsystem



The subsystem in Figure 2-3 contains the following components that you may want to use as you learn to use coreAssembler:

- DW_memctl
- DW_ahb
- AHB Master

The AHB Master is meant to be exported out of the design and then replaced by a real AHB Master – such as a CPU – later in the design process; at least one exported AHB master is required in a subsystem if you intend to do a basic simulation that tests connections.

2.3.2 Configuring the DW_memctl within a Subsystem

The “Parameters” chapter on page 125 describes the DW_memctl hardware configuration parameters that you configure using the coreAssembler GUI. Corresponding databooks for the other components in a subsystem contain “Parameters” chapters that describe their respective configuration parameters.

The “Creating the RTL View of a Subsystem” chapter in the *coreAssembler User Guide* discusses how to configure subsystem components and automatically connect them using the coreAssembler GUI.

2.3.3 Creating Gate-Level Netlists within coreAssembler

The “Creating the Gate-Level Netlist for a Subsystem” chapter in the *coreAssembler User Guide* discusses how to create a translation of the RTL view into a technology-specific netlist for a subsystem.

2.3.4 Verifying the DW_memctl within coreAssembler

The “Verification” chapter on page 205 provides an overview of the testbench available for DW_memctl verification using the coreAssembler GUI.

The “Verifying Subsystems and Components” chapter in the *coreAssembler User Guide* discusses how to simulate a subsystem.

2.3.5 Running Leda on Generated Code with coreAssembler

When you select **Verify Component > Run Leda Coding Checker for */i_component*** from the Activity List, the corresponding Activity View appears. In this Activity View you select rules configuration file and define Leda command line switches.

2.4 Database Files

The following subsections describe some key files created in coreConsultant and coreAssembler activities.

2.4.1 Design/HDL Files

The following sections describe the design and HDL files that are produced by coreConsultant and coreAssembler when configuring and verifying a DesignWare Synthesizable Component. The following files are created in different directories by coreConsultant and coreAssembler:

- coreConsultant – *workspace/* directory
- coreAssembler – *workspace/components/i_component/* directory

2.4.1.1 RTL-Level Files

The following table describes the RTL files that are generated by the Create RTL activity. They are encrypted except where otherwise noted. Any Synopsys synthesis tool or simulator can read encrypted RTL files.

Table 2-3 RTL-Level Files

Files	Encrypted?	Purpose
<i>./src/component_cc_constants.v</i>	No	Includes definitions and values of all configuration parameters that you have specified for the component.
<i>./src/component.v</i>	No	Top-level HDL file. Include the DesignWare libraries by using the following options in your simulator invocation: +libext+.v+.V -y \${SYNOPSYS}/packages/gtech/src_ver -y \${SYNOPSYS}/dw/sim_ver
<i>./src/component_submodule.v</i>	Yes	Sub-modules of component
<i>./src/component_constants.v</i>	No	Includes the constants used internally in the design.
<i>./src/component_undef.v</i>		Includes an undef for each of the definitions found in the <i>component_cc_constants.v</i> file; compiled in after the last file listed in <i>./src/components.lst</i> when compiling multiple instances of the same IP.
<i>./src/component.lst</i>	No	Lists the order in which the RTL files should be read into tools, such as simulators or dc_shell. For example, use the following option to read the design into VCS: vcs +v2k -f component.lst

2.4.1.2 Simulation Model Files

The following table includes files generated for the component during the Generate GTECH Simulation activity. These files are needed when you are using a non-Synopsys simulator (when you can not use the encrypted RTL).

Table 2-4 Simulation Model Files

Files	Encrypted?	Purpose
<code>./gtech/final/db/component.v</code>	No	Simulation model of the component for use with non-Synopsys simulators. A technology-independent, gate-level netlist; VHDL and Verilog versions are generated. Include the DesignWare libraries by using the following options in your simulator invocation: +libext+.v+.V -y \${SYNOPSYS}/packages/gtech/src_ver -y \${SYNOPSYS}/dw/sim_ver

2.4.2 Synthesis Files

The following table includes files generated after the Create Gate-Level Netlist activity is performed on a component.

Table 2-5 Synthesis Files

Files	Encrypted?	Purpose
<code>./syn/auxScripts</code>	No	Auxiliary files for synthesis.
<code>./syn/final/db/component.db</code>	Binary format	Synopsys .db files (gate level) that can be read into dc_shell for further synthesis, if desired.
<code>./syn/final/db/component.v</code>	No	Gate-level netlist that is mapped to technology libraries that you specify.
<code>./syn/constrain/script/*.*</code>	No	Constraint files for the components.
<code>./syn/final/report/*.*</code>	No	Synthesis result files.

2.4.3 Verification Reference Files

Files described in the following table include information pertaining to the component's operation so that you can verify installation and configuration of the component has been successful. These files are not for re-use during system-level verification.

Table 2-6 Verification Reference Files

Files	Encrypted?	Purpose
<code>./sim/runtest</code>	No	Perl script that runs the Setup and Run Simulations activity from the command line.
<code>./sim/runtest.log</code>	No	The overall result of simulation, including pass/fail results.
<code>./sim/test_testname/test.result</code>	No	Pass/fail of individual test.

Table 2-6 Verification Reference Files

Files	Encrypted?	Purpose
./sim/test_ <i>testname</i> /test.log	No	Log file for individual test.

3

Architecture

This chapter provides an overview of the DW_memctl and design flow.

You can configure the DW_memctl to work as:

- Generic SDRAM controller, which supports SDR-SDRAM, Mobile SDR-SDRAM, Mobile DDR-SDRAM (LPDDR), DDR-SDRAM
- Generic SDRAM controller, which can support an SDR-SDRAM/Mobile
- Generic Static memory controllers, which support SRAMs and FLASHes (both can coexist)
- Both an SDRAM controller (SDR-SDRAM or Mobile SDR-SDRAM or Mobile DDR-SDRAM or DDR-SDRAM) and a Static memory controller



DW_memctl does not support 2-bank mobile SDRAM.

The DW_memctl provides an interface between each of the following memory devices and an AMBA AHB 2.0 bus:

- JEDEC-standard SDR-SDRAM
- JEDEC-standard Mobile SDR-SDRAM
- JEDEC-standard Mobile DDR-SDRAM (LPDDR)
- JEDEC-standard DDR-SDRAM
- Asynchronous SRAM, with or without page-mode
- Asynchronous FLASH, with or without page-mode
- Limited synchronous SRAM and FLASH interface support

These interfaces enable you to integrate the DW_memctl into embedded applications for system-on-a-chip (SOC) designs. The DW_memctl is an AHB slave in SOC designs and provides all the functionality for read/write transactions from multiple AHB masters to the interface memory device.

As an SDRAM controller, the DW_memctl can support up to sixteen row address bits, fifteen column address bits, and four bank address bits. As a Static memory controller, the DW_memctl can support up to thirty-two address bits.

The DW_memctl can support up to a maximum of eight different groups of external memory, each connected to a single chip select and responding to the same commands; these groups of external memory are sometimes also referred to as memory “banks.” Only one set of timing registers control SDRAMs, so different SDRAM banks must have the same timings. For Static memories, there are three different sets of timing registers.

Using the coreAssembler tool, you specify parameters based on your design specifications to describe the memory controller; coreAssembler then automatically configures, synthesizes, and verifies the components for your design environment. Additionally, if you want to use DesignWare Memory Models (DWMMs), coreAssembler generates preverified DWMMs and also automatically configures the DW_memctl core with the timing parameters for that particular DWMM.

3.1 Automation of DW_memctl Flow

The DW_memctl uses Synopsys DesignWare Memory Models (DWMMs) to configure the default parameters of the DW_memctl. The models are used for initializing the signal and timing parameters, as well as for automatically configuring the verification environment.

The DW_memctl coreKit is shipped with a “DWMMtable” file which lists hundreds of vendor-specific memories for which Synopsys has a DWMM model. The DWMMtable file has timing and signal information for all memory models supported by the DW_memctl. coreAssembler uses the most current DWMMtable file, which is located in the *workspace/components/instance_name/DWMM* directory.

If you think you have a supported model but it is not included in the DWMMtable file, then you can locate the memory models online at:

<http://www.synopsys.com/dw/dwmm.php>

If you need to request an updated DWMMtable, you need to open a Case through Solvnet:

<https://solvnet.synopsys.com>

Using the DWMMs, the DW_memctl design flow is straightforward and automatic. The process is as follows:

1. In the configuration window, coreAssembler lists all memories in the DWMM table.
2. For each memory group, you map one of the memories types in the list which you plan to use
3. The coreAssembler tool is used to download models of all the memories you have chosen from the Synopsys Web site. All selected models can be downloaded if the “Download DesignWare Memory Model” box is checked. coreAssembler launches an external web browser to the Synopsys Web Site, where you need to enter your SolvNet ID and password. The download must finish before the Component Configuration activity can be completed.



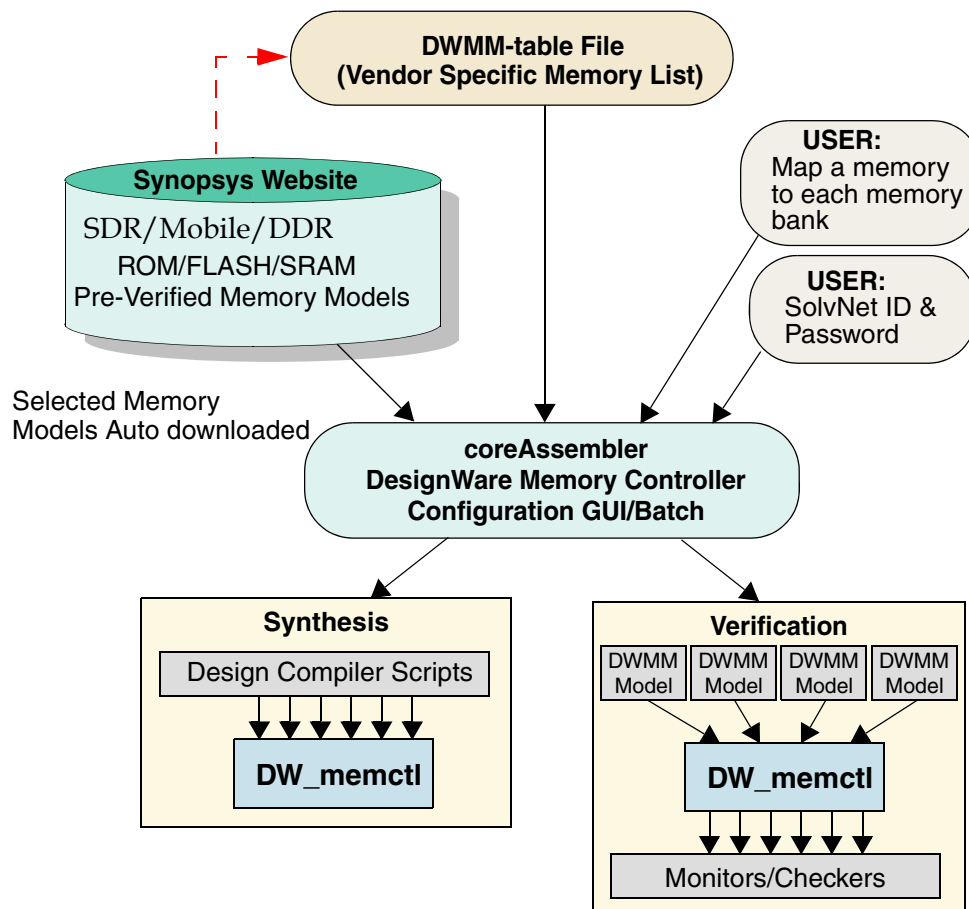
Note

If you do not intend to use the supplied DW_memctl simulation testbench (Verify Component activity group) nor the DesignWare Memory Models in your own testbench, then there is no need to check “Download DesignWare Memory Models”.

Depending on your AHB frequency selection, the default power-on values of the DW_memctl timing and signal configuration registers are automatically extracted from the DWMMtable and set. Setting other parameters, such as the number of chip selects and memory address mapping, completes the required data entry. You can set up all SDRAM-related parameters with default values, and you can configure them at runtime as well.

Figure 3-1 illustrates the DW_memctl design flow.

Figure 3-1 DW_memctl Memory Controller Design Flow



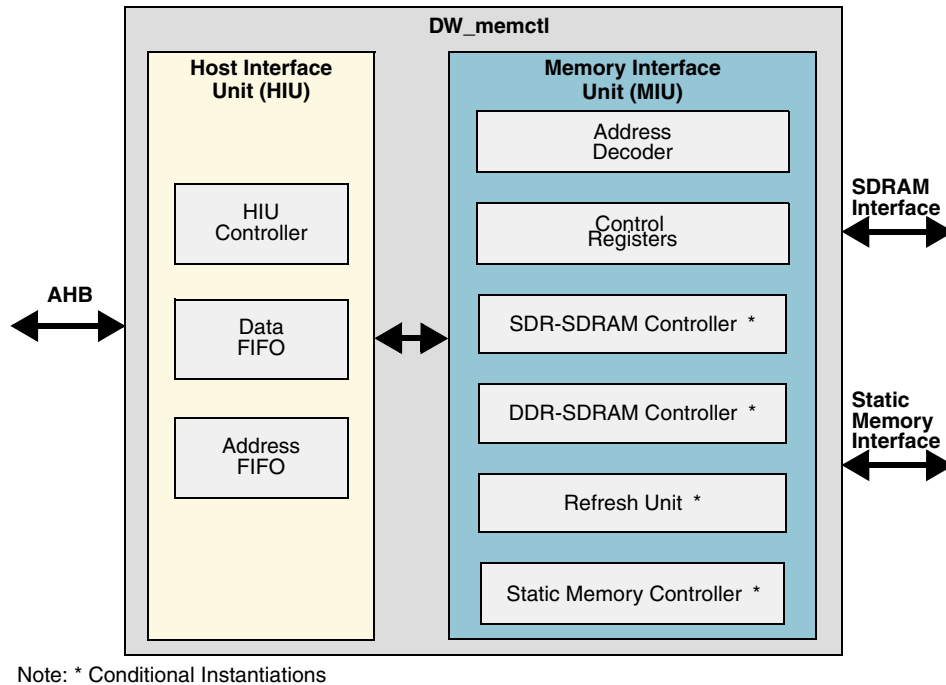
Synopsys' coreAssembler creates a verification environment, configuring it according to your parameter selections. For all memories you select, the automated testbench instantiates the required number of the corresponding DWMMs.

3.2 Block Diagram

The DW_memctl consists of two main functional blocks, illustrated in [Figure 3-2](#):

- AMBA Host Interface Unit (HIU)
- Memory Interface Unit (MIU)

Figure 3-2 DW_memctl Block Diagram



The host interface unit (HIU) identifies a register access request or a memory access request from the AHB bus to the DW_memctl slave. It has an internal address FIFO through which it channels all the requests to the memory interface unit (MIU). All requests move through the address FIFO in a pipeline fashion. For example, if an SRAM write occurs in the MIU and there is a pending FLASH read on the AHB interface, wait states for the FLASH read are inserted on the AHB bus until the SRAM writes are complete and the controller begins receiving the first data from the FLASH.

The DW_memctl as an AHB slave on the AMBA bus requires these select signals:

- One for the register select (hsel_reg)
- One for selecting the memory (hsel_mem)

Because it has two device select signals, the DW_memctl – although one physical AMBA slave device – is two logical AMBA slave devices. The select signal for the memory is common to the SDRAM/FLASH/SRAM. You specify the kind of memory to be connected to each of the chip selects, the base addresses, and the block sizes. The address decoder uses this information to compare the host address with chip select registers. It then generates corresponding requests to the various state machines, as well as the chip selects and control signals to the various memories.

Table 3-1 lists the design hierarchy of the DW_memctl.

Table 3-1 DW_memctl Design Hierarchy

File Name	Block Name
DW_memctl †	Top-level design
DW_memctl_hiu	Host interface unit (HIU)
DW_memctl_hiu_ctl	HIU control unit
DW_memctl_hiu_rbuf	HIU read buffer unit
DW_memctl_hiu_afifo	HIU address FIFO
DW_memctl_hiu_acore	HIU address FIFO core
DW_memctl_hiu_dfifo	HIU data FIFO
DW_memctl_hiu_dcore	HIU data FIFO core
DW_memctl_miu	Memory interface unit (MIU)
DW_memctl_miu_sdc	MIU SDR-SDRAM/Mobile SDR-SDRAM controller *
DW_memctl_miu_dsdcc *	MIU DDR-SDRAM controller
DW_memctl_miu_ddrwr †	MIU DDR-SDRAM 2x clock domain write data path
DW_memctl_miu_smc	MIU Static memory controller
DW_memctl_miu_dmc	MIU dummy ready controller
DW_memctl_miu_refctl	MIU refresh controller
DW_memctl_miu_cr	MIU control registers unit
DW_memctl_miu_smcr	MIU Static memory control register
DW_memctl_miu_addrdec	MIU address decoder

* — DW_memctl_miu_sdc, DW_memctl_miu_sfc, and DW_memctl_miu_dsdcc are conditionally instantiated. Only one is present in a controller.

† — The DW_memctl and DW_memctl_miu_ddrwr are provided in unencrypted source format.

3.3 AMBA Host Interface Unit (HIU)

The AMBA Host Interface Unit (HIU) is the interface between the DW_memctl and the AMBA Advanced High-performance Bus (AHB). The HIU generates memory read/write requests or control register read/write requests to the MIU block, which correspond to transfers on the AMBA bus; the HIU does not distinguish between an SDRAM request and an SRAM/FLASH request.

The HIU has the following functions:

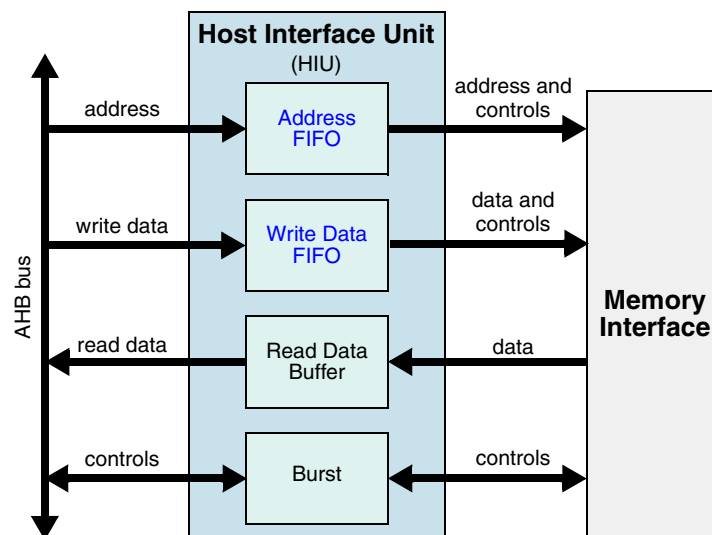
- Buffers register/memory access requests and sends them to the DW_memctl MIU
- Converts an AMBA burst size into a memory burst size, depending on the AMBA/memory device bus-width ratio
- Supports all fixed-length bursts (SINGLE, WRAP4, INCR4, WRAP8, INCR8, WRAP16, and INCR16) as well as undefined-length bursts (INCR)
- Supports AMBA early-burst termination
- Breaks AMBA wrapping burst into two separate memory bursts
- Supplies the wrapping address before the slave sees it on the AMBA in order to save overhead cycles between two bursts
- Detects memory page boundaries; terminates the current burst and reissues a new burst
- Masks invalid bytes for transfers that are narrower than the width of the AMBA bus

The HIU consists of the following sub-blocks:

- **Address FIFO** – Buffers the request of the AMBA AHB and sends memory/register access requests to the MIU; also contains some control information for a read/write transfer
- **Write Data FIFO** – Buffers write data to the memory and control registers
- **Read Data Buffer** – Buffers the read data from the memory
- **Burst Control** – Controls all the HIU sub-blocks by generating the control logic for read and write transfers

Figure 3-3 illustrates the relationship between the HIU sub-blocks and the interfaces to the MIU with which it communicates.

Figure 3-3 Host Interface Unit (HIU) Block Diagram



In order to coordinate register access requests and memory access requests, the register requests are queued through the address FIFO and the write data FIFO.

Since the register access shares the same read/write data path as the memory – for cases where the ratio of the AHB data width and SDRAM data width is 2:1 – register access requires two internal cycles between the HIU and MIU to complete the transfer. For example, if the AHB data width is 32 and the SDRAM data width is 16, it takes two 16-bit write cycles from the HIU to the MIU.

You can choose to configure the DW_memctl as only a Static memory controller, in which case the ratio of the register interface data width to the AHB data width is 1:1, regardless of the width of the Static memory data.

3.3.1 Address FIFO

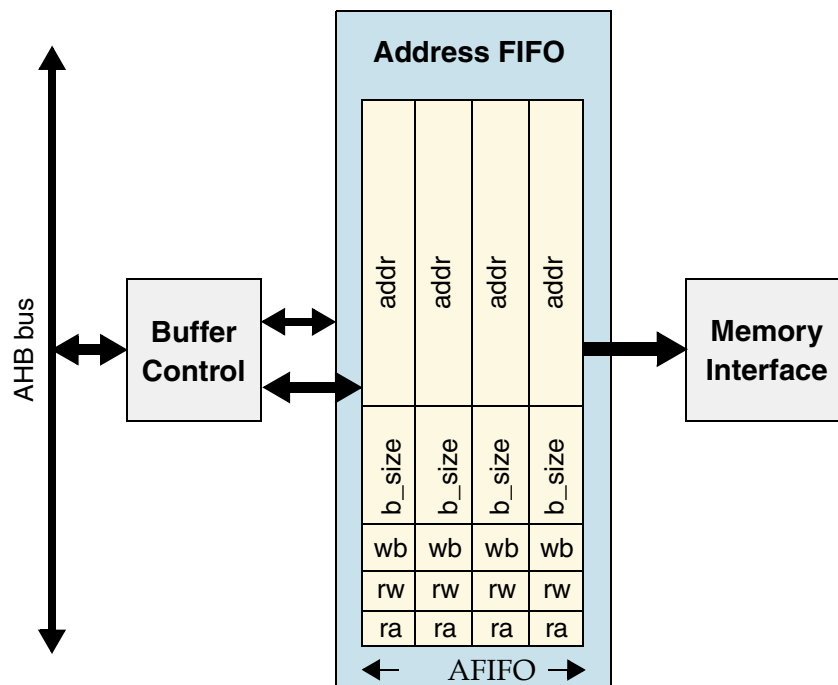
The address FIFO stores the AMBA address that the MIU needs to decode for data transfers; only the start address associated with a burst is stored, except under the following circumstances:

- If the start address is not aligned, a wrapping burst (WRAP4, WRAP8, or WRAP16) takes two FIFO entries.
- If the SDRAM page boundary is crossed, the SDRAM burst is broken in two and takes two FIFO entries.

Once the burst is complete, the next write or read transfer start address is sent to the MIU.

Figure 3-4 illustrates the structure of an address FIFO.

Figure 3-4 Host Interface Unit (HIU) Address FIFO



Each FIFO entry consists of:

- addr (32 bits) – AMBA burst start address or register address
- b_size (6 bits) – burst count in terms of memory burst
- wb (1 bit) – wrapping burst flag

- rw (1 bit) – 0 -> write; 1 -> read
- ra (1 bit) – 0 -> memory access; 1 -> register access

The depth of the address FIFO can be between 4 and 32; 4 is the default value.

You should specify the depth of the address FIFO using your knowledge of typical system behavior. In general, however, the default 4-deep address FIFO should be enough to prevent unwanted AMBA wait cycles. A read transfer needs only a 2-deep address FIFO and is not a limiting factor, since a read transfer holds the AMBA bus and wait cycles are expected in the case of a read.

In the case of a write transfer, however, if you decide to initiate many short back-to-back write bursts, you should specify the depth of the address FIFO to equal the number of back-to-back write bursts so that there will be no wait cycles.

In all other cases, the default depth should be deep enough.

3.3.2 Write Data FIFO

The write data FIFO contains the write data to the memory device and control information associated with the write transfer generated by the burst control. The depth of the write data FIFO can be between 4 and 32; 8 is the default value.

You should select the depth of the write data FIFO using SDRAM latency, which includes the cycles necessary to do refresh, precharge (tRP), active-to-read or active-to-write delay (tRCD), and so on. If the AMBA/SDRAM bus ratio is 2:1, you should allow 2 cycles to read/write one set of AMBA data; deeper FIFO depth is recommended.

Select the recommended depth using the following formulas to ensure reasonable latency:

$$1:1 \text{ ratio} - \text{Data FIFO Depth} = 4 + \text{tRP} + \text{tRCD} = d$$

$$2:1 \text{ ratio} - \text{Data FIFO Depth} = d + (\text{HSIZE} - d)/2 = (\text{HSIZE} + d)/2$$

where:

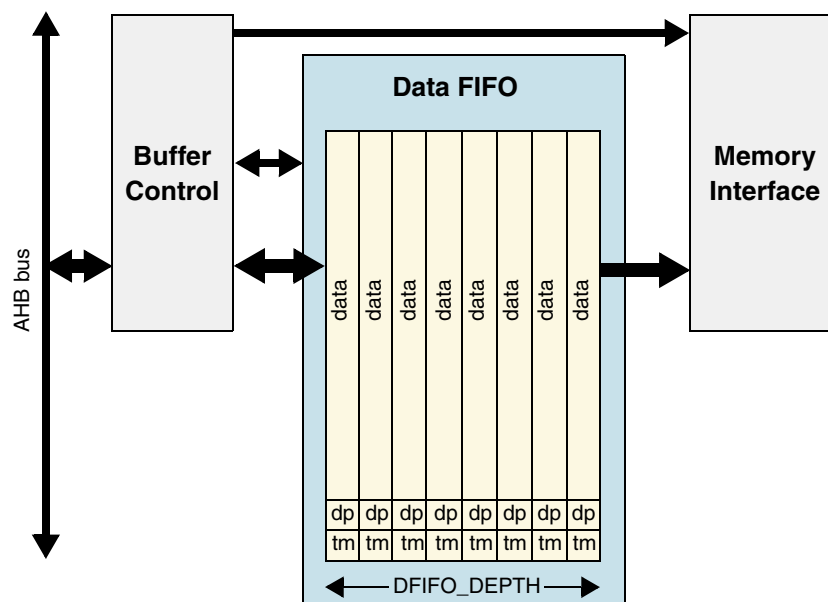
- tRP = precharge delay in cycles
- tRCD = RAS-to-CAS delay in cycles

In a system that has write-back cache, it is recommended that you set the data FIFO depth equal to the cache line-size.

Figure 3-5 illustrates the structure of a data FIFO when there is a 1:1 correspondence between S_DATA_WIDTH and H_DATA_WIDTH; that is, when the bus ratio is set to 1. Each FIFO entry consists of:

- data (S_DATA_WIDTH bits) – memory register write data
- dp (1 bit) – double pop flag
- tm (1 bit) – termination flag

Figure 3-5 Host Interface Unit (HIU) Data FIFO (bus ratio = 1)

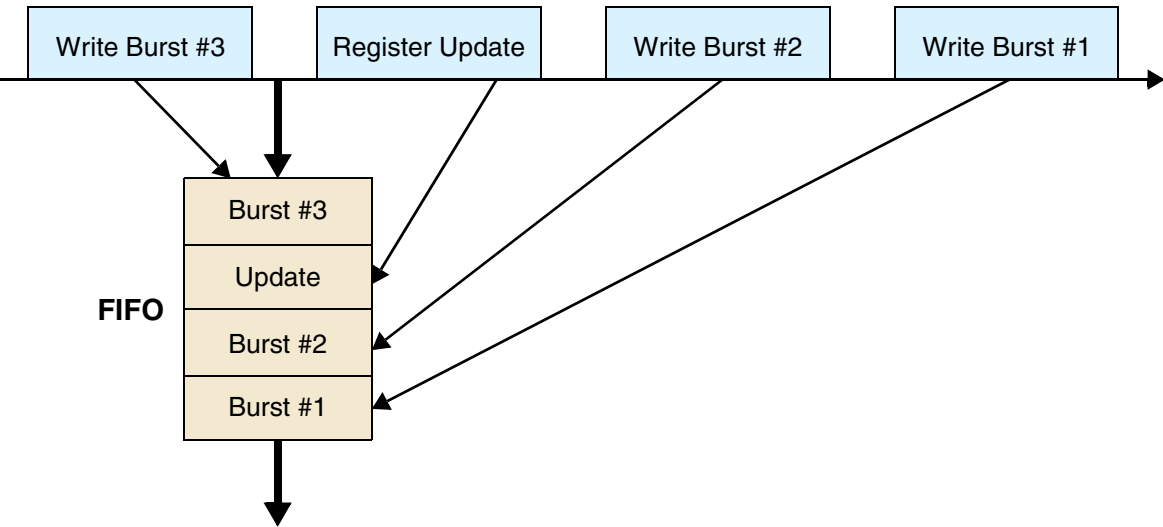


3.3.3 Control Register Access

The control register read and write data paths share the same path as the memory read/write. Reading the registers can be delayed, based on pending memory writes in the write data FIFO. [Figure 3-6](#) shows how the memory/register accesses are sequenced through the address FIFO. Write bursts 1 and 2 are writes to memory, which are followed by a write request to the control register.

The FIFO in [Figure 3-6](#) shows the request queue. The register is updated only after memory burst 1 and memory burst 2 are complete.

Figure 3-6 Register and Memory Access Through Address FIFO



The overall delay for a register read (from AMBA NONSEQ to HRDATA) is:

- 5 cycles – data width ratio is 1:1 between AMBA and SDRAM
- 8 cycles – data width ratio is 2:1 between AMBA and SDRAM



Note If you choose to have only the Static memory controller and not the SDRAM controller, then the register write data width has a 1:1 ratio with the AHB data width.

[Table 3-2](#) lists the timing diagrams for register read/writes.

Table 3-2 Register Read/Write Timing Diagrams

Timing Diagram	Description
Figure 3-7 on page 43	Register write cycle, data width ratio 1:1
Figure 3-8 on page 44	Register write followed by read, data width ratio 1:1
Figure 3-9 on page 45	Register write cycle, data width ratio 2:1
Figure 3-10 on page 46	Register write followed by read cycle, data width ratio 2:1

Figure 3-7 illustrates a single transfer register write cycle on the AHB bus, which is a write to the `STMG0R` (Address 0x4). The register is updated at the rising edge of clock cycle 5.

Figure 3-7 Register Write Cycle, Data Width Ratio 1:1

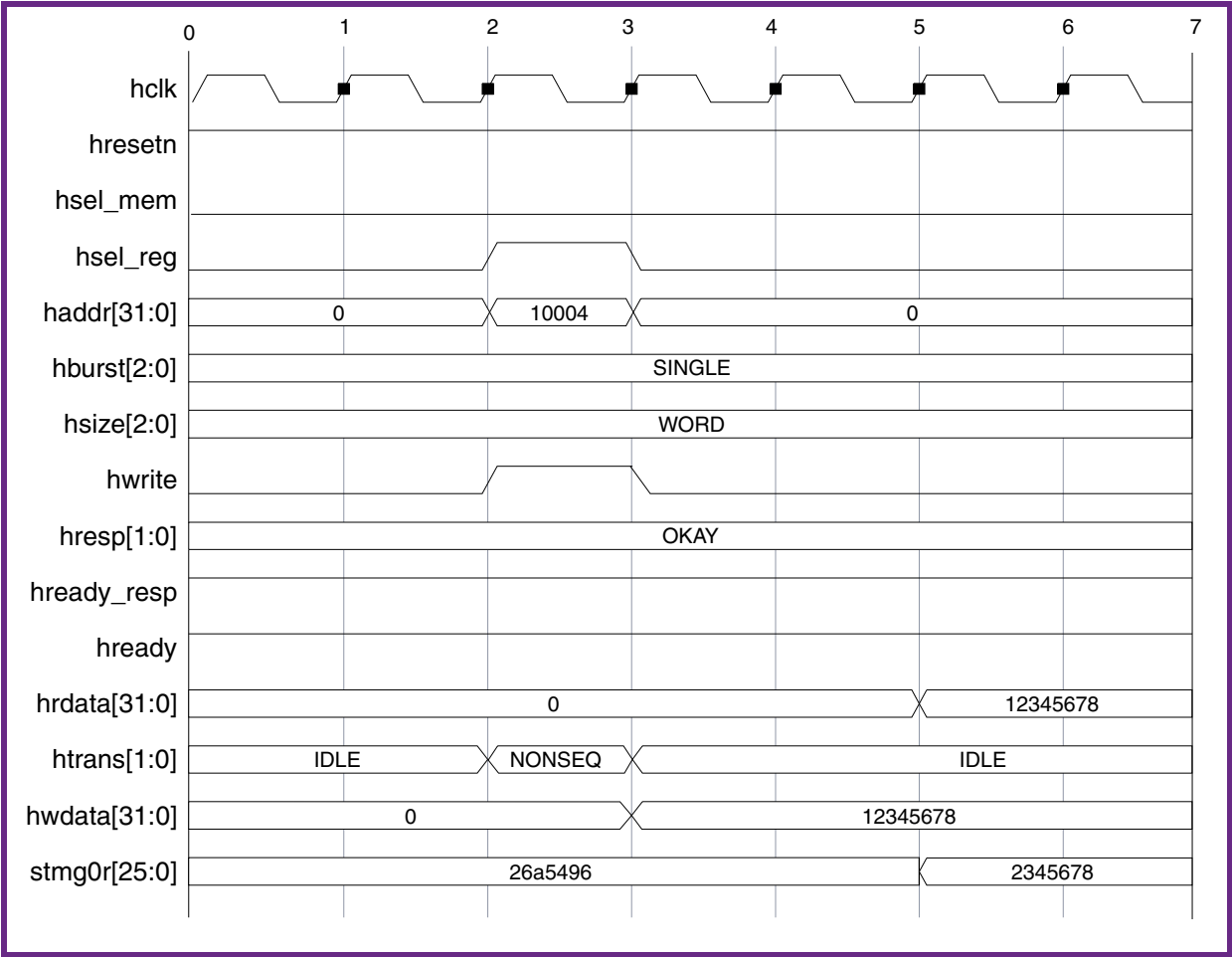


Figure 3-8 illustrates a single transfer register write cycle, followed by a read cycle on the AHB bus, which is a read to the **STMG0R** (Address 0x4). The data is written to the register on the rising edge of clock 5; for a read, hready_resp goes high on the rising edge of clock 7.

Figure 3-8 Register Write Followed by Read, Data Width Ratio 1:1

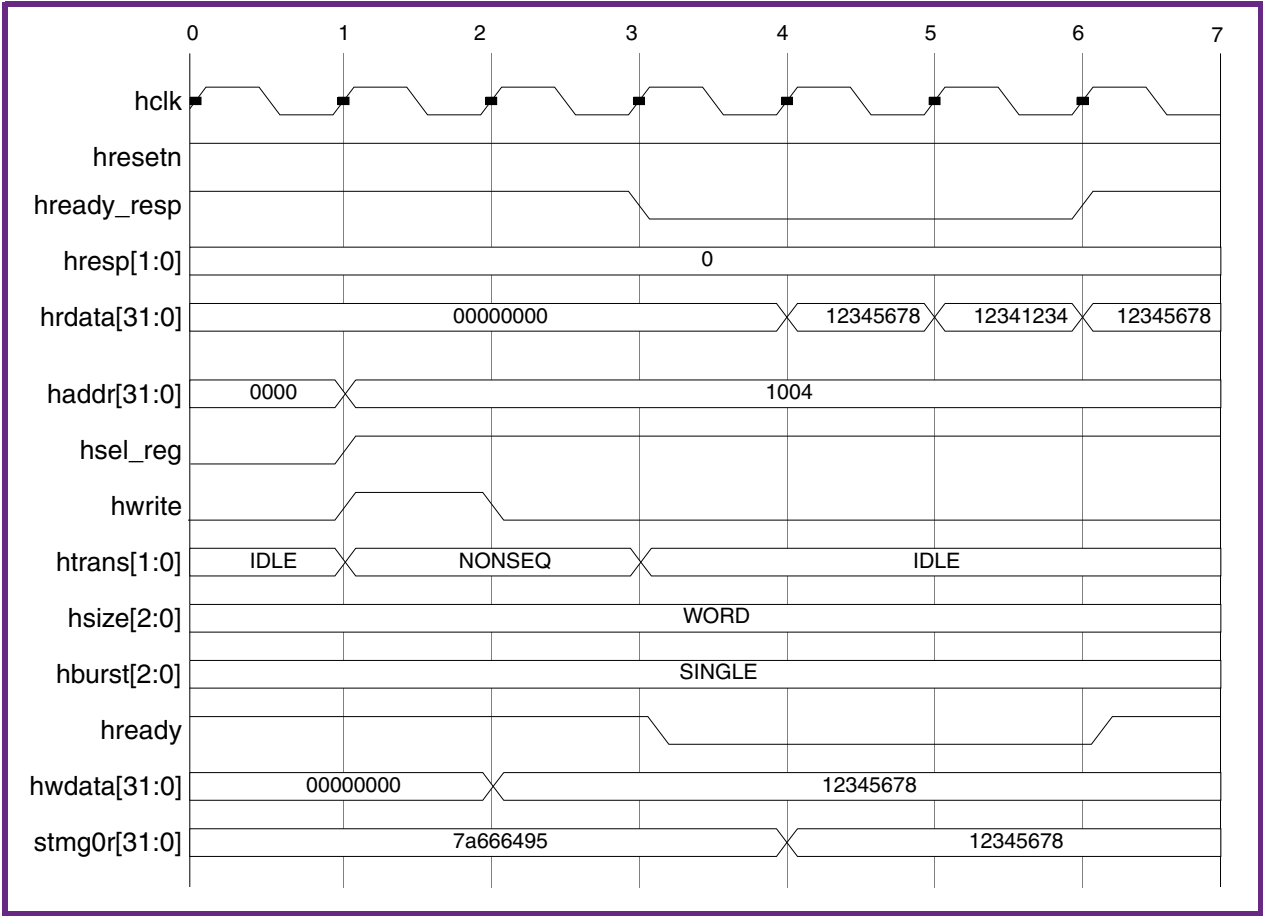


Figure 3-9 illustrates a single transfer register write cycle on the AHB bus. Since the data width ratio is 2:1, and because of the fact that the register read/write shares the same data path with the memory read/write, internally there are two write cycles. In this case, the **STMG0R** (Address 0x4) is written.

Clock cycles 5 and 7 show that there are two internal pop requests to the write buffer; data is written to the register at the rising edge of clock cycle 7.

Figure 3-9 Register Write Cycle, Data Width Ratio 2:1

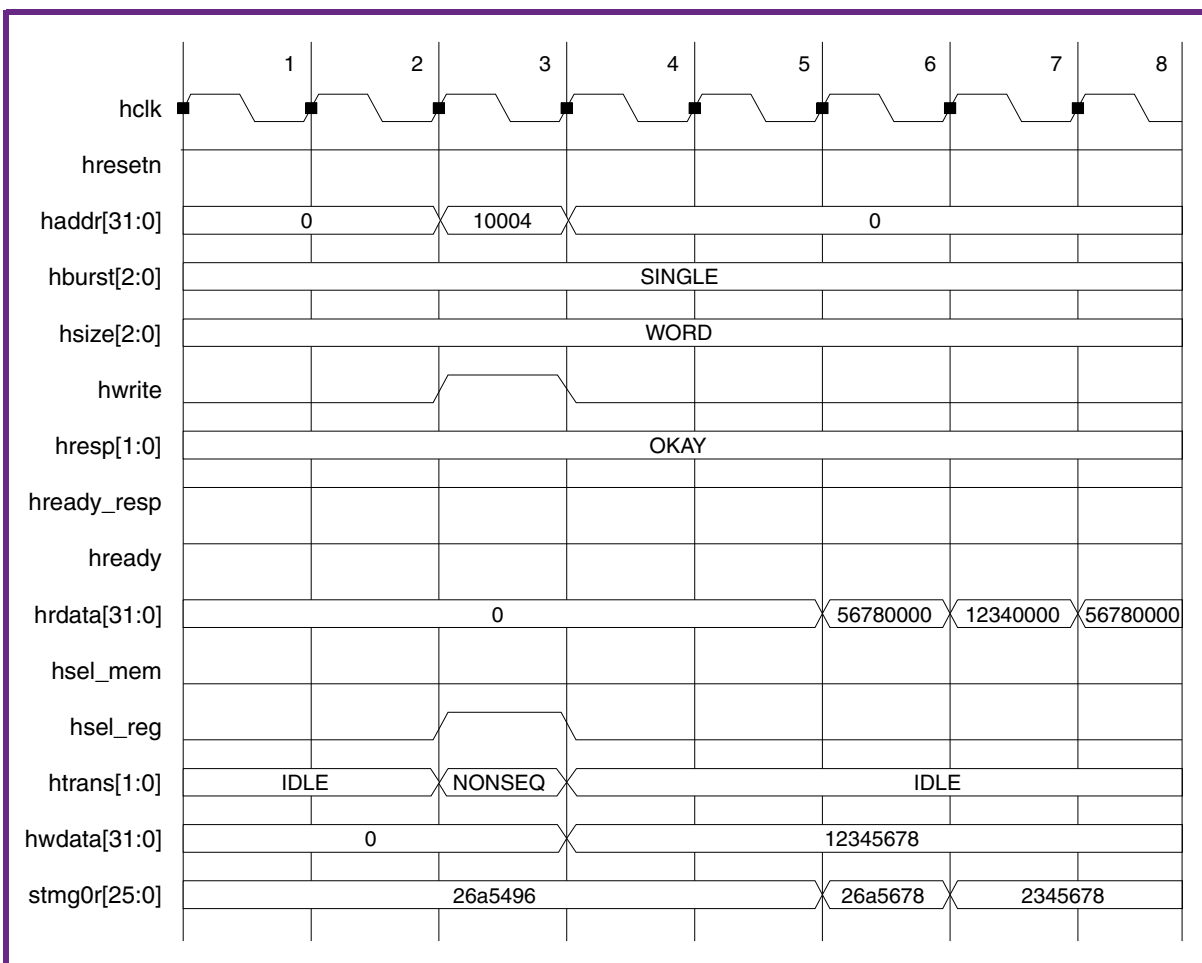
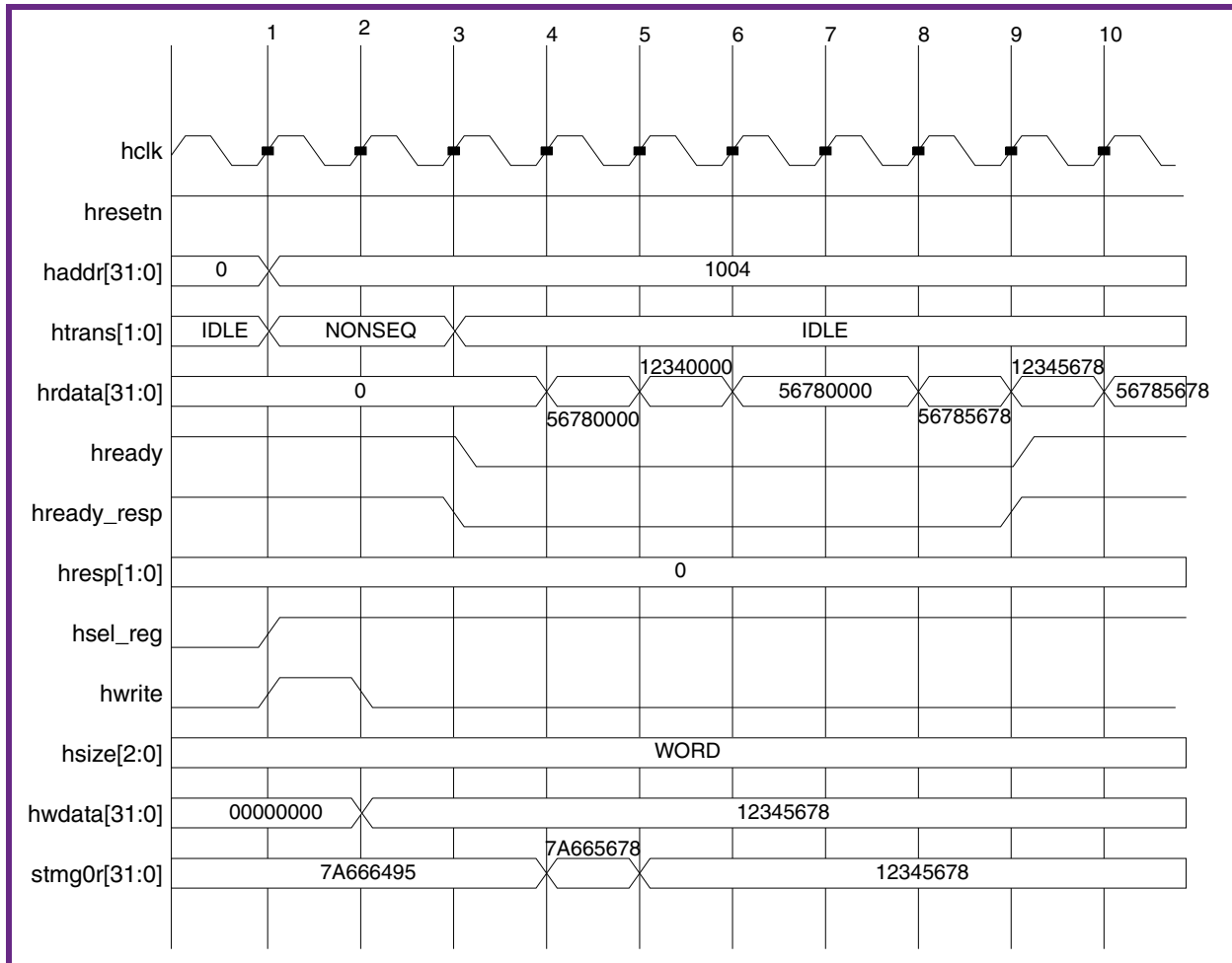


Figure 3-10 illustrates a single transfer register write cycle, followed by a read cycle on the AHB bus. Since the data width ratio is 2:1 and the register read/write shares the same data path with the memory read/write, internally there are two cycles. The data is written to the register on the rising edge of clock 6; for the read, hready_resp is available on the rising edge of clock 10.

Figure 3-10 Register Write Followed by Read Cycle, Data Width Ratio 2:1



3.4 Memory Interface Unit (MIU)

The memory interface unit (MIU) is the interface for both SDRAM and Static memories; it generates appropriate address, data, and control signals corresponding to memory read/write transfers. The MIU contains two sets of modules, which are enabled depending on whether you choose the SDRAM or Static memory.

If you choose the SDRAM controller, the MIU includes the following modules:

- SDRAM control unit – Generates the SDRAM control signals
- Refresh unit – Generates the SDRAM refresh request at appropriate intervals
- Address decoder unit – Generates the row, column, and bank addresses that correspond to the logical address provided by the host interface

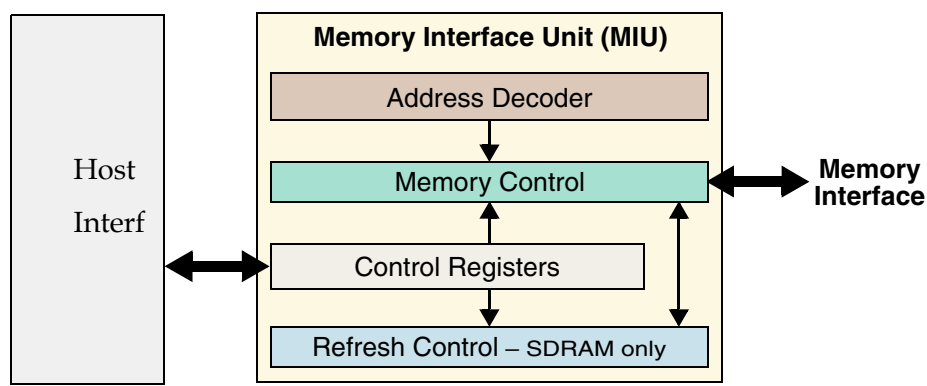
- Control register unit – Holds the DW_memctl SDRAM control and configuration registers, as well as address decoder registers

If you choose the Static memory controller, the MIU activates the following modules:

- Static control unit – Generates the SRAM/FLASH control signals
- Address decoder unit – Decodes and generates the address to SRAM/FLASH from the AHB address
- Control register unit – Holds the control registers and timing registers for Static memories; there are three sets of timing registers in this control unit

You can use three separate Static memories (SRAM/FLASH) or multiple memories of the same type; for example, three different SRAMs with different timings. [Figure 3-11](#) illustrates the block diagram of the MIU.

Figure 3-11 Memory Interface Unit (MIU) Block Diagram



For more information on details of the MIU, refer to [“DW_memctl as SDRAM Controller Functional Description”](#) on page 49 or [“DW_memctl as Static Memory Controller Functional Description”](#) on page 101.

3.5 Bus Sharing and the EBI

The SDRAM and Static memory interfaces can have their own separate addresses and data buses, or they can have a common shared address and/or data pins for both interfaces; this is controlled by the ENABLE_DATABUS_SHARING and ENABLE_ADDRBUS_SHARING configuration parameters. Bus sharing addresses the needs of both high-performance designs, as well as pin-limited designs; however, the control signals cannot be shared.

In addition to sharing pins between SDRAM and Static memories, you can also share these pins with other devices in your system through an external bus interface (EBI), if needed. The DW_memctl provides request/grant signals to support external arbitration and sharing of address and data pins through an external EBI block. For more details, refer to [“DW_memctl Integration”](#) on page 223 and [“External Bus Interface”](#) on page 293.

4

DW_memctl as SDRAM Controller Functional Description

This chapter describes the functional details of the DW_memctl as an SDRAM controller. Under the SDRAM category, the memory controller supports four different devices – SDR-SDRAM, Mobile SDR-SDRAM, and DDR-SDRAM. You can choose only one type of these device for each instantiation of the DW_memctl.

4.1 SDR-SDRAM and Mobile SDR-SDRAM Controller

This section discusses the details of how the DW_memctl supports SDR-SDRAM and Mobile SDR-SDRAM devices.

4.1.1 Power-On Initialization

The SDR-SDRAM controller follows the JEDEC-recommended SDR-SDRAM power-on initialization sequence as follows:

1. Apply power and start clock; maintain a NOP condition at the inputs
2. Maintain stable power, stable clock, and NOP input conditions for a minimum of t_{init} clock cycles
3. Issue precharge commands for all banks of the device
4. Issue auto-refresh commands, depending on the value num_init_ref in the programmable register
5. Issue a set-mode register command to initialize the mode register
6. Issue a set-extended-mode register command to initialize the extended-mode register (valid only for Mobile SDR-SDRAM)

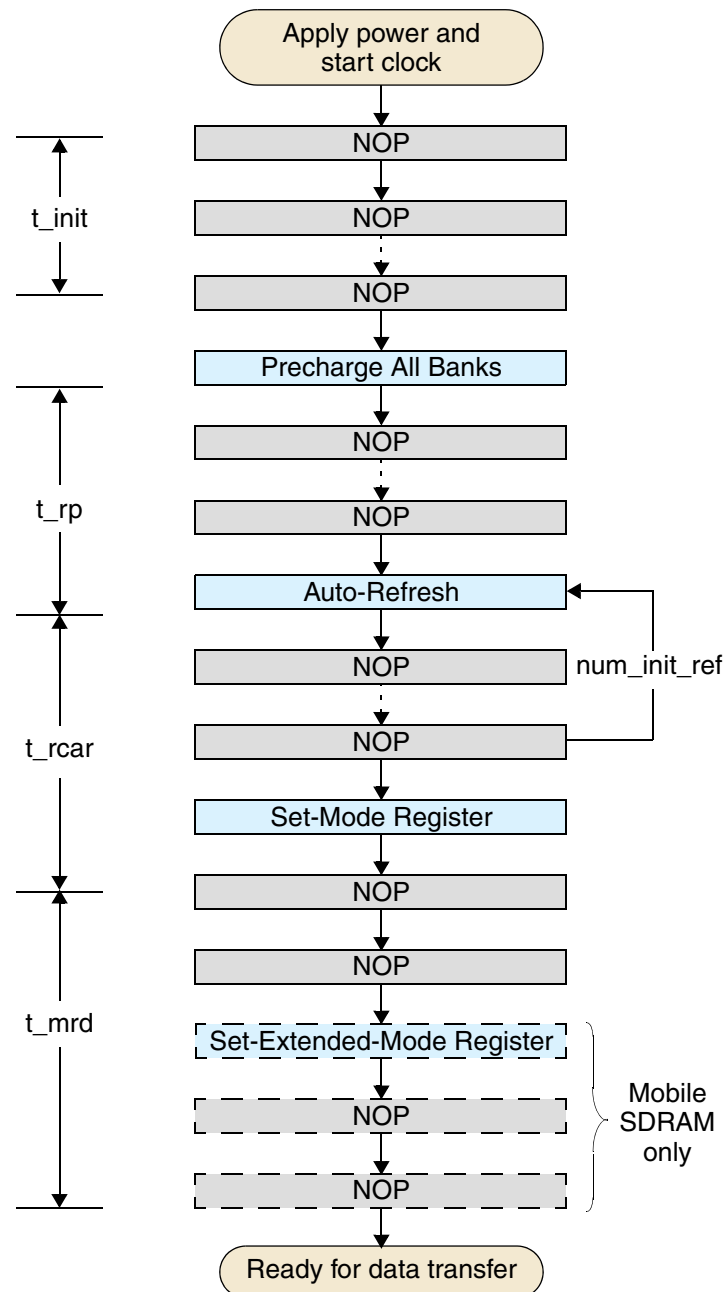
The DW_memctl performs a power-on sequence of the SDRAM under these circumstances:

- Immediately after reset
- When the programmable initialize bit (bit 0 of SCTL0) is set, the DW_memctl resets the bit when it comes out of initialization

All SDRAM read/write requests that occur during initialization are queued in the memory controller.

Figure 4-1 illustrates the commands issued to the SDRAM by the controller during the power-on initialization.

Figure 4-1 SDR/Mobile SDR-SDRAM Power-On Command Sequence



The T_{INIT} , T_{RP} , and T_{RCAR} compile-time parameters are the default values for t_{init} , t_{rp} and t_{rcar} , respectively, which you can use for initialization. If you feel that the reset time of the system is long enough to take care of the t_{init} time, then you can assign a value of zero to the T_{INIT} parameter.

The DW_memctl initializes the SDRAM after reset using the specified default compile-time timing parameters. After reset, if you feel that these timing parameters are not adequate, then you can program the timing parameters accordingly and then program the initialize bit – that is, bit 0 of SCTLR – to 1. This forces the DW_memctl to initialize the SDRAM.

The t_{mrd} is fixed at a value of 3 clock cycles, according to the JEDEC standard.

4.1.2 Read/Write Operations

The DW_memctl converts all AHB bursts to four-word bursts on the SDRAM side. The memory bursts are concatenated to achieve continuous data flow for long AHB bursts. You can terminate the memory read/write burst with either a precharge command or terminate command, depending on which precharge mode – immediate precharge or delayed precharge – that you program. You can also terminate the write burst with a subsequent write burst.



Attention

An AHB INCR write/read transfer termination inserts additional memory cycles. There are one or two extra clock cycles inserted on the memory bus at the end of an INCR AHB write/read full burst; this is due to the registered termination that is implemented for better timing. Extra write cycles have write enable and byte select signals masked, which does not cause data corruption. However, there is a potential performance issue with small INCR bursts. Extra read cycles at the end of INCR are not propagated to the bus interface.

The DW_memctl does not use auto-precharge mode.

The DW_memctl supports two precharge modes – immediate precharge and delayed precharge. If you program for an immediate precharge mode, then the DW_memctl closes the open row after a read or write access. If you program for a delayed precharge mode, then the DW_memctl keeps the row open after an access. The DW_memctl can keep multiple banks open at the same time, depending on the value of num_open_bank in the programmable register. When the number of open banks reaches the num_open_bank and an access to a new bank comes, the DW_memctl will close the oldest bank (the bank opened first) before opening the new bank.

Table 4-1 on page 52 lists the DW_memctl performance during read/writes to the SDRAM in various circumstances, with the following assumptions:

- DW_memctl is idle; that is, no pending read/write request is in the address FIFO
- All control signals to the SDRAM are registered
- In the case of a read, timing information is relative to the latency from the time the select signal is asserted on the AMBA bus to when the first data is available on the AMBA bus
- In the case of a write, timing information is relative to the latency from the time the select signal is asserted on the AMBA bus to when the first data is written to the SDRAM

- Timing parameters in [Table 4-1](#) are:
 - tINT – internal delay before a command is sent to the memory device; 2 clock cycles when ADDRDEC_PIPE is not defined; 3 clock cycles when ADDRDEC_PIPE is defined
 - tRCD – active-to-read/write command time; assumed to be 3
 - tCAS – CAS latency; assumed to be 3

Table 4-1 Latencies for Read/Write

Transfer Type	Condition	Timing	Latency
Writes	Page hit	tINT=2	2 (Figure 4-2 on page 53)
	Page miss	(tINT=2) + (tRCD= 3)	5 (Figure 4-3 on page 54)
Reads	Page hit	(tINT=2) + (tCAS=3)	5 (Figure 4-5 on page 56)
	Page miss	(tINT=2) + (tRCD= 3) + (tCAS=3)	8 (Figure 4-6 on page 57)

[Table 4-2](#) lists timing diagrams for simple read/write operations to the SDRAM; all read/writes are assumed to have delayed precharge. In those timing diagrams, the ADDRDEC_PIPE is not defined. The number of idle clock cycles between back-to-back transfers will increase by one when the ADDRDEC_PIPE is defined.

Table 4-2 SDR/Mobile SDR-SDRAM Read/Write Timing Diagrams

Timing Diagram	Description
Figure 4-2 on page 53	Page-hit single write
Figure 4-3 on page 54	Page-miss single write
Figure 4-4 on page 55	Page-hit burst write
Figure 4-5 on page 56	Page-hit single read
Figure 4-6 on page 57	Page-miss single read
Figure 4-7 on page 58	Page-hit burst read
Figure 4-8 on page 59	Page-hit write followed by write
Figure 4-9 on page 60	Page-hit write followed by read
Figure 4-10 on page 61	Page-hit read followed by read
Figure 4-11 on page 62	Page-hit read followed by write

Figure 4-2 illustrates a page-hit single-cycle write (hburst == SINGLE) transfer to the SDR-SDRAM from the AHB bus.

Figure 4-2 SDR/Mobile SDR-SDRAM Page-Hit Single Write

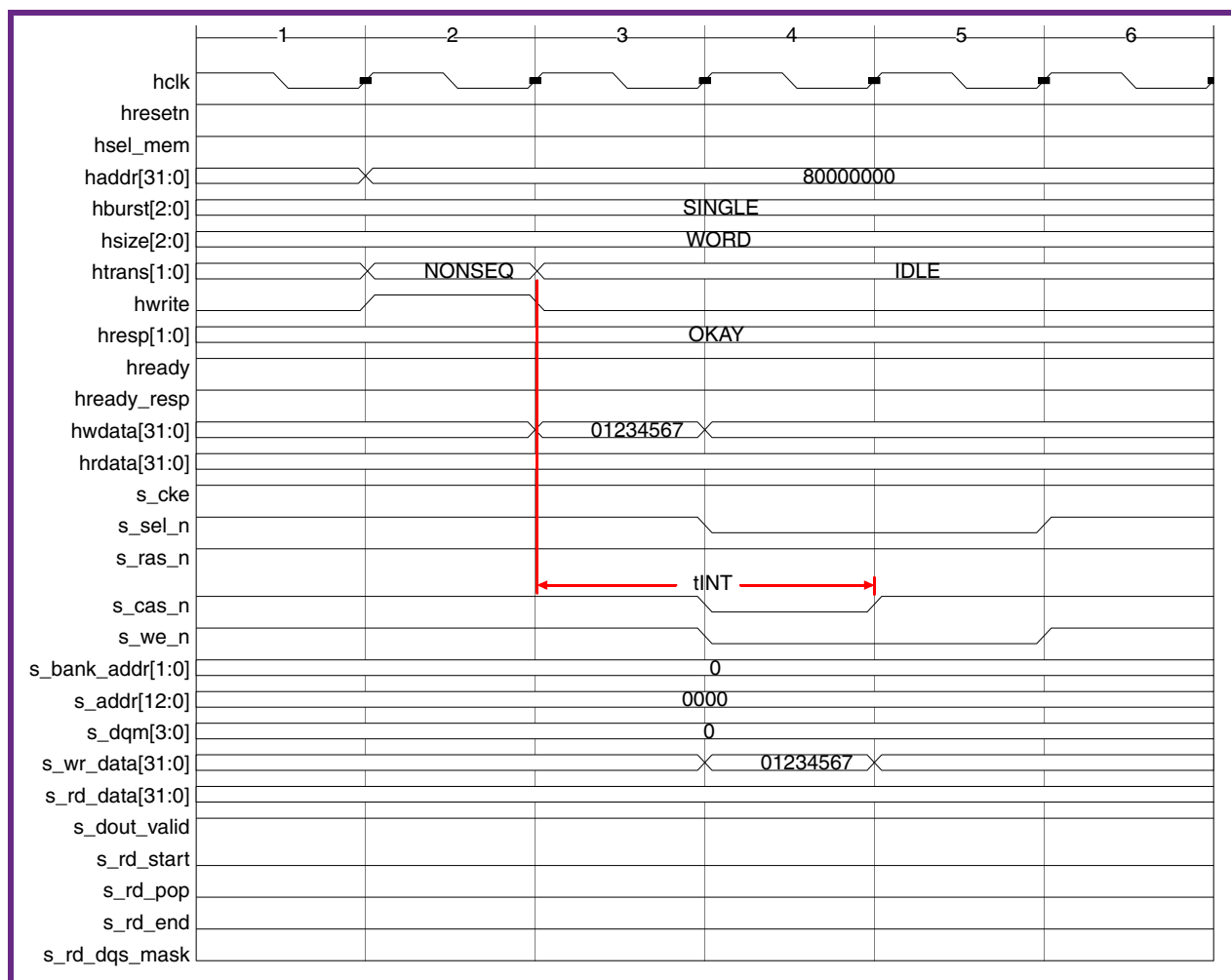


Figure 4-3 illustrates a page-miss single-cycle write (hburst == SINGLE) transfer to the SDR-SDRAM from the AHB bus. DW_memctl takes two internal cycles (tINT) before the activate command is generated on the SDRAM side; s_ras goes low on the rising edge of clock 4.

Figure 4-3 SDR/Mobile SDR-SDRAM Page-Miss Single Write

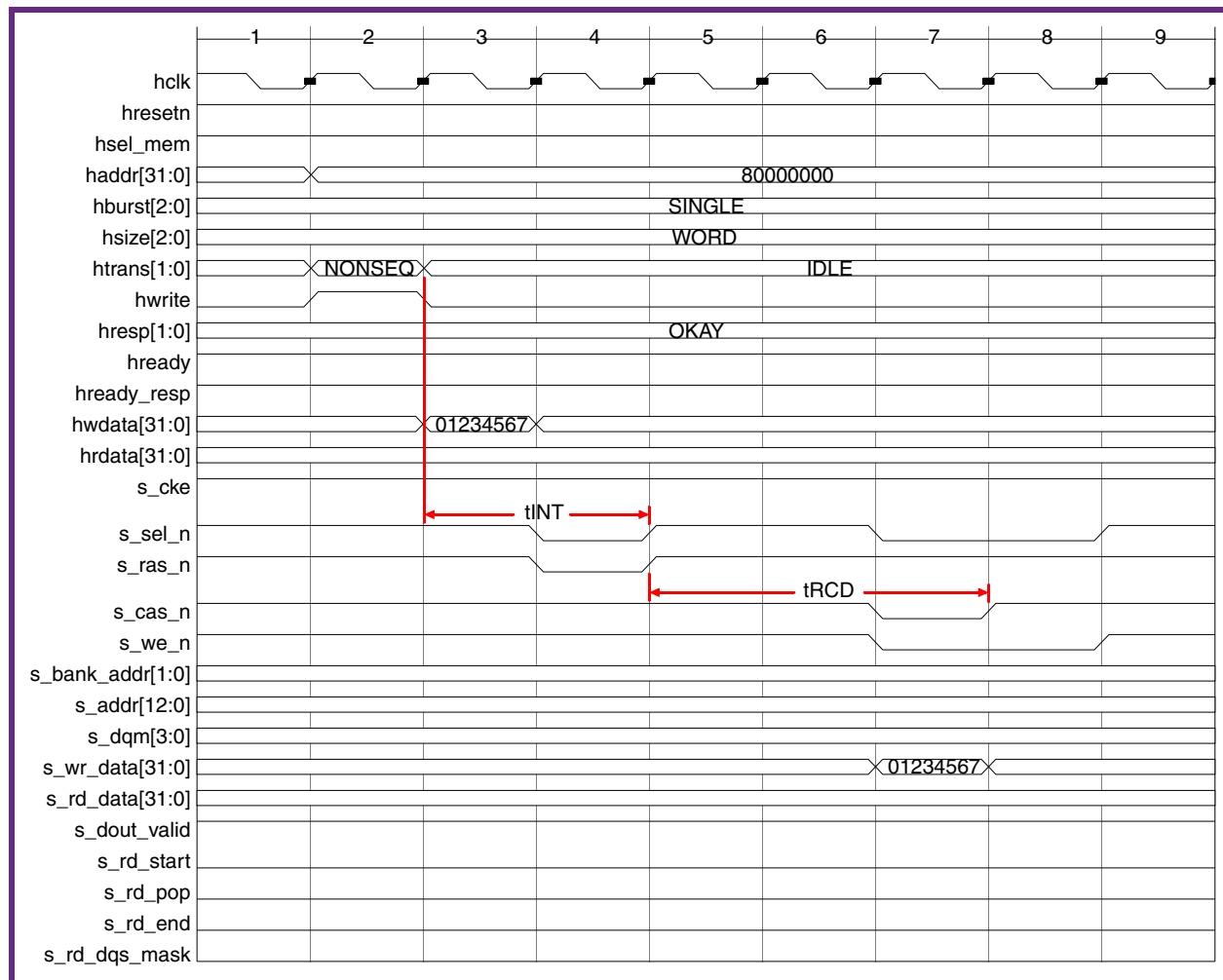


Figure 4-4 illustrates a page-hit burst write (hburst == INCR8) transfer to the SDR-SDRAM from the AHB bus.

Figure 4-4 SDR/Mobile SDR-SDRAM Page-Hit Burst Write

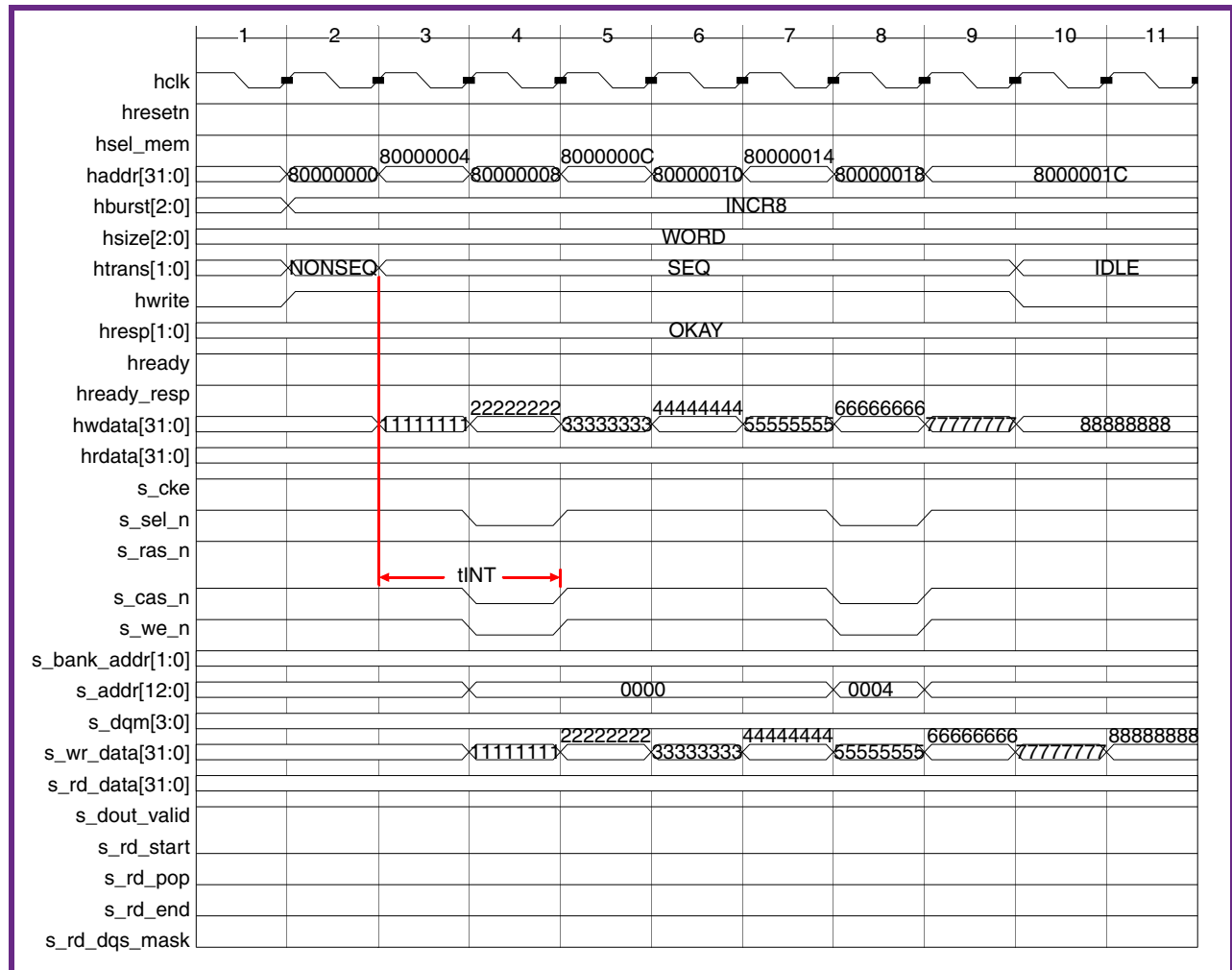


Figure 4-5 illustrates a page-hit single-cycle read (hburst == SINGLE) transfer to the SDR-SDRAM from the AHB bus.

Figure 4-5 SDR/Mobile SDR-SDRAM Page-Hit Single Read

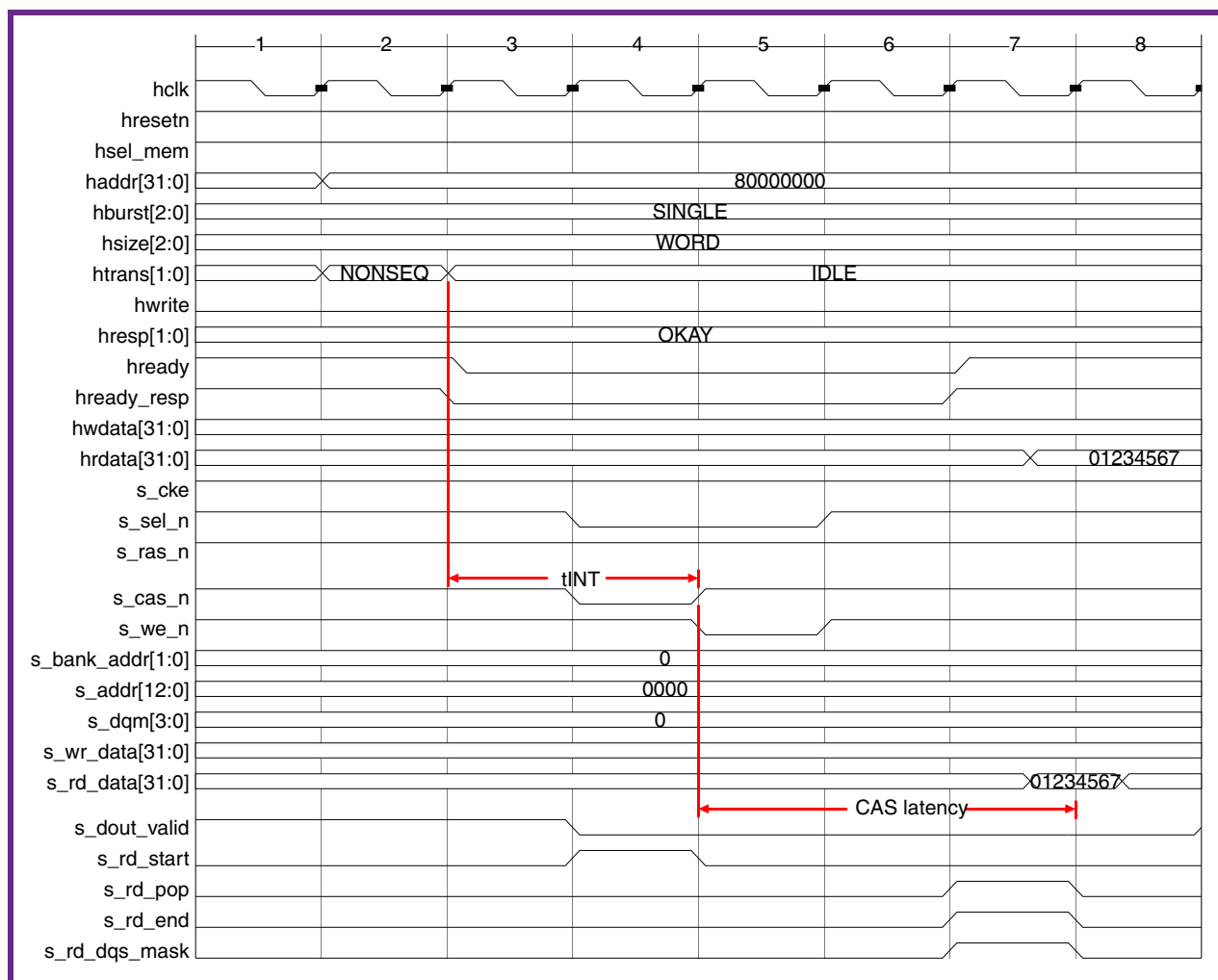


Figure 4-6 illustrates a page-miss single-cycle read (hburst == SINGLE) transfer to the SDR-SDRAM from the AHB bus. DW_memctl takes two internal cycles (tINT) before the activate command is generated on the SDRAM side; s_ras goes low on the rising edge of clock 4.

Figure 4-6 SDR/Mobile SDR-SDRAM Page-Miss Single Read

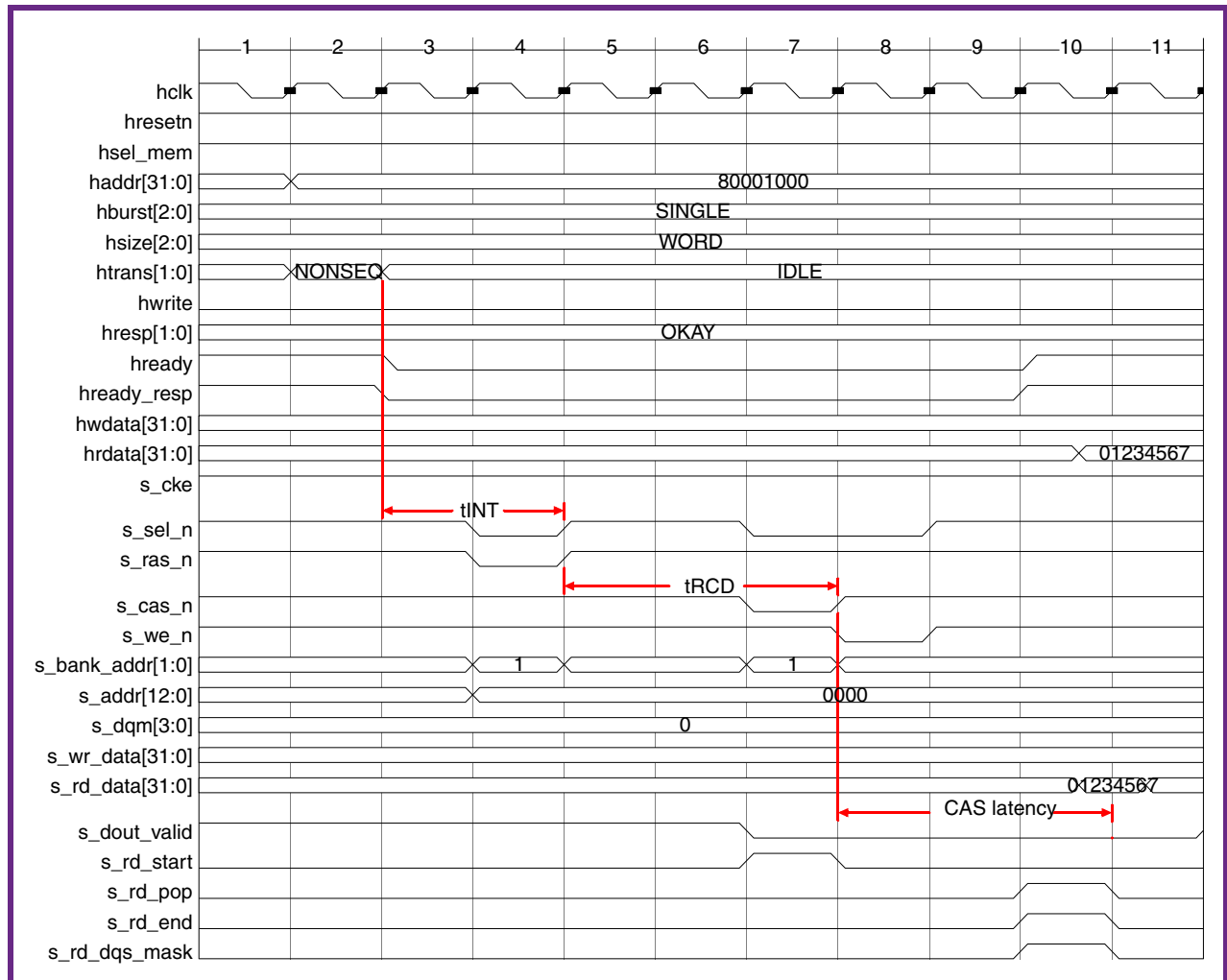


Figure 4-7 illustrates a page-hit burst read (hburst == INCR8) transfer to the SDR-SDRAM from the AHB bus.

Figure 4-7 SDR/Mobile SDR-SDRAM Page-Hit Burst Read

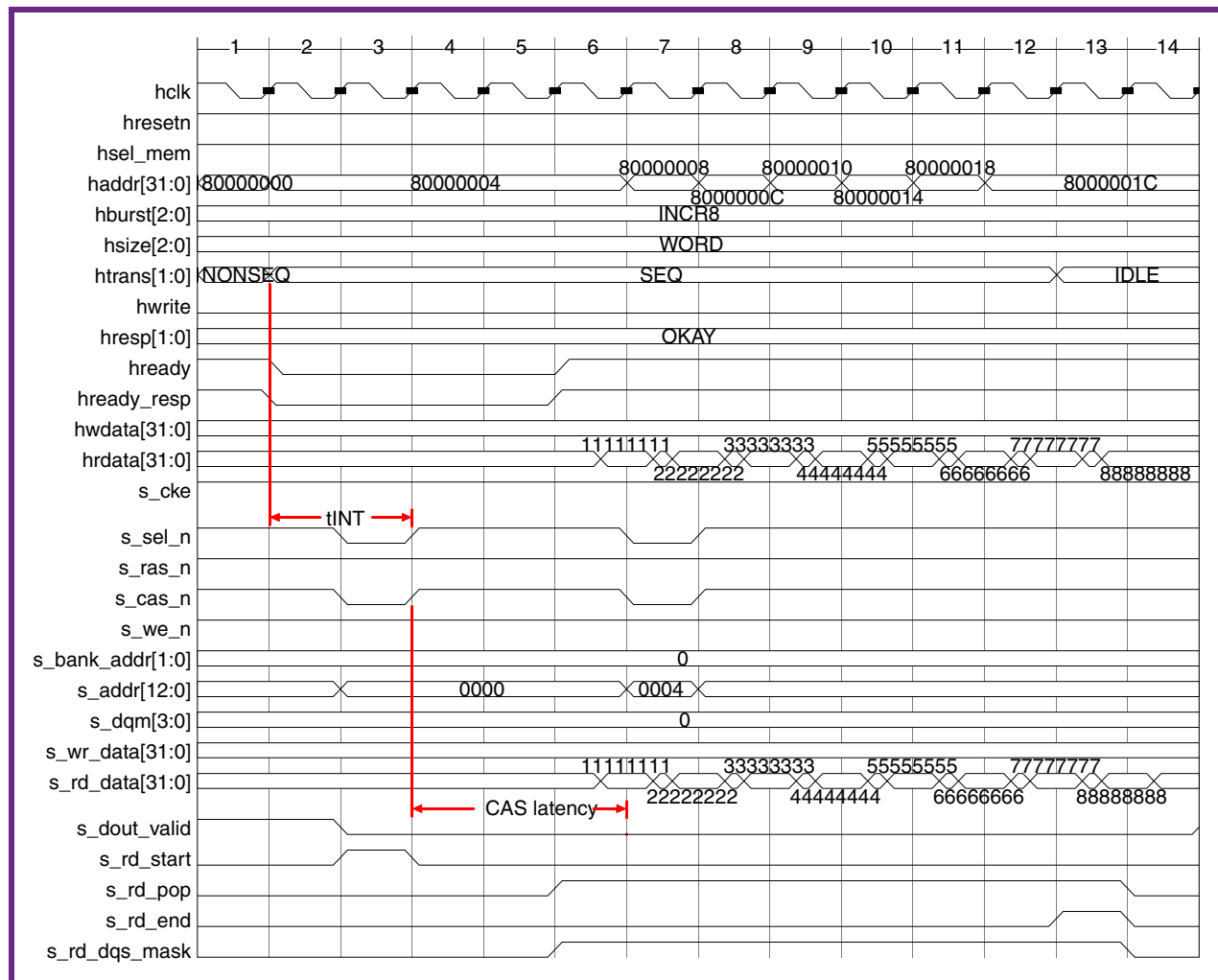


Figure 4-8 illustrates a page-hit single write (hburst == SINGLE) transfer, followed by a page-hit single write transfer.

Figure 4-8 SDR/Mobile SDR-SDRAM Page-Hit Write Followed by Write

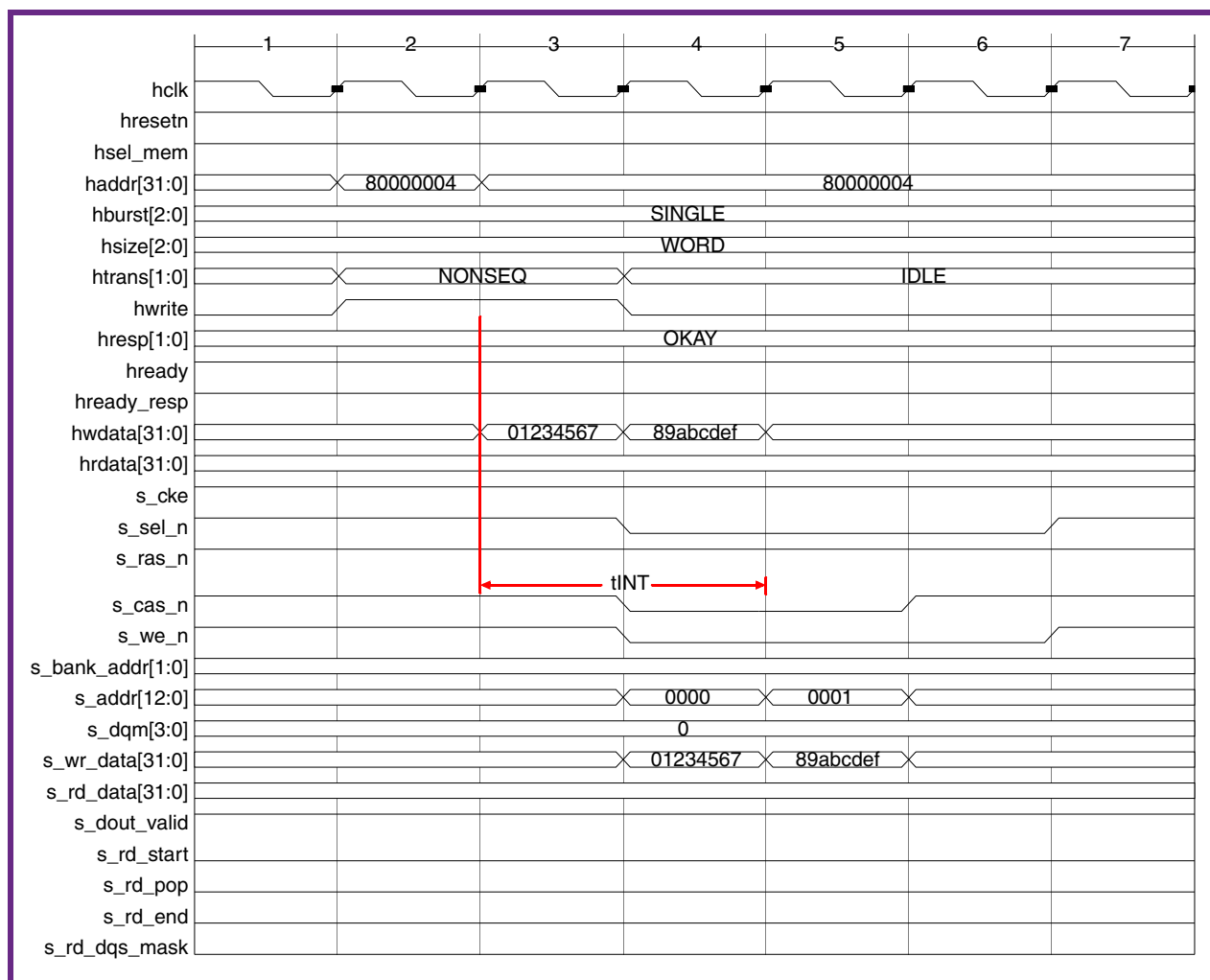


Figure 4-9 illustrates a page-hit single write (hburst == SINGLE) transfer, followed by a page-hit single read transfer.

Figure 4-9 SDR/Mobile SDR-SDRAM Page-Hit Write Followed by Read

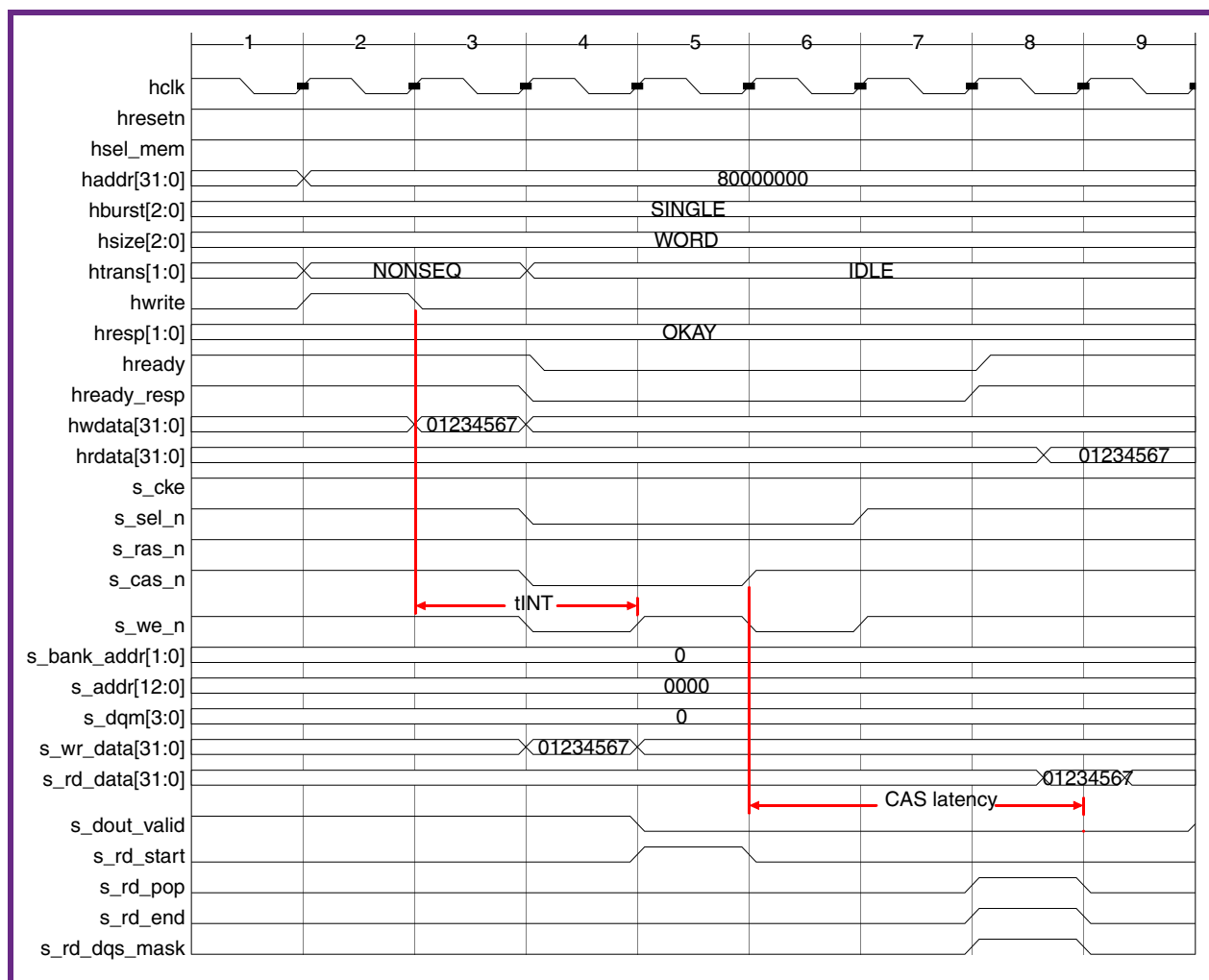


Figure 4-10 illustrates a page-hit single-cycle read (hburst == SINGLE) transfer, followed by a page-hit single read transfer.

Figure 4-10 SDR/Mobile SDR-SDRAM Page-Hit Read Followed by Read

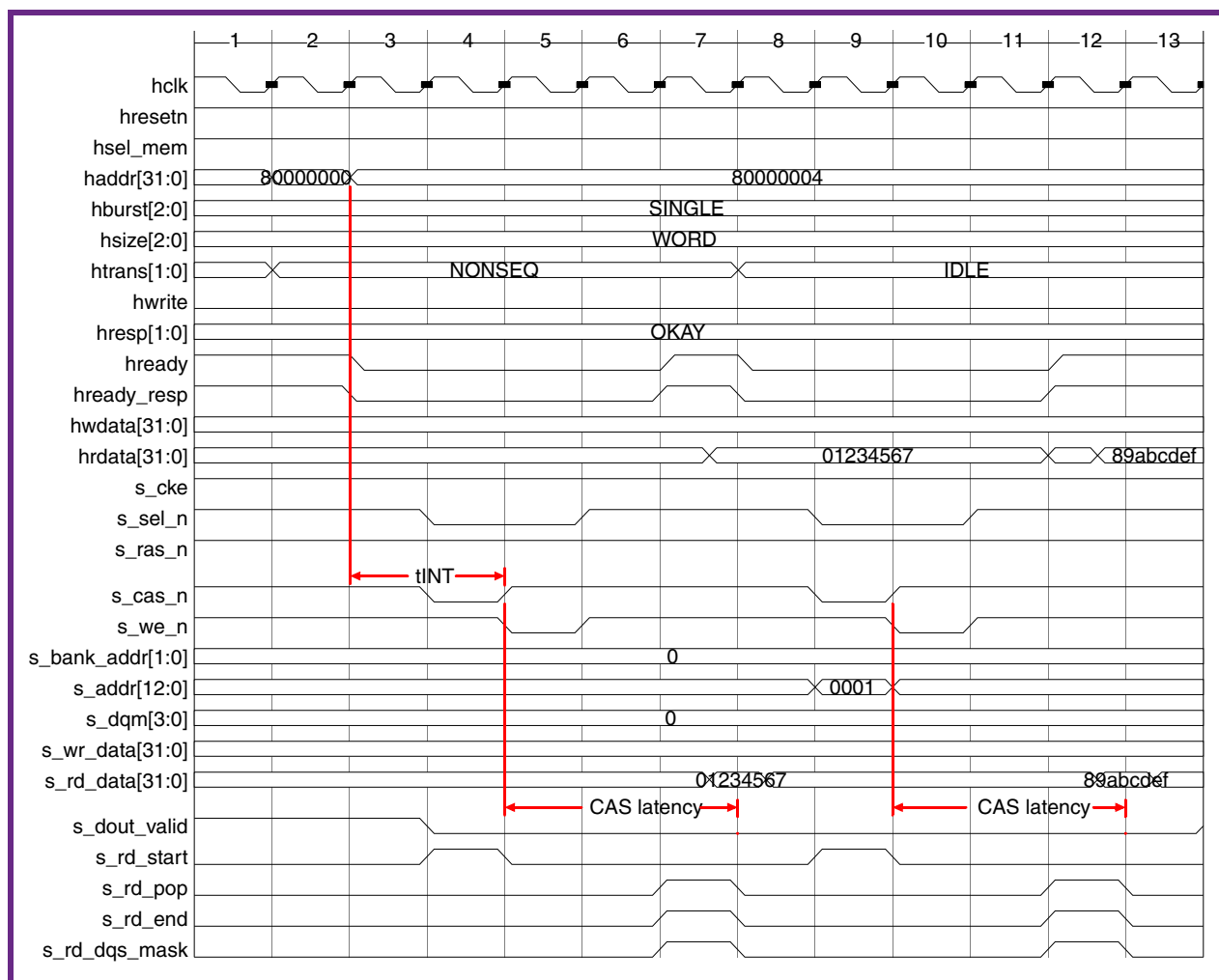
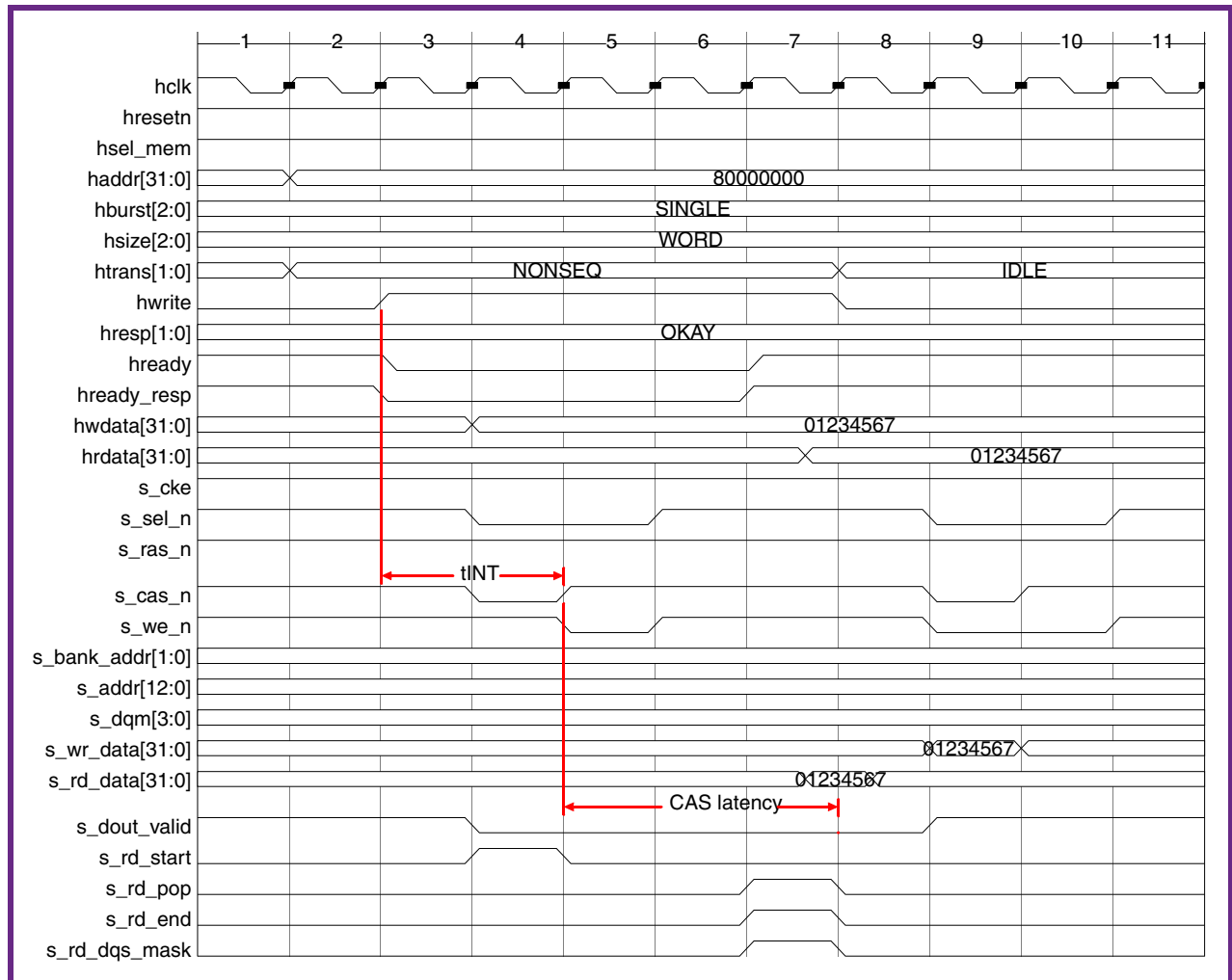


Figure 4-11 illustrates a page-hit single cycle read (hburst == SINGLE) transfer, followed by a page-hit single write transfer.

Figure 4-11 SDR/Mobile SDR-SDRAM Page-Hit Read Followed by Write



4.1.3 Set-Mode and Extended-Mode Register

The DW_memctl automatically sets the SDR-SDRAM mode register and extended-mode register (extended-mode register is only for Mobile SDR-SDRAM) during the power-up initialization. During normal operation, if you want to set the mode register or extended-mode register, you need to set set_mode_reg (bit 9 of SCTL) or exn_mode_reg_update (bit 18 of SCTL) in the control register (SCTL) to 1. After the memory controller finishes the mode register setting, it clears the set_mode_reg or the exn_mode_reg_update bit to 0.

The “burst length” field and the “burst type” field of the SDR-SDRAM-mode register are fixed by the DW_memctl to “010” (burst length 4) and “0” (sequential burst), respectively. The DW_memctl programs the “CAS latency” field and the “operating mode” field of the mode register according to the values provided by the user in the control and timing registers.

The DW_memctl programs the extended-mode register of the Mobile SDR-SDRAM according to the value provided by the user in the EXTN_MODE_REG (address 32'hxxx_xxAC).

4.1.4 Auto-Refresh

Auto-refresh commands are issued when the refresh control block issues refresh requests. During normal refresh operations, the DW_memctl always refreshes one row at a time. It is important for the user to program the tREF refresh interval register after a reset.

If you need to refresh the SDRAM while a burst is active, normally the DW_memctl will issue the refresh command after the ongoing burst completes. However, if the ongoing burst is an AHB INCR burst, the DW_memctl will stop the burst, issue the refresh command, and then resume the burst.

The DW_memctl takes into account the maximum time it takes to complete a worst-case burst. This is the time to complete a read burst corresponding to an INCR16 burst on the AMBA bus, and with an AMBA-to-SDRAM data width ratio of 2:1. It is reasonable to assume 50 cycles for this worst-case burst, with 32 cycles for the data and the remaining 17 cycles for various latencies for the worst case.

The t_ref value can be calculated using the following equation:

$$t_{\text{ref}} = \text{refresh_period} / \text{clock_period}$$

where refresh_period = typically 7.8/15.6 μs .

The tREF is the value of a free-running counter that the refresh logic in the DW_memctl operates on. When the count expires, the refresh logic gives a refresh request to the SDRAM control.

Since the 64 ms refresh period is the same for most SDRAMs, the total number of rows in the SDRAM limits the minimum operating frequency for the DW_memctl. While calculating the minimum frequency, use the following equation:

$$t_{\text{REF}} > 50 * (1/f)$$

Typically, the refresh cycle is 15.6 μs or 7.8 μs , depending on the refresh rate; [Table 4-3](#) summarizes this.

Table 4-3 Minimum Frequency Requirement

Number of Rows	tREF	Min Frequency
64K	(64ms-(50/f))/65536	51Mhz
32K	(64ms-(50/f))/32768	26Mhz
16K	(64ms-(50/f))/163904	13Mhz
8K	(64ms-(50/f))/8192	6Mhz
4K	(64ms-(50/f))/4096	3Mhz
2K	(64ms-(50/f))/2048	1.5Mhz

The refresh logic in the DW_memctl is inactive when the DW_memctl forces the SDRAM into self-refresh or power-down mode.

Figure 4-12 illustrates the command sequence issued by the SDRAM controller when performing an auto-refresh.

Figure 4-12 SDR/Mobile SDR-SDRAM Auto-Refresh Command Sequence

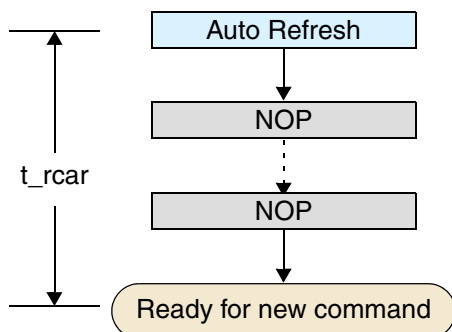
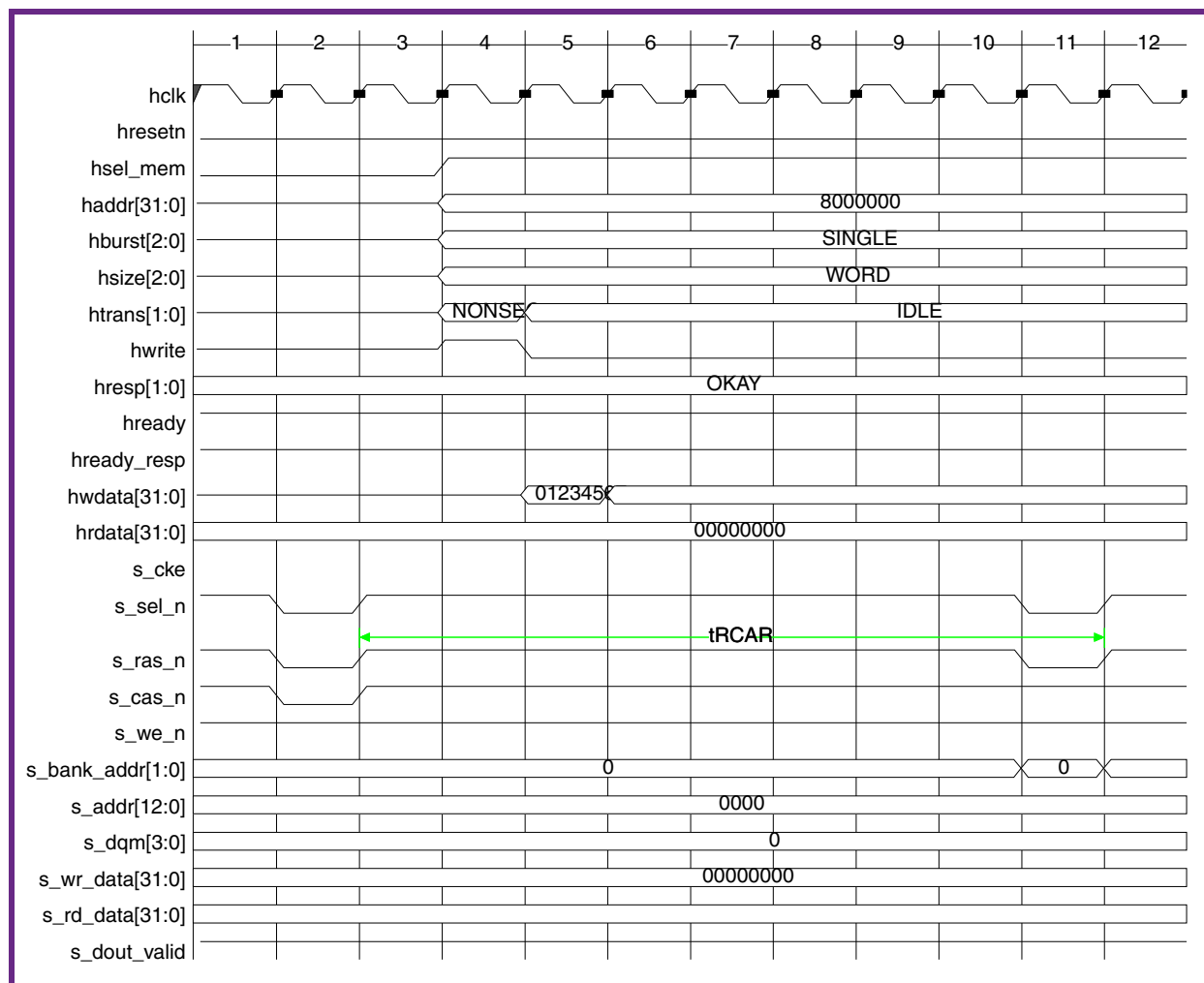


Figure 4-13 shows the timing diagram of an auto-refresh followed by a write.

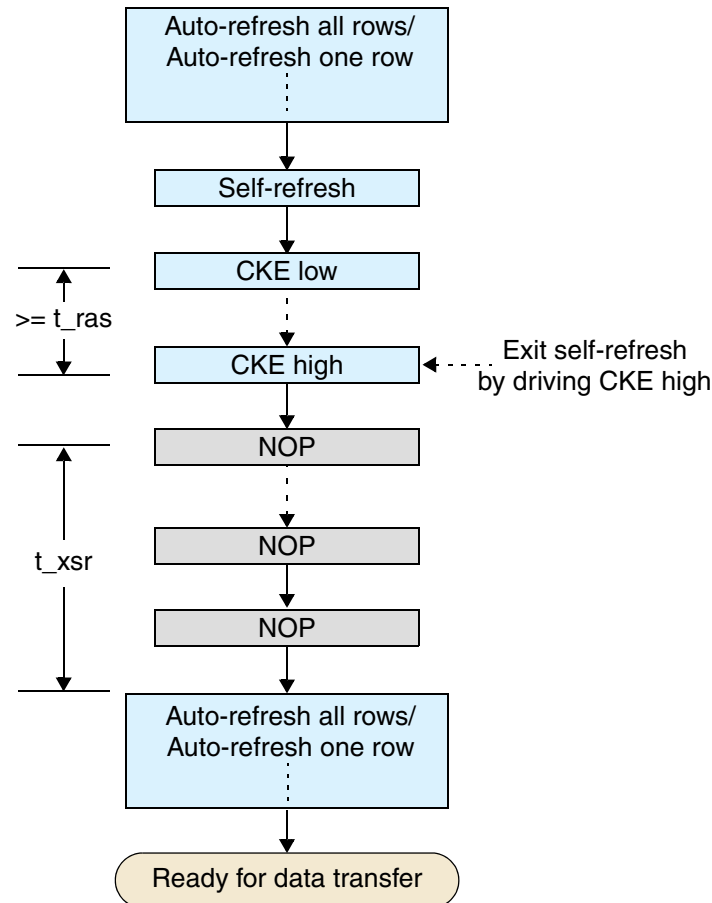
Figure 4-13 SDR/Mobile SDR-SDRAM Auto-Refresh Timing



4.1.5 Self-Refresh

You can put the SDRAM into self-refresh mode, at which point the SDRAM retains data without external clocking and auto-refresh. Figure 4-14 illustrates the command sequence issued by the SDRAM controller to initiate, maintain, and exit the self-refresh mode.

Figure 4-14 SDR/Mobile SDR-SDRAM Self-Refresh Command Sequence



You can force the DW_memctl to enter self-refresh mode by programming bit 1 of the SDRAM control register (SCTLR) (Address 32'hxxx_x0C). The DW_memctl forces the SDRAM to come out of self-refresh mode when bit 1 of the SCTLR is set to 0. You can set bit 1 of the SCTLR to 0 in software by programming the SDRAM control register. You can set bit 1 of the SCTLR to 0 with hardware by driving the clear_sr_dp input pin to high. You can use the clear_sr_dp pin when the code resides in the SDRAM, and the SDRAM itself is in self-refresh mode.

Bits 4 and 5 of the SCTLR specify the type of refresh done by the DW_memctl just prior to entering self-refresh mode and just after entering self-refresh mode. Programming bit 4 of the SCTLR to 0 forces the DW_memctl to refresh only one row before putting the SDRAM into self-refresh mode. The default value of 1 forces the DW_memctl to perform auto-refreshes for all rows. Bit 5 does the same, except that it controls the refresh pattern just after coming out of self-refresh mode.

Since it takes time between programming the control register bit to the SDRAM entering self-refresh mode, the DW_memctl provides a read-only register bit (bit 11 of the SDRAM control register) to indicate that the SDRAM is already in self-refresh mode. If you want to gate off the clock to the DW_memctl when the SDRAM is in self-refresh mode, you should ensure this bit is set to 1 before you stop the clock.

The SDRAM must remain in self-refresh mode for a minimum period of t_{ras} and can remain in self-refresh mode for an indefinite period of time. After the SDRAM exits self-refresh mode, the DW_memctl issues NOP commands for t_{xsr} before it issues any other command. The t_{ras} and t_{xsr} are programmable register values and have default values. These registers can be programmed only once after reset.

When an AHB read/write request to the SDRAM occurs while the SDRAM is in self-refresh mode, the DW_memctl generates dummy ready signals to the AHB without accessing external memory; no error response is generated on the AHB bus. While in self refresh mode, AHB reads return read data of undeterministic value and AHB write data is discarded. An AHB response of OKAY is provided for all memory reads and writes while in self refresh mode.

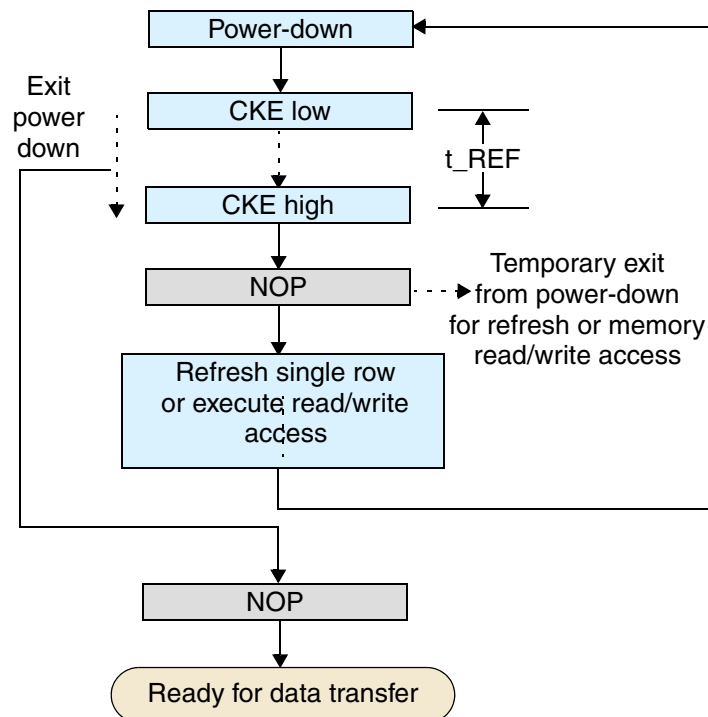
4.1.6 Power-Down

The SDRAM can be put into power-down mode to save power. There are two ways to force the DW_memctl to put the SDRAM in power-down mode:

- Program bit 2 of SCTL to 1; program back to 0 to bring the SDRAM out of power-down mode.
- Use the power_down input pin; can be driven by an external power management unit; the SDRAM will be in power-down mode as long as this signal stays high.

Figure 4-15 illustrates the command sequence issued by the SDRAM controller to initiate, maintain, and exit power-down mode.

Figure 4-15 SDR/Mobile SDR-SDRAM Power-Down Command Sequence



When in SDRAM power-down mode, the DW_memctl keeps switching the device back and forth between power-down and refresh mode. It remains in power-down for a t_{ref} period of time, then comes out of power-down and does a single-row refresh; then it again goes into power-down mode. The DW_memctl keeps the SDRAM in this periodical power-down/refresh/power-down sequence until it is commanded to exit power-down mode (by programming bit 2 of SCTL to 0).

When an AHB read/write request to the SDRAM occurs while the SDRAM is in power-down mode, the DW_memctl brings the SDRAM out of power-down mode and issues the read/write access to the SDRAM. DW_memctl then puts the SDRAM back to power-down mode after the read/write access.

4.2 DDR-SDRAM and Mobile DDR-SDRAM Controller

This section discusses the details of how the DW_memctl supports DDR-SDRAM and Mobile DDR-SDRAM devices.

4.2.1 DDR-SDRAM Power-On Initialization

The DDR-SDRAM controller follows the JEDEC-recommended DDR-SDRAM power-on initialization sequence (illustrated on [page 68](#)):

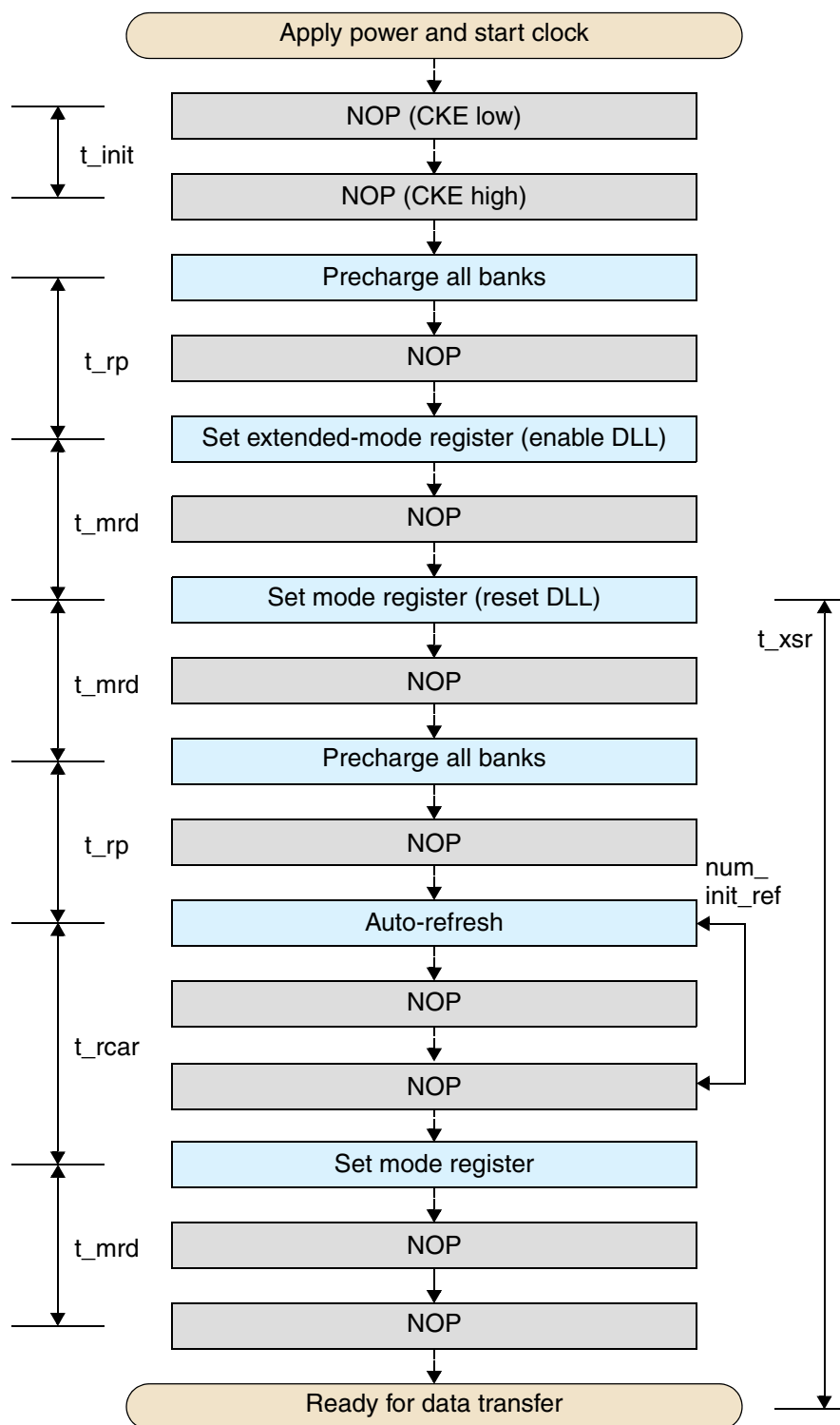
1. Apply power and start clock; maintain a NOP condition at the inputs
2. Maintain stable power, stable clock, low CKE, and NOP input conditions for t_{init} clock cycles
3. Bring CKE high with the NOP command
4. Issue precharge commands for all banks of the device
5. Set the extended-mode register to enable the DLL
6. Set the mode register to reset the DLL and program the operating parameters
7. Issue precharge commands for all banks of the device
8. Issue auto-refresh commands, depending on the value `num_init_ref` in the programmable register
9. Issue a mode-register-set command to program the operating parameters with the reset DLL bit deactivated.



The DDR-SDRAM requires a period of time (t_{xsr}) between the DLL reset (step 6) and any READ command. The requirement is automatically satisfied by the DW_memctl.

Figure 4-16 illustrates the commands issued to the DDR-SDRAM by the controller during the power-on initialization.

Figure 4-16 DDR-SDRAM Power-On Command Sequence



The T_INIT, T_RP, and T_RCAR compile-time parameters are the default values for t_init, t_rp and t_rcar, shown in [Figure 4-16](#) on page 68, which you can use for initialization. If you feel that the reset time of the system is long enough to take care of the t_init time, then you can assign a value of zero to the T_INIT parameter. The t_mrd is fixed at a value of 3 clock cycles.

The DW_memctl initializes the DDR-SDRAM after reset using the specified default compile-time timing parameters. After reset, if you feel that these timing parameters are not adequate, then you can program the timing parameters accordingly and then program the initialize bit – that is, bit 0 of SCTL0 – to 1. This forces the DW_memctl to initialize the DDR-SDRAM.

4.2.2 Mobile DDR-SDRAM Power-On Initialization

The Mobile DDR-SDRAM controller uses the following sequence (illustrated on [page 70](#)):

1. Apply power and start clock; maintain a NOP condition at the inputs
2. Maintain stable power, stable clock, high CKE and NOP input conditions for t_init clock cycles
3. Issue precharge commands for all banks of the device
4. Issue auto-refresh commands, depending on the value num_init_ref in the programmable register
5. Issue a mode-register-set command to program the operating parameters.
6. Issue an extended mode-register-set command

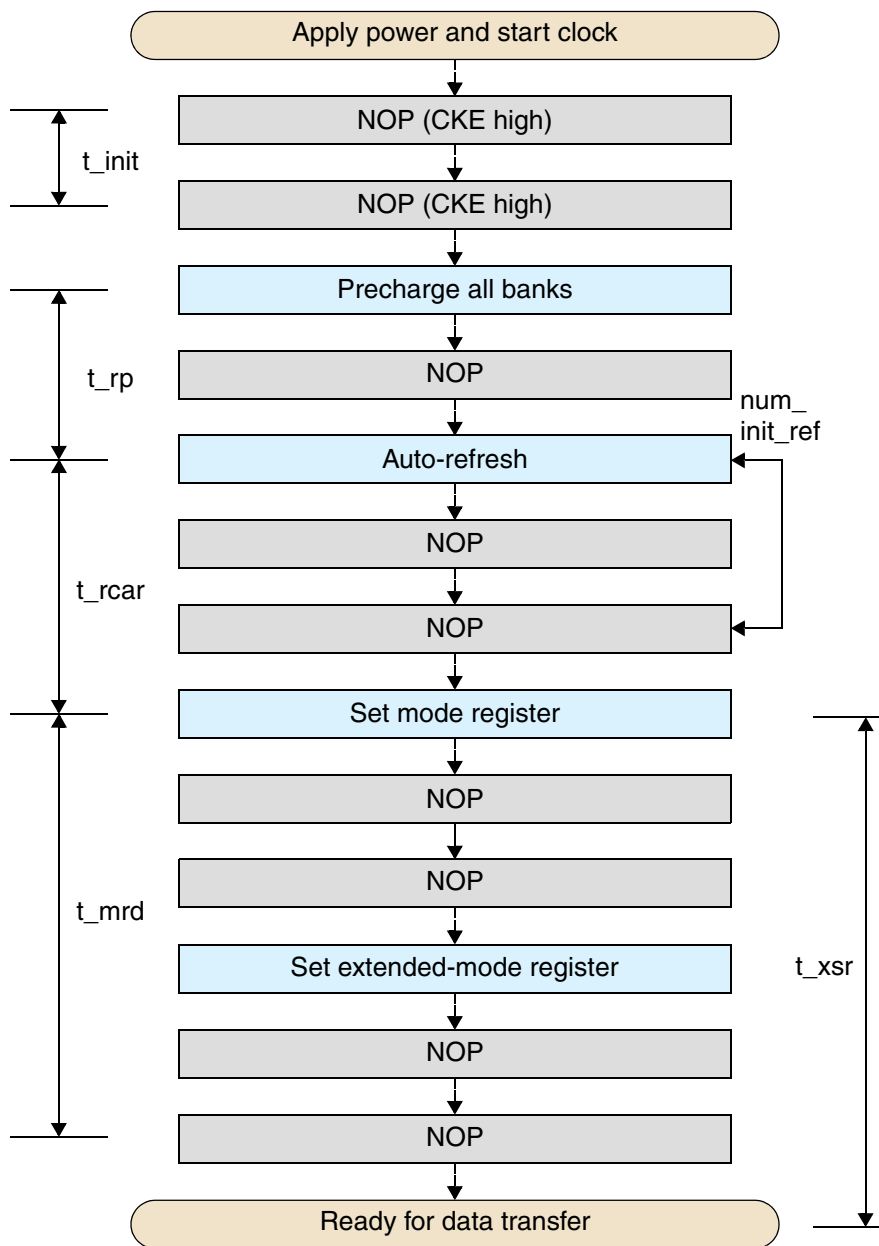
The DW_memctl performs a power-on sequence of the DDR-SDRAM under these circumstances:

- Immediately after reset
- When the programmable initialize bit (bit 0 of SCTL0) is set, the DW_memctl resets the bit when it comes out of initialization
- Upon exiting of the deep power-down mode when using Mobile DDR-SDRAM devices

All SDRAM read/write requests that occur during initialization are queued in the memory controller.

Figure 4-17 illustrates the commands issued to the Mobile DDR-SDRAM by the controller during the power-on initialization.

Figure 4-17 Mobile DDR-SDRAM Power-On Command Sequence



The T_INIT, T_RP, and T_RCAR compile-time parameters are the default values for t_init, t_rp and t_rcar, as shown in [Figure 4-17](#), which you can use for initialization. If you feel that the reset time of the system is long enough to take care of the t_init time, then you can assign a value of zero to the T_INIT parameter. The t_mrd is fixed at a value of 3 clock cycles.

The DW_memctl initializes the Mobile DDR-SDRAM after reset using the specified default compile-time timing parameters. After reset, if you feel that these timing parameters are not adequate, then you can program the timing parameters accordingly and then program the initialize bit – that is, bit 0 of SCTL0 – to 1. This forces the DW_memctl to initialize the Mobile DDR-SDRAM.

4.2.3 Read/Write Operations

The DW_memctl converts all AHB bursts to 4-word bursts on the DDR-SDRAM side. The memory bursts are concatenated to achieve continuous data flow for long AHB bursts. The memory read burst can be terminated with either a precharge command or terminate command, depending on which precharge mode – immediate precharge or delayed precharge – you program. The write burst can be terminated with a subsequent write burst.



Attention

An AHB INCR write/read transfer termination inserts additional memory cycles. There are one or two extra clock cycles inserted on the memory bus at the end of an INCR AHB write/read full burst; this is due to the registered termination that is implemented for better timing. Extra write cycles have write enable and byte select signals masked, which does not cause data corruption. However, there is a potential performance issue with small INCR bursts. Extra read cycles at the end of INCR are not propagated to the bus interface.

The DW_memctl does not use auto-precharge mode.

The DW_memctl supports two precharge modes – immediate precharge mode and delayed precharge mode. If you program for an immediate precharge mode, then the DW_memctl closes the open row after a read or write access. If you program for a delayed precharge mode, then the DW_memctl keeps the row open after an access. The DW_memctl can keep multiple banks open at the same time, depending on the value of num_open_bank in the programmable register. When the number of open banks reaches the num_open_bank (bits 16:12 of SCTL0) and an access to a new bank comes, the DW_memctl will close the oldest bank (the bank opened first) before opening the new bank.

[Table 4-4](#) lists the DW_memctl performance during read/writes under various circumstances, with the following assumptions:

- DW_memctl is idle; that is, no pending read/write request is in the address FIFO
- All control signals to the DDR-SDRAM are registered
- In the case of a read, timing information is relative to the latency from the time the select signal is asserted on the AMBA bus to when the first data is available on the AMBA bus
- In the case of a write, timing information is relative to the latency from the time the select signal is asserted on the AMBA bus to when the first data is written to the DDR-SDRAM

- Timing parameters in [Table 4-4](#) are:
 - tINT – internal delay before a command is sent to the memory device; 2 clock cycles when ADDRDEC_PIPE is not defined; 3 clock cycles when ADDRDEC_PIPE is defined
 - tRCD – active-to-read/write command time; assumed to be 3 clock cycles
 - tCAS – CAS latency; assumed to be 2 clock cycles

Transfer Type	Condition	Timing	Latency
Writes	Page hit	tINT=2	2 (Figure 4-18 on page 73)
	Page miss	(tINT=2) + (tRCD= 3)	5 (Figure 4-19 on page 74)
Reads	Page hit	(tINT=2) + (tCAS=2)	4 (Figure 4-21 on page 76)
	Page miss	(tINT=2) + (tRCD= 3) + (tCAS=2)	7 (Figure 4-22 on page 77)

[Table 4-4](#) lists timing diagrams for simple read/write operations to SDRAMs; all read/writes are assumed to have delayed precharge. In those timing diagrams, the ADDRDEC_PIPE is not defined. The number of idle clock cycles between back-to-back transfers will increase by one when the ADDRDEC_PIPE is defined.

Table 4-4 DDR-SDRAM Read/Write Timing Diagrams

Timing Diagram	Description
Figure 4-18 on page 73	Page-hit single write
Figure 4-19 on page 74	Page-miss single write
Figure 4-20 on page 75	Page-hit burst write
Figure 4-21 on page 76	Page-hit single read
Figure 4-22 on page 77	Page-miss single read
Figure 4-23 on page 78	Page-hit burst read
Figure 4-24 on page 79	Page-hit write followed by write
Figure 4-25 on page 80	Page-hit write followed by read
Figure 4-26 on page 81	Page-hit read followed by read
Figure 4-27 on page 82	Page-hit read followed by write
Figure 4-28 on page 83	Write data path

Figure 4-18 illustrates a page-hit single-cycle write (hburst == SINGLE) transfer to the DDR-SDRAM from the AHB bus.

Figure 4-18 DDR-SDRM Page-Hit Single Write

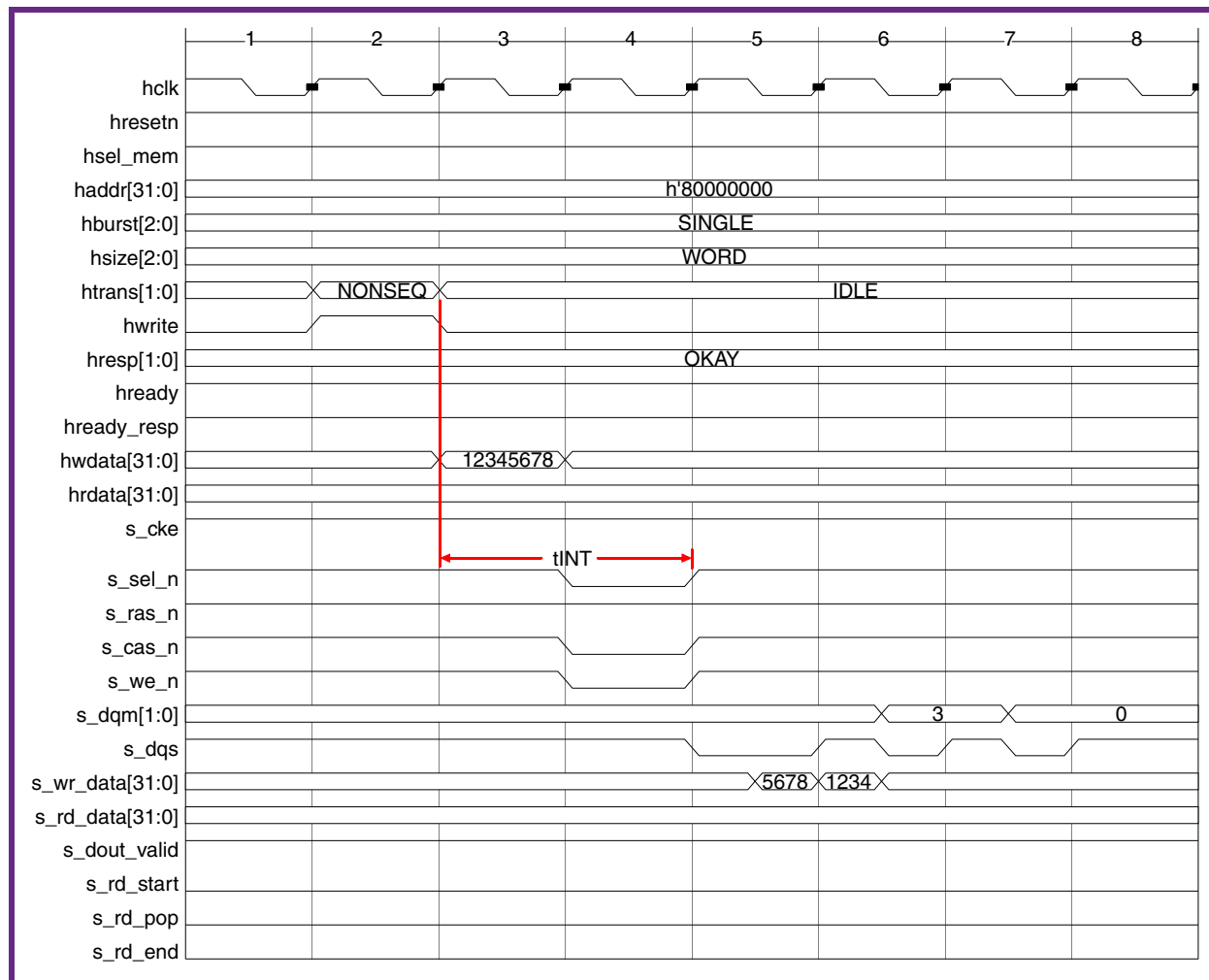


Figure 4-19 illustrates a page-miss single-cycle write (hburst == SINGLE) transfer to the DDR-SDRAM from the AHB bus. DW_memctl takes two internal cycles (tINT) before the activate command is generated on the SDRAM side; s_ras goes low on the rising edge of clock 4.

Figure 4-19 DDR-SDRM Page-Miss Single Write

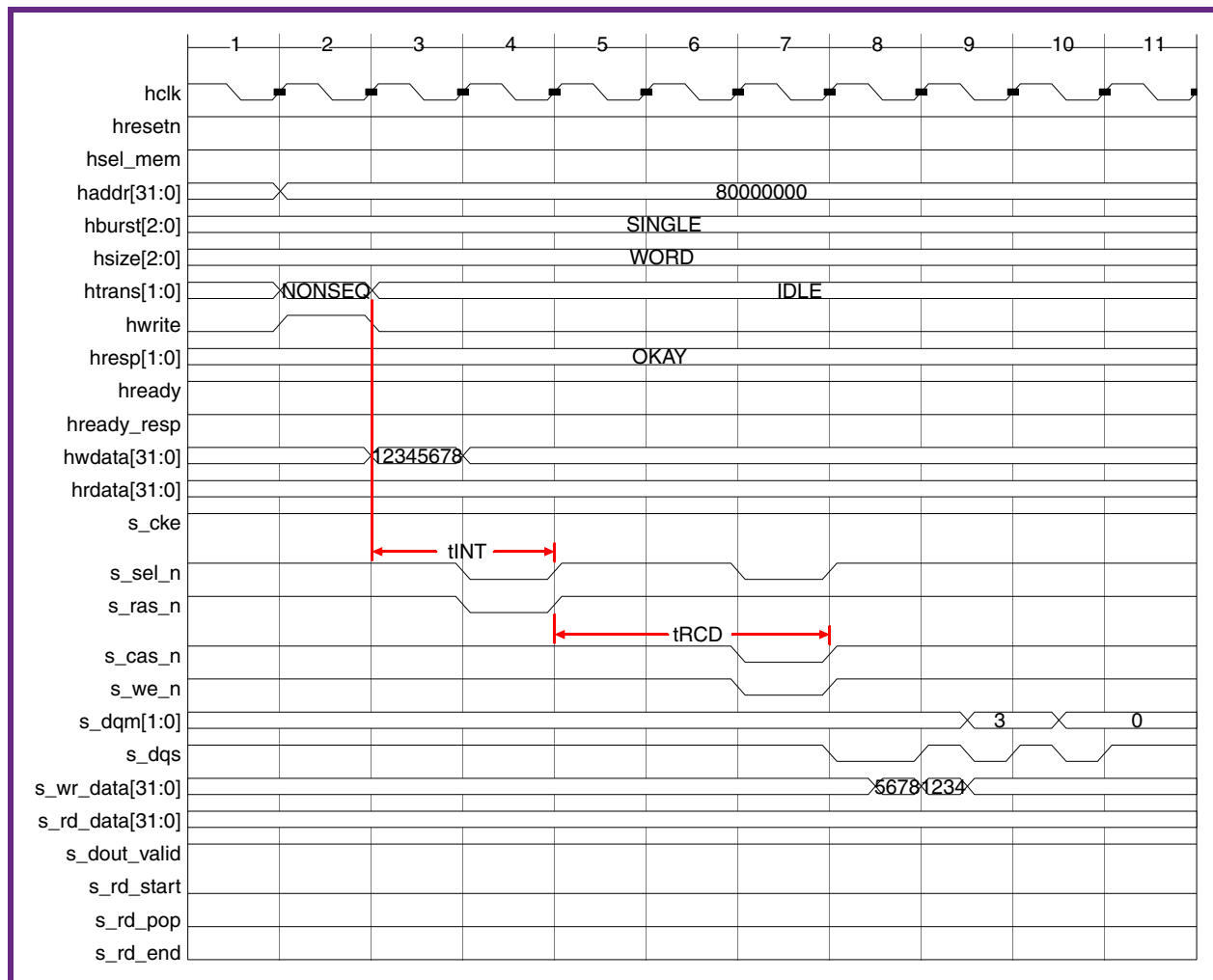


Figure 4-20 DDR-SDRM Page-Hit Burst Write

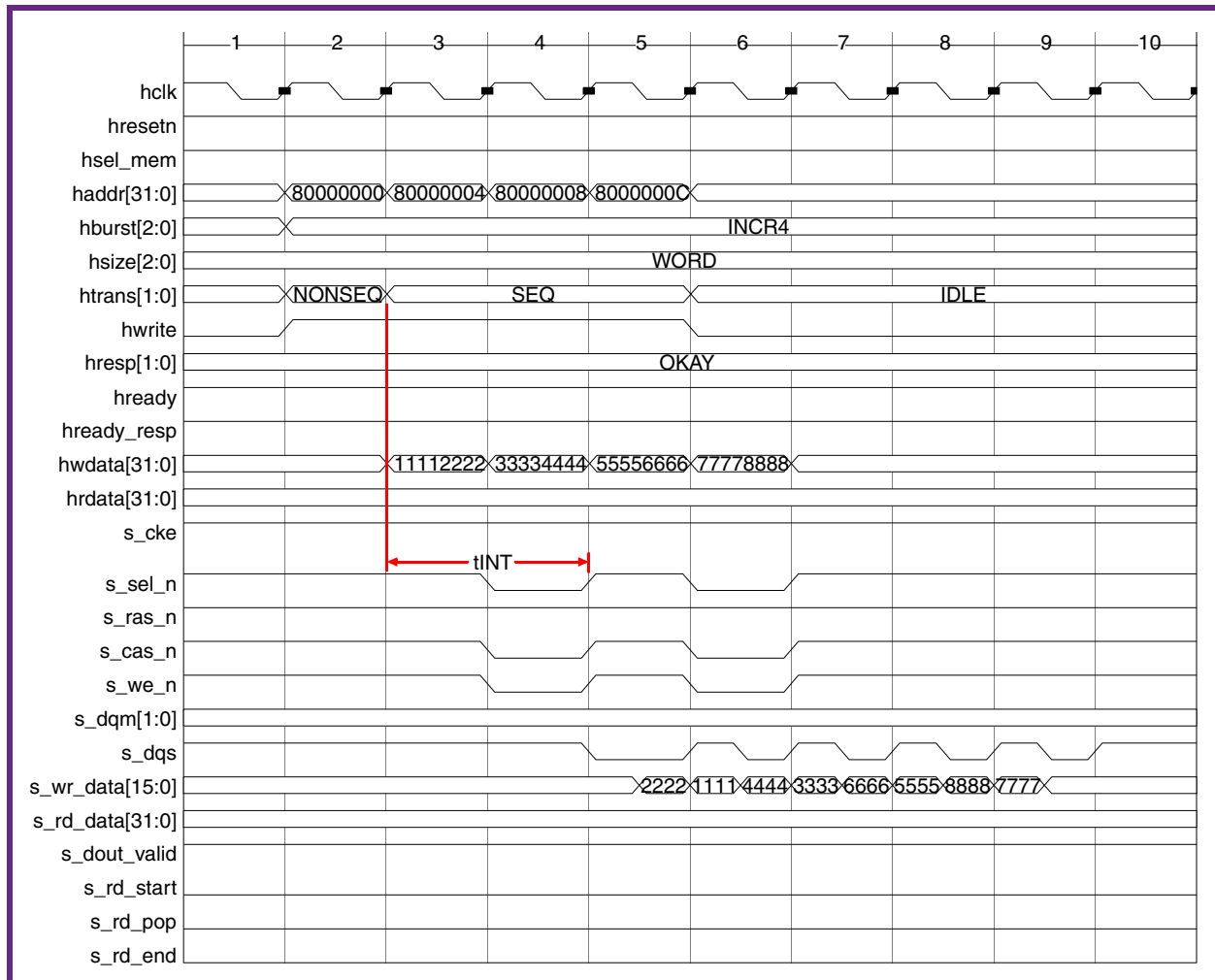


Figure 4-21 illustrates a page-hit single-cycle read (hburst == SINGLE) transfer to the DDR-SDRAM from the AHB bus.

Figure 4-21 DDR-SDRM Page-Hit Single Read

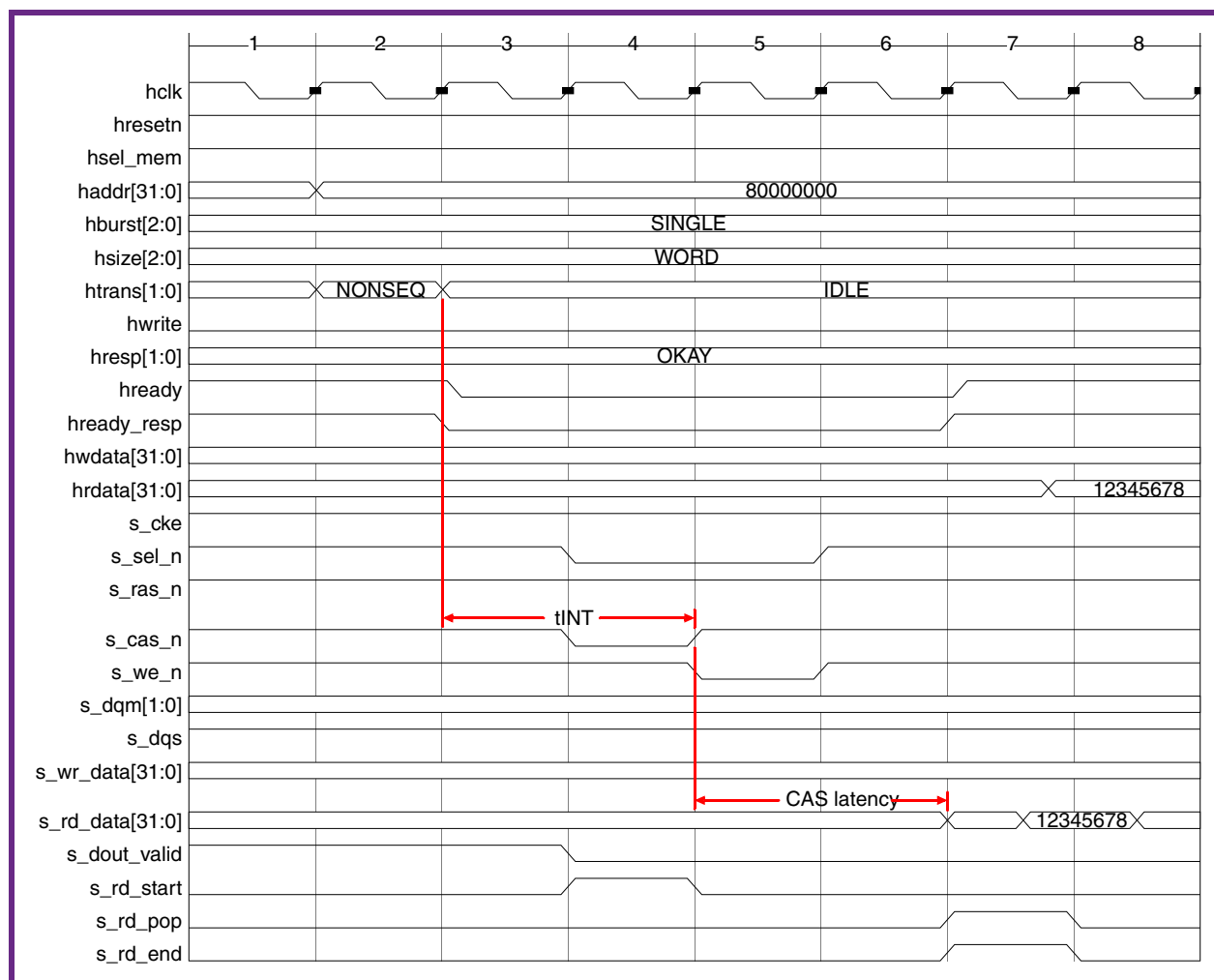


Figure 4-22 illustrates a page-miss single-cycle read (hburst == SINGLE) transfer to the DDR-SDRAM from the AHB bus. DW_memctl takes two internal cycles (tINT) before the activate command is generated on the SDRAM side; s_ras goes low on the rising edge of clock 4.

Figure 4-22 DDR-SDRM Page-Miss Single Read

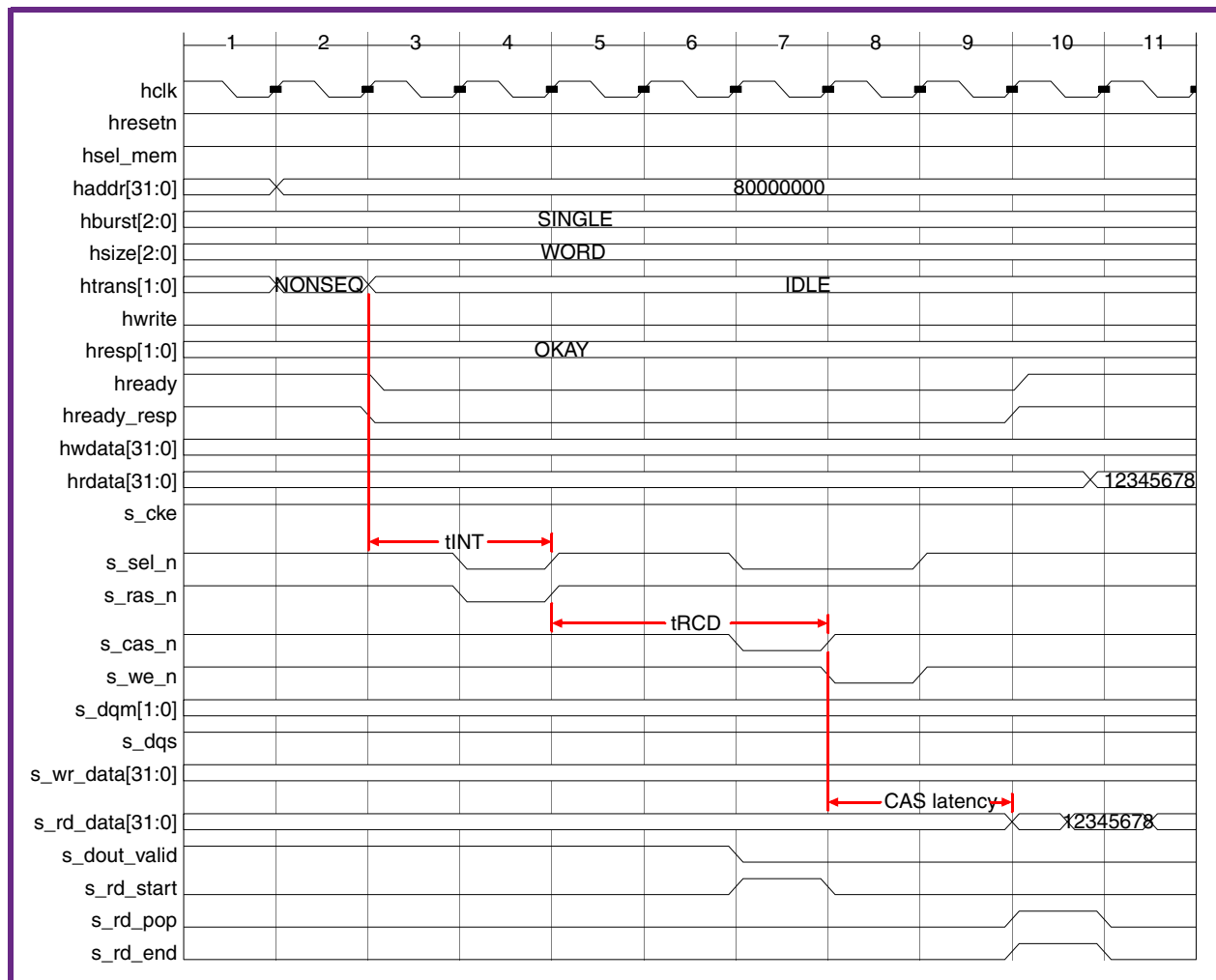


Figure 4-23 illustrates a page-hit burst read (hburst == INCR4) transfer to the DDR-SDRAM from the AHB bus.

Figure 4-23 DDR-SDRM Page-Hit Burst Read

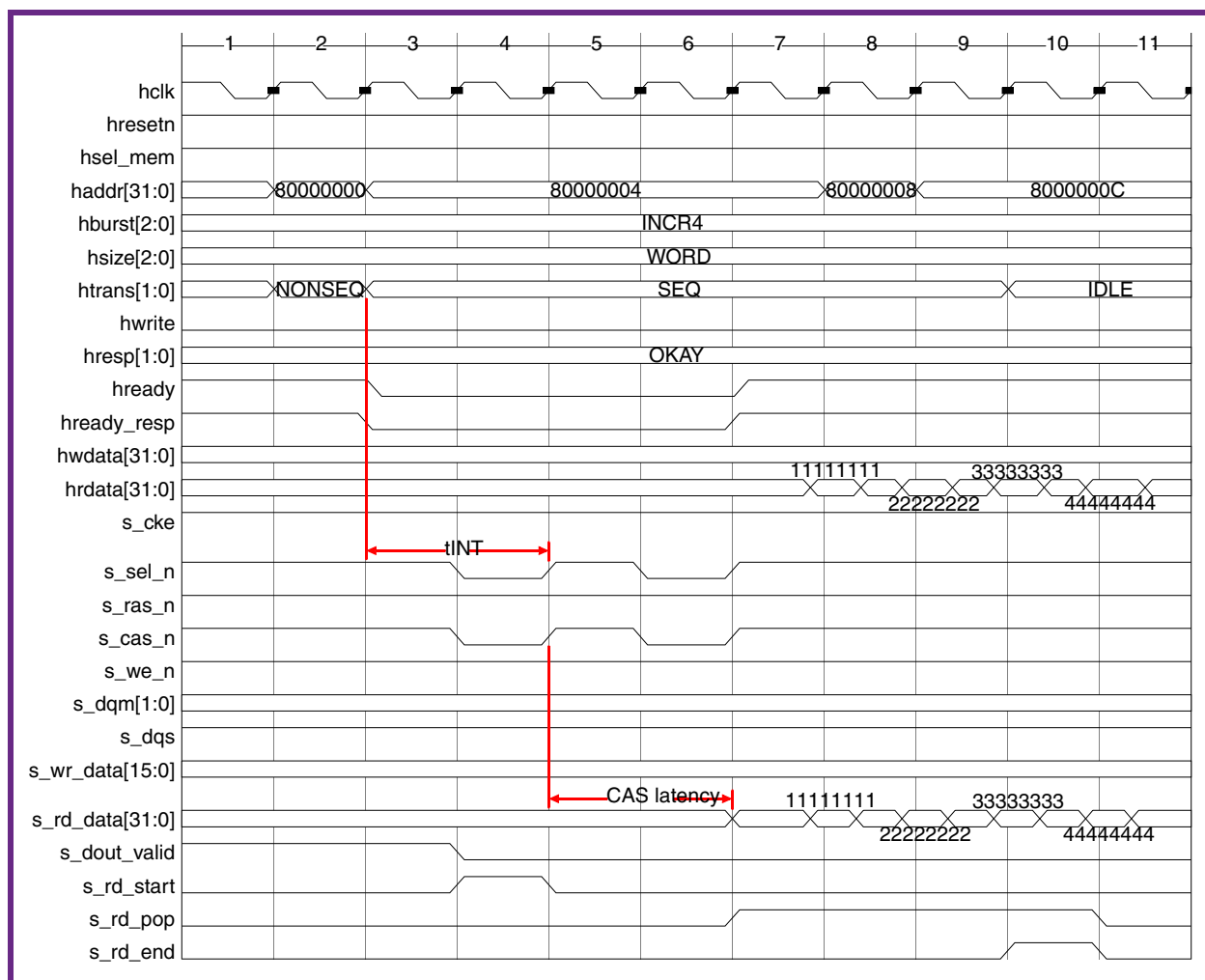


Figure 4-24 illustrates a page-hit single write (hburst == SINGLE) transfer, followed by a page-hit single write transfer.

Figure 4-24 DDR-SDRM Page-Hit Write Followed by Write

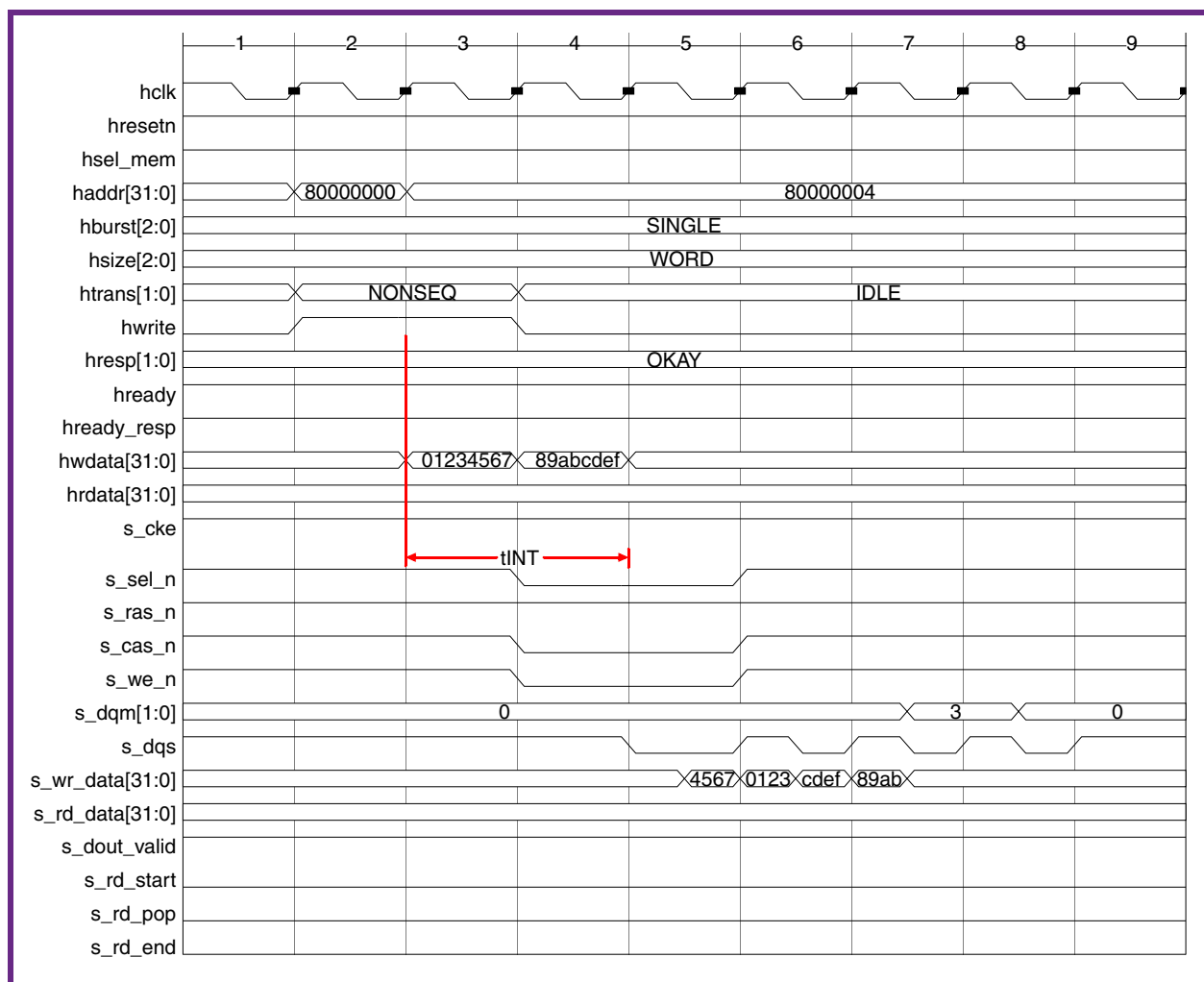


Figure 4-25 illustrates a page-hit single write (hburst == SINGLE) transfer, followed by a page-hit single read transfer.

Figure 4-25 DDR-SDRM Page-Hit Write Followed by Read

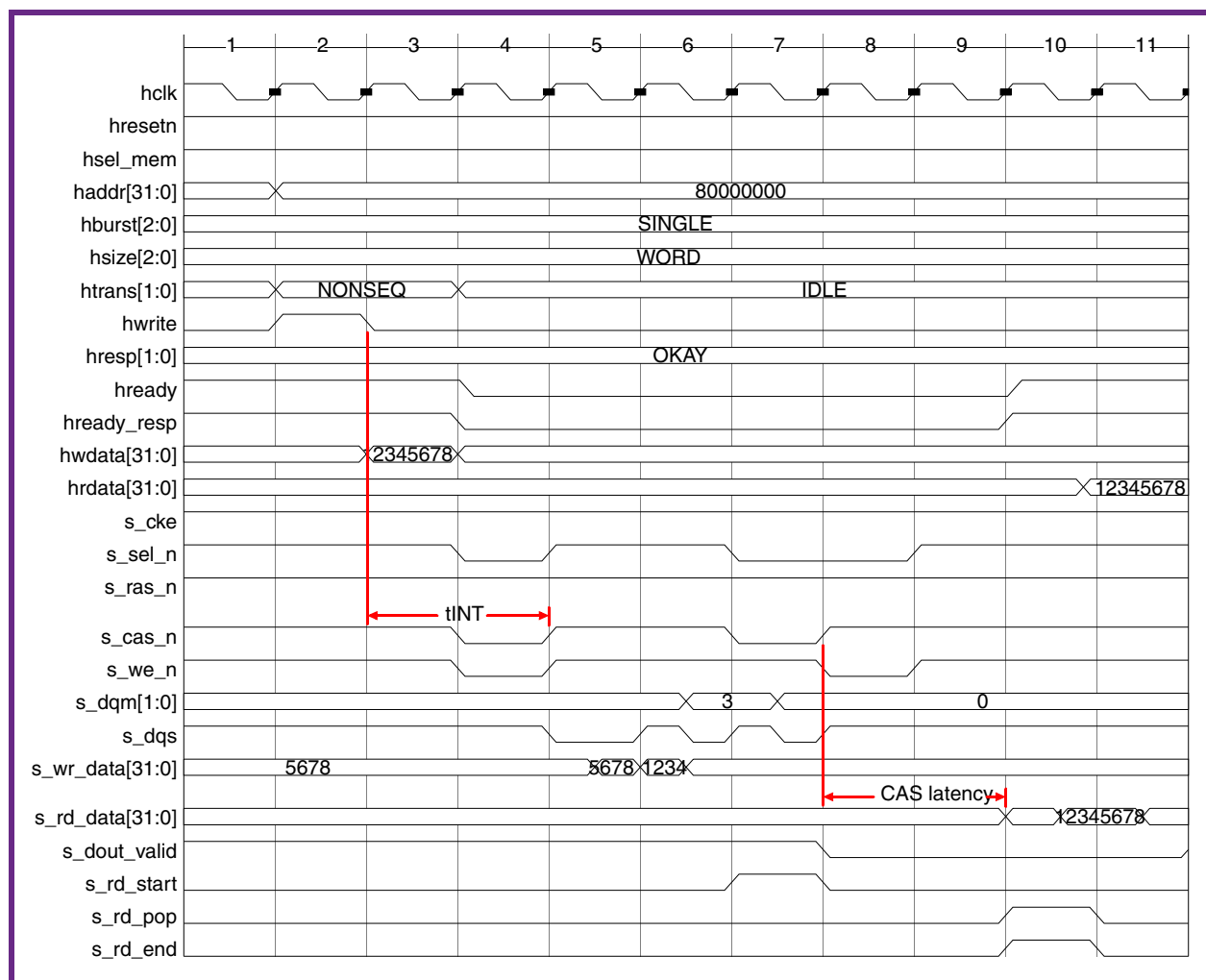


Figure 4-26 illustrates a page-hit single-cycle read (hburst == SINGLE) transfer, followed by a page-hit single read transfer.

Figure 4-26 DDR-SDRM Page-Hit Read Followed by Read

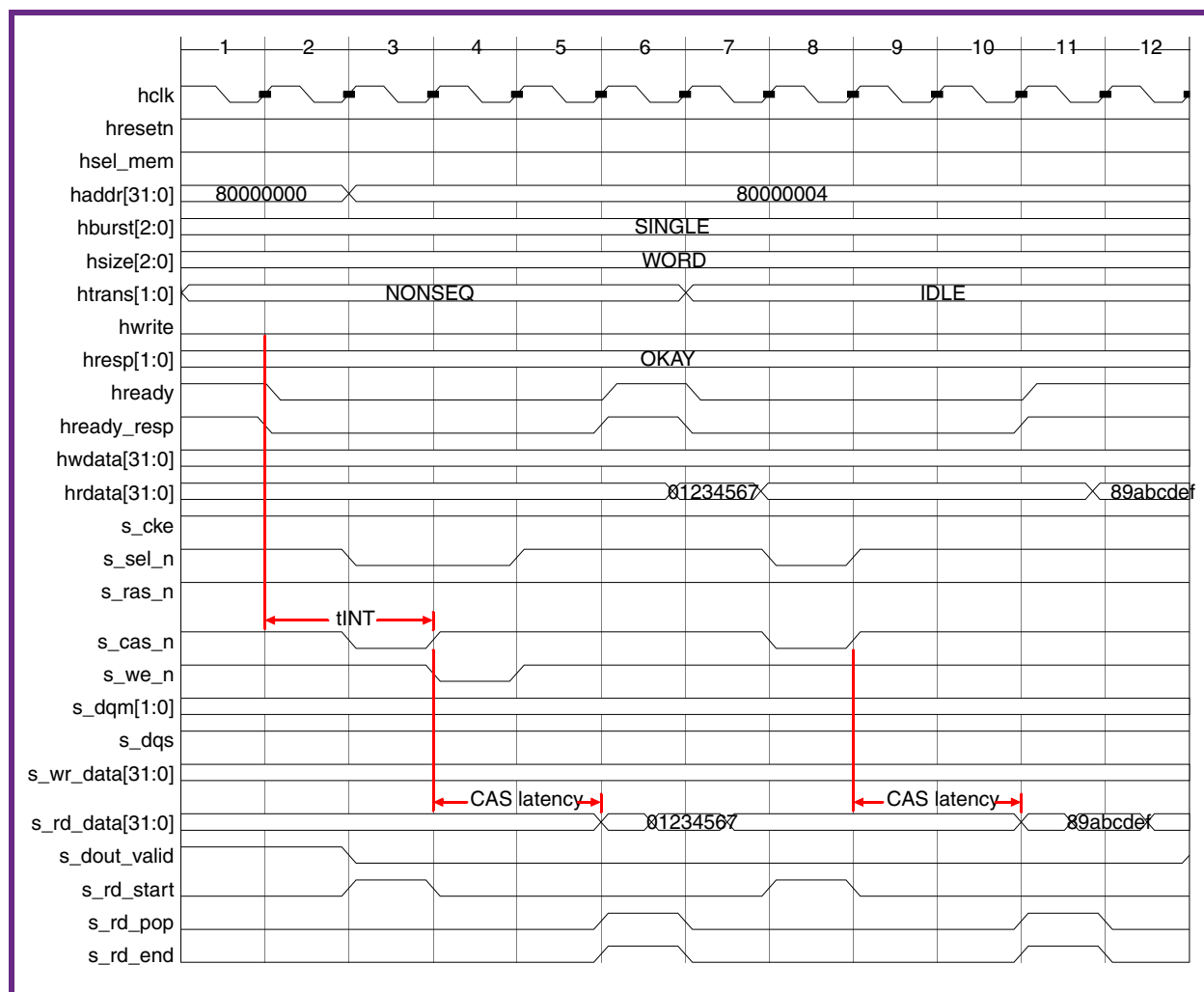
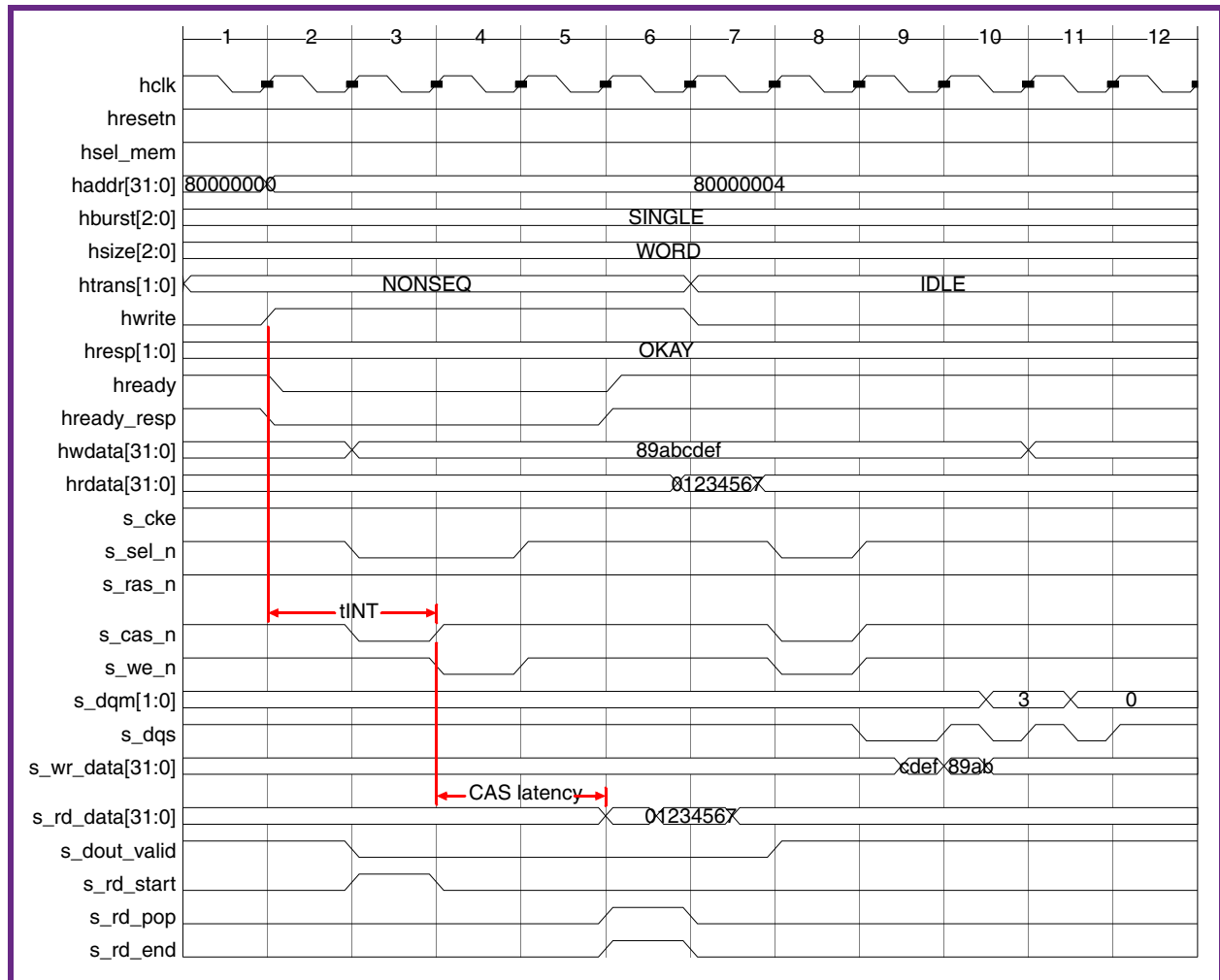


Figure 4-27 illustrates a page-hit single-cycle read (hburst == SINGLE) transfer, followed by a page-hit single write transfer.

Figure 4-27 DDR-SDRM Page-Hit Read Followed by Write



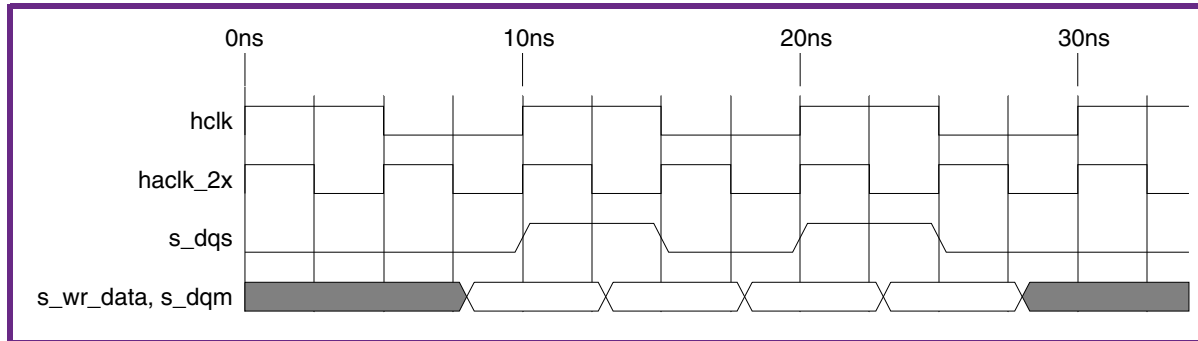
4.2.4 DDR-SDRAM Write Data Path

Note that the DDR-SDRAM write data (**s_wr_data**) and data mask (**s_dqm**) signals are clocked out of the DW_memctl on the falling edge of **hclk_2x** in order to meet setup and hold with regard to the data-strobe signal (**s_dqs**) that is generated on the rising edge of **hclk_2x**.

The DDR-SDRAM signals that are in the 2x clock domain (**hclk_2x**) are coded only in the DW_memctl_miu_ddrwr.v block. This file, along with the top-level file DW_memctl.v, is provided in unencrypted source format for user reference.

In order to improve scan test coverage of the flip-flops clocked by `hclk_2x`, an additional test mode pin (`scan_mode`) is used when the DDR-SDRAM is selected. The logic associated with the `scan_mode` pin is limited to the `DW_memctl_miu_ddrwr.v` module.

Figure 4-28 DDR-SDRAM Write Data Path



4.2.5 Set-Mode and Extended-Mode Register

The `DW_memctl` automatically sets the DDR-SDRAM mode register and extended-mode register during the power-up initialization. During the normal operation, if the user wants to set the mode register or extended-mode register, he needs to set `set_mode_reg` (bit 9 of `SCTLR`) or `exn_mode_reg_update` (bit 18 of `SCTLR`) in the control register (`SCTLR`) to 1. After the memory controller finishes the mode register setting, it clears the `set_mode_reg` or the `exn_mode_reg_update` bit to 0.

The “burst length” field and the “burst type” field of the DDR-SDRAM-mode register are fixed by the `DW_memctl` to “010” (burst length 4) and “0” (sequential burst), respectively. The `DW_memctl` programs the “CAS latency” field and the “operating mode” field of the mode register according to values provided by the user in the control and timing registers.

The `DW_memctl` programs the DDR-SDRAM extended-mode register according to the value provided by the user in the `EXTN_MODE_REG` (address 32'hxxxx_xxAC).

4.2.6 Auto-Refresh

Auto-refresh commands are issued when the refresh control block issues refresh requests. During normal refresh operations, the `DW_memctl` always refreshes one row at a time. It is important for the user to program the `tREF` refresh interval register after a reset, or to set it correctly before synthesis.

If you need to refresh the DDR-SDRAM while a burst is active, normally the `DW_memctl` will issue the refresh command after the ongoing burst completes. However, if the ongoing burst is an AHB INCR burst, the `DW_memctl` will stop the burst, issue the refresh command, and then resume the burst.

The `DW_memctl` takes into account the maximum time it takes to complete a worst-case burst. This is the time to complete a read burst corresponding to an INCR16 burst on the AMBA bus, and with an AMBA-to-DDR-SDRAM data-width ratio of 2:1. It is reasonable to assume 50 cycles for this worst-case burst, with 32 cycles for the data and the remaining 17 cycles for various latencies for the worst case.

The `t_ref` value can be calculated using the following equation:

$$t_{\text{ref}} = \text{refresh_period} / \text{clock_period}$$

where `refresh_period` is typically 7.8/15.6 μs .

The t_{ref} is the value of a free-running counter that the refresh logic in the DW_memctl operates on. When the count expires, the refresh logic gives a refresh request to the DDR-SDRAM control.

Since the 64 ms refresh period is the same for most DDR-SDRAMs, the total number of rows in the DDR-SDRAM limits the minimum operating frequency for the DW_memctl. While calculating the minimum frequency, use this equation:

$$t_{REF} > 50 * (1/f)$$

Typically, the refresh cycle is 15.6 μ s or 7.8 μ s, depending on the refresh rate; [Table 4-5](#) summarizes this.

Table 4-5 Minimum Frequency Requirement

Number of Rows	t_{REF}	Min Frequency
64K	$(64\text{ms} - (50/f))/65536$	51Mhz
32K	$(64\text{ms} - (50/f))/32768$	26Mhz
16K	$(64\text{ms} - (50/f))/163904$	13Mhz
8K	$(64\text{ms} - (50/f))/8192$	6Mhz
4K	$(64\text{ms} - (50/f))/4096$	3Mhz
2K	$(64\text{ms} - (50/f))/2048$	1.5Mhz

The refresh logic in the DW_memctl is inactive when the DW_memctl forces the DDR-SDRAM into self-refresh mode.

[Figure 4-29](#) illustrates the command sequence issued by the DDR-SDRAM controller when performing an auto-refresh.

Figure 4-29 DDR-SDRM Auto-Refresh Command Sequence

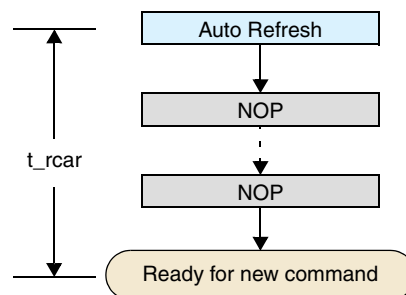
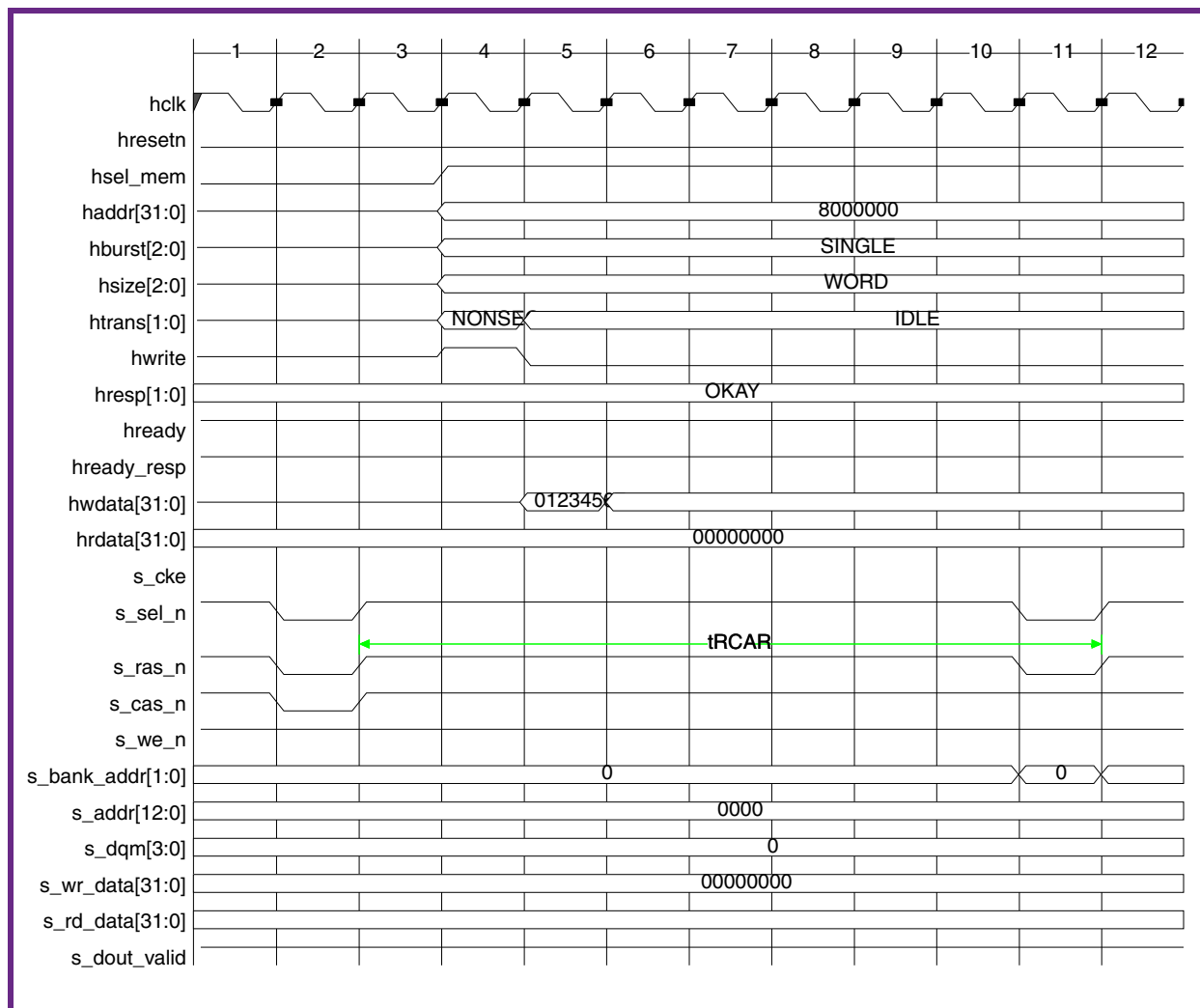


Figure 4-30 shows the timing diagram of an auto-refresh followed by a write.

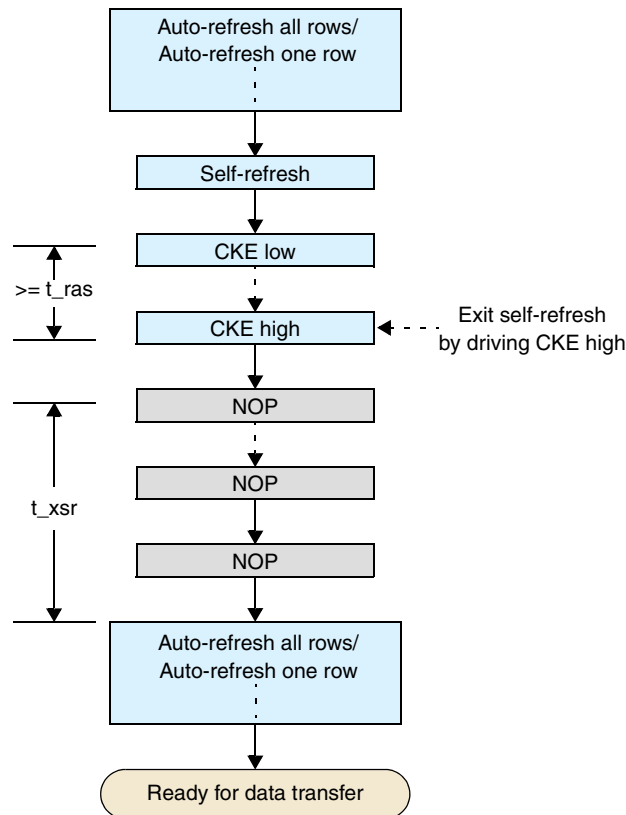
Figure 4-30 DDR-SDRAM Auto-Refresh Timing



4.2.7 Self-Refresh

You can put the DDR-SDRAM into self-refresh mode, at which point the DDR-SDRAM retains data without external clocking and auto-refresh. Figure 4-31 illustrates the command sequence issued by the DDR-SDRAM controller to initiate, maintain, and exit the self-refresh mode.

Figure 4-31 DDR-SDRM Self-Refresh Command Sequence



You can force the DW_memctl to enter self-refresh mode by programming bit 1 of the SDRAM control register (SCTLR) (Address 32'hxxxx_xx0C). The DW_memctl forces the DDR-SDRAM to come out of self-refresh mode when bit 1 of the SCTLR is set to 0. You can set bit 1 of the SCTLR to 0 in software by programming the SDRAM control register. You can set bit 1 of the SCTLR to 0 with hardware by driving the clear_sr_dp input pin to high. You can use the clear_sr_dp pin when the code resides in the DDR-SDRAM and the DDR-SDRAM itself is in self-refresh mode.

Bits 4 and 5 of the SCTLR specify the type of refresh done by the DW_memctl just prior to entering self-refresh mode and just after entering self-refresh mode. Programming bit 4 of the SCTLR to 0 forces the DW_memctl to refresh only one row before putting the DDR-SDRAM into self-refresh mode. The default value of 1 forces the DW_memctl to perform auto-refreshes for all rows. Bit 5 does the same, except that it controls the refresh pattern just after coming out of self-refresh mode.

The DDR-SDRAM can remain in self-refresh mode for an indefinite period of time. The DW_memctl issues NOP commands for at least t_{xsr} before it issues any other commands; t_{xsr} is a programmable register value and has a default value. This value can be programmed only once after reset.

Since it takes time between programming the control register bit to the DDR-SDRAM entering self-refresh mode, the DW_memctl provides a read-only register bit (bit 11 of the SDRAM control register) to indicate that DDR-SDRAM is already in self-refresh mode. If you want to gate off the clock to the DW_memctl when the DDR-SDRAM is in self-refresh mode, you should ensure that this bit is set to 1 before you stop the clock.

When an AHB read/write request to the DDR-SDRAM occurs while the DDR-SDRAM is in self-refresh mode, the DW_memctl generates dummy ready signals to the AHB without accessing external memory; no error response is generated on the AHB bus. While in self refresh mode, AHB reads return read data of undeterministic value and AHB write data is discarded. An AHB response of OKAY is provided for all memory reads and writes while in self refresh mode.

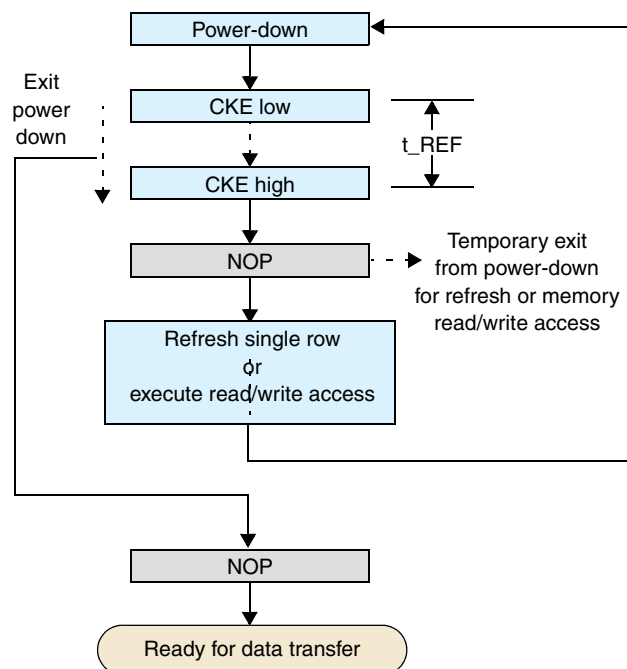
4.2.8 Power-Down

The DDR-SDRAM can be put into power-down mode to save power. There are two ways to force the DW_memctl to put the DDR-SDRAM in power-down mode (bit 19 of SCTL0 must be programmed to 0 to select standard power down):

- Program bit 2 of SCTL0 to 1; program back to 0 to bring the DDR-SDRAM out of power-down mode.
- Use the power_down input pin; can be driven by an external power management unit; the DDR-SDRAM will be in power-down mode as long as this signal stays high.

Figure 4-32 illustrates the command sequence issued by the SDRAM controller to initiate, maintain, and exit power-down mode.

Figure 4-32 DDR-SDRM Power-Down Command Sequence



When in DDR-SDRAM power-down mode, the DW_memctl keeps switching the device back and forth between power-down and refresh mode. It remains in power-down for a t_{ref} period of time, then comes out of power-down and does a single-row refresh; then it again goes into power-down mode. The

DW_memctl keeps the DDR-SDRAM in this periodic power-down/refresh/power-down sequence until it is commanded to exit power-down mode (by programming bit 2 of SCTL0 to 0, or by de-asserting the power_down input pin).

When an AHB read/write request to the DDR-SDRAM occurs while the DDR-SDRAM is in power-down mode, the DW_memctl brings the DDR-SDRAM out of power-down mode and issues the read/write access to the DDR-SDRAM. The DW_memctl then puts the DDR-SDRAM back to power-down mode after the read/write access.

4.2.9 Deep Power-Down (Mobile DDR-SDRAM Only)

Deep power-down is an operating mode used to achieve maximum power reduction by eliminating the power of the memory array. Data is not retained once the device enters deep power-down mode. Deep power-down is only supported when DW_memctl is configured as a Mobile DDR-SDRAM controller. It should only be enabled when the attached memory device supports deep power-down.

To enter this mode:

- Program bits 2 and 19 of SCTL0 to 1; program bit 2 back to 0 to bring the SDRAM out of deep power-down mode.
- Drive the power_down input pin high, with bit 19 of SCTL0 already programmed to 1; power_down can be driven by an external power management unit; the SDRAM will be in deep power-down mode as long as the power_down input signal stays high.

When an AHB read/write request to the Mobile DDR-SDRAM occurs while in deep power-down mode, the DW_memctl generates dummy ready signals to the AHB without accessing external memory; no error response is generated on the AHB bus. While in deep power-down mode, AHB reads return read data of undeterministic value and AHB write data is discarded. An AHB response of OKAY is provided for all memory reads and writes while in deep power-down mode.

Upon exit from deep power-down, the Mobile DDR-SDRAM initialization sequence is executed, as described in “[Mobile DDR-SDRAM Power-On Initialization](#)” on page 69. Note that t_init is usually 200us for Mobile DDR-SDRAM initialization, so if the T_INIT configuration parameter is not set to achieve this number of initialization cycles, it can be programmed by software in bits 15:0 of STMG1R.



Note

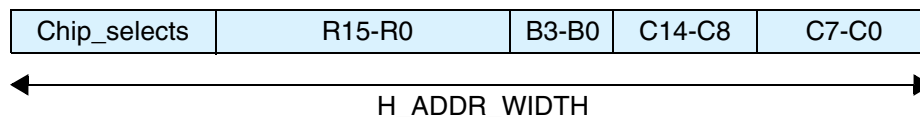
If the T_INIT configuration parameter is not set to a value equal to or greater than t_init/clock_period, the supplied DW_memctl Deep Power Down test (test_mobile_sdram_dpd) will report errors from the DesignWare Memory Models. These errors are reported when there are commands (other than NOP/DESELECT) issued to the Mobile DDR-SDRAM before the required t_init initialization period is completed.

4.3 Address Multiplexing

The DW_memctl performs address multiplexing in order to convert the address generated by the host interface to the SDRAM row address, column address, and bank address.

Figure 4-33 shows a host address format where all SDRAM address bits supported by the DW_memctl are used; that is, column address C0-C14, row address R0-R15, and bank address B0-B3.

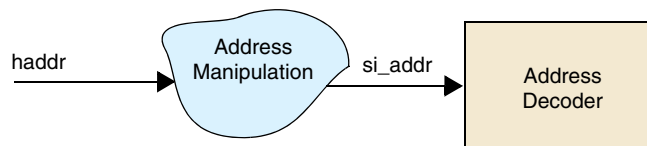
Figure 4-33 Host Address Format



The DW_memctl supports a minimum page size of 256 words; thus, only bits C0-C7 are fixed. The position of all other address bits can change, depending on how many column, row, and bank address bits are used.

Because AMBA addressing is always in bytes and SDRAM addressing depends on the data width of the supported SDRAM, some address manipulation is required before giving it as input to the address decoding unit, as illustrated in Figure 4-34.

Figure 4-34 Address Path in DW_memctl



In Table 4-6, haddr is the address received from the AMBA interface, and si_addr is the input to the address decoder.

Table 4-6 Host Address Manipulation

Memory data width	si_addr
16	haddr[H_ADDR_WIDTH:1]
32	haddr[H_ADDR_WIDTH:2]
64	haddr[H_ADDR_WIDTH:3]
128	haddr[H_ADDR_WIDTH:4]

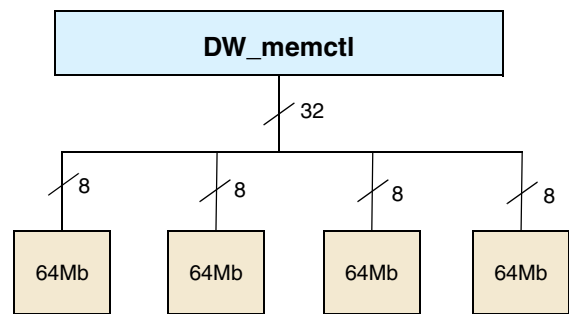
Table 4-7 lists the address format for one memory device configuration.

Table 4-7 Example Configuration 1

H_DATA_WIDTH	S_DATA_WIDTH	Memory Configuration	Col_addr_width	Row_addr_width	Bank_addr_width	Block Size [†]
32	32	8M x 8 x 4 = (64Mb) x 4	9	12	2	10 (8MB * 4 = 32MB)
[†] Block size (bits 4:0 of the mask registers for chip select SMSKR0-7)						

Figure 4-35 shows the configuration of this SDRAM.

Figure 4-35 Example 1 Configuration



Since the S_DATA_WIDTH is 32, the effective address input to the address decoder is haddr [31:2]. Figure 4-36 shows the format of the address input to the address decoder, relative to the host address.

Figure 4-36 Example 1 Host Address Format

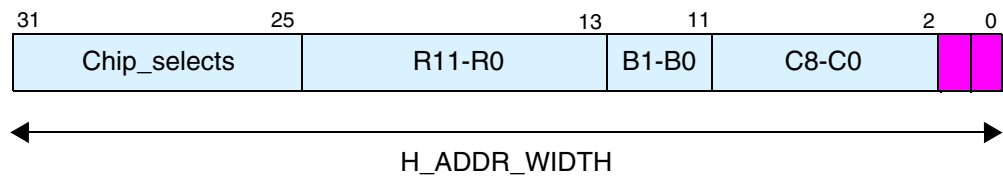


Table 4-8 lists the address format for another memory device configuration.

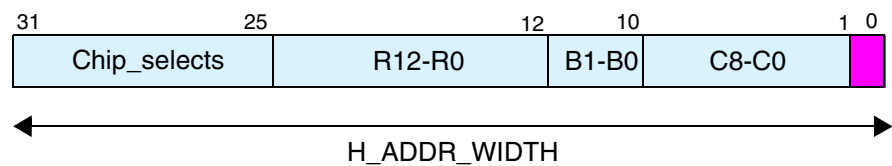
Table 4-8 Example Configuration 2

H_DATA_WIDTH	S_DATA_WIDTH	Memory Configuration	Col_addr_width	Row_addr_width	Bank_addr_width	Block Size [†]
32	16	16Mx16 (64Mb)	9	13	2	10(32MB)

[†] Block size (bits 4:0 of the mask registers for chip select SMSKR0-7)

Since the S_DATA_WIDTH is 16, the effective address input to the address decoder is haddr [31:1]. Figure 4-37 shows this format of the address input to the address decoder, relative to the host address.

Figure 4-37 Example 2 Host Address Format



4.3.1 Use of Higher AHB Address for Bank Address Decoding

DW_memctl has a feature that uses the higher address bits of the AHB address to generate the bank address of the SDRAM. You can enable this mode by setting the compile-time parameter `HIGHER_ADDR_FOR_BANKS` to 1. This mode is especially useful for Mobile SDR-SDRAM and Mobile DDR-SDRAM devices, while it is not of much use for SDR-SDRAMs and DDR-SDRAMs.

The Mobile SDR-SDRAMs and Mobile DDR-SDRAMs enable you to selectively switch off banks. Using higher address bits for banks provides contiguous system address space, even if some banks are switched off. Enabling this mode can reduce the page-hit ratio during CAS address crossovers.

The AHB address format, with respect to the SDRAM address format when this mode is enabled, is as follows:

$haddr = \{chip-sel, bank, ras, cas, byte-addr\}$

The AHB address format, with respect to SDRAM address format when this mode is disabled, is as follows:

$haddr = \{chip-sel, ras, bank, cas, byte-addr\}$

4.4 Chip Select Decoding

You can select the number of chip selects during compile time using the `N_CS` parameter. You should specify the size of the memory connected to each chip select in bits 4:0 of the corresponding Address Mask Registers (`SMSKRn`) (address 32'hxxxx_xx54-70). [Table 4-9](#) lists the number of address bits that are used to generate a chip select, which is dependent on the block size programmed in the mask register.

Table 4-9 Number of Bits for Generating Chip Selects

Mask Register Bits	Block Size	Address Bits for Addressing a Block	Bits used in Address Comparison
00000	Unused		
00001	64KB	16	H_ADDR_WIDTH-1:16
00010	128KB	17	H_ADDR_WIDTH-1:17
00011	256KB	18	H_ADDR_WIDTH-1:18
00100	512KB	19	H_ADDR_WIDTH-1:19
00101	1MB	20	H_ADDR_WIDTH-1:20
00110	2MB	21	H_ADDR_WIDTH-1:21
00111	4MB	22	H_ADDR_WIDTH-1:22
01000	8MB	23	H_ADDR_WIDTH-1:23
01001	16MB	24	H_ADDR_WIDTH-1:24
01010	32MB	25	H_ADDR_WIDTH-1:25
01011	64MB	26	H_ADDR_WIDTH-1:26

[†] No address comparison in 32-bit AHB address mode

Table 4-9 Number of Bits for Generating Chip Selects

Mask Register Bits	Block Size	Address Bits for Addressing a Block	Bits used in Address Comparison
01100	128MB	27	H_ADDR_WIDTH-1:27
01101	256MB	28	H_ADDR_WIDTH-1:28
01110	512MB	29	H_ADDR_WIDTH-1:29
01111	1GB	30	H_ADDR_WIDTH-1:30
10000	2GB	31	H_ADDR_WIDTH-1:31
10001	4GB	32	H_ADDR_WIDTH-1:32 [†]

[†] No address comparison in 32-bit AHB address mode

The number of bits in the chip-select register used for address comparison depends on the mask register value; only those bits will be compared with the host address for generating chip selects.

**Note**

The base address specified for a particular chip select must be in terms of the block size chosen for that particular chip select. For example, if you choose a block size of 4 MB for chip select0, then the base address has to be at a block boundary. Therefore you can have a base address such as 0x0000_0000, 0x0040_0000, 0x0080_0000, and so on.

The DW_memctl supports address aliasing and remapping, which can be done only on chip select0 and chip select1.

When aliasing is enabled, the chip select becomes active when the AHB address matches either the CHIP_SELECTn_BASE_ADDRESS or the CHIP_SELECTn_ALIAS_ADDRESS. Similarly, when remapping is enabled, the chip select becomes active when the AHB address matches the CHIP_SELECTn_BASE_ADDRESS and remap pin is 0, or when the remap pin is 1 and the AHB address matches the CHIP_SELECTn_REMAP_ADDRESS.

ALIAS REGISTER0 (CSALIAS0_LOW) controls the aliasing for chipselect0. ALIAS REGISTER1 (CSALIAS1_LOW) controls the aliasing for chip select1.

4.4.1 Scenario 1

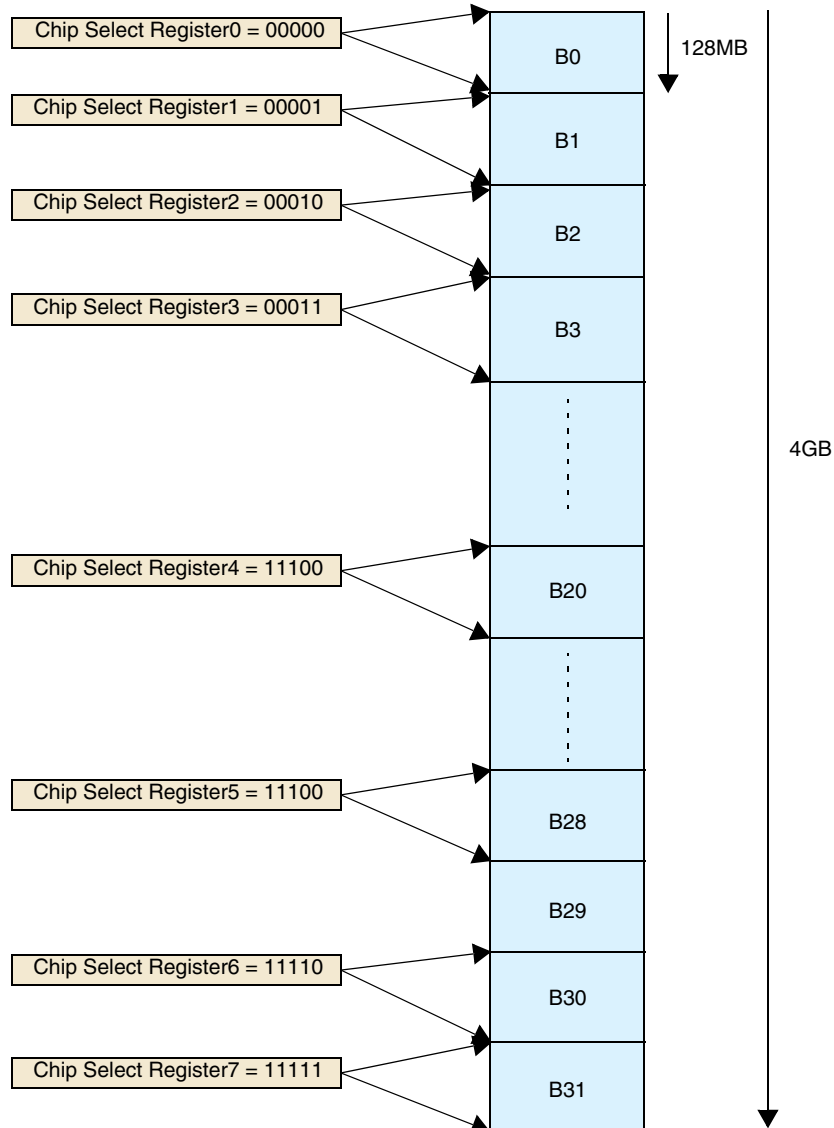
- AMBA address width = 32
- 128MB SDRAM connected to each of the eight chip selects

In this case, since each of the eight chip selects are connected to an SDRAM of the same size (128MB), you should program bits 4:0 of the mask register that corresponds to each of the chip selects to a value of 12 (128MB).

With an address width of 32, AMBA can theoretically access 4GB of memory. If each of the SDRAMs connected to the chip selects are 128MB each, then this represents 8 out of the possible 32 blocks of 128MB in the 4GB address space.

Figure 4-38 shows that chip selects 0-3 are mapped to blocks 0-3, chip select 4 is mapped to block 20, and chip selects 5-7 are mapped to blocks 28, 30, and 31. Figure 4-38 also shows the corresponding values of the chip-select base address.

Figure 4-38 Chip Select Generation Example 1



4.4.2 Scenario 2

- H_ADDR_WIDTH = 32
- N_CS = 2
- S_ROW_ADDR_WIDTH = 13
- S_COL_ADDR_WIDTH = 9
- S_BANK_ADDR_WIDTH = 2
- S_DATA_WIDTH = 16
- Block size chosen for chip select0 = 32MB (value of 10 in the mask register)
- Block size chosen for chip select1 = 32MB (value of 10 in the mask register)
- Programmed value for row_address_width = 11
- Programmed value of column address_width = 8
- Programmed value of bank address_width = 1
- Programmed value of SDRAM data width = 16

This scenario chooses a block size of 32MB for both chip select0 and chip select1, but uses only a 2MB memory device for each chip select. Since the block size is 32MB, only bits 31:25 of the host address are compared for generating the chip select. Additionally, only the 21 LSB bits generate the address within a block.

Figure 4-39 illustrates the format of the host address; the address decoder ignores bits 24:21 in the host address.

Figure 4-39 Host Address Seen by Address Decoding Logic – Example 2

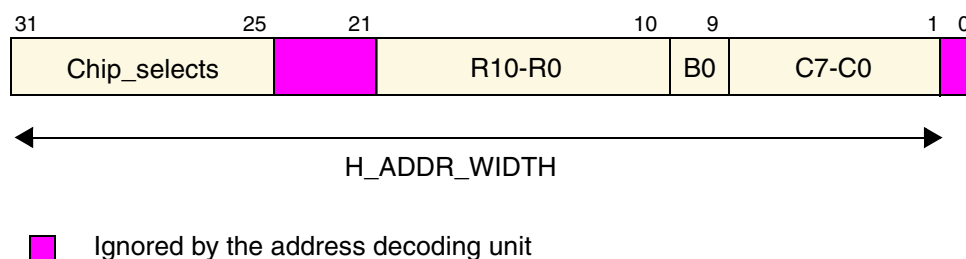
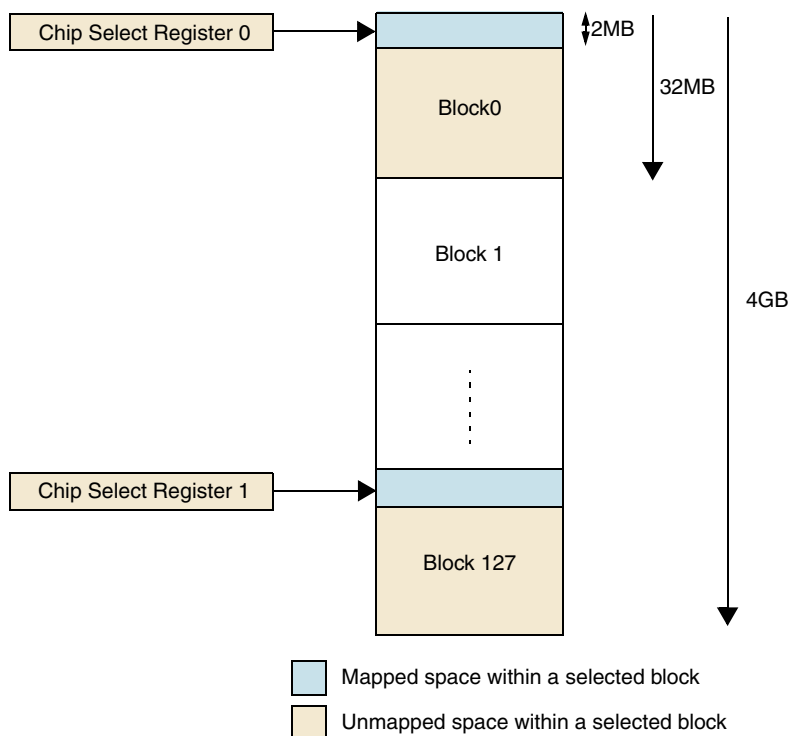


Figure 4-40 illustrates the memory mappings of the configuration in this scenario. If you try to access a location that is offset more than 2MB from the base address, it will be mapped to a location within 2MB from the base address. Thus you can use a smaller memory device than the specified block size indicates, but you must be careful with the addressing.

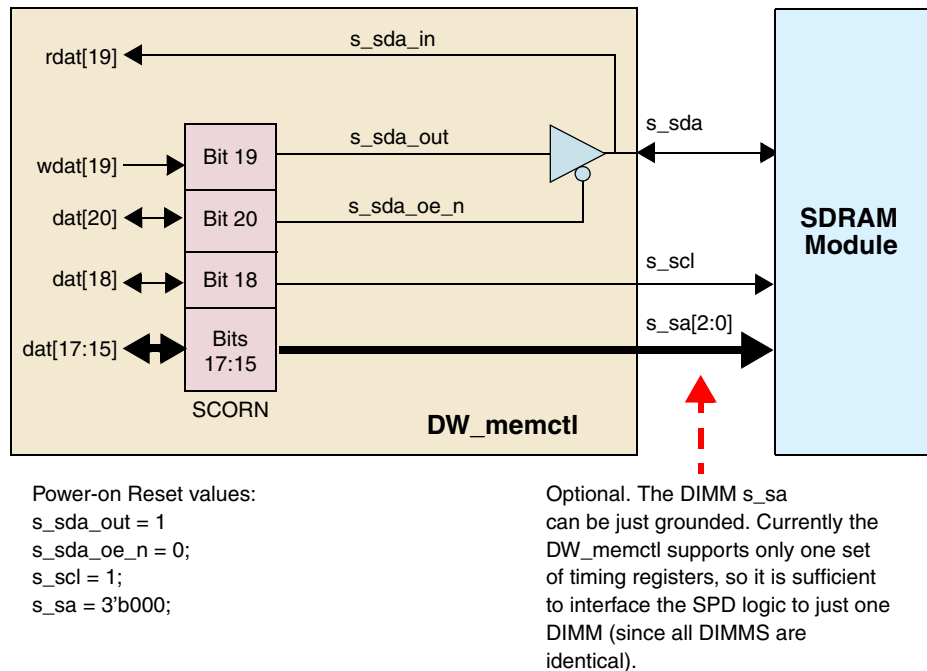
Figure 4-40 Memory Mapping in Example 2



4.5 Interfacing DW_memctl with SDRAM SIMM/DIMM Modules

The DW_memctl supports interfacing with SDRAM SIMM/DIMM modules. The differentiating factor for interfacing to SDRAM modules is the Serial Presence Detect (SPD) feature support, as illustrated in Figure 4-41.

Figure 4-41 SPD Interface to SDRAM Modules



The DW_memctl does not have a built-in I²C serial EPROM interface to read the SPD feature. Instead, it is interfaced to the SPD through a parallel port-like interface, which can be controlled by writes and reads to the SCONR register.

You can:

- Use bits 20:15 of the SDRAM Config Register (SCONR) for the SPD interface.
- Program the address of the SPD device in bits 17:15; this is optional and need not be brought out of the chip.
- Toggle the clock input to the SPD using bit 18 of the SCONR
- Toggle the write-data input to the SPD using bit 19 and the tri-state control bit 20 of the SCONR; read serial using data using bit 19 of this register

For details about the I²C protocol, refer to the *I²C EEPROM Data Sheet*. Also refer to the “Memory Module Serial Presence-Detect” application notes provided by Micron at:

http://www.micron.com/support/part_info/technotes

The I²C EEPROM data can be read as:

1. Issue the I²C “START” command.
2. Issue Sequential Read command (and device address):
for i=0; i < 128 (or 256) read bytes
3. Issue I²C “STOP” command.

The following is pseudo-code for reading the I²C EEPROM data:

```

bit [32:0] x;
array [128] bit[32:0] EEData;
bit [7:0] DAddr, RcvDat

x = peek32(SCORN)
x = x & 32'hFFEF_FFFF; // s_sda_oe_n = 0; enable s_sda drive
poke32(SCORN, x)

// IIC START Command
x = x & 32'hFFE7_FFFF; // s_sda = 0; (when s_scl = 1, indicated IIC "Start")
poke32(SCORN, x); wait(4_micro_sec);

// Send device Address & Read command - MSB first LSB last
DAddr = 8'b1010_0001; // [7:4] = 4'b1010 - eeprom requirement; [3:1] = device_addr; [0] - read
for(i=7; i >= 0 ; i--) {
x = x & 32'hFFEB_FFFF; // s_scl = 0 - clock low
poke32(SCORN, x); Wait(4_micro_sec);

x = {x[31:20], DAddr[i], x[18:0]}; // s_sda_out = DAddr[i]
poke32(SCORN, x); wait(4_micro_sec);

x = x | 32'h0004_0000; // s_scl = 1- clock high
poke32(SCORN, x); Wait(4_micro_sec);
}
// ACK cycle by EEPROM
x = x & 32'hFFEB_FFFF; // s_scl = 0 - clock low
poke32(SCORN, x);
x = x | 32'h0010_0000; // s_sda_oe_n = 1; tristate s_sda
poke32(SCORN, x); wait(4_micro_sec);
x = x | 32'h0004_0000; // s_scl = 1- clock high
poke32(SCORN, x); Wait(4_micro_sec);

// Receive all the eeprom bytes - MSB first, LSB last
for(j=0; j < 128; j++) {

RcvData = 0;
for(i=7; i >= 0 ; i--) {
x = x & 32'hFFFB_FFFF; // s_scl = 0 - clock low
poke32(SCORN, x); Wait(4_micro_sec);

x = x | 32'h0004_0000; // s_scl = 1- clock high
poke32(SCORN, x); Wait(4_micro_sec);

x= peek32(SCORN);
RcvData[i] = x[19];
}
EEData[j] = RcvData;

```

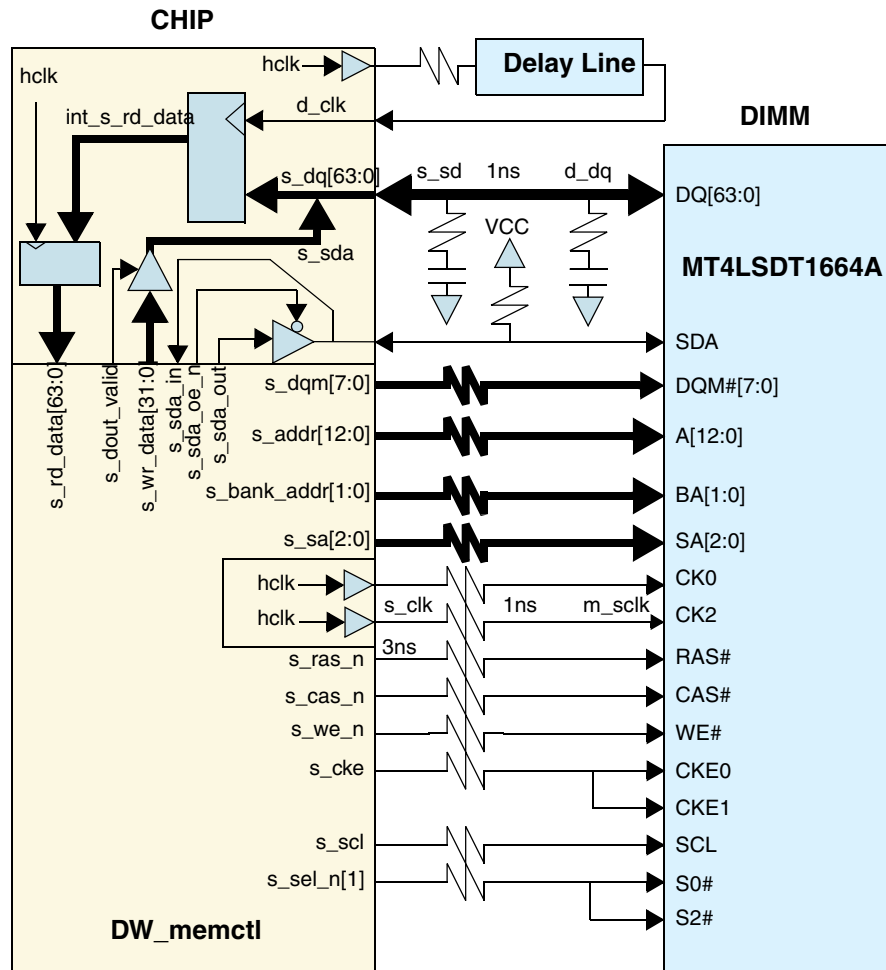
```
// ACK cycle from DW_memctl
x = x & 32'hFFFB_FFFF; // s_scl = 0 - clock low
poke32(SCORN, x);
x = x & 32'hFFFE7_FFFF; // s_sda = 0 - ack; s_sda_oe_n = 0 - drive s_sda
poke32(SCORN, x); wait(4_micro_sec);
x = x | 32'h0004_0000; // s_scl = 1- clock high
poke32(SCORN, x); Wait(4_micro_sec);
}

// IIC STOP Command
x = x | 32'h000C_0000; // s_sda = 1; (when s_scl = 1, indicates IIC "stop").
poke32(SCORN, x); Wait(4_micro_sec);
```

4.6 Interfacing DW_memctl to Single Bank DIMMS

Figure 4-42 illustrates the connection interfacing the DW_memctl to a single-bank DIMM, which in this example is the Micron DIMM module MT4LSDT1664A, a 128MB module.

Figure 4-42 DW_memctl Interface to Single-Bank DIMM

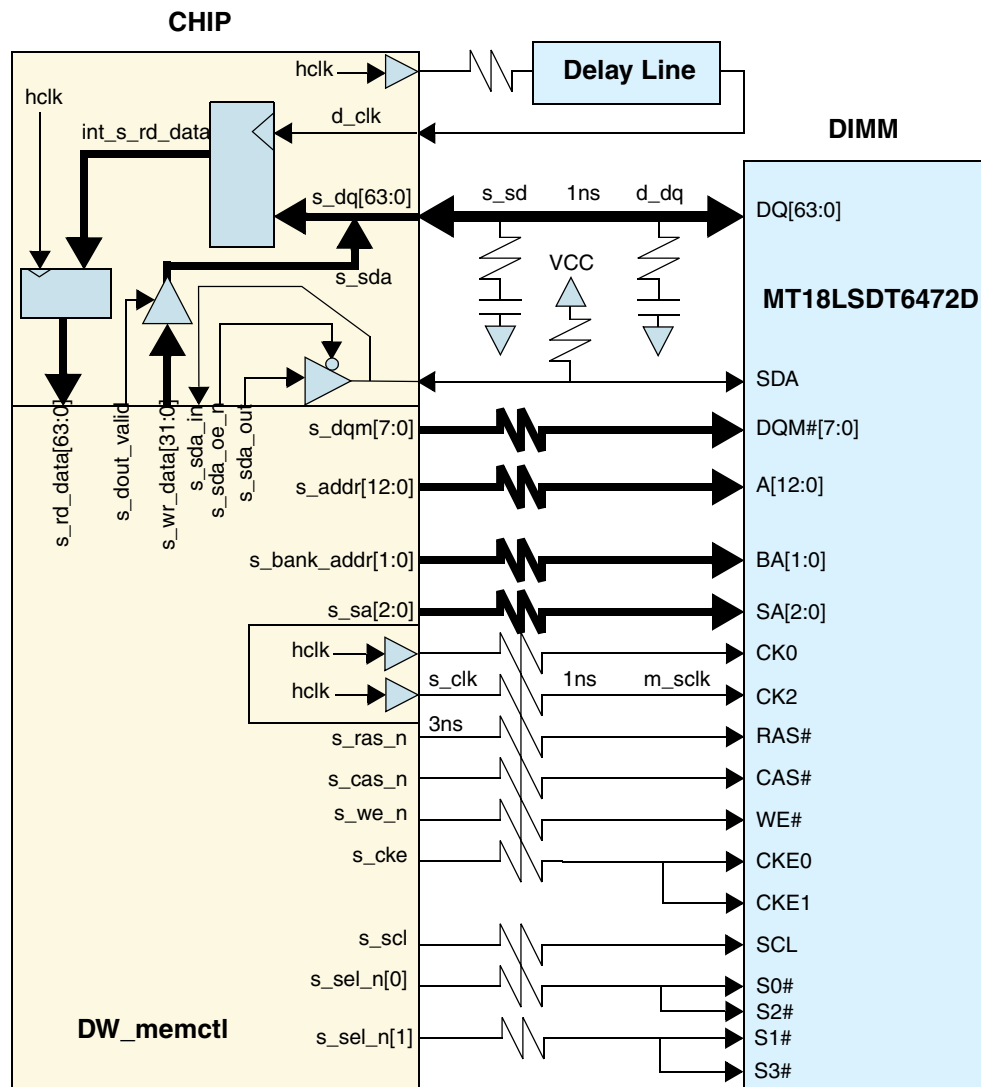


As shown in the illustration, the single chip select from the DW_memctl is connected to both chip selects of the DIMM module.

4.7 Interfacing DW_memctl to Two-Bank DIMMS

Figure 4-43 illustrates the connection interfacing the DW_memctl to a two-bank DIMM, which in this example is the Micron DIMM module MT18LSDT6472D, a 256MB module.

Figure 4-43 DW_memctl Interface to Two-Bank DIMM



In this case, the DW_memctl needs to be configured for two chip selects. If chip selects 0 and 1 are used, then **s_sel_n[0]** should be connected to chip selects **s0#** and **s2#** of the DIMM. The DW_memctl chip select **s_sel_n[1]** should be connected to DIMM chip selects **s1#** and **s3#**.

Additionally, the chip select block size parameters **BLOCK_SIZE0** and **BLOCK_SIZE1** should be specified as half the size of the DIMM - 128MB in this case.

5

DW_memctl as Static Memory Controller Functional Description

This chapter describes the functional details of the DW_memctl as a Static memory controller. Under the Static memory category, the memory controller supports asynchronous SRAMs, asynchronous FLASH memories, synchronous SRAMs (non-ZBT), synchronous FLASH memories, and non-memory devices with ready pin.

**Note**

Unless otherwise noted, “Static memory” refers to SRAMs and NOR-FLASH memories, unless otherwise noted; NAND-FLASH memories are not supported.

5.1 Configuring DW_memctl for Static Memory

The DW_memctl supports Static memories of various data widths. The DW_memctl has three sets of timing registers for controlling the Static memory:

- Static Memory Timing Register – Set 0 (SMTMGR_SET0) (address 32'hxxxx_xx94)
- Static Memory Timing Register – Set 1 (SMTMGR_SET1) (address 32'hxxxx_xx98)
- Static Memory Timing Register – Set 2 (SMTMGR_SET2) (address 32'hxxxx_xx9C)

You can use the following compile-time and programmable parameters when configuring the DW_memctl for Static memories:

- Maximum data width – The MAX_SM_DATA_WIDTH compile-time parameter specifies the width of the Static memory data bus interface for the DW_memctl. This parameter defines the size of the Static memory data bus.

You can choose the AHB data width (H_DATA_WIDTH) to Static Memory data width (SM_DATA_WIDTH_SET0/1/2) ratio as 1:1, 2:1, 4:1, 8:1, or 16:1. The minimum Static Memory data width is 8; supported AHB data widths are 32, 64, and 128.

- Programmable data widths – You can dynamically specify the Static memory data widths by programming the Static Memory Control Register (SMCTLR) – (address 32'hxxxx_xxA4). Bits 9:7 specify the Static memory data bus width of the memory associated with timing register set0; bits 12:10 and 15:13 specify the Static memory data bus width of the memory associated with timing registers set1 and set2, respectively. If you configure the DW_memctl as both a Static memory

controller and an SDRAM controller, then the Static memory data width must be less than or equal to the SDRAM data width. The SM_DATA_WIDTH_SET0, SM_DATA_WIDTH_SET1 and SM_DATA_WIDTH_SET3 compile-time parameters specify the SMCTLR reset values for the Static memory data widths.

- Address width – The MAX_SM_ADDR_WIDTH compile-time parameter specifies the width of the Static memory address bus interface of the DW_memctl, which supports up to 32 bits of address bus for Static memories.
- Programmable read latencies – You can dynamically specify the read latencies – which are required if you put extra flip-flops in the read data path to the Static memory – by programming the Static Memory Timing Registers (SMTMGR_SET0, SMTMGR_SET1 and SMTMGR_SET2). The SM_READ_PIPE_SET0, SM_READ_PIPE_SET1 and SM_READ_PIPE_SET2 compile-time parameters specify the reset values for the Static memory read latencies; values are 0-3.
- Write Latency – The SM_WRITE_PIPE compile-time parameter specifies write latency, which is required if you put extra flip-flops in the write data path to the Static memory; values are 0-1.

5.1.1 Static Memory Chip Selection

The DW_memctl supports up to eight chip selects. If you configure the DW_memctl as only a Static memory controller – by setting DYNAMIC_RAM_TYPE to 6 and ENABLE_STATIC to 1 – then you can use all the chip selects to enable Static memories. The Address Mask Registers (MSKR_n) (addresses 0x54 to 0x70) and the Chip Select Registers (CSLR_n) (addresses 32'hxxxx_xx14 to 32'hxxxx_xx30) control the chip select selection.

Bits 4:0 of the mask register specify the size of the memory assigned to a particular chip select. The DW_memctl supports Static memory sizes from 64KB to 4GB. Bits 6:5 specify the type of memory connected to that particular chip select. Bits 8:7 specify the Static Memory Timing Register set with which this particular memory is associated. There are compile-time parameters that act as default values for these register bits. For more details, refer to the appropriate descriptions in “[Registers](#)” on page 167.

The chip select base address registers hold the base address values that correspond to each chip select. The DW_memctl compares the AHB address with the chip select base address register values in order to generate the chip select.

5.1.2 FLASH Reset/Power-Down Control

When the sm_rp_n pin – connected to a FLASH memory – is driven low, the following happens:

- FLASH internal status registers are cleared
- Many internal circuits are turned off
- Device goes into power-down mode

In this mode, all inputs to the FLASH except sm_rp_n have a value of “Don’t Care,” and all outputs from the FLASH are high-impedance.

During reset, the DW_memctl asserts sm_rp_n, which is de-asserted by the memory controller immediately after reset. After reset, all requests to the FLASH will be queued until the t_rpd timer for the FLASH expires. During normal operation, the FLASH Timing Register (FLASH_TRPDR) (address 32'hxxxx_xxA0), the sm_power_down pin, and the Static Memory Control Register (SMCTLR) enable you to control the reset/power-down mode of the FLASH.

Even though the DW_memctl can support up to three different FLASH memories with different timing parameters, there is only one register for specifying tRPD.

There are two ways to control the reset/power-down mode of a FLASH memory:

- Program bit 0 of Static Memory Control Register (address 32'hxxx_xxA4) – A 0 commands the DW_memctl to put the FLASH in reset/power-down mode; a 1 commands the DW_memctl to take the FLASH out of reset/power-down mode.
- Use sm_power_down input pin – FLASH will be in reset/ power-down mode as soon as this signal stays high.

5.1.3 FLASH Memory Write Protection

Some FLASH memories have a write protection pin that protects important system information in the boot block. For the DW_memctl, you can control this pin by programming the Static Memory Control Register (SMCTLR) (address 0x84). Writing 0 to bits 3:1 of SMCTLR forces a 0 on the WP pin of the FLASH memory.

The FLASH memory write protect pin is a 1-bit input. For DW_memctl, SMCTLR[1] controls the write protect pin of the FLASH memory controlled by Register Set0, SMCTLR[2] for Register Set1, and SMCTLR[3] for Register Set2.

5.1.4 FLASH Memory Status Information

Some FLASH memories have a status pin that indicates the status of the internal state machine of the FLASH memory. The DW_memctl does not have dedicated pins for the status inputs from the FLASH memories; however you can connect the FLASH status pins to the General Purpose Input (GPI) pins of the DW_memctl and get the status information by reading corresponding bits in the SDRAM Refresh Interval Register (SREFR) (address 0x10). You do not always have to sample the status pin to get a FLASH device status; instead you can directly read the Status registers of a FLASH device.

FLASH Status pins are single-bit outputs from a FLASH. If you use three different FLASH memories controlled by register sets 0, 1, and 2, then you can connect the Status pins of the three FLASH memories to GPI [2:0].

5.1.5 Static Memory Timing Diagrams

[Table 5-1](#) lists the timing diagrams for read/write operations to Static memories.

Table 5-1 Static Memory Read/Write Timing Diagrams

Timing Diagram	Description
Figure 5-1 on page 104	Read timing of SRAM and FLASH
Figure 5-2 on page 105	Page read timing of FLASH
Figure 5-3 on page 106	Write timing of SRAM and FLASH
Figure 5-4 on page 107	External memory data bus turnaround timing
Figure 5-5 on page 108	First read after reset/power-down

**Note**

All of these timing diagrams assume an internal delay of two AHB clock cycles, which is the delay for an AHB cycle to be active on the AHB bus to the clock cycle where the corresponding memory command is seen on the memory bus. If you also set the PIPE_ADDRESS_DECODER parameter to 1, which controls the pipelining of address decoder outputs, then there will be an extra clock delay.

Figure 5-1 illustrates the timing for a read operation of an SRAM and FLASH, where:

- tRC – read cycle time.

Figure 5-1 Read Timing of SRAM and FLASH

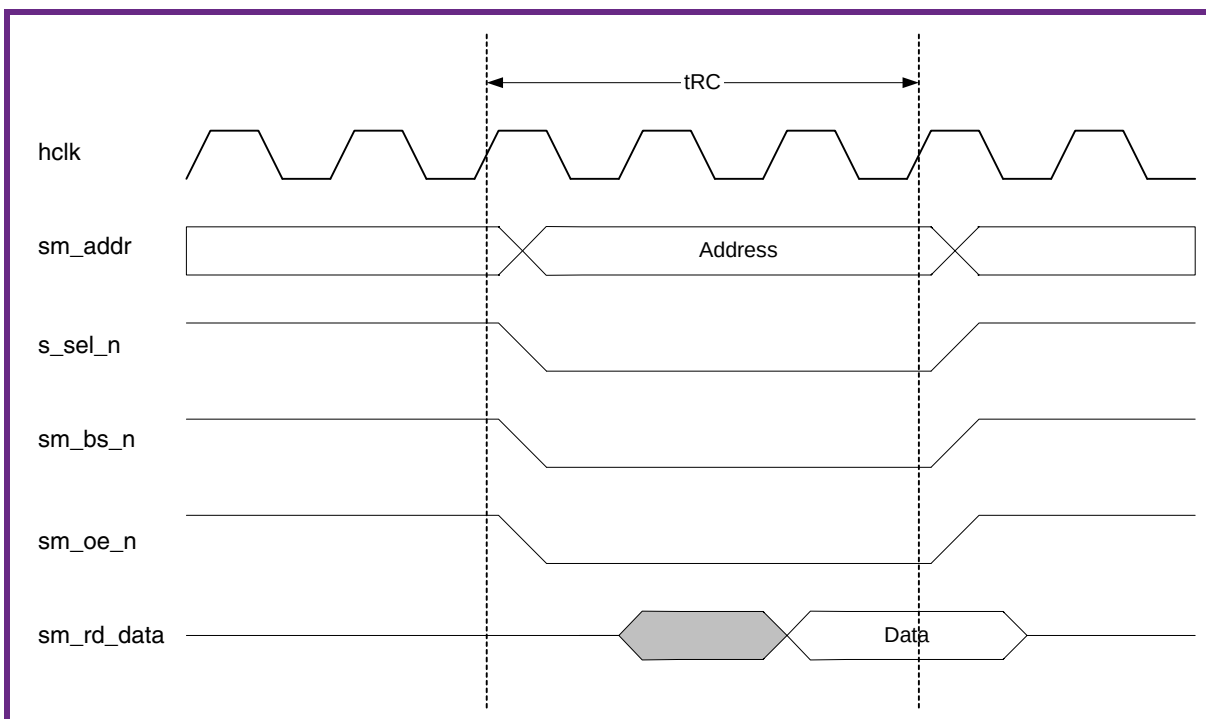


Figure 5-2 illustrates the timing for a page read operation of a FLASH, where:

- tRC – read cycle time.
- tPRC – page mode read cycle time

Figure 5-2 Page Read Timing of FLASH

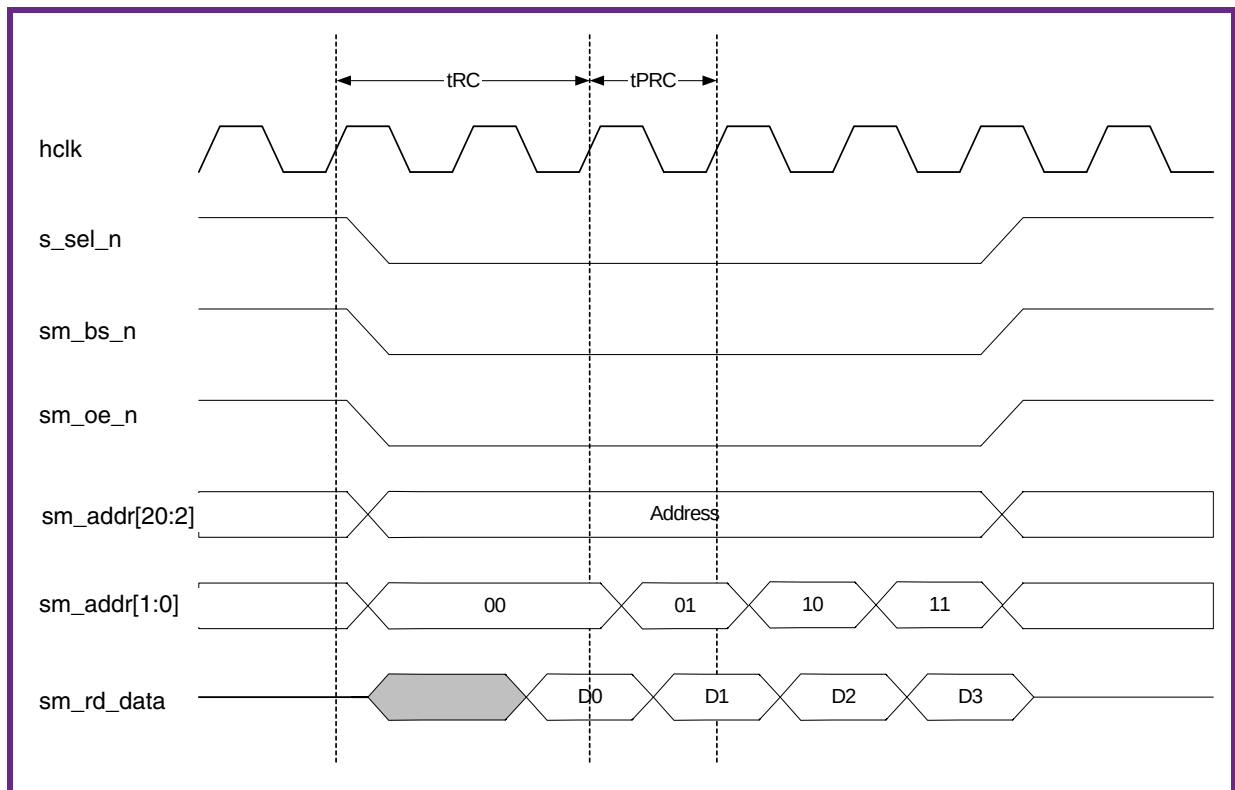


Figure 5-3 illustrates the timing for a write operation of an SRAM and FLASH, where:

- t_{AS} – address setup time
- t_{WP} – write pulse period
- t_{WR} – write recovery time

Figure 5-3 Write Timing of SRAM and FLASH

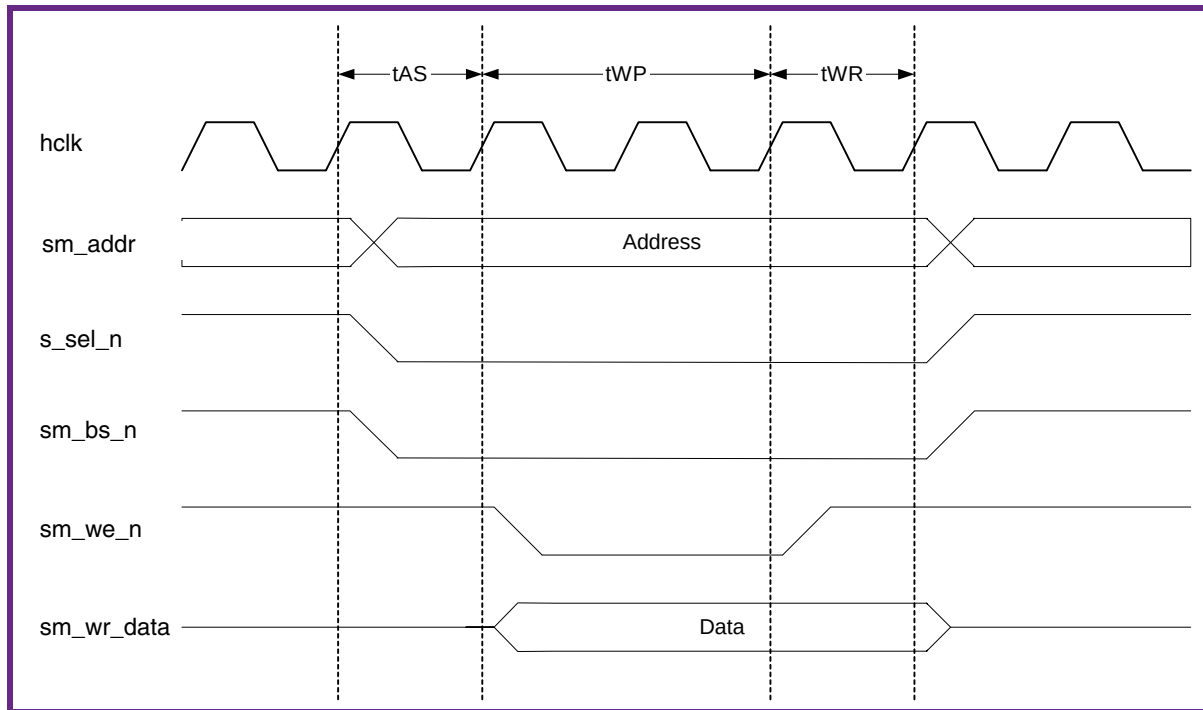


Figure 5-4 illustrates an example of inserting one idle clock cycle for memory data bus turnaround time, where:

- tBTA – number of idle clock cycles for the external memory data bus turnaround time

Figure 5-4 External Memory Data Bus Turnaround Timing

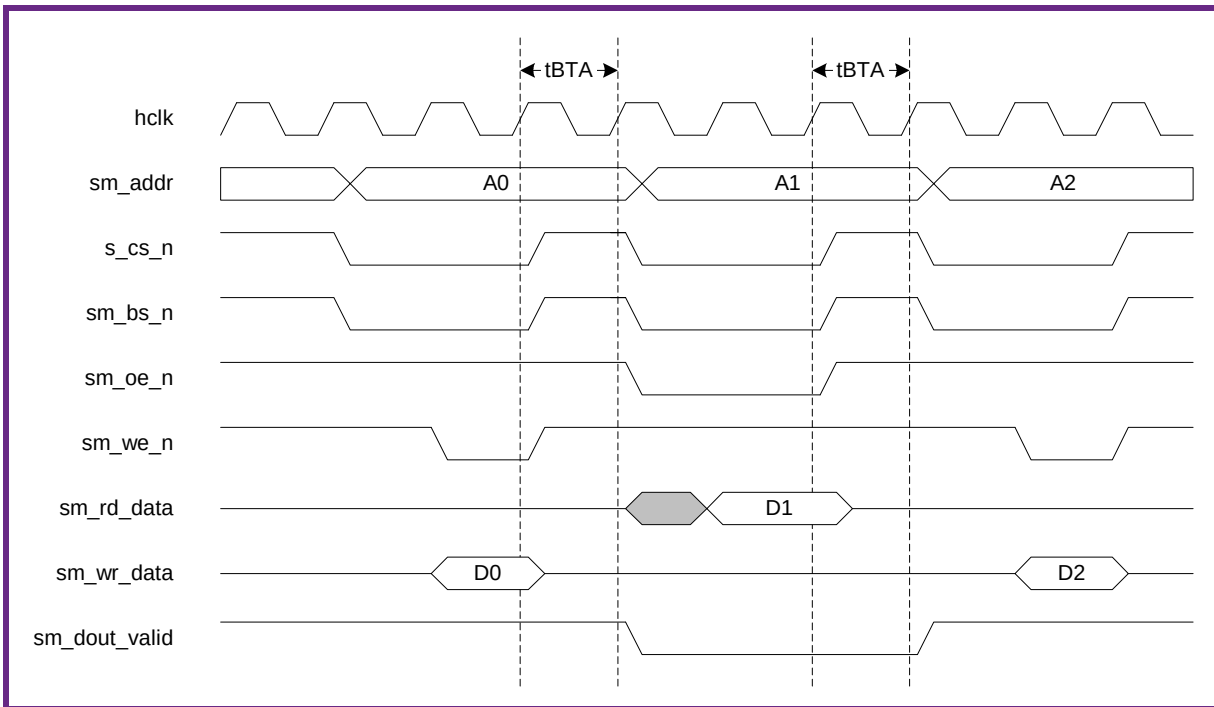
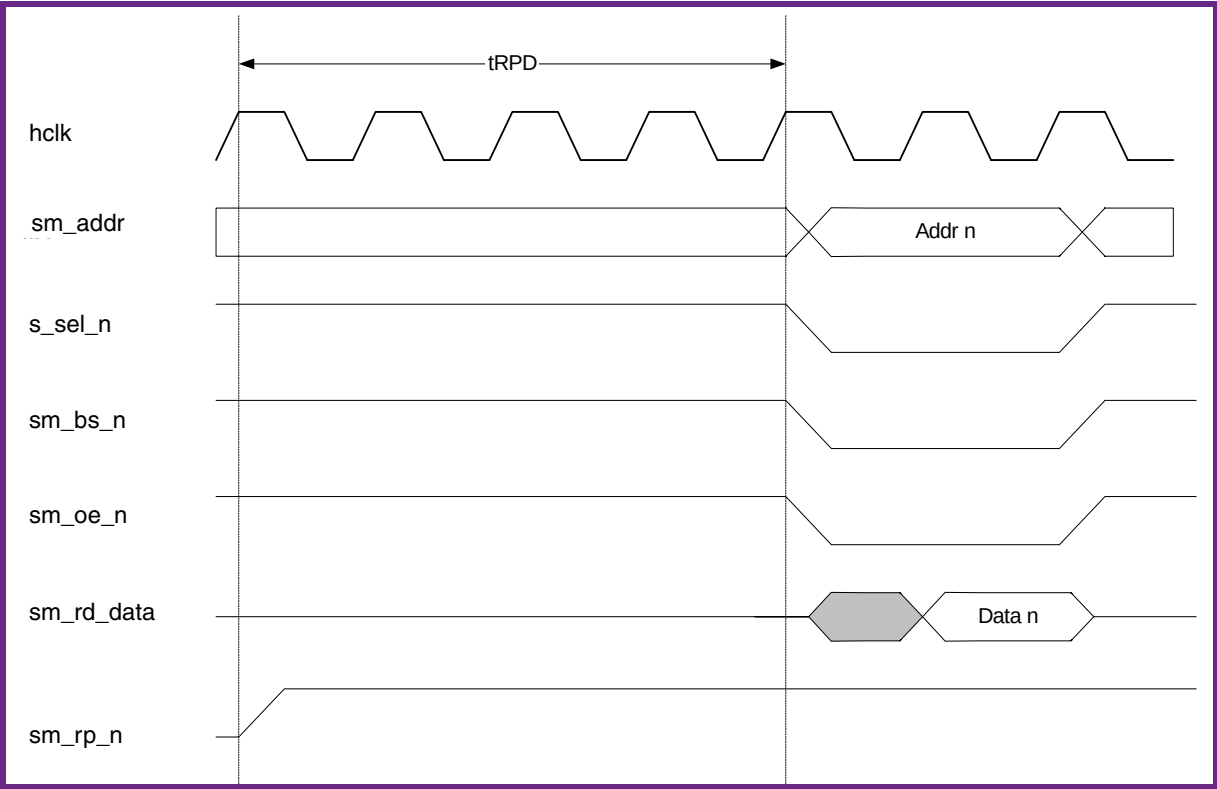


Figure 5-5 illustrates how, after exiting reset/power-down mode, certain delays (tRPD) should be met before any read/write accesses.

Figure 5-5 First Read After Reset/Power-Down



5.2 Address Multiplexing

DW_memctl performs address multiplexing in order to convert the address generated by the host interface (AHB HADDR) to the static memory address (sm_addr). The sm_addr signal is dependent on the sm_data_width to which the memory has been programmed. The general rule for sm_addr generation is detailed in Table 5-2.

Table 5-2 Host Address Generation

Memory Data Width	sm_addr
8	haddr[MAX_SM_ADDR_WIDTH – 1:0]
16	haddr[MAX_SM_ADDR_WIDTH – 1:1]
32	haddr[MAX_SM_ADDR_WIDTH – 1:2]
64	haddr[MAX_SM_ADDR_WIDTH – 1:3]
128	haddr[MAX_SM_ADDR_WIDTH – 1:4]

5.3 Interfacing with Asynchronous SRAMs

The DW_memctl supports asynchronous SRAMs.

5.3.1 Specifying the SRAM Timing Parameters

You can specify SRAM timing parameters by programming the Static Memory Timing Register - Set 0 (SMTMGR_SET0), Static Memory Timing Register - Set 1 (SMTMGR_SET1), or Static Memory Timing Register - Set 2 (SMTMGR_SET2), depending on which one of the three register sets should control the SRAM.

You can use the following SRAM timing parameters:

- Read cycle time (tRC) – Bits 5:0 of SMTMGR_SET0/1/2 specify the read cycle time; use the T_RC_SET0/T_RC_SET1/T_RC_SET2 compile-time parameters to specify default values.
- Address setup time (tAS) – Bits 7:6 of the Static Memory Timing Register SMTMGR_SET0/1/2; use the T_AS_SET0/T_AS_SET1/T_AS_SET2 compile-time parameters to specify default values.
- Write pulse width (tWP) – Bits 15:10 of SMTMGR_SET0/1/2; use the T_WP_SET0/T_WP_SET1/T_WP_SET2 compile-time parameters to specify default values.
- Write recovery time (tWR) – Bits 9:8 of the Static Memory Timing Register SMTMGR_SET0/1/2; use the T_WR_SET0/T_WR_SET1/T_WR_SET2 compile-time parameters to specify default values.
- Bus turnaround time (tBTA) – Bits 18:16 of SMTMGR_SET0/1/2; use the T_BTA_SET0/T_BTA_SET1/T_BTA_SET2 compile-time parameters to specify default values.

This parameter forces the DW_memctl to insert *tBTA* number of cycles between back-to-back read/ writes. If the address setup time is significant, tBTA is not necessary.

5.3.2 I/O Interface Between SRAM and DW_memctl

The following are conditions for interfacing SRAM pins to DW_memctl pins:

- Address pins – Connect the address pins to the DW_memctl sm_addr pins.
- Chip select pin – Connect the chip select pin to one of the DW_memctl chip select pins, s_sel_n. Specify which chip select is connected to the SRAM in the memory type bits (bits 6:5) of the mask register (SMSKR0-7) that correspond to a particular chip select.
- Output enable pin – Connect the output-enable pin of the SRAM to the DW_memctl sm_oe_n pin.
- Write enable pin – Connect the write-enable pin of the SRAM to the DW_memctl sm_we_n pin.
- Byte control pins – Connect the byte enable pins to the DW_memctl sm_bs_n[SM_DATA_WIDTH/8-1:0] pins. If the SRAM has LB (lower byte enable) and UB (upper byte enable), the LB should be connected to sm_bs_n[0] and UB should be connected to sm_bs_n[1].

- Data inputs/outputs – Drive the bidirectional pin of the SRAM using the following DW_memctl pins:
 - Write data output – sm_wr_data
 - Read data input – sm_rd_data
 - Output data valid – sm_dout_valid

Use the data out valid signal as the enable signal for the tri-state that drives the data.

5.3.3 Using CellularRAM in Asynchronous Burst Mode

DW_memctl supports CellularRAM when used in CellularRAM's asynchronous operating mode (the default). CellularRAM asynchronous page mode reads should also function correctly, however, have not been validated with this release of DW_memctl due to model limitations. Verification of the CellularRAM page mode operational mode is the responsibility of the end user.

DW_memctl does not support CellularRAM's synchronous burst mode or mixed-mode operation. Burst mode and mixed-mode both require use of a WAIT signal, which is the equivalent of the sm_ready signal on DW_memctl. DW_memctl supports use of the sm_ready signal only for asynchronous transfers.

5.4 Interfacing with Asynchronous FLASH Memories

The DW_memctl supports asynchronous FLASH memories.

5.4.1 Specifying FLASH Memory Timing Parameters

You can specify FLASH timing parameters by programming the Static Memory Timing Register - Set 0 (SMTMGR_SET0), Static Memory Timing Register - Set 1 (SMTMGR_SET1), or Static Memory Timing Register - Set 2 (SMTMGR_SET2), depending on which of the three register sets should control the FLASH.

You can use the following FLASH timing parameters:

- Read/write cycle time (tRC) – Bits 5:0 of SMTMGR_SET0/1/2 specify the read/write cycle time; use the T_RC_SET0/T_RC_SET1/T_RC_SET2 compile-time parameters to specify default values.

Equivalent parameters:

- INTEL-FLASH – t_AVAV
- AMD-FLASH – t_RC

- Page read-cycle time (tPRC) – Bits 22:19 of the Static Memory Timing Register SMTMGR_SET0/1/2 specify the page read-cycle time; use the T_PRC_SET0/T_AS_SET1/T_PRC_SET2 compile-time parameter to specify default values.

Equivalent parameters:

- INTEL-FLASH – t_APA
- AMD-FLASH – t_PACC

- Address setup time (tAS) – Bits 7:6 of the Static Memory Timing Register SMTMGR_SET0/1/2 specify the address setup time; use the T_AS_SET0/T_AS_SET1/T_AS_SET2 compile-time parameters to specify default values.
Equivalent parameters:
 - INTEL-FLASH – t_ELWL
 - AMD-FLASH – max(t_AS,t_CS)
- Write pulse width (tWP) – Bits 15:10 of the Static Memory Timing Register SMTMGR_SET0/1/2 specify the write pulse width; use the T_WP_SET0/T_WP_SET1/T_WP_SET2 compile-time parameters to specify default values.
Equivalent parameters:
 - INTEL-FLASH – t_WLWH/t_WP
 - AMD-FLASH – t_WLWH
- Write recovery time (tWR) – Bits 9:8 of the Static Memory Timing Register SMTMGR_SET0/1/2 specify the address hold time when write enable goes high; use the T_WR_SET0/T_WR_SET1/T_WR_SET2 compile-time parameters to specify default values.
Equivalent parameters:
 - For Intel FLASH – tWHAX
 - Micron FLASH – tAH
 - INTEL-FLASH – max(t_WHAX,t_WHDX)
 - AMD-FLASH – t_DH
- Bus turnaround time (tBTA) – Bits 18:16 of SMTMGR_SET0/1/2 force the DW_memctl to insert tBTA number of cycles between back-to-back read/writes; use the T_BTA_SET0/T_BTA_SET1/T_BTA_SET2 compile-time parameters to specify default values.
If the address setup time is significant, the tBTA is not necessary.
- rp pin high to read/write delay (T_RPD) – Bits 11:0 of the FLASH_TRPD register (address 0x7C) specify RP high-to-output delay; use the FLASH_TRPD compile-time parameter to specify a default value.

5.4.2 I/O Interface Between FLASH Memory and DW_memctl

The following are conditions for interfacing FLASH pins to DW_memctl pins:

- FLASH address pins – Connect the address pins to the DW_memctl sm_addr pins
- Chip select pin – Connect the chip select pin to the DW_memctl chip select pin, s_sel_n. Specify which chip select is connected to the FLASH in the memory type bits (bits 6:5) of the mask register (SMSKR1-8) that correspond to a particular chip select
- Output enable pin – Connect the output enable pin to the DW_memctl sm_oe_n pin
- Write enable pin – Connect the write enable pin to the DW_memctl sm_we_n pin.
- Write protect pin – Connect the write protect pin to the DW_memctl sm_wp_n pin

- Reset power-down pin – Connect the reset power-down pin to the DW_memctl sm_rp_n pin
- Status pin – Connect the status pins to the DW_memctl GPI pins

5.5 Interfacing to Synchronous SRAMs

Although as a Static memory controller, the DW_memctl is designed mainly used for interfacing to external asynchronous Static memories, you can still interface on-chip and non-ZBT-type off-chip Synchronous SRAMs (SSRAMs) to it. This interface is not part of the automated DWMM configuration and testbench flow.

The external asynchronous memory access normally consists of multiple clock cycles. Additionally, back-to-back asynchronous SRAM writes need at least a dead cycle between writes in order to generate the write pulse. Also, in order to support AHB early-burst termination, the minimum cycle time for an asynchronous SRAM write is 2 – a minimum of one clock address setup time and one clock write-pulse width.

On-chip SSRAMs do not have these limitations. In a synchronous SRAM, you can do a write access every clock, and you can also use combination logic to terminate a write cycle (on seeing the AHB terminate), since the write signal is not sampled by SSRAMs until the rising edge of the clock.

The pipeline feature provided by the SM_READ_PIPE_SET0/1/2 parameters enables you to efficiently interface the DW_memctl to synchronous SRAMs; this guarantees that during a read, there are no wait states within a read burst.

5.5.1 Interfacing to an On-Chip SSRAM

Through the coreAssembler GUI, you should set the address setup time (T_{AS_SETn}), write recovery (T_{WR_SETn}), and data bus turnaround (T_{BTA_SETn}) to “0,” while you set the write pulse width (T_{WP_SETn}) and read cycle time (T_{RC_SETn}) to “1.”

Since the Static memory read data is registered inside the DW_memctl before being sent onto the AHB bus, you can directly connect the SSRAM read data bus to the DW_memctl sm_rd_data signal without any registers, provided it can meet an approximate 40% input setup time requirement. This means that there is only one pipe in the SRAM read data path – one due to the SSRAM – so you must set the SM_READ_PIPE_SET n parameter to “1;” you need to change only the timing register set with which you map your on-chip SSRAM.

Figure 5-6 illustrates the memory instantiation for this scenario.

Figure 5-6 SSRAM Interface

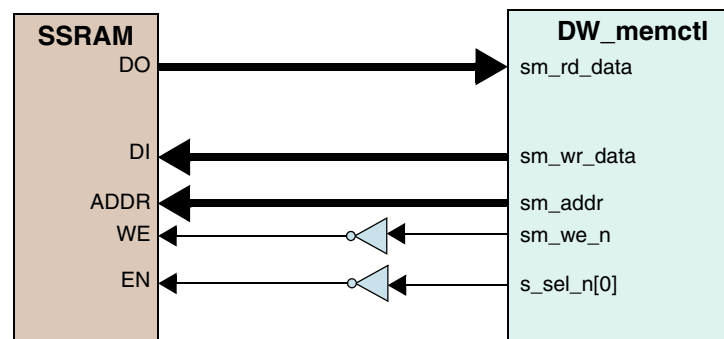
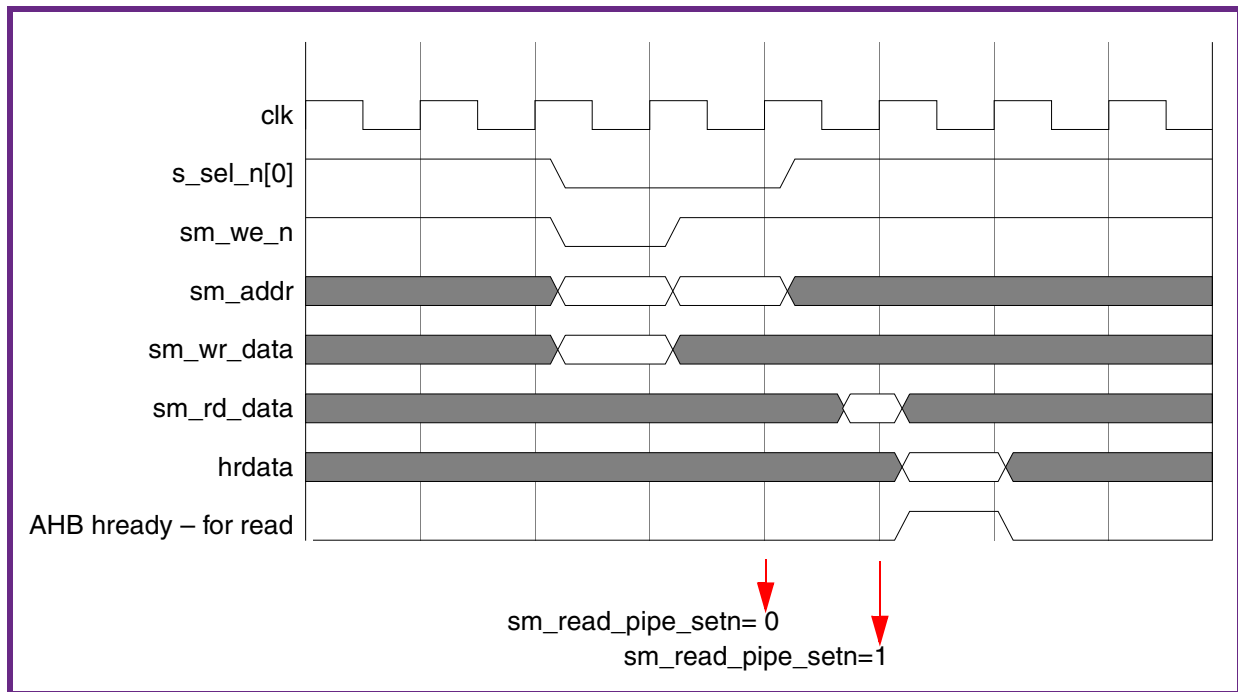


Figure 5-7 illustrates the waveforms for this scenario.

Figure 5-7 Write Followed by Read Cycle; SM_READ_PIPE_SETN = 1



If you want to simulate your SSRAM in the Synopsys-supplied test environment, then you should edit the sim/testbench/mem_banks_auto.v file to instantiate the memory models.

The following sample code shows how you can interface the Xilinx Block SSRAM; note that this example does not cover byte-writes. In an actual system, you would need to instantiate four byte-size SSRAMs; the write signal can be generated by ORing the sm_we_n and sm_bs_n[x] signals together.

```

wire [1:0] s_sel_n;
wire [31:0] sm_rd_data;
wire [31:0] sm_wr_data;
wire [14:0] sm_addr;

DW_memctl U_memctl (
  .hclk (clk),
  .hresetn (rst_n),
  ....
  ....
  .s_sel_n (s_sel_n),
  .sm_rd_data (sm_rd_data),
  .sm_wr_data (sm_wr_data),
  .sm_we_n (sm_we_n),
);

// 1024 x 32 Block SSRAM
RAMB4_S32 U_ssram(
  .ADDR (sm_addr[9:0]),
  .DI (sm_rd_data),
  .DO (sm_wr_data),
  .CLK (clk)

```

```

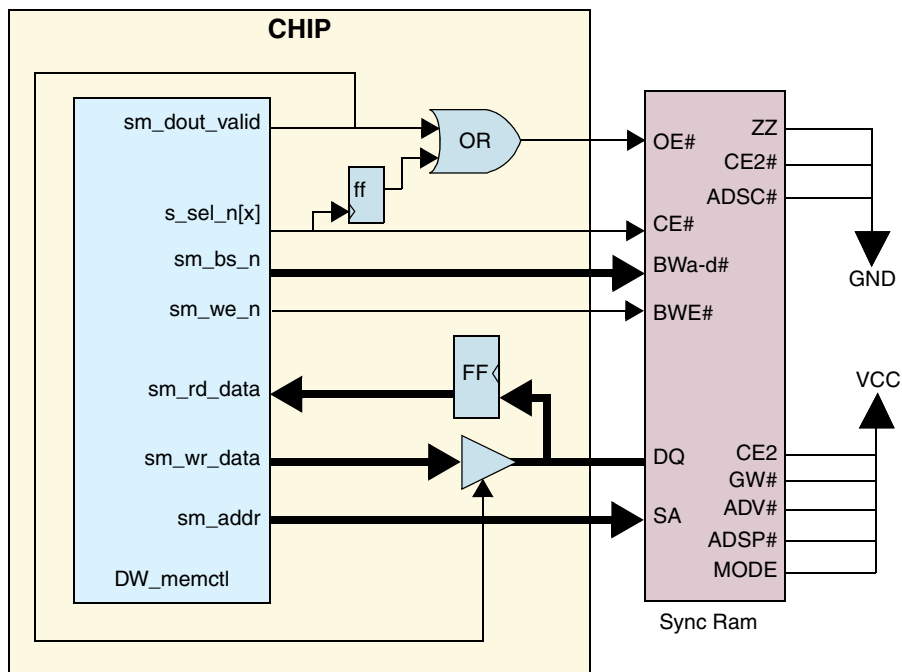
.WE (~sm_we_n),
.EN (~s_sel_n[0]),
.RST (1'b0)
);

```

5.5.2 Interfacing to an Off-Chip SSRAM

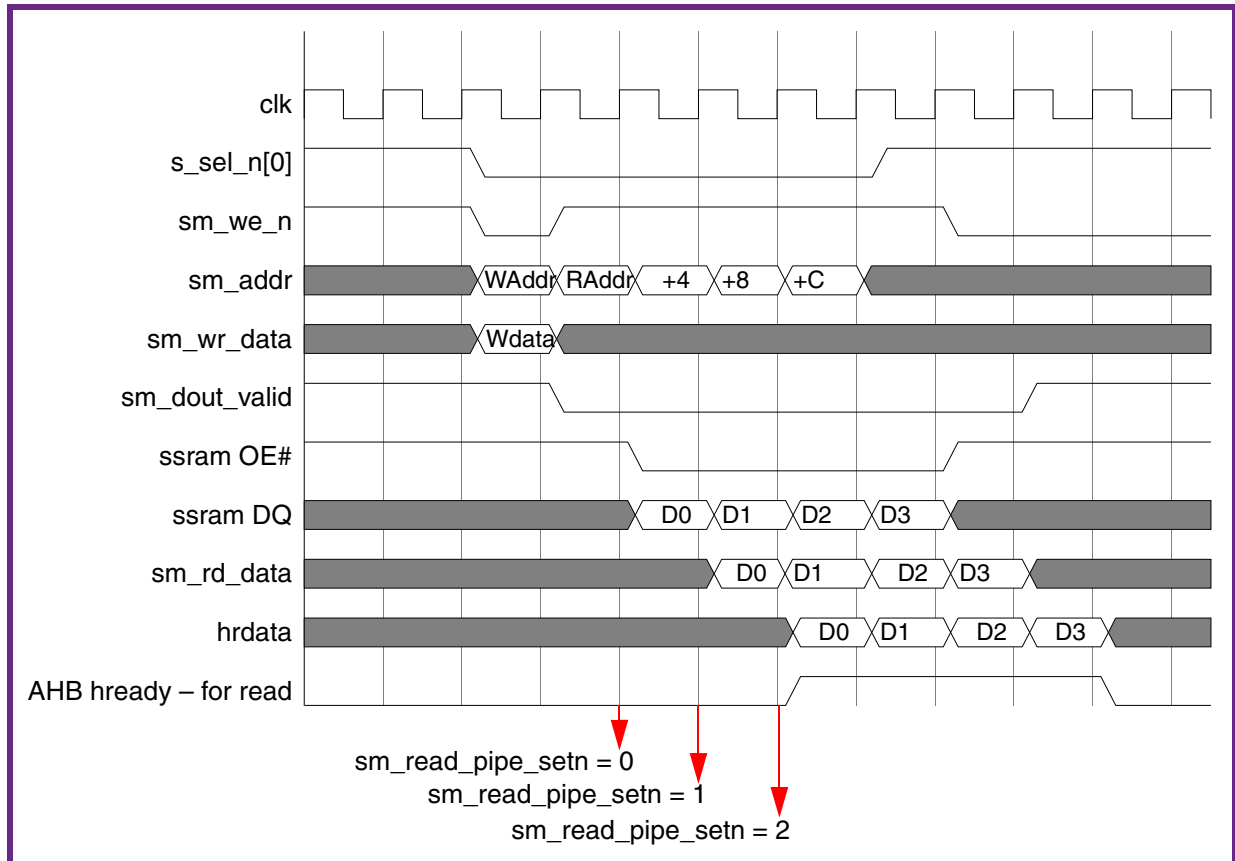
You can interface an off-chip SSRAM to the DW_memctl. [Figure 5-8](#) illustrates how to interface the DW_memctl to a flow-through SyncBurst SRAM.

Figure 5-8 Interfacing DW_memctl to Flow-Through SyncBurst SRAM



As in the on-chip example, set T_{AS_SETn} , T_{WR_SETn} , and T_{BTA_SETn} to "0." Set T_{WP_SETn} and T_{RC_SETn} to "1." To meet timing, you may need to add a register in the sm_rd_data , as shown in [Figure 5-9](#), so you should change the $SM_READ_PIPE_SETn$ to "2" - one due to flow-through SDRAM and one more due to the register in the read path.

Figure 5-9 Write Followed by INC-4 Read Burst Cycle; $SM_READ_PIPE_SETn = 2$



5.6 Interfacing with Synchronous FLASH Memories

The DW_memctl supports synchronous FLASH memories. Most synchronous FLASH memories support both synchronous read mode and asynchronous read mode. The DW_memctl supports both read modes. For write access, since most synchronous FLASH memories support only asynchronous writes, the DW_memctl supports only asynchronous writes.

Simulation of Synchronous FLASH is not supported through the supplied \$SYNOPSIS test environment.

5.6.1 I/O Interface Between Synchronous FLASH Memory and DW_memctl

Figure 5-10 shows how to interface the DW_memctl to a synchronous FLASH when the FLASH operates in asynchronous-read mode. For the asynchronous read/write timing diagrams and timing parameter setting, refer to “Interfacing with Asynchronous FLASH Memories” on page 110.

Figure 5-10 Interfacing DW_memctl to Synchronous FLASH in Asynchronous Read Mode

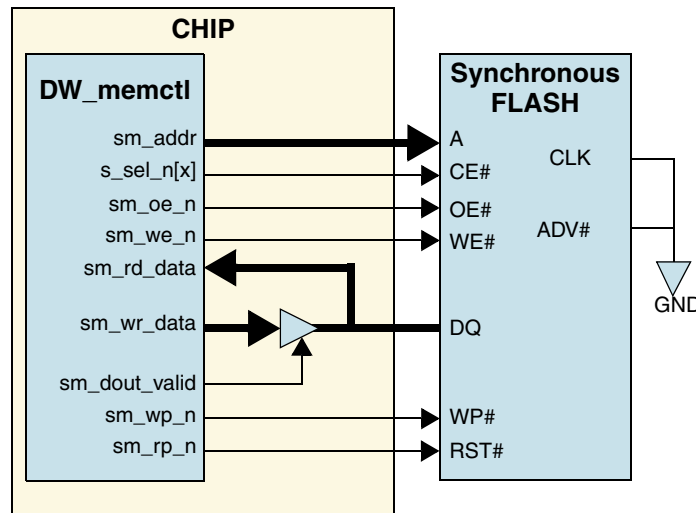
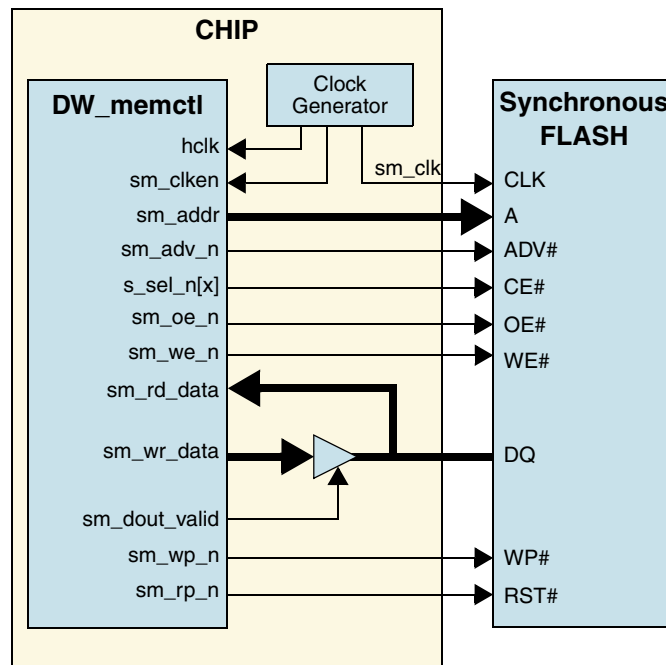


Figure 5-11 shows how to interface the DW_memctl to a synchronous FLASH when the FLASH operates in synchronous-read mode.

Figure 5-11 Interfacing DW_memctl to Synchronous FLASH in Synchronous Read Mode



Most synchronous FLASH memories support both synchronous read mode, as well as asynchronous read mode. If you use only the asynchronous read mode, you should directly connect the FLASH ADV# pin to logic low. When you want to use the synchronous read mode, you should connect the FLASH ADV# pin to the DW_memctl sm_adv_n pin. When the sm_adv_n pin is connected to the FLASH ADV# pin, the DW_memctl:

- Can support synchronous single read
- Can support synchronous page read
- Can support asynchronous single read
- Might not support asynchronous page read operation (if the device ADV# pin is required to be kept low during the entire asynchronous page read process)

The power-on default read mode of most synchronous FLASH memories is asynchronous read mode. Use the following steps to change the asynchronous read mode to synchronous read mode:

1. Change the FLASH read mode to synchronous mode by writing to the FLASH internal configuration register
2. Set the page_mode bit in the Static Timing Register Set0/1/2 (SMTMGR_SET0/1/2) to enable the burst read (the power-on default mode should be non-page/burst mode)

5.6.2 Timing Diagrams of Synchronous Reads

The synchronous FLASH clock frequency can be a sub-multiple of the AHB clock frequency – that is, 1, 1/2, 1/3, 1/4, and so on. When the synchronous FLASH clock frequency is lower than the AHB clock frequency, the user needs to set the `low_freq_sync_device` bit `SMTMGR_SETn[27]` in `SMTMGR_SETn` to “1.”

Figure 5-12 shows the timing diagram of a synchronous FLASH burst read, where the synchronous FLASH clock frequency is the same as the AHB clock frequency. When the synchronous FLASH clock frequency is the same as the AHB clock frequency, you can either set the `low_freq_sync_device` (bit 27 in `SMTMGR_SETn`) to 0 or set the bit to 1 and connect `sm_clken` to logic 1. After `tRC` (read initial access time), the first word `D0` is available at the rising edge of clock 5; the subsequent words are available one per clock.

Figure 5-12 Synchronous FLASH Burst Read

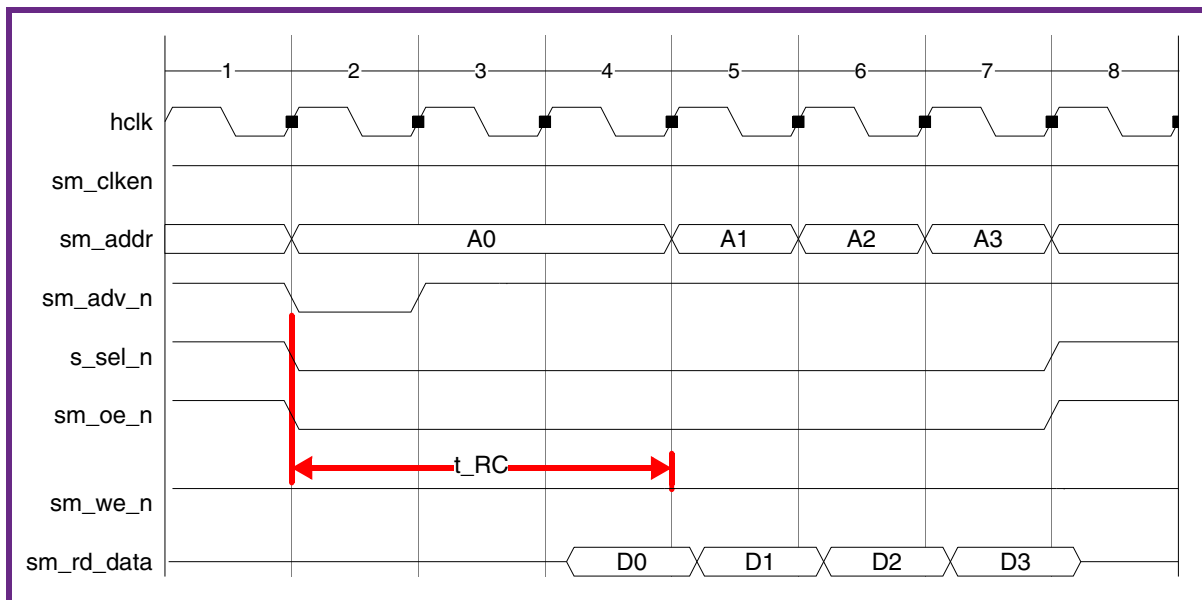
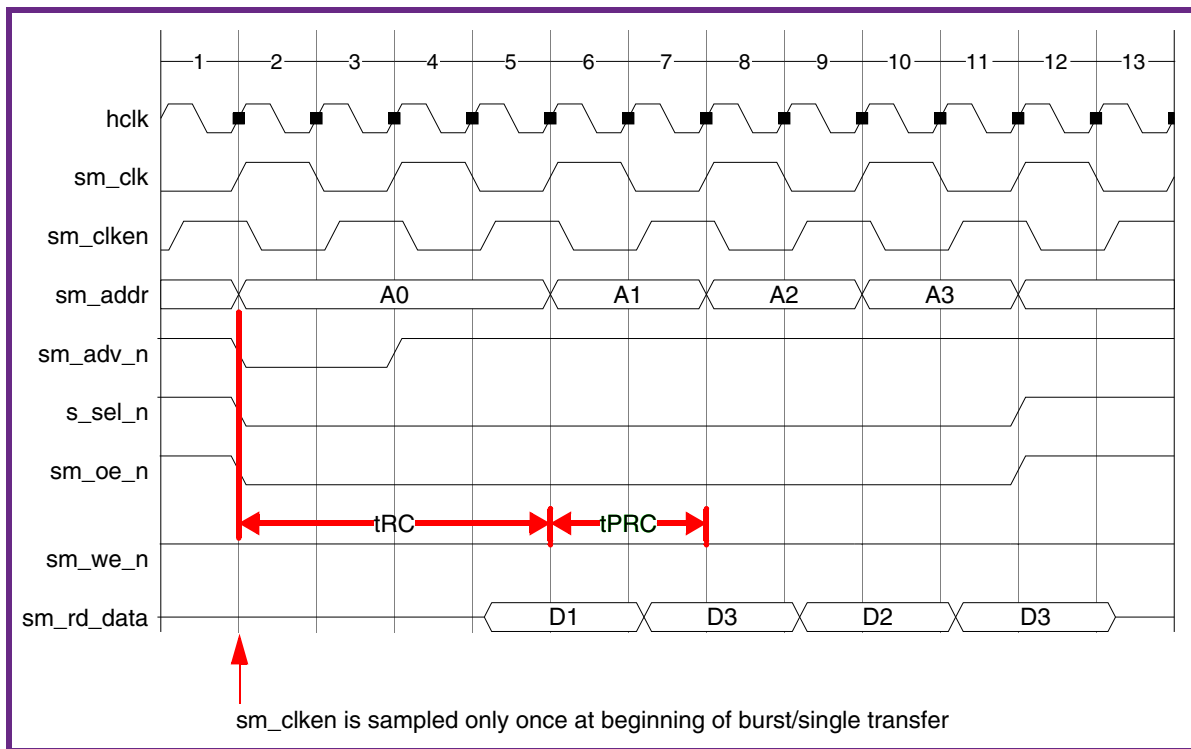


Figure 5-13 shows the timing diagram of a synchronous FLASH burst read, where the synchronous FLASH clock frequency is half of the AHB clock frequency. The DW_memctl always works on the AHB clock.

Figure 5-13 Low-Frequency Synchronous FLASH Burst Read



The user needs to provide the clock (sm_clk) to the synchronous FLASH and provide the appropriate clock enable signal (sm_clken) to the DW_memctl. A high level on sm_clken at the rising edge of hclk indicates a rising edge of the sm_clk. Before the DW_memctl starts a read burst, it waits for a high level on sm_clken to ensure that control signals to the FLASH are aligned with the sm_clk. Once the burst has started, the DW_memctl relies on the timing parameters in SMTMGR_SET0/1/2 to align the control signals to the FLASH. Note that the memory clock (sm_clk) should be a free-running sub-multiple AHB clock.

For information on how to specify these timing parameters, refer to [“Specifying Synchronous FLASH Memory Timing Parameters”](#) on page 119.

5.6.3 Specifying Synchronous FLASH Memory Timing Parameters

You can specify synchronous FLASH timing parameters by programming the Static Memory Timing Register - Set 0 (SMTMGR_SET0), Static Memory Timing Register - Set 1 (SMTMGR_SET1), or Static Memory Timing Register - Set 2 (SMTMGR_SET2), depending on which of the three register sets should control the synchronous FLASH.

You can use the following synchronous FLASH timing parameters:

- Read access time (tRC) – Bits 5:0 of SMTMGR_SET0/1/2 specify the initial read data time in AHB clock cycles. The value of the tRC should be multiples of the AHB clock frequency to FLASH clock frequency ratio.

The T_RC_SET0/T_RC_SET1/T_RC_SET2 compile-time parameters specify the default value of tRC.

Equivalent parameters:

- INTEL-FLASH – t_AVQV
- AMD-FLASH – t_IACC

- Burst read-cycle time (tPRC) – Bits 22:19 of the Static Memory Timing Register SMTMGR_SET0/1/2 specify the burst read-cycle time, which is equal to the AHB clock frequency to FLASH clock frequency ratio – for example, 1 in [Figure 5-11](#) on page 116 and 2 in [Figure 5-12](#) on page 118).

The T_PRC_SET0/T_AS_SET1/T_PRC_SET2 compile-time parameters specify the default value of tPRC.

- Write address setup time (tAS) – Bits 7:6 of the Static Memory Timing Register SMTMGR_SET0/1/2 specify the write address setup time in AHB clock cycles.

The T_AS_SET0/T_AS_SET1/T_AS_SET2 compile-time parameters specify the default value of tAS.

Equivalent parameters:

- INTEL-FLASH – t_ELWL
- AMD-FLASH – max(t_AS, t_CS)

- Write pulse width (tWP) – Bits 15:10 of the Static Memory Timing Register SMTMGR_SET0/1/2 specify the write pulse width in AHB clock cycles.

The T_WP_SET0/T_WP_SET1/T_WP_SET2 compile-time parameters specify the default value of tWP.

Equivalent parameters:

- INTEL-FLASH – t_WLWH/t_WP
- AMD-FLASH – t_WLWH

- Write recovery time (tWR) – Bits 9:8 of the Static Memory Timing Register SMTMGR_SET0/1/2 specify the address hold time (in AHB clock cycles) when the write enable goes high.

The T_WR_SET0/T_WR_SET1/T_WR_SET2 compile-time parameters specify the default value of tWR.

Equivalent parameters:

- For Intel FLASH – tWHAX
- Micron FLASH – tAH
- INTEL-FASLH – max(t_WHAX, t_WHDX)
- AMD-FLASH – t_DH

- Bus turnaround time (tBTA) – Bits 18:16 of SMTMGR_SET0/1/2 force the DW_memctl to insert tBTA number of cycles between back-to-back read/writes.

The T_BTA_SET0/T_BTA_SET1/T_BTA_SET2 compile-time parameters specify the default value of tBTA. If the address setup time is significant, then tBTA is not necessary.

- rp pin high to read/write delay (T_RPD) – Bits 11:0 of the FLASH_TRPD register (address 0x7C) specify RP High-to-output delay in AHB clock cycles.

The FLASH_TRPD compile-time parameter specifies the default value of T_RPD.

5.7 Interfacing with Non-Memory Devices with Ready Pin

The DW_memctl supports non-memory devices with a ready pin. This type of device has the same interface as the asynchronous SRAM, except that it has a ready pin to indicate that the read data is available on the data bus or that the write data is accepted by the device.

For information on how to connect the device to the DW_memctl, refer to the [“Interfacing with Asynchronous SRAMs”](#) on page 109. The ready pin of the device should be connected to the DW_memctl sm_ready pin. The ready_mode bit of the Static Timing Register (SMTMGR_SET0/1/2 – bit 26) should be set to 1.

The DW_memctl converts all AHB bursts to single read/write accesses on the device side.

5.7.1 Timing Diagrams of Read/Write Accesses

Figure 5-14 shows the timing diagram of a read access. The DW_memctl checks sm_ready after the tRC read access time. You need to ensure that the sm_ready signal is being driven with respect to the AHB clock hclk. This avoids a possible race condition if sm_ready is driven by a different clock.

When sm_ready is high, the DW_memctl latches read data at the next rising clock edge.

Figure 5-14 Read Access of the Device with Ready Signal

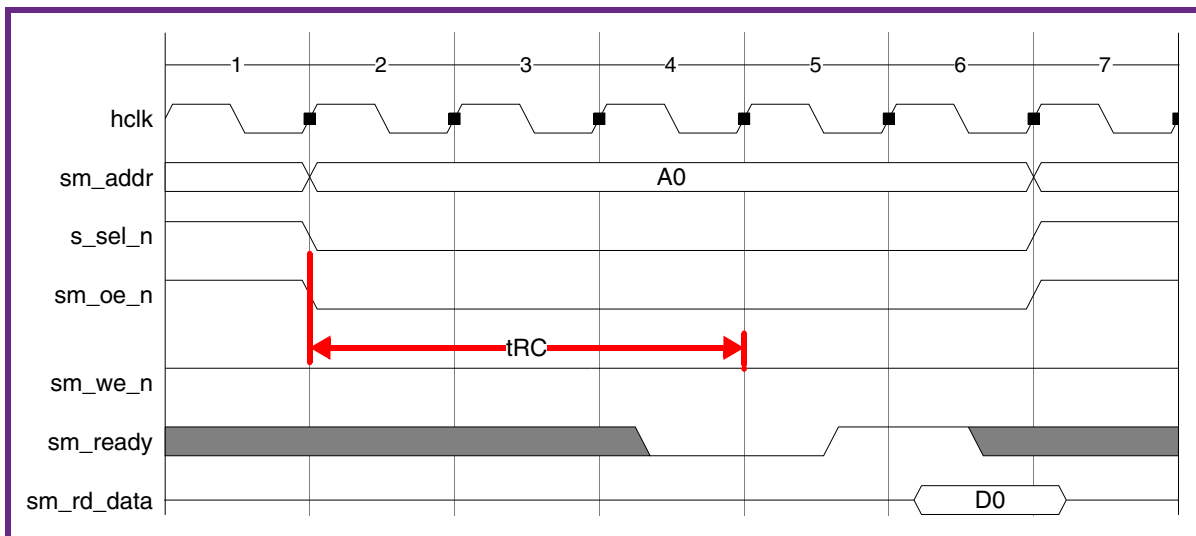
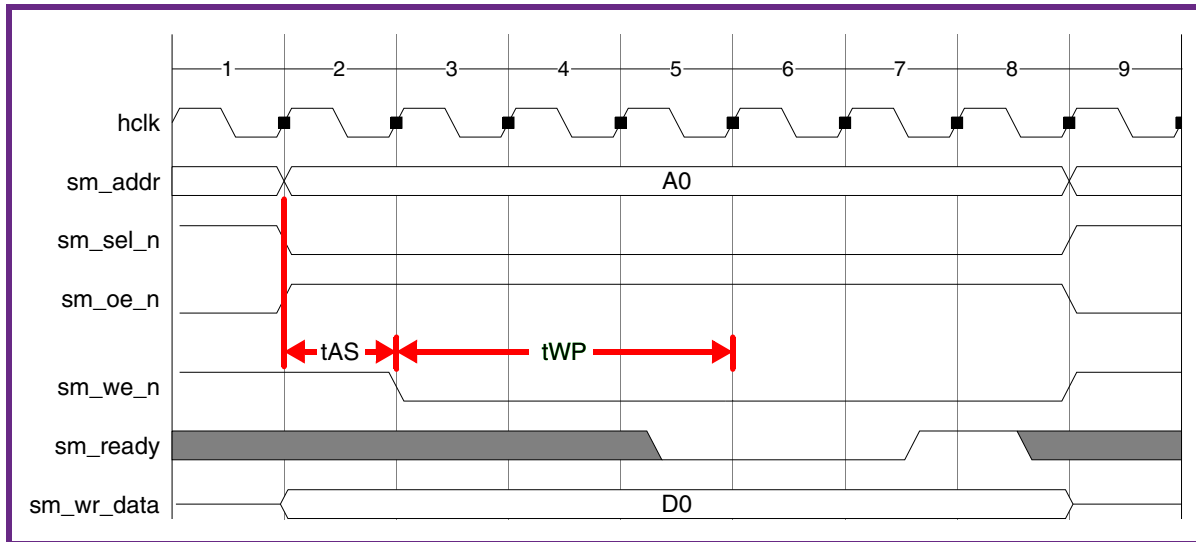


Figure 5-15 shows the timing diagram of a write access. The DW_memctl checks sm_ready after a time equal to "tAS(address setup time) + tWP(write period)." When the sm_ready is high, the write is finished. You need to ensure that the sm_ready signal is being driven with respect to the AHB clock hclk. This avoids a possible race condition if sm_ready is driven by a different clock.

Figure 5-15 Write Access of the Device with Ready Signal



5.7.2 Aborted Pre-Fetched Reads or Write Address Setup (with Ready Signal)

For undefined-length INCR AHB read transactions, DW_memctl initiates the read to the next sequential memory address even before the next SEQ transaction is seen on the AHB bus, pre-fetching the read data. This decreases latency for the read data when the next SEQ transaction does arrive. If the next transaction turns out to be a NONSEQ, therefore, ending the previous undefined-length INCR series of reads, DW_memctl aborts the memory read transaction (deasserts s_sel_n) – regardless of the value of sm_ready – because that pre-fetched memory read data will not be needed.

An aborted memory read transaction may occur in other circumstances as well, such as when an AHB "Early Burst Termination" occurs in the middle of an AHB read. In these circumstances, DW_memctl terminates the memory read transaction, regardless of the value of sm_ready.

Similarly, the tAS (write address setup) phase of a memory write may be aborted, in certain circumstances, such as Early Burst Termination or an AHB Master inserting BUSY cycles into a write transfer. No memory data corruption will occur, since sm_we_n will never be asserted for that aborted memory write.

5.7.3 Specifying the Timing Parameters

You can specify timing parameters of the device with the ready pin by programming the Static Memory Timing Register - Set 0 (SMTMGR_SET0), Static Memory Timing Register - Set 1 (SMTMGR_SET1), or Static Memory Timing Register - Set 2 (SMTMGR_SET2), depending on which of the three register sets should control the device.

You can use the following timing parameters:

- Read cycle time (tRC) – Bits 5:0 of SMTMGR_SET0/1/2 specify the read cycle time.
The T_RC_SET0/T_RC_SET1/T_RC_SET2 compile-time parameters specify the default value for tRC.
- Address setup time (tAS) – Bits 7:6 of the Static Memory Timing Register SMTMGR_SET0/1/2.
The T_AS_SET0/T_AS_SET1/T_AS_SET2 compile-time parameters specify the default value for tAS.

**Note**

tAS must be set to a value of 1 or greater when using sm_ready. tAS=0 is not supported when using sm_ready.

- Write period (tWP) – Bits 15:10 of SMTMGR_SET0/1/2.
The T_WP_SET0/T_WP_SET1/T_WP_SET2 compile-time parameters specify default value for tWP.
- Bus turnaround time (tBTA) – Bits 18:16 of SMTMGR_SET0/1/2 force the DW_memctl to insert tBTA number of cycles between back-to-back read/writes.
The T_BTA_SET0/T_BTA_SET1/T_BTA_SET2 compile-time parameters specify the default value for tBTA.

6

Parameters

This chapter describes the configuration parameters used by the DW_memctl. The settings of the configuration parameters determine the I/O signal list and internal functionality of the DW_memctl peripheral. You use coreConsultant or coreAssembler to configure the following parameters and generate the configured code.



Attention

When using coreConsultant or coreAssembler, you can right-click on a parameter label to access a “What’s This” popup dialog that will tell you the details for that particular parameter. The information in each What’s This dialog essentially matches the information in the parameter descriptions below.

6.1 Parameter Descriptions

The DW_memctl provides many configurable parameters that you can modify to tailor the memory controller to your requirements. There are two types of parameters:

- **Configurable Parameters** – Dependent upon your system requirements and must be modified before synthesizing the core; these parameters are translated to hardware. You specify the value for this type of parameter prior to synthesis; there is no way you can change it after the design is converted to gates.

The WRITE_FIFO_DEPTH parameter is an example. This parameter determines how many flip-flops are synthesized for the write FIFO. You may want to consider your future upgrade requirements when configuring these parameters.

- **Power-On Reset Value Parameters** – Initialize the reset values of different control registers in the controller. These parameters provide the initial/reset values, but after the design has been converted to gates, they can still be changed by programming the registers via software following power-on-reset. You must define some of these parameters yourself, and some of them are automatically configured, depending on the DWMMs that you pick.

For example, the CHIP_SELECT_n_BASE_ADDRESS is a parameter that you need to specify, while CAS_LATENCY is automatically configured from the DWMM you select.

You can also manually override configurable parameters that are automatically set from the chosen DWMM, if necessary. If you ensure all reset values are correct during the configuration process, you do not need to reprogram any of the registers later using software.

In addition, if your memory parameters are fixed and will not change, you can hardwire a group of configurable parameters to save area and improve synthesis timing. For example, if your memory address space is fixed, then setting `HARD_WIRE_CIPSELECT_PARAMETERS` will hardwire all the chip-select address decoder-related registers.

The following are categories of register power-on reset parameters:

- ❑ **SDRAM-Specific Power-On Reset Value Parameters** – Timing parameters specific to the `DW_memctl` as an SDRAM controller.
- ❑ **Static Memory-Specific Power-On Reset Value Parameters** – Default and maximum values for the Static Memory Timing register.
- ❑ **Chip Select-Specific Power-On Reset Value Parameters** – Power-on reset values for registers.

If you plan to boot from a FLASH, ensure in your configuration that the default reset value parameters for the FLASH are correct. The chip-select base address, block size, read access time, and so on, need to be correct for the memory controller to boot from FLASH after reset. It is better to provide conservative timing values so that you can boot with a lower-speed device. You can later change the timing parameters to optimal values through programming.

6.1.1 Configurable Parameters

[Table 6-1](#) lists configurable parameters that you can modify before synthesizing the core so that the `DW_memctl` controls the kind of memory that you intend to use.

Table 6-1 Configurable Parameters

Label	Parameter Definition
DWMM Installation	
Download and install DWMM Technology Software Package?	Parameter name: <code>Installdwmm</code> Legal Values: 0, 1 Default Value: 0 Description: Download and install the DesignWare Memory Model Technology Software Package. If you already have the DWMM installed, you can deselect this option and point to the existing DWMM TSP installation instead of creating a new installation.
Specify the path to install DWMM TSP:	Parameter name: <code>TSPpath</code> Legal Values: pathname Default Value: local directory Description: Directory path where to install the downloaded DWMM TSP. You need to have write access to the path provided.
* Synthesis compile-time parameter; cannot be changed later. % Hardwired if “Hardwire SDRAM Parameters” is enabled \$ Hardwired if “Hardwire Static Memory Parameters” is enabled ~ Hardwired if “Hardwire Chip Select Parameters” is enabled + Automatically extracted and loaded from DWMMs @ These parameters are automatically extracted and loaded from DesignWare Memory Models	

Table 6-1 Configurable Parameters

Label	Parameter Definition
Specify your LMC_HOME path:	Parameter name: LMCpath Legal Values: pathname Default Value: local directory Description: Existing DWMM TSP installation directory path.
Specify Memory Controller Type	
Specify DWMMtable File	Parameter name: SpecifyDWMMtable Legal Values: pathname Default Value: configuration directory Description: This file is the interface between DW_memctl and DesignWare Memory Models. This file can be found in your <i>configuration/DWMM</i> directory. The version number of this table file is the first line of this file. This file will be updated at the Synopsys MemoryCentral web site if any of the model information has changed. (http://www.synopsys.com/products/designware/memorycentral/dwmc.html) If you find that a new version of this file is on the web, then you should download this table file and then provide the download path here.
Specify SDRAM Controller Type	Parameter name: DYNAMIC_RAM_TYPE Legal Values: 0 – SDR-SDRAM 1 – DDR-SDRAM 3 – Mobile SDR-SDRAM 7 – Mobile DDR-SDRAM 6 – Disabled Default Value: 0 (SDR-SDRAM) Description: Select any of the SDRAM memory controller types.
Enable Static Memory Controller	Parameter name: ENABLE_STATIC Legal Values: 0 – Disable 1 – Enable Default Value: 1 Description: Enables or disables the Static memory controller.
Number of Chip Selects	Parameter name: N_CS Legal Values: 1-8 Default Value: 5 Description: Specifies the total number of Chip selects to be enabled. The maximum of 8 chip selects are available to be shared between the different types of memories.

* Synthesis compile-time parameter; cannot be changed later.

% Hardwired if “Hardwire SDRAM Parameters” is enabled

\$ Hardwired if “Hardwire Static Memory Parameters” is enabled

~ Hardwired if “Hardwire Chip Select Parameters” is enabled

+ Automatically extracted and loaded from DWMMs

@ These parameters are automatically extracted and loaded from DesignWare Memory Models

Table 6-1 Configurable Parameters

Label	Parameter Definition
System Frequency	Parameter name: SysFrequency Legal Values: Default Value: 100 Description: Specifies the system frequency in MHz. This value will be used for automatically setting the memory timing parameters. The testbench will use this frequency for running simulations.
Choose Memory Models	
Download DesignWare Memory Model	Parameter name: DownloadMemoryModel Legal Values: pathname Required Value: <i>workspace/download</i> for coreConsultant; <i>workspace/components/instance/download</i> for coreAssembler Description: If you have not already downloaded the memory models for your test, then you should enable this activity to download the models now. launches external web browser and connects to Synopsys web site where you need to enter SolvNet ID and password. Download must finish before completing the Component Configuration activity.
Specify Memory Model for Chip Select n	Parameter name: CHIP_SELECT n _MEM, where n is 0-7 Legal Values: 0 – SDRAM 1 – SRAM 2 – FLASH 3 – ROM 4 – Combo Default Value: 0 Description: Specify the memory model you want to connect to chip select n ; default value for bits 7:5 of register <i>MSKRN</i> .
Specify Memory Model for Chip Select n	Parameter name: REG_SELECT n Legal Values: 0 – SET0 1 – SET1 2 – SET2 Default Value: 2 Description: This specifies the timing register set associated for the static memory connected to Chip Select n .

* Synthesis compile-time parameter; cannot be changed later.

% Hardwired if “Hardwire SDRAM Parameters” is enabled

\$ Hardwired if “Hardwire Static Memory Parameters” is enabled

~ Hardwired if “Hardwire Chip Select Parameters” is enabled

+ Automatically extracted and loaded from DWMMs

@ These parameters are automatically extracted and loaded from DesignWare Memory Models

Table 6-1 Configurable Parameters

Label	Parameter Definition
Controller Configuration – AHB	
AHB Data Width *	Parameter name: H_DATA_WIDTH Legal Values: 32, 64, 128 Default Value: 32 Description: Specifies the width of the AHB data bus.
AHB Address Width *	Parameter name: H_ADDR_WIDTH Legal Values: 32, 64 Default Value: Fixed at 32 Description: Specifies the AHB address width.
Controller Configuration – CONTROLLER	
Add pipeline to address decoder outputs *	Parameter name: PIPE_ADDRESS_DECODER Legal Values: 0, 1 Default Value: 0 Description: Address decoder logic has the most critical timing in DW_memctl. Setting this option will add registers at the address decoder outputs and will enable the design to meet in excess of 200 MHz. If you enable this option, then this will add an extra cycle for memory access. By default, this is not set. If you see that the DW_memctl has a problem in meeting timing with the library that you are using, you should enable this option.
Address FIFO Depth *	Parameter name: ADDR_FIFO_DEPTH Legal Values: 4-32 Default Value: 4 Description: Specifies the depth of the address FIFO in the Host Interface Unit. The start address of Memory/Register access is stored in the FIFO. The recommended Address FIFO depth should be selected using system-typical behavior knowledge. In general, the default 4-deep address FIFO should be enough to prevent unwanted AMBA wait cycles.
Write FIFO Depth *	Parameter name: WRITE_FIFO_DEPTH Legal Values: 4-32 Default Value: 8 Description: The Write Data FIFO stores the write data. The depth of the Write Data FIFO can be between 4 and 32, with the default value being 8. The recommended Write Data FIFO depth should be selected using memory latency. If the AMBA/memory bus ratio is 2:1, it takes two cycles to read/write one AMBA datum. Therefore, a deeper FIFO depth is recommended. The width of the FIFO will correspond to the AMBA data width.

* Synthesis compile-time parameter; cannot be changed later.

% Hardwired if “Hardwire SDRAM Parameters” is enabled

\$ Hardwired if “Hardwire Static Memory Parameters” is enabled

~ Hardwired if “Hardwire Chip Select Parameters” is enabled

+ Automatically extracted and loaded from DWMMs

@ These parameters are automatically extracted and loaded from DesignWare Memory Models

Table 6-1 Configurable Parameters

Label	Parameter Definition
Enable EBI Interface *	Parameter name: EBI_INTERFACE Legal Values: 0 – Disable 1 – Enable Default Value: 0 Description: Enabling this will activate the External Bus Interface Logic and signals. This logic needs to be enabled only if you plan to share the memory controller I/O pads with other devices in your chip.
Enable SDRAM and Static Memory Data bus sharing *	Parameter name: ENABLE_DATABUS_SHARING Legal Values: 0 – Disable 1 – Enable Default Value: 0 Description: This is for enabling the sharing of the data bus between the Dynamic RAM and the Static Memory Interface. The data bus sharing feature cannot be used with DDR-SDRAMs due to critical timing and different clock domains in the data path.
Enable SDRAM and Static Memory Address bus sharing *	Parameter name: ENABLE_ADDRBUS_SHARING Legal Values: 1 – Enable 0 – Disable Default Value: 0 Description: This enables sharing the Address bus between the Dynamic RAM and the Static Memory Interface. The Address bus sharing feature may not be used with DDR-SDRAM, since it has SSTL2 pads. If selected, one may need to use external SSTL2 to TTL/LVTTL converters outside the chip to interface to Static memories.

* Synthesis compile-time parameter; cannot be changed later.

% Hardwired if “Hardwire SDRAM Parameters” is enabled

\$ Hardwired if “Hardwire Static Memory Parameters” is enabled

~ Hardwired if “Hardwire Chip Select Parameters” is enabled

+ Automatically extracted and loaded from DWMMs

@ These parameters are automatically extracted and loaded from DesignWare Memory Models

Table 6-1 Configurable Parameters

Label	Parameter Definition
Hardwire SDRAM Parameters *	Parameter name: HARD_WIRE_SDRAM_PARAMETERS Legal Values: 0, 1 Default Value: 0 Description: Setting this parameter to 1 will always drive the SDRAM parameters to constant values and will disable the programming feature to them. This parameter is intended for designs in which the SDRAM memory type is fixed. Setting this parameter will reduce area slightly and improve synthesis timing. The parameters which are set constant by this parameter are: S_DATA_WIDTH, READ_PIPE, EXTENDED_MODE_REG, S_BANK_ADDR_WIDTH, S_ROW_ADDR_WIDTH, S_COL_ADDR_WIDTH, OPEN_BANKS, CAS_LATENCY, T_RAS_MIN, T_RCD, T_RP, T_WR, T_WTR, T_RCAR, T_XSR, T_RC, T_INIT, NUM_INIT_REF.
Hardwire Static Memory Parameters *	Parameter name: HARD_WIRE_STATIC_PARAMETERS Legal Values: 0, 1 Default Value: 0 Description: Setting this parameter to 1 will always drive the Static memory timing parameters to constant values and will disable the programming feature to these. This parameter is intended for designs in which the Static memory type is fixed. Setting this parameter will reduce area slightly and improve synthesis timing. The parameters which are set constant by this parameter are: SM_DATA_WIDTH, SM_READ_PIPE_SETn, S_DATA_WIDTH, T_RPD, T_RC_SETn, T_AS_SETn, T_WR_SETn, T_WP_SETn, T_BTA_SETn, T_PRC_SETn, PAGE_MODE_SETn, PAGE_SIZE_SETn, LOW_FREQ_DEV_SETn, READY_MODE_SETn.
Hardwire Chip Select Parameters *	Parameter name: HARD_WIRE_CIPSELECT_PARAMETERS Legal Values: 0, 1 Default Value: 0 Description: Setting this parameter to 1 will always drive the Chip Select parameters to constant values and will disable the programming feature to these. This parameter is intended for designs in which the memory address is fixed. Setting this parameter will reduce area slightly and improve synthesis timing. The parameters which are set constant by this parameter: CHIP_SELECTn_BASE_ADDRESS, BLOCK_SIZEn, CHIP_SELECTn_MEM, REG_SELECTn, CHIP_SELECTn_ALIAS_ADDRESS, CHIP_SELECTn_REMAP_ADDRESS

* Synthesis compile-time parameter; cannot be changed later.

% Hardwired if “Hardwire SDRAM Parameters” is enabled

\$ Hardwired if “Hardwire Static Memory Parameters” is enabled

~ Hardwired if “Hardwire Chip Select Parameters” is enabled

+ Automatically extracted and loaded from DWMMs

@ These parameters are automatically extracted and loaded from DesignWare Memory Models

Table 6-1 Configurable Parameters

Label	Parameter Definition
Controller Configuration – SDRAM	
Write Pipe for SDRAM *	Parameter name: WRITE_PIPE Legal Values: 0-3 Default Value: 0 Description: Specifies the delay in clock cycles due to register insertion on the SDRAM write data bus outside the DW_memctl. This value, along with cas-latency and READ_PIPE, tells the memory controller when to expect the read data.
Read Pipe for SDRAM %	Parameter name: READ_PIPE Legal Values: 0-7 Default Value: 2 Description: Specifies the delay in clock cycles due to register insertion on the SDRAM read data bus outside the DW_memctl. This value, along with cas-latency and WRITE_PIPE, tells the memory controller when to expect the read data.
Number of Clock Cycles between SDRAM refresh %	Parameter name: T_REF Legal Values: 0 - 65535 Default Value: 1040 Description: The default value for the number of clock cycles between consecutive refresh cycles (this is typically 15.6μs or 7.8μs in the case of the newer larger SDRAMs). The user can change this value later by programming the SREFR register.
SDRAM INIT Cycles after power up %	Parameter name: T_INIT Legal Values: 0 - 65535 Default Value: 8 Description: The default value for the number of clock cycles to hold the SDRAM in puts stable after power up before issuing any commands. The user can change this value later by programming the STMGR1 register. This is Power Up VDD and CLK stable time for the SDRAMs, which is typically around 100μs. If your system reset already satisfies this time, then this could be programmed to just 1.
SDRAM Data Bus Width (Maximum) *	Parameter name: MAX_S_DATA_WIDTH Legal Values: 8, 16, 32, 64, 128 Default Value: 32 Description: Specifies the maximum width of the SDRAM data bus. For SDR-SDRAM and Mobile SDR-SDRAM, the typical values supported by DW_memctl are: 16, 32, 64, or 128 in ratio 1:1 or 1:2 with H_DATA_WIDTH For DDR-SDRAM and Mobile DDR-SDRAM, the supported values are: 8, 16, 32, or 64 in ratio 1:2 or 1:4 with H_DATA_WIDTH

* Synthesis compile-time parameter; cannot be changed later.

% Hardwired if “Hardwire SDRAM Parameters” is enabled

\$ Hardwired if “Hardwire Static Memory Parameters” is enabled

~ Hardwired if “Hardwire Chip Select Parameters” is enabled

+ Automatically extracted and loaded from DWMMs

@ These parameters are automatically extracted and loaded from DesignWare Memory Models

Table 6-1 Configurable Parameters

Label	Parameter Definition
SDRAM Data Bus Width (Default Value) %	Parameter name: S_DATA_WIDTH Legal Values: 8, 16, 32, 64, 128 Default Value: 32 Description: This specifies the default reset value for SDRAM data width. This corresponds to bits [41:13] of the SCORN register. For example, when designing a chip, one could select a 32-bit MAX_S_DATA_WIDTH, but later connect only a 16-bit SDRAM to it and program the SDRAM data width to 16-bits. For SDR-SDRAM and Mobile SDR-SDRAM, the typical values supported by DW_memctl are: 16, 32, 64, or 128 in ratio 1:1 or 1:2 with H_DATA_WIDTH For DDR-SDRAM and Mobile DDR-SDRAM the supported values are: 8, 16, 32, or 64 in ratio 1:2 or 1:4 with H_DATA_WIDTH
SDRAM Address Width (Maximum) *	Parameter name: MAX_S_ADDR_WIDTH Legal Values: 11-16 Default Value: 16 Description: Address width for SDRAM. The s_addr port will have this width.
Bank Address Width (Maximum) *	Parameter name: MAX_S_BANK_ADDR_WIDTH Legal Values: 1-4 Default Value: 2 Description: Maximum width of the SDRAM bank address bits. The s_bank_addr signal has this width. Values 1 to 4 address 2 to 16 banks.
Use Higher AHB Address bits for SDRAM Bank *	Parameter name: HIGHER_ADDR_FOR_BANKS Legal Values: 0, 1 Default Value: 0 Description: When enabled, higher AHB address bits are used for generating the SDRAM bank address. [haddr = {chip-sel, bank, ras, cas, byte-addr} instead of haddr = {chip-sel, ras, bank, cas, byte-addr}]; a useful feature in Mobile SDR/DDR-SDRAM memory systems. Since Mobile SDR/DDR-SDRAM allows one to selectively switch off banks, using higher address bits for banks will provide contiguous system address space, even if some banks are switched off. Similarly, SyncFlash pages will be mapped to continuous system address space. Enabling this could reduce the page-hit ratio during cas address crossovers.
Number of Data Strokes for DDR SDRAM *	Parameter name: NUM_DQS Legal Values: 0-128 Default Value: 4 Description: Specifies the number of data strobe pins for DDR-SDRAMs.

* Synthesis compile-time parameter; cannot be changed later.

% Hardwired if "Hardwire SDRAM Parameters" is enabled

\$ Hardwired if "Hardwire Static Memory Parameters" is enabled

~ Hardwired if "Hardwire Chip Select Parameters" is enabled

+ Automatically extracted and loaded from DWMMs

@ These parameters are automatically extracted and loaded from DesignWare Memory Models

Table 6-1 Configurable Parameters

Label	Parameter Definition
Extended Mode Register Value %	Parameter name: EXTENDED_MODE_REG Legal Values: 32'h0000_0000 to 32'hfff_ffff Default Value: 32'h00000000 Description: Default value for extended-mode register values for DDR-SDRAM or Mobile DDR/SDR-SDRAM. This goes in as the default value for bits 11:0 of DW_memctl Extended Mode register EXN_MODE_REG (address 0x0000_00B0).
Controller Configuration – STATIC	
Write Pipe for Static Memory *	Parameter name: SM_WRITE_PIPE Legal Values: 0-3 Default Value: 0 Description: Specifies the delay in clock cycles due to register insertion on the Static memory write data bus outside the DW_memctl. This parameter, along with read access time (T_RC) and SM_READ_PIPE, tells the memory controller when to expect the read data.
Static Memory Data Bus Width (Maximum) *	Parameter name: MAX_SM_DATA_WIDTH Legal Values: 8, 16, 32, 64, or 128 Default Value: 32 Description: Maximum size of Static memory data bus width. Static memory data signals have this width. When configured as both a Static memory controller and an SDRAM controller, Static memory data width must be less than or equal to SDRAM memory data width (S_DATA_WIDTH).
Static Memory Address Width (Maximum) *	Parameter name: MAX_SM_ADDR_WIDTH Legal Values: 11-32 Default Value: 23 Description: Maximum size of Static memory address width. The sm_addr signal has this width.
FLASH t_rpd delay \$	Parameter name: T_RPD Legal Values: 0 - 4095 Default Value: 200 Description: Static memory FLASH reset/power-down high to read/write delay.

* Synthesis compile-time parameter; cannot be changed later.

% Hardwired if “Hardwire SDRAM Parameters” is enabled

\$ Hardwired if “Hardwire Static Memory Parameters” is enabled

~ Hardwired if “Hardwire Chip Select Parameters” is enabled

+ Automatically extracted and loaded from DWMMs

@ These parameters are automatically extracted and loaded from DesignWare Memory Models

Table 6-1 Configurable Parameters

Label	Parameter Definition
MEMCTL Source Code Configuration	
Use DesignWare Foundation Synthesis Library (active only when source license available)	Parameter Name: USE_FOUNDATION Legal Values: True (1) or False (0) Default Value: True; only if Source license is available. Dependencies: Must have Source license. Description: The component code utilizes DesignWare Foundation parts for optimal Synthesis QoR. Customers with only a DesignWare license <i>must</i> use Foundation parts. Customers with both Source and DesignWare licenses have the option of using Foundation parts.
SDRAM Specific	
Row Address Width + %	Parameter name: S_ROW_ADDR_WIDTH Legal Values: 11-16 Default Value: 12 Description: Width of the SDRAM row address bus. This directly controls the width of the SDRAM address bus. The supported SDRAM should have at least 2K rows. If the user is using DesignWare Memory Models, then this parameter will be automatically loaded.
Column Address Width + %	Parameter name: S_COL_ADDR_WIDTH Legal Values: 8-15 Default Value: 9 Description: Width of the SDRAM column address bus. The supported SDRAM should have at least 256 columns. If the user is using the DesignWare Memory Model, then this parameter will be automatically loaded.
DDR-SDRAM Precharge Bit +*	Parameter name: A8_FOR_PRECHARGE Legal Values: 1 – A8 used for precharge 0 – A10 used for precharge Default Value: 0 Description: Specifies the Precharge bit used by the DDR-SDRAM. Some DDR SDRAMs use A8 bit for precharge, whereas other vendors use the A10 bit for Precharge.

* Synthesis compile-time parameter; cannot be changed later.

% Hardwired if “Hardwire SDRAM Parameters” is enabled

\$ Hardwired if “Hardwire Static Memory Parameters” is enabled

~ Hardwired if “Hardwire Chip Select Parameters” is enabled

+ Automatically extracted and loaded from DWMMs

@ These parameters are automatically extracted and loaded from DesignWare Memory Models

Table 6-1 Configurable Parameters

Label	Parameter Definition
Bank Address Width ⁺ %	Parameter name: S_BANK_ADDR_WIDTH Legal Values: 1-4 Default Value: 2 Description: Width of the SDRAM bank address. DW_memctl supports a maximum of 16 banks. Values 1 to 4 address 2 to 16 banks. If the user is using the DesignWare Memory Models, then this parameter will be automatically loaded.
Number of Banks to be kept open at any time ⁺ %	Parameter name: OPEN_BANKS Legal Values: 1-16 Default Value: 4 Description: This specifies the number of active banks to be kept open at any time. The user can choose a value from 1 to 16 banks.
Cas Latency ⁺ %	Parameter name: CAS_LATENCY Legal Values: 1 – 1 clock 2 – 2 clocks 3 – 3 clocks 4 – 4 clocks 5 – 1.5 clocks 6 – 2.5 clocks Default Value: 3 Description: The default value in clock cycles for the SDRAM Cas latency. This value can be changed later by programming the STMG1R timing register. If the user is using the DesignWare Memory Model, then this parameter will be automatically loaded.
t_ras_min ⁺ %	Parameter name: T_RAS_MIN Legal Values: 1-16 Default Value: 6 Description: The default value in clock cycles for the minimum delay between the ACTIVE and the PRECHARGE commands. The user can change this value later by programming the STMGR1 register. If the user is using DesignWare Memory Models, then this parameter will be automatically loaded.
t_rcd ⁺ %	Parameter name: T_RCD Legal Values: 1-8 Default Value: 3 Description: The default value in clock cycles for the minimum delay between the ACTIVE and READ/WRITE commands. The user can change this value later by programming the STMGR1 register. If the user is using the DesignWare Memory Models, then this parameter will be automatically loaded.

* Synthesis compile-time parameter; cannot be changed later.

% Hardwired if “Hardwire SDRAM Parameters” is enabled

\$ Hardwired if “Hardwire Static Memory Parameters” is enabled

~ Hardwired if “Hardwire Chip Select Parameters” is enabled

+ Automatically extracted and loaded from DWMMs

@ These parameters are automatically extracted and loaded from DesignWare Memory Models

Table 6-1 Configurable Parameters

Label	Parameter Definition
$t_{rp}^{+ \%}$	Parameter name: T_RP Legal Values: 1-8 Default Value: 3 Description: Default value for SDRAM precharge period
$t_{wr}^{+ \%}$	Parameter name: T_WR Legal Values: 1-4 Default Value: 3 Description: The default value in clock cycles for the delay from the last data in (in case of writes) to the next precharge command. The user can change this value later by programming the STMGR1 register. If the user is using the DesignWare Memory Models, then this parameter will be automatically loaded.
$t_{rcar}^{+ \%}$	Parameter name: T_RCAR Legal Values: 1-16 Default Value: 10 Description: The default value in clock cycles for the Auto Refresh period. This is the minimum time between two auto refresh commands. The user can change this value later by programming the STMGR1 register. If the user is using the DesignWare Memory Models, then this parameter will be automatically loaded.
$t_{xsr}^{+ \%}$	Parameter name: T_XSR Legal Values: 1-512 Default Value: 11 Description: The default value for the Exit self-refresh to Active or auto-refresh command time. This is the minimum time the controller should wait after taking the SDRAM out of the self_refresh mode before issuing any Active- or Auto-refresh commands. The user can change this value later by programming the STMGR1 register. If the user is using DesignWare Memory Models, then this parameter will be automatically loaded.
$t_{rc}^{+ \%}$	Parameter name: T_RC Legal Values: 1-16 Default Value: 10 Description: Active-to-active command period. Values of 0-15 correspond to t_{rc} of 1 to 16 clocks. The user can change this value later by programming the STMGR1 register. If the user is using the DesignWare Memory Model, then this parameter will be automatically loaded.
* Synthesis compile-time parameter; cannot be changed later. % Hardwired if “Hardwire SDRAM Parameters” is enabled \$ Hardwired if “Hardwire Static Memory Parameters” is enabled ~ Hardwired if “Hardwire Chip Select Parameters” is enabled + Automatically extracted and loaded from DWMMs @ These parameters are automatically extracted and loaded from DesignWare Memory Models	

Table 6-1 Configurable Parameters

Label	Parameter Definition
t _{xp} ⁺ %	Parameter name: T_XP Legal Values: 2-7 Default Value: 5 Description: DDR and Mobile DDR Exit power down to next valid command delay. Valid values are 2 to 7 clocks. There is no corresponding timing register for this value, and therefore it cannot be changed later.
Number of refresh during Init %	Parameter name: NUM_INIT_REF Legal Values: 1-16 Default Value: 8 Description: The default value for the number of auto refreshes during initialization. Values 0-15 correspond to 1-16 auto refresh. The user can change this value later by programming the STMGR2 register. Users have to manually specify this parameter, which is not automatically loaded, even if they use DesignWare Memory Models.
Write to Read Delay for DDR-SDRAM %	Parameter name: T_WTR Legal Values: 1-4 Default Value: 1 Description: This is the internal write-to-read command delay for DDR-SDRAMs.
Enable Ready Mode Read Operation (Use s _{rd_ready}) %	Parameter name: S_READY_VALID Legal Values: 0, 1 Default Value: 0 Description: When enabled, indicates that the SDRAM read data should be sampled after the s _{rd_ready} input goes active. This goes in as the default value for bit 17 of the SCTL register.
Set s _{dout_valid} signal to be deasserted when controller is idle *	Parameter name: S_DOUT_VALID_LOW Legal Values: 0, 1 Default Value: 0 Description: s _{dout_valid} signal enables the output buffers DQS and DQ. Setting this parameter enables s _{dout_valid} to be driven to logic 0 (de-asserted) when the controller is idle; for example, during initialization, power-down, self-refresh.

* Synthesis compile-time parameter; cannot be changed later.

% Hardwired if “Hardwire SDRAM Parameters” is enabled

\$ Hardwired if “Hardwire Static Memory Parameters” is enabled

~ Hardwired if “Hardwire Chip Select Parameters” is enabled

+ Automatically extracted and loaded from DWMMs

@ These parameters are automatically extracted and loaded from DesignWare Memory Models

Table 6-1 Configurable Parameters

Label	Parameter Definition
STATIC Memory SETn	
t_{rc_setn} @ \$	Parameter name: T_RC_SETn Legal Values: 1-64 Default Value: t_{rc_set0} – 2 t_{rc_set1} – 28 t_{rc_set2} – 28 Description: Specifies Read Cycle Time for the static memory associated with register setn. This is the minimum time the Address bus must be stable. This goes in as the default value of bits 5:0 of the Static Memory timing register SMTMGR_SETn. If the user is using the DesignWare Memory Model, then this parameter will be automatically loaded. Corresponding parameters are: AMD-FLASH – trc INTEL-FLASH – tavav Samsung SRAMs – trc
t_{as_setn} @ \$	Parameter name: T_AS_SETn Legal Values: 0-3 Default Value: 1 Description: Specifies the Write Address setup time for Static memory associated with register setn. This is the minimum time the address must be stable before Chip Enable or Write Enable is asserted. If the user is using the DesignWare Memory Model, then this parameter will be automatically loaded. This parameter goes in as the default value for bits 7:6 of the Static Memory Timing Register Setn SMTMGR_SETn. Corresponding timing parameters are: AMD FLASH – max(tas,tcs) Intel Flash – telwl Samsung SRAMs – tas
* Synthesis compile-time parameter; cannot be changed later. % Hardwired if “Hardwire SDRAM Parameters” is enabled \$ Hardwired if “Hardwire Static Memory Parameters” is enabled ~ Hardwired if “Hardwire Chip Select Parameters” is enabled + Automatically extracted and loaded from DWMMs @ These parameters are automatically extracted and loaded from DesignWare Memory Models	

Table 6-1 Configurable Parameters

Label	Parameter Definition
t_wr_setn @ \$	Parameter name: T_WR_SETn Legal Values: 0-3 Default Value: t_wr_set0 – 0 t_wr_set1 – 3 t_wr_set2 – 3 Description: Specifies the Write Recovery time for the Static memory associated with register setn. This is same as the minimum time Address/Data must be stable after a Chip Enable or Write Enable is de-asserted. This goes in as bits 9:8 of Static Memory Timing Register setn SMTMGR_SETn. If the user is using the DesignWare Memory Model, then this parameter is automatically loaded. Corresponding timing parameters are: AMD-FLASH – t_{dh} INTEL-FLASH – max(t_{whax}, t_{whdx}) Samsung SRAMs – t_{ha}
t_wp_setn @ \$	Parameter name: T_WP_SETn Legal Values: 1-64 Default Value: t_wp_set0 – 2 t_wp_set1 – 20 t_wp_set2 – 20 Description: Specifies the Write Pulse Width for Static memory associated with register setn. This goes in as the default value for bits 15:10 of Static Memory Timing register set0 SMTMGR_SETn. If the user is using the DesignWare Memory Model, then this parameter is automatically loaded. Corresponding parameters are: AMD-FLASH – t_{wp} INTEL-FLASH – t_{wlwh} Samsung SRAMs – t_{pwe}

* Synthesis compile-time parameter; cannot be changed later.
 % Hardwired if “Hardwire SDRAM Parameters” is enabled
 \$ Hardwired if “Hardwire Static Memory Parameters” is enabled
 ~ Hardwired if “Hardwire Chip Select Parameters” is enabled
 + Automatically extracted and loaded from DWMMs
 @ These parameters are automatically extracted and loaded from DesignWare Memory Models

Table 6-1 Configurable Parameters

Label	Parameter Definition
t_bta_setn @ \$	Parameter name: T_BTA_SETn Legal Values: 0-7 Default Value: t_bta_set0 – 1 t_bta_set1 – 4 t_bta_set2 – 4 Description: Specifies the Data Bus turnaround time for Static memory associated with register setn. This goes in as the default value for bits 18:16 of the SMTMGR_SETn register. The user has to manually enter this parameter. This parameter is not automatically loaded, even if you are using the DesignWare Memory Model.
t_prc_setn @ \$	Parameter name: T_PRC_SETn Legal Values: 1-16 Default Value: t_prc_set0 – 1 t_prc_set1 –16 t_prc_set2 – 4 Description: Specifies the Page Mode read cycle time for Static memory associated with register setn. This is the delay from the LSBs of the address bus to the corresponding Data Bus change. This goes in as the default value for bits 22:19 of SMTMGR_SETn. If you are using the DesignWare Memory Model, then this parameter will be automatically loaded. Corresponding parameters are: AMD-FLASH – taccl INTEL-FLASH – tapa
Enable Page/Burst mode for set n \$	Parameter name: PAGE_MODE_SETn Legal Values: 0 – Device Page/Burst mode disabled 1 – Device Page/Burst mode enabled Default Value: 0 Description: Specifies if the Static memory device associated with register setn supports page mode or not. This goes in as the default value for bit 23 of the SMTMGR_SETn register.

* Synthesis compile-time parameter; cannot be changed later.

% Hardwired if “Hardwire SDRAM Parameters” is enabled

\$ Hardwired if “Hardwire Static Memory Parameters” is enabled

~ Hardwired if “Hardwire Chip Select Parameters” is enabled

+ Automatically extracted and loaded from DWMMs

@ These parameters are automatically extracted and loaded from DesignWare Memory Models

Table 6-1 Configurable Parameters

Label	Parameter Definition
page_size_setn ^{\$}	Parameter name: PAGE_SIZE_SETn Legal Values: 0 – 4-word page 1 – 8-word page 2 – 16-word page 3 – 32-word page Default Value: 0 Description: Specifies the page size for the Static memory device associated with register setn. The user has to manually enter this parameter, which is not automatically set even if you use the DesignWare Memory Model. This goes in as the default value of bits 25:24 of the SMTMGR_SETn register.
Enable Ready mode Operation for setn (Use sm_ready) ^{\$}	Parameter name: READY_MODE_SETn Legal Values: 0 – Enable ready-mode operation 1 – Disable ready-mode operation Default Value: 0 Description: Specifies if the device associated with register setn needs a ready-mode handshake or not. This feature is meant for non-memory type of device interface. In ready mode, DW_memctl takes read data after sm_ready signal goes active. During write, the write cycle ends only after the sm_ready is sampled active. This goes in as the default value for bit 26 of the SMTMGR_SETn register. This parameter is not automatically set if the user is using the DesignWare Memory Model.
Enable Low Frequency static device for setn (use sm_clken signal) ^{\$}	Parameter name: LOW_FREQ_DEV_SETn Legal Values: 0 or 1 Default Value: 0 Description: This bit is valid if register setn is used to control a synchronous memory. When interfacing to a synchronous memory, the device frequency can be a sub-multiple of the AHB frequency (1, 1/2, 1/3, 1/4, or 1/5, and so on). This bit indicates if the device operates on a lower frequency than the AHB Clock frequency.
Read Pipe for Static Memory setn ^{\$}	Parameter name: SM_READ_PIPE_SETn Legal Values: 0-3 Default Value: 0 Description: Specifies the delay in clock cycles due to register insertion on the memory read data bus, outside the DW_memctl for the Static memory associated with Register setn. This parameter, along with read access time (T_RC) and SM_WRITE_PIPE, tells the memory controller when to expect the read data.

* Synthesis compile-time parameter; cannot be changed later.

% Hardwired if “Hardwire SDRAM Parameters” is enabled

\$ Hardwired if “Hardwire Static Memory Parameters” is enabled

~ Hardwired if “Hardwire Chip Select Parameters” is enabled

+ Automatically extracted and loaded from DWMMs

@ These parameters are automatically extracted and loaded from DesignWare Memory Models

Table 6-1 Configurable Parameters

Label	Parameter Definition																					
Static Memory Data Bus Width for set n (default value) \$	Parameter name: SM_DATA_WIDTH_SETn Legal Values: 8, 16, 32, 64, or 128 Default Value: 32 Description: This specifies the default reset value for Static Memory data width. For example when designing a chip one could select a 32-bit MAX_SM_DATA_WIDTH, but later connect only 16-bit Static memory to it and strap the sm_data_width_setn port to 000 to select data width of 16-bits.																					
Controller Chip Selects																						
ChipSelect n Base Address ~	Parameter name: CHIP_SELECTn_BASE_ADDRESS Legal Values: 32'h0000_xxxx to 32'hffff_xxxx Default Value: 0x80000000 Description: This specifies the default value for the base address register corresponding to Chip Selectn. This field is enabled only if the number of selected chip selects is greater than 1.																					
Block Size for ChipSelect n ~	Parameter name: BLOCK_SIZEn Legal Values: 0-17 <table><tr><td>0 – None</td><td>7 – 4MB</td><td>14 – 512MB</td></tr><tr><td>1 – 64KB</td><td>8 – 8MB</td><td>15 – 1GB</td></tr><tr><td>2 – 128KB</td><td>9 – 16MB</td><td>16 – 2GB</td></tr><tr><td>3 – 256KB</td><td>10 – 32MB</td><td>17 – 4GB</td></tr><tr><td>4 – 512KB</td><td>11 – 64MB</td><td>Others reserved</td></tr><tr><td>5 – 1MB</td><td>12 – 128MB</td><td></td></tr><tr><td>6 – 2MB</td><td>13 – 256MB</td><td></td></tr></table> Default Value: 12 – 128MB Description: The default block size for memory connected to Chip Selectn. For an SDRAM, minimum block size is 2MB.	0 – None	7 – 4MB	14 – 512MB	1 – 64KB	8 – 8MB	15 – 1GB	2 – 128KB	9 – 16MB	16 – 2GB	3 – 256KB	10 – 32MB	17 – 4GB	4 – 512KB	11 – 64MB	Others reserved	5 – 1MB	12 – 128MB		6 – 2MB	13 – 256MB	
0 – None	7 – 4MB	14 – 512MB																				
1 – 64KB	8 – 8MB	15 – 1GB																				
2 – 128KB	9 – 16MB	16 – 2GB																				
3 – 256KB	10 – 32MB	17 – 4GB																				
4 – 512KB	11 – 64MB	Others reserved																				
5 – 1MB	12 – 128MB																					
6 – 2MB	13 – 256MB																					
Enable Remap for Chipselect n *	Parameter name: CHIP_SELn_REMAP_ENABLE Legal Values: 0, 1 Default Value: 0 Description: This enables the remap logic for Chip Selectn. When enabled, and when the remap input is logic 1, the chip select will be generated if there is a match between the host address for memory address and the value in the remap register for Chip Selectn; refer to page 181 for details.																					

* Synthesis compile-time parameter; cannot be changed later.

% Hardwired if “Hardwire SDRAM Parameters” is enabled

\$ Hardwired if “Hardwire Static Memory Parameters” is enabled

~ Hardwired if “Hardwire Chip Select Parameters” is enabled

+ Automatically extracted and loaded from DWMMs

@ These parameters are automatically extracted and loaded from DesignWare Memory Models

Table 6-1 Configurable Parameters

Label	Parameter Definition
Enable Aliasing for Chipselect n *	Parameter name: CHIP_SEL n _ALIAS_ENABLE Legal Values: 0, 1 Default Value: 0 Description: This enables the aliasing logic for Chip Select n . When enabled, Chip Select n will be generated under two conditions: (1) when the host memory address matches the value in the SCSLR n register, and (2) when the host memory address matches the value in the ALIAS n register; refer to page 181 for details.
ChipSelect n Alias Address ~	Parameter name: CHIP_SELECT n _ALIAS_ADDRESS Legal Values: 32'h0000_XXXX to 32'hffff_XXXX Default Value: 32'h18000000 Description: This specifies the default value for the alias address register that corresponds to Chip Select n . This field is enabled only if the parameter CHIP_SEL n _ALIAS_ENABLE is set to 1; refer to page 181 for details.
ChipSelect n Remap Address ~	Parameter name: CHIP_SELECT n _REMAP_ADDRESS Legal Values: 32'h0000_XXXX to 32'hffff_XXXX Default Value: 32'h12000000 Description: This specifies the default value for the Remap address register that corresponds to Chip Select n . This field is enabled only if the parameter CHIP_SEL n _REMAP_ENABLE is set to 1; refer to page 181 for details.

* Synthesis compile-time parameter; cannot be changed later.

% Hardwired if “Hardwire SDRAM Parameters” is enabled

\$ Hardwired if “Hardwire Static Memory Parameters” is enabled

~ Hardwired if “Hardwire Chip Select Parameters” is enabled

+ Automatically extracted and loaded from DWMMs

@ These parameters are automatically extracted and loaded from DesignWare Memory Models

7

Signals

The following subsections describe the DW_memctl I/O signals.

7.1 DW_memctl Interface Diagrams

The DW_memctl I/O signals depend on these configuration parameters:

```
DYNAMIC_RAM_TYPE
ENABLE_STATIC
ENABLE_DATABUS_SHARING
ENABLE_ADDRBUS_SHARING
EBI_INTERFACE
```

The combination of values on these parameters determines how the DW_memctl is configured.

[Table 7-1](#) lists the different configurations for which you can use the DW_memctl and the illustrations showing the respective I/O signals.

Table 7-1 Parameter-Dependent Configurations

Configuration	Figure
SDRAM and Static memory controller with separate addresses and data buses for each interface	Figure 7-1 on page 146
SDRAM and Static controller with shared address and data pins	Figure 7-2 on page 147
DW_memctl interface with shared address bus	Figure 7-3 on page 148
DW_memctl interface with shared data bus	Figure 7-4 on page 149
SDRAM controller only	Figure 7-5 on page 150
Static memory controller only	Figure 7-6 on page 151

Figure 7-1 shows the DW_memctl as an SDRAM and a Static memory controller with separate addresses and data buses for each interface; refer to Table 7-2 on page 152 for I/O signal descriptions.

- DYNAMIC_RAM_TYPE < 4; ENABLE_STATIC=1; EBI_INTERFACE=1
- ENABLE_ADDRBUS_SHARING=0; ENABLE_DATABUS_SHARING=0

Figure 7-1 DW_memctl Interface with Separate Address/Data Bus

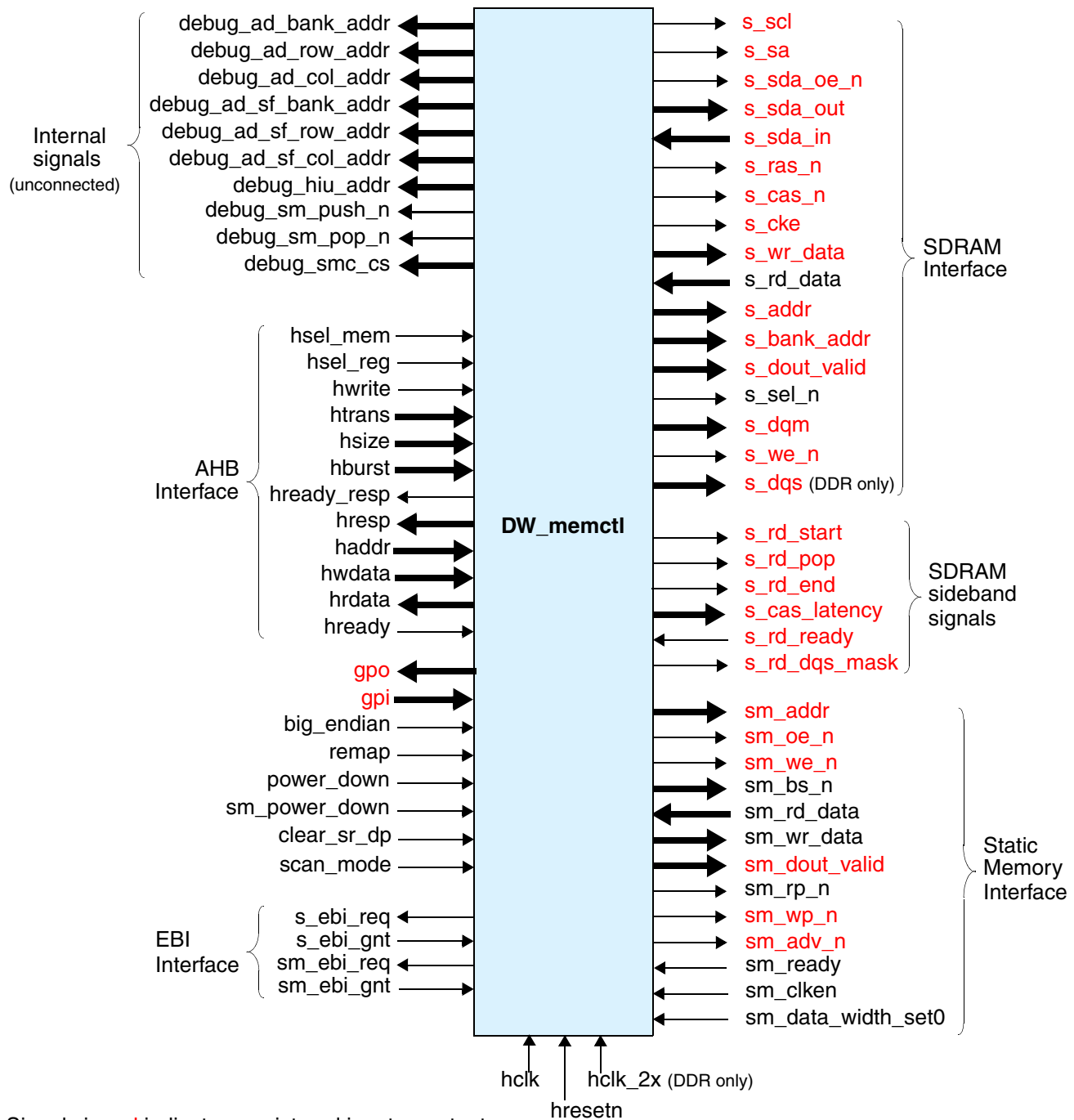


Figure 7-2 shows the DW_memctl as both an SDRAM and Static controller with shared address and data pins; refer to Table 7-2 on page 152 for I/O signal descriptions.

- DYNAMIC_RAM_TYPE < 4; ENABLE_STATIC=1; EBI_INTERFACE=1
- ENABLE_ADDRBUS_SHARING=1; ENABLE_DATABUS_SHARING=1

Figure 7-2 DW_memctl Interface with Shared Address/Data Bus

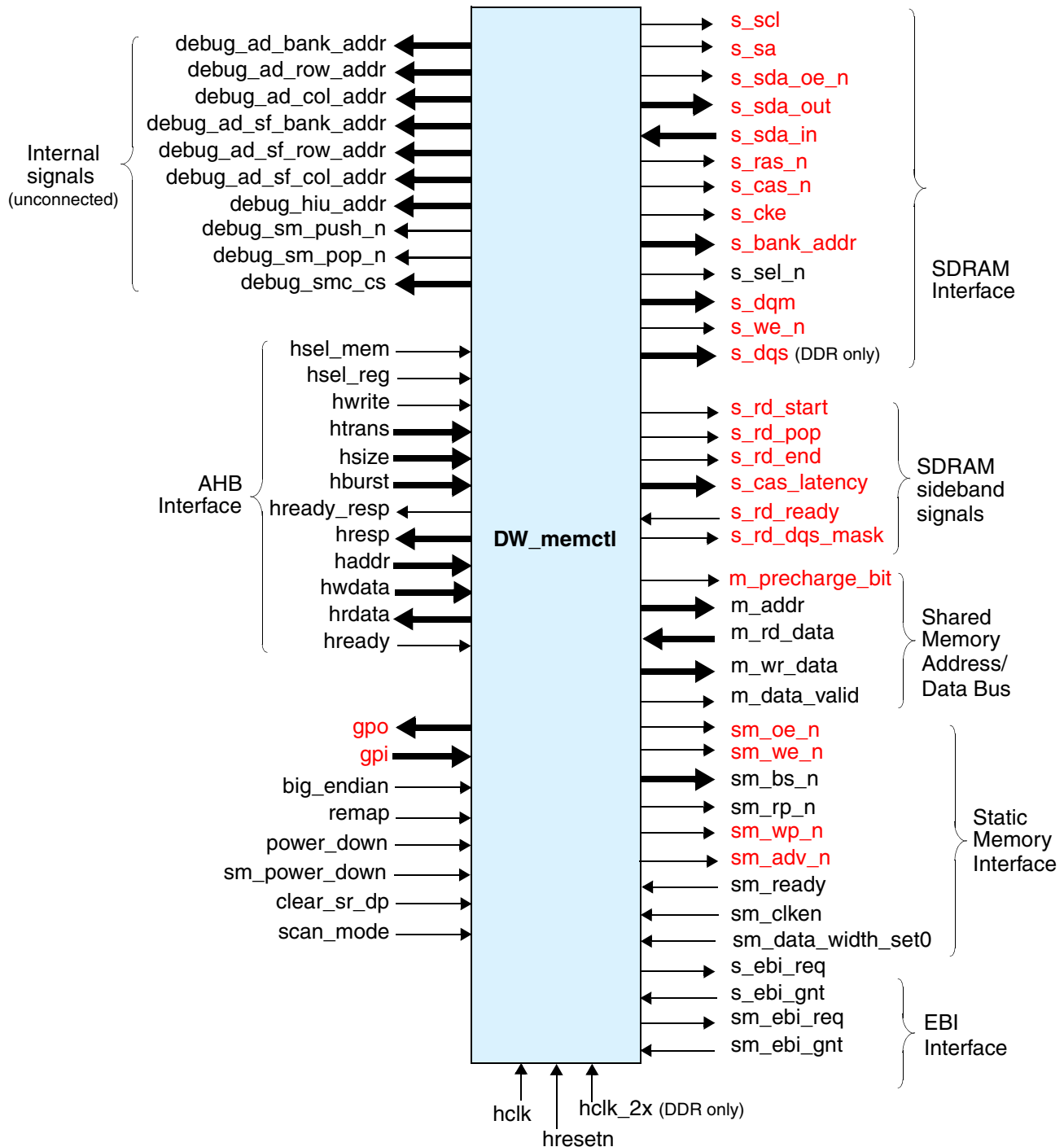


Figure 7-3 shows the DW_memctl as both an SDRAM and Static controller with shared address pins; refer to Table 7-2 on page 152 for I/O signal descriptions.

- DYNAMIC_RAM_TYPE < 4; ENABLE_STATIC=1; EBI_INTERFACE=1
- ENABLE_ADDRBUS_SHARING=1; ENABLE_DATABUS_SHARING=0

Figure 7-3 DW_memctl Interface with Shared Address Bus

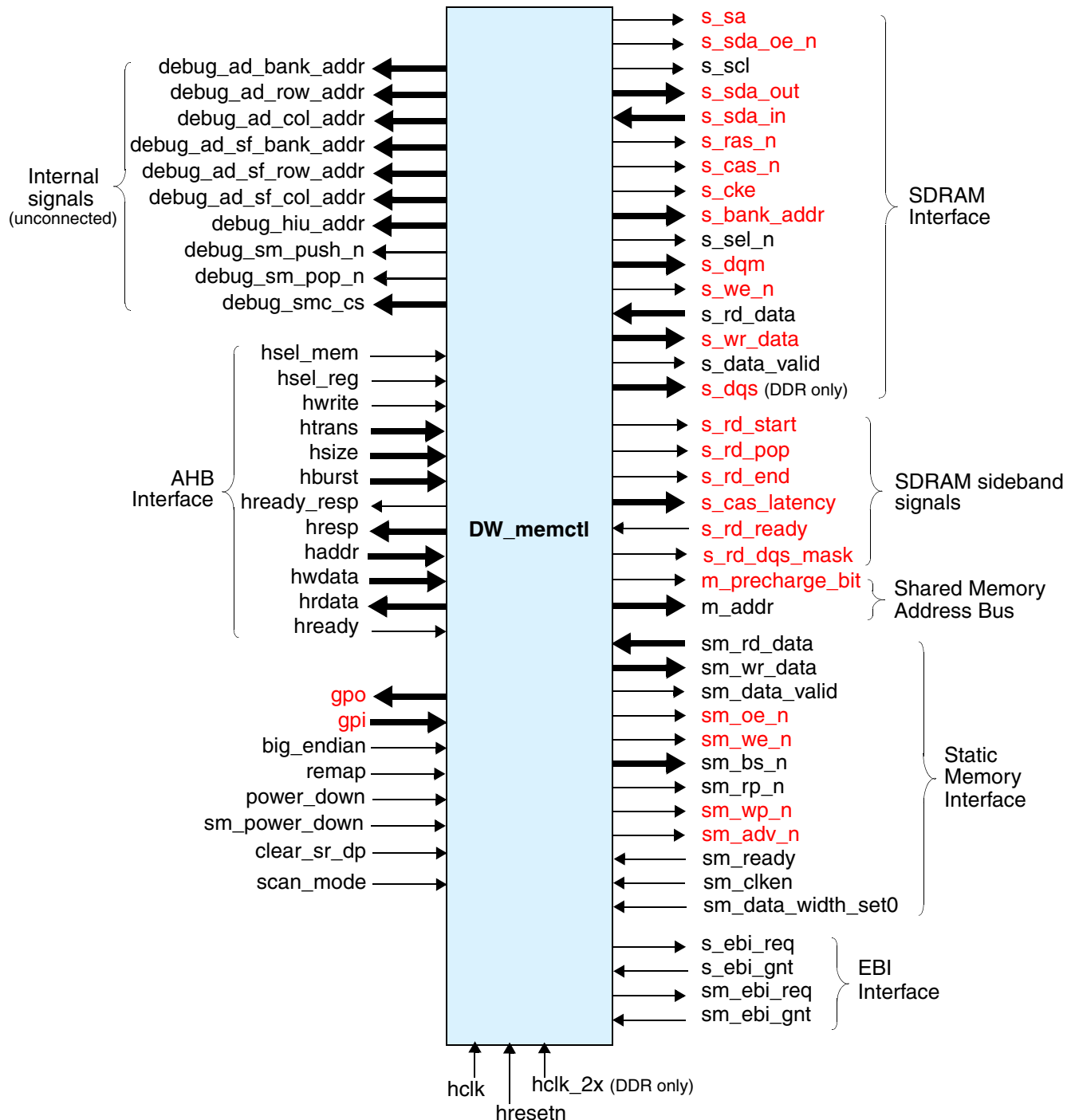


Figure 7-4 shows the DW_memctl as both an SDRAM and Static controller with shared data pins; refer to Table 7-2 on page 152 for I/O signal descriptions.

- DYNAMIC_RAM_TYPE < 4; ENABLE_STATIC=1; EBI_INTERFACE=1
- ENABLE_ADDRBUS_SHARING=0; ENABLE_DATABUS_SHARING=1

Figure 7-4 DW_memctl Interface with Shared Data Bus

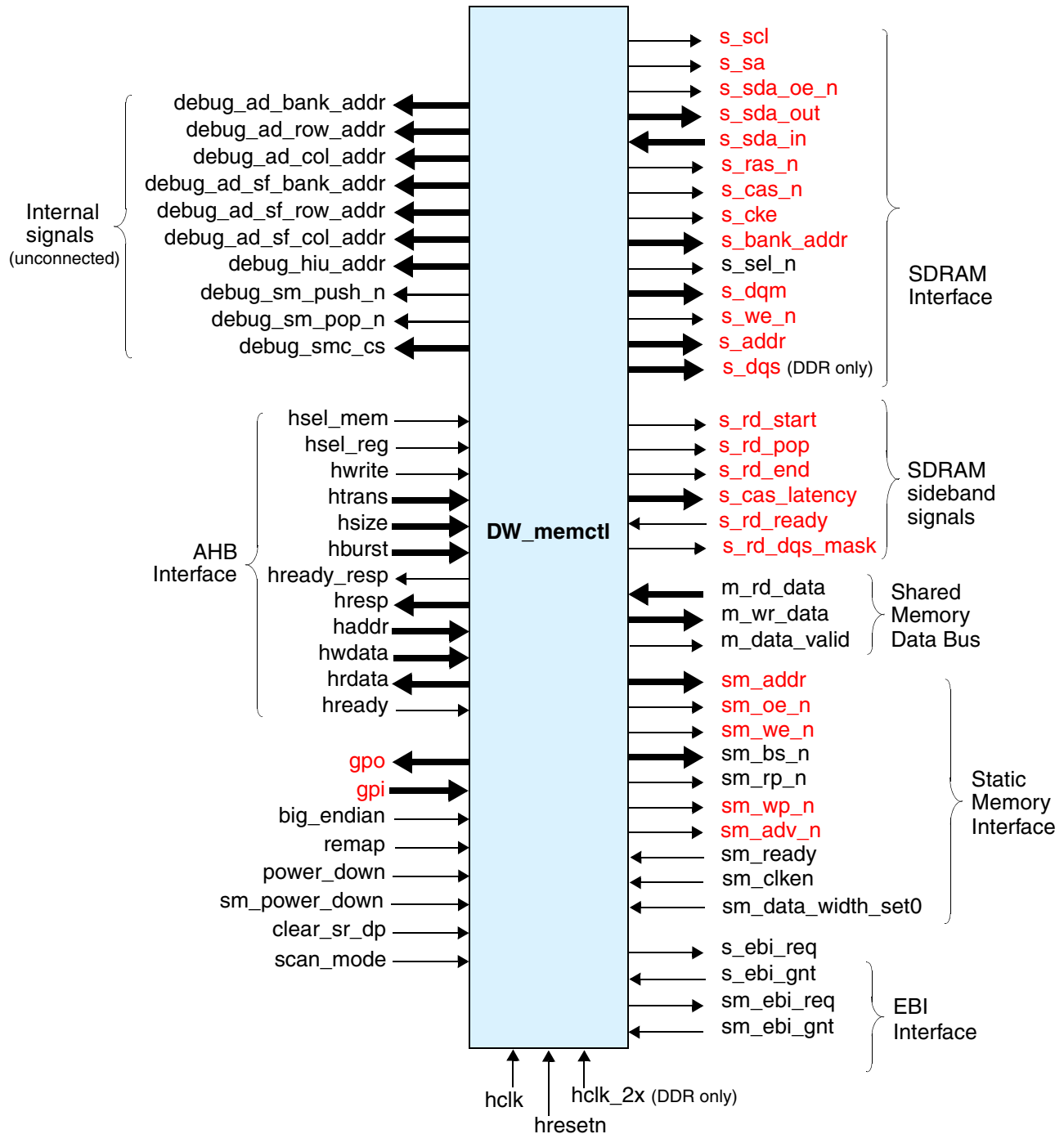
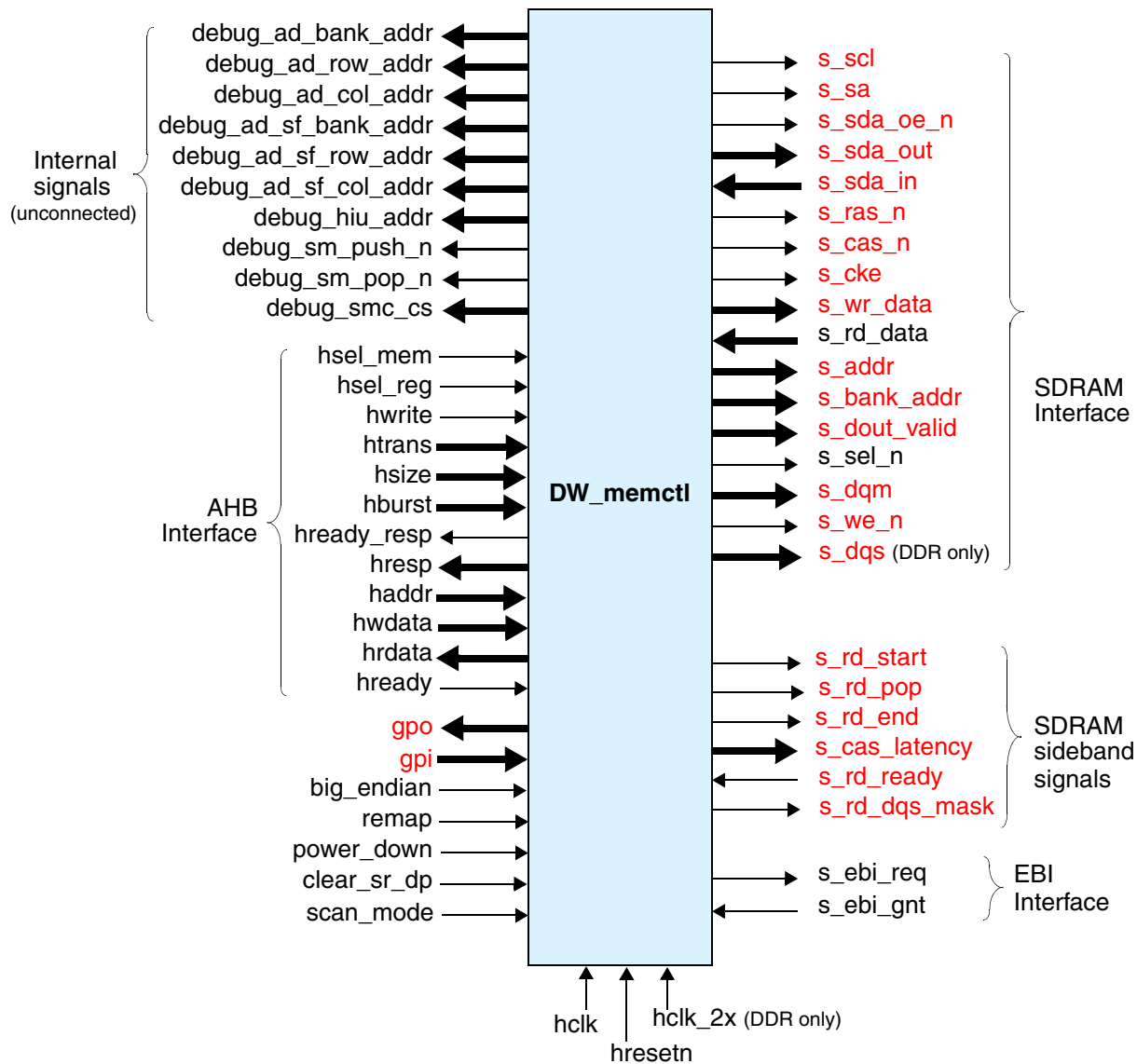


Figure 7-5 shows the DW_memctl as an SDRAM-only controller; refer to Table 7-2 on page 152 for I/O signal descriptions.

- DYNAMIC_RAM_TYPE < 4; ENABLE_STATIC = 0; EBI_INTERFACE=1
- ENABLE_ADDRBUS_SHARING=0; ENABLE_DATABUS_SHARING=0

Figure 7-5 DW_memctl Interface as an SDRAM Controller

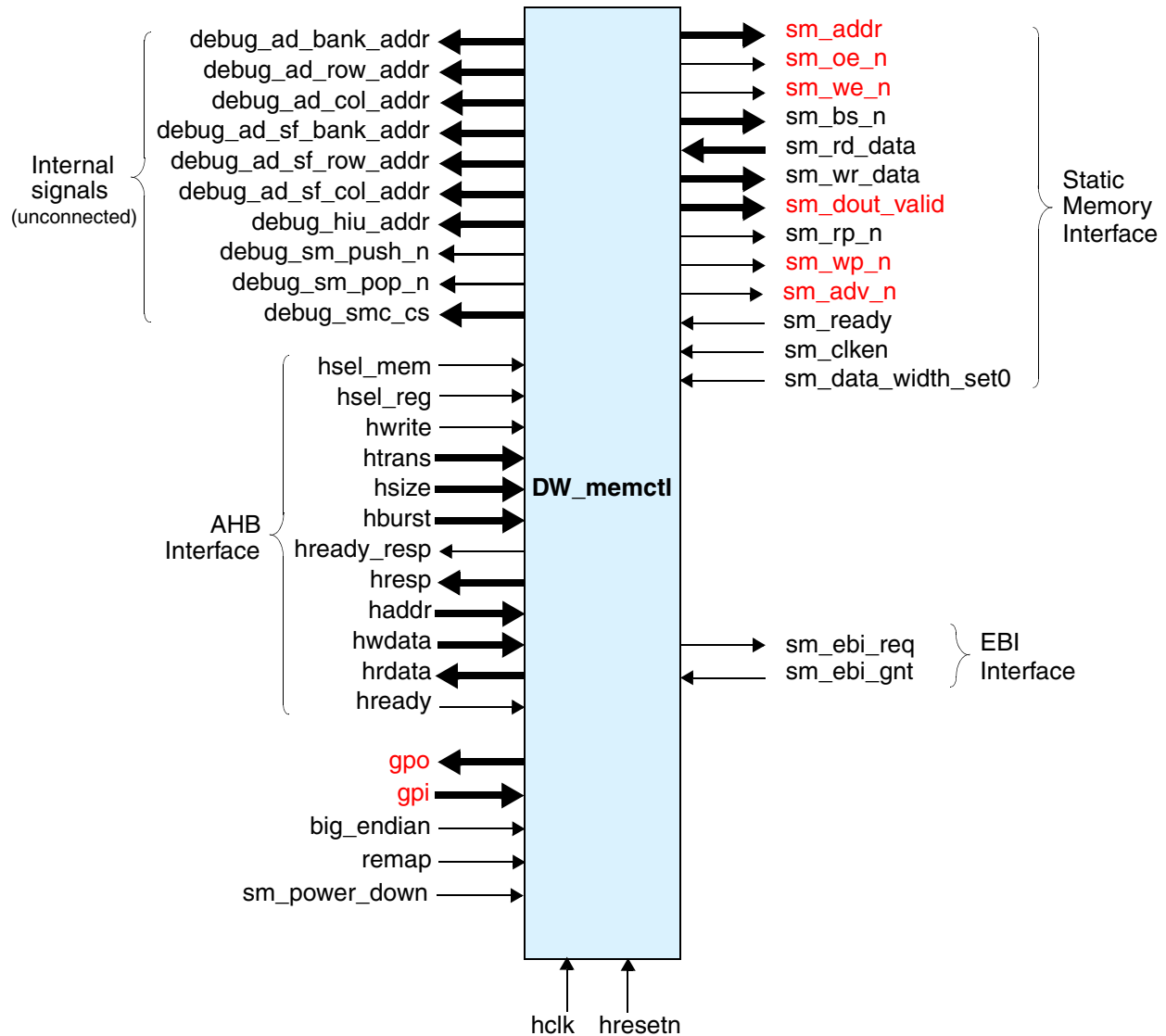


Signals in red indicate a registered input or output.

Figure 7-6 shows the DW_memctl as a Static-only memory controller; refer to Table 7-2 on page 152 for I/O signal descriptions.

- DYNAMIC_RAM_TYPE =4; ENABLE_STATIC=1; EBI_INTERFACE =1;
- ENABLE_ADDRBUS_SHARING=0;ENABLE_DATABUS_SHARING=0

Figure 7-6 DW_memctl Interface as a Static Memory Controller



Signals in red indicate a registered input or output.

7.2 DW_memctl Signal Descriptions

Table 7-2 identifies the signals that are associated with the DW_memctl.



Note

The Description column in Table 7-2 provides detailed information about each signal.

In the **Registered** field, a “Yes” indicates whether an I/O signal is directly connected to an internal register and nothing else. An I/O signal is also considered to be registered if the signal is connected to one or more inverters or buffers between the I/O port and internal register, but not connected to any logic that involves another signal.

The **Input/Output Delay** field provides the percentage of the clock cycle assumed to be used by logic outside this design. The given value is used to automatically define the default synthesis constraints for input/output delay. You can override these default values in the Specify Port Constraints activity in coreConsultant or coreAssembler.

Table 7-2 DW_memctl I/O Signals

Name	Width	I/O	Description
AHB Interface Signals			
hclk	1 bit	In	AHB bus interface clock. Active State: High. Active on rising edge. Registered: N/A Default Input Delay: N/A
hclk_2x	1 bit	In	2x AHB bus interface clock—needed for DDR-SDRAM only; used only inside unencrypted DW_memctl_miu_ddrwr.v file. Active State: High Registered: N/A Default Input Delay: N/A Dependencies: Both edges of hclk_2x are used in the design
hresetn	1 bit	In	AHB bus active-low reset pin; asynchronously asserted and synchronously removed. Active State: Low Registered: N/A Default Input Delay: 20%
hsel_mem	1 bit	In	AHB select signal for DW_memctl. Active State: High Synchronous to: hclk Registered: No Default Input Delay: 65%
hsel_reg	1 bit	In	AHB select signal for DW_memctl registers. Active State: High Synchronous to: hclk Registered: No Default Input Delay: 70%

Table 7-2 DW_memctl I/O Signals

Name	Width	I/O	Description
hwrite	1 bit	In	AHB bus interface: 0 – read 1 – write Active State: High Synchronous to: hclk Registered: No Default Input Delay: 50%
htrans	2 bit	In	AHB bus transfer type. Active State: High Synchronous to: hclk Registered: No Default Input Delay: 50%
hsize	3 bit	In	AHB bus transfer size. Active State: High Synchronous to: hclk Registered: No Default Input Delay: 50%
hburst	3 bit	In	AHB bus burst type. Active State: High Synchronous to: hclk Registered: No Default Input Delay: 50%
hready_resp	1 bit	Out	AHB bus data ready response. Active State: High Synchronous to: hclk Registered: No Default Output Delay: 70%
hready	1 bit	In	AHB bus data ready input. Active State: High Synchronous to: hclk Registered: No Default Input Delay: 50%
hresp	2 bit	Out	AHB bus transfer response. Active State: High Synchronous to: hclk Registered: No Default Output Delay: 75%

Table 7-2 DW_memctl I/O Signals

Name	Width	I/O	Description
haddr	See Description	In	AHB bus interface address bus. Width: H_ADDR_WIDTH Active State: High Synchronous to: hclk Registered: No Default Input Delay: 50%
hwdata	See Description	In	AHB bus interface write data. Width: H_DATA_WIDTH Active State: High Synchronous to: hclk Registered: No Default Input Delay: 70%
hrdata	See Description	Out	AHB bus interface read data. Width: H_DATA_WIDTH Active State: High Synchronous to: hclk Registered: No Default Output Delay: 70%
Miscellaneous Signals			
big_endian	1 bit	In	Endian AHB bus interface: 1 – Big Endian AHB bus interface 0 – Little Endian AHB bus interface Active State: High Synchronous to: hclk Registered: No Default Input Delay: 5%
remap	1 bit	In	Remap input for chip select0 and chip select1 logic; if 1, then address comparison is done with remap registers. Active State: High Synchronous to: hclk Registered: No Default Input Delay: 20%
gpo	8 bits	Out	General Purpose Outputs. Active State: High Synchronous to: hclk Registered: Yes Default Output Delay: 75%

Table 7-2 DW_memctl I/O Signals

Name	Width	I/O	Description
gpi	8 bits	In	General Purpose Inputs; Status pins from Flash could be connected to gpi pins and can be read by reading the SREFR register bits 31:24. Active State: High Synchronous to: hclk Registered: Yes Default Input Delay: 50%
power_down	1 bit	In	External power-down input for SDRAM; generated by external power management unit. SDRAM is put in power-down mode by either activating this signal or by programming power_down_mode bit of SDRAM Control Register (SCTLR). Active State: High Synchronous to: hclk Registered: No Default Input Delay: 20%
clear_sr_dp	1 bit	In	Clear self_refresh bit of SDRAM Control Register (SCTLR) to exit self-refresh mode of SDRAM; generated by external power management unit. Active State: High Synchronous to: hclk Registered: No Default Input Delay: 20%
sm_power_down	1 bit	In	External power-down input for FLASH memory; generated by external power management unit. FLASH is put in power-down mode by either activating this signal or by programming sm_rp_n bit of Static Memory Control Register (SMCTLR)). Active State: High Synchronous to: hclk Registered: No Default Input Delay: 20%
scan_mode	1 bit	In	Scan mode. This signal is used to bypass an internally generated asynchronous reset with hresetn in scan mode. This signal ensures that all flip-flops in the component are controllable and observable during scan testing, during which time the signal must be asserted. At all other times, it must be de-asserted. Active State: N/A Registered: No Synchronous to: hclk Default Input Delay: 2.5% for hclk; 5% for hclk_2x Dependencies: Must be asserted during scan testing.

Table 7-2 DW_memctl I/O Signals

Name	Width	I/O	Description
debug_ad_bank_addr	See Description	Out	Debug/internal testing; (unconnected). Width: MAX_S_BANK_ADDR_WIDTH-1:0 Active State: N/A Synchronous to: hclk Registered: No Default Output Delay: 50%
debug_ad_row_addr	See Description	Out	Debug/internal testing; (unconnected). Width: MAX_S_ADDR_WIDTH-1:0 Active State: N/A Synchronous to: hclk Registered: No Default Output Delay: 50%
debug_ad_col_addr	See Description	Out	Debug/internal testing; (unconnected). Width: MAX_S_ADDR_WIDTH-1:0 Active State: N/A Synchronous to: hclk Registered: No Default Output Delay: 50%
debug_ad_sf_bank_addr	See Description	Out	Debug/internal testing; (unconnected). Width: MAX_S_BANK_ADDR_WIDTH-1:0 Active State: N/A Synchronous to: hclk Registered: No Default Output Delay: 50%
debug_ad_sf_row_addr	See Description	Out	Debug/internal testing; (unconnected). Width: MAX_S_ADDR_WIDTH-1:0 Active State: N/A Synchronous to: hclk Registered: No Default Output Delay: 50%
debug_ad_sf_col_addr	See Description	Out	Debug/internal testing; (unconnected). Width: MAX_S_ADDR_WIDTH-1:0 Active State: N/A Synchronous to: hclk Registered: No Default Output Delay: 50%

Table 7-2 DW_memctl I/O Signals

Name	Width	I/O	Description
debug_hiu_addr	See Description	Out	Debug/internal testing; (unconnected). Width: H_ADDR_WIDTH-1:0 Active State: N/A Synchronous to: hclk Registered: No Default Output Delay: 50%
debug_sm_burst_done	1 bit	Out	Debug/internal testing; (unconnected). Active State: N/A Synchronous to: hclk Registered: No Default Output Delay: 50%
debug_sm_pop_n	1 bit	Out	Debug/internal testing; (unconnected). Active State: N/A Synchronous to: hclk Registered: No Default Output Delay: 50%
debug_sm_push_n	1 bit	Out	Debug/internal testing; (unconnected). Active State: N/A Synchronous to: hclk Registered: No Default Output Delay: 50%
debug_smc_cs	4 bits	Out	Debug/internal testing; (unconnected). Width: H_DATA_WIDTH Active State: N/A Synchronous to: hclk Registered: No Default Output Delay: 50%
SDRAM Serial Presence Detect EEPROM Interface Signals			
s_scl	1 bit	Out	SDRAM module serial-presence-detect clock. Active State: High Synchronous to: hclk Registered: Yes Default Output Delay: 75%
s_sa[2:0]	3 bits	Out	SDRAM module serial-presence-detect address. Active State: High Synchronous to: hclk Registered: Yes Default Output Delay: 75%

Table 7-2 DW_memctl I/O Signals

Name	Width	I/O	Description
s_sda_oe_n	1 bit	Out	SDRAM module active-low output enable for serial-presence-detect data. Active State: Low Synchronous to: hclk Registered: Yes Default Output Delay: 70%
s_sda_out	1 bit	Out	SDRAM module serial-presence-detect data out. Active State: High Synchronous to: hclk Registered: Yes Default Output Delay: 70%
s_sda_in	1 bit	In	SDRAM module serial-presence-detect data in. Active State: High Synchronous to: hclk Registered: Yes Default Input Delay: 50%
SDRAM Interface Signals			
s_ras_n	1 bit	Out	SDRAM row address select. Active State: Low Synchronous to: hclk Registered: Yes Default Output Delay: 75%
s_cas_n	1 bit	Out	SDRAM column address select. Active State: Low Synchronous to: hclk Registered: Yes Default Output Delay: 75%
s_cke	1 bit	Out	SDRAM clock enable. Active State: High Synchronous to: hclk Registered: Yes Default Output Delay: 75%

Table 7-2 DW_memctl I/O Signals

Name	Width	I/O	Description
s_rd_data	See Description	In	SDRAM read data from external SDRAM. Width: MAX_S_DATA_WIDTH for SDR-SDRAM and Mobile SDR-SDRAM MAX_S_DATA_WIDTH*2 for DDR-SDRAM and Mobile DDR-SDRAM Active State: High Synchronous to: hclk Registered: No Default Input Delay: 10%
s_wr_data	See Description	Out	SDRAM write data to external SDRAM; in DDR mode, is driven out by negative edge of hclk_2x to provide setup time with regard to s_dqs. Refer to DW_memctl_miu_ddrwr.v file for more details. Width: S_DATA_WIDTH Active State: High Synchronous to: hclk; hclk_2x in DDR Mode only Registered: Yes Default Output Delay: 75%
s_addr	See Description	Out	SDRAM address bus to external SDRAM (also see the m_precharge_bit pin details). Width: S_ADDR_WIDTH Active State: High Synchronous to: hclk Registered: Yes Default Output Delay: 75%
s_bank_addr	See Description	Out	SDRAM bank address to external SDRAM. Width: S_BANK_ADDR_WIDTH Active State: High Synchronous to: hclk Registered: Yes Default Output Delay: 75%
s_dout_valid	See Description	Out	SDRAM valid signal for write data to SDRAM; decides direction of data flow. All bits are identical; one bit per byte of data provided to improve drive strength, routing, and timing closure. Width: MAX_S_DATA_WIDTH/8 Active State: High Synchronous to: hclk Registered: Yes Default Output Delay: 75%

Table 7-2 DW_memctl I/O Signals

Name	Width	I/O	Description
s_sel_n	N_CS	Out	SDRAM or Static memory chip selects. Active State: Low Synchronous to: hclk Registered: No Default Output Delay: 75%
s_dqm	See Description	Out	SDRAM input mask for write access and output enable for read access; in DDR mode, is driven out by negative edge of hclk_2x to provide setup time with regard to s_dqs. Refer to DW_memctl_miu_ddrwr.v file for more details. Width: S_DATA_WIDTH/8 Active State: High Synchronous to: hclk; hclk_2x in DDR Mode only Registered: Yes Default Output Delay: 75%
s_we_n	1 bit	Out	SDRAM write enable. Active State: Low Synchronous to: hclk Registered: Yes Default Output Delay: 75%
s_dqs	NUM_DQS	Out	SDRAM data strobe driven out by positive edge of hclk_2x clock. Refer to DW_memctl_miu_ddrwr.v file for more details. Active State: High Synchronous to: hclk; ; hclk_2x in DDR Mode only Registered: Yes Default Output Delay: 70%; hclk_2x only
s_rd_ready	1 bit	In	SDRAM read data ready – sideband signal; s_rd_data is sampled after s_rd_ready goes active when “Ready Mode Read operation” is enabled. For more information on this signal, refer to “Typical Interfaces” on page 244. Active State: High Synchronous to: hclk Registered: Yes Default Input Delay: 20%

Table 7-2 DW_memctl I/O Signals

Name	Width	I/O	Description
s_rd_start	1 bit	Out	SDRAM read command issued – sideband signal. For more information on this signal, refer to “ Typical Interfaces ” on page 244. Active State: High Synchronous to: hclk Registered: Yes Default Output Delay: 50%
s_rd_pop	1 bit	Out	SDRAM read data pop – sideband signal. For more information on this signal, refer to “ Typical Interfaces ” on page 244. Active State: High Synchronous to: hclk Registered: Yes Default Output Delay: 50%
s_rd_end	1 bit	Out	SDRAM last read data in a burst – sideband signal. For more information on this signal, refer to “ Typical Interfaces ” on page 244. Active State: High Synchronous to: hclk Registered: Yes Default Output Delay: 50%
s_cas_latency	3 bits	Out	SDRAM CAS latency – sideband signal. Active State: High Synchronous to: hclk Registered: Yes Default Output Delay: 75%
s_rd_dqs_mask	1 bit	Out	SDRAM read DQS mask – sideband signal. For more information on this signal, refer to “ Typical Interfaces ” on page 244. Active State: High Synchronous to: hclk Registered: Yes Default Output Delay: 70%; hclk_2x only
s_read_pipe	3 bits	Out	SDRAM sideband signal. Holds the value of SCTL[8:6], read_pipe bitfield. Active State: High Synchronous to: hclk Registered: Yes Default Output Delay: 50%

Table 7-2 DW_memctl I/O Signals

Name	Width	I/O	Description
EBI Signals			
s_ebi_req	1 bit	Out	SDRAM controller request to external bus interface. Active State: High Synchronous to: hclk Registered: No Default Output Delay: 40%
s_ebi_gnt	1 bit	In	SDRAM controller grant from external bus interface. Active State: High Synchronous to: hclk Registered: No Default Input Delay: 40%
sm_ebi_req	1 bit	Out	Static memory controller request to external bus interface. Active State: High Synchronous to: hclk Registered: No Default Output Delay: 40%
sm_ebi_gnt	1 bit	In	Static memory controller grant from external bus interface. Active State: High Synchronous to: hclk Registered: No Default Input Delay: 40%
Static Memory Interface Signals			
sm_addr	See Description	Out	Static memory address bus. Width: MAX_SM_ADDR_WIDTH Active State: High Synchronous to: hclk Registered: Yes Default Output Delay: 70%
sm_oe_n	1 bit	Out	Static memory output enable. Active State: Low Synchronous to: hclk Registered: Yes Default Output Delay: 60%
sm_we_n	1 bit	Out	Static memory write enable. Active State: Low Synchronous to: hclk Registered: Yes Default Output Delay: 60%

Table 7-2 DW_memctl I/O Signals

Name	Width	I/O	Description
sm_bs_n	See Description	Out	Static memory byte select. Width: MAX_SM_DATA_WIDTH/8 Active State: Low Synchronous to: hclk Registered: No Default Output Delay: 60%
sm_rd_data	See Description	In	Static memory read data from external memory. Width: MAX_SM_DATA_WIDTH Active State: High Synchronous to: hclk Registered: No Default Input Delay: 60%
sm_wr_data	See Description	Out	Static memory write data to external memory. Width: MAX_SM_DATA_WIDTH Active State: High Synchronous to: hclk Registered: No Default Output Delay: 50%
sm_dout_valid	See Description	Out	Static memory output data valid; decides direction of data flow. All bits are identical; one bit per byte of data provided to improve drive strength, routing, and timing closure. Width: MAX_SM_DATA_WIDTH/8 Active State: High Synchronous to: hclk Registered: Yes Default Output Delay: 60%
sm_rp_n	1 bit	Out	Static memory FLASH reset/power-down. Active State: Low Synchronous to: hclk Registered: No Default Output Delay: 50%
sm_wp_n	3 bits	Out	Static memory FLASH write protection. Active State: Low Synchronous to: hclk Registered: Yes Default Output Delay: 60%

Table 7-2 DW_memctl I/O Signals

Name	Width	I/O	Description
sm_ready	1 bit	In	Static memory data ready; used for non-memory type device interface. Active State: High Synchronous to: hclk Registered: No Default Input Delay: 50%
sm_clken	1 bit	In	Static memory controller clock enable; used for synchronous SRAM devices working on a sub-multiple AHB hclk. Active State: High Synchronous to: hclk Registered: No Default Input Delay: 20%
sm_adv_n	1 bit	Out	Static memory controller address valid Active State: Low Synchronous to: hclk Registered: Yes Default Output Delay: 50%
sm_data_width_set0	3 bits	In	Configures Set0 Static memory data bus width on reset. When the clock after hresetn goes high, this value is loaded into SMCTLR[9:7] and the register value is used in the core, provided HARD_WIRE_STATIC_PARAMETERS = 0. When HARD_WIRE_STATIC_PARAMETERS = 1, the pin is directly used in the core. 000 – 16 bits 001 – 32 bits 010 – 64 bits 011 – 128 bits 100 – 8 bits Active State: High Synchronous to: hclk Registered: No Default Input Delay: 5%

Table 7-2 DW_memctl I/O Signals

Name	Width	I/O	Description
Shared Address Mode Pins			
m_addr	See Description	Out	Shared address pins for SDRAM and Static memory; pin active only if compile-time parameter ENABLE_ADDRBUS_SHARING is set. For more information on this signal, refer to “Data Connections” on page 226. Width: M_ADDR_WIDTH Active State: High Synchronous to: hclk Registered: No Default Output Delay: 70%
m_precharge_bit	1 bit	Out	SDRAM address bit 10 or 8 in shared mode; to support refresh even when Static memory is accessed; active only if compile-time ENABLE_ADDRBUS_SHARING parameter is set. Some DDR-SDRAMs use A8 during precharge instead of A10. The A8_FOR_PRECHARGE parameter defines whether A8 to A10 is driven on m_precharge_bit. For more information on this signal, refer to “Data Connections” on page 226. Active State: High Synchronous to: hclk Registered: Yes Default Output Delay: 75%
Shared Data Mode Pins			
m_wr_data	See Description	Out	Shared write data for SDRAM and Static memory; active only if compile-time parameter ENABLE_DATABUS_SHARING is set. Width: M_DATA_WIDTH Active State: High Synchronous to: hclk Registered: No Default Output Delay: 50%
m_rd_data	See Description	In	Shared read data for SDRAM and Static memory; active only if compile-time parameter ENABLE_DATABUS_SHARING is set. Width: M_DATA_WIDTH Active State: High Synchronous to: hclk Registered: No Default Input Delay: 10%

Table 7-2 DW_memctl I/O Signals

Name	Width	I/O	Description
m_dout_valid	See Description	Out	Shared memory data bus data enable output; valid only if compile-time parameter ENABLE_DATABUS_SHARING is set. All bits are identical; one bit per byte of data provided to improve drive strength, routing, and timing closure. Width: M_DATA_WIDTH/8 Active State: High Synchronous to: hclk Registered: No Default Output Delay: 60%

8

Registers

This chapter describes the programmable registers of the DW_memctl.

All registers are 32 bits wide; each non-implemented bit is reserved for future use. Static memory timing registers are selectively enabled, depending on the ENABLE_STATIC compile-time parameter. Unused bits should always be programmed to 0. In the case of SDRAM registers, they are enabled depending on the SDRAM you have chosen – SDR-SDRAM, or DDR-SDRAM.

All DW_memctl registers that are software-accessible and supported are documented in this chapter. Any addresses not listed in this chapter, but still accessible within the DW_memctl register address space, are not supported and should not be accessed by software.

Compile-time parameters set default values for corresponding registers; you can change the values of these registers after reset. However, for all except the control registers, it is recommended that you change the register values only once after reset and not between memory cycles.

**Note**

The DW_memctl supports only 32-bit register read/writes, which is true even if the AHB data width is 128 or 64 bits wide.

8.1 Register Memory Map

Table 8-1 lists the memory map for the DW_memctl.

Table 8-1 Memory Map

Register Name	Address	R/W	Width	Description
SDR-SDRAM, Mobile SDR-SDRAM, Mobile DDR-SDRAM, and DDR-SDRAM Registers				
SCONR	32'hxxxx_xx00	R/W	32 bits	SDRAM configuration register
STMG0R	32'hxxxx_xx04	R/W	32 bits	SDRAM timing register0
STMG1R	32'hxxxx_xx08			SDRAM timing register1
SCTLR	32'hxxxx_xx0C	R/W	32 bits	SDRAM control register
SREFR	32'hxxxx_xx10	R/W	32 bits	SDRAM refresh register
EXN_MODE_REG	32'hxxxx_xxAC	R/W	32 bits	DDR/Mobile-SDR/Mobile-DDR extension mode register
SCSLR0_LOW	32'hxxxx_xx14	R/W	32 bits	Chip select register0 (lower 32bits)
SCSLR1_LOW	32'hxxxx_xx18			Chip select register1 (lower 32bits)
SCSLR2_LOW	32'hxxxx_xx1C			Chip select register2 (lower 32bits)
SCSLR3_LOW	32'hxxxx_xx20			Chip select register3 (lower 32bits)
SCSLR4_LOW	32'hxxxx_xx24			Chip select register4 (lower 32bits)
SCSLR5_LOW	32'hxxxx_xx28			Chip select register5 (lower 32bits)
SCSLR6_LOW	32'hxxxx_xx2C			Chip select register6 (lower 32bits)
SCSLR7_LOW	32'hxxxx_xx30			Chip select register7 (lower 32bits)
SMSKR0	32'hxxxx_xx54	R/W	32 bits	Mask register 0
SMSKR1	32'hxxxx_xx58			Mask register 1
SMSKR2	32'hxxxx_xx5C			Mask register 2
SMSKR3	32'hxxxx_xx60			Mask register 3
SMSKR4	32'hxxxx_xx64			Mask register 4
SMSKR5	32'hxxxx_xx68			Mask register 5
SMSKR6	32'hxxxx_xx6C			Mask register 6
SMSKR7	32'hxxxx_xx70			Mask register 7
CSALIAS0_LOW	32'hxxxx_xx74	R/W	32 bits	Alias register for chip select0 (lower 32 bits)
CSALIAS1_LOW	32'hxxxx_xx78			Alias register for chip select1 (lower 32 bits)
CSREMAP0_LOW	32'hxxxx_xx84	R/W	32 bits	Remap register for chip select0 (lower 32 bits)
CSREMAP1_LOW	32'hxxxx_xx88			Remap register for chip select0 (lower 32 bits)

Note: During register access (when hreg_sel is active), the memory controller uses only address bits haddr[7:0]; other address bits are "don't care." The register base address depends on how the AHB address decoder maps hsel_reg. For example, if the AHB decoder maps to hsel_reg a 16K area from 32'h4560_2000 to 32'h4560_2fff, then the system address – 32'h4560_2000, 32'h4560_2100, and so on (32'h4560_2x00) – is aliased to the DW_memctl register 0 (SCONR).

Table 8-1 Memory Map

Register Name	Address	R/W	Width	Description
SRAMs and NOR-FLASH Registers				
SMTMGR_SET0	32'hxxxx_xx94	R/W	32 bits	Static memory timing register Set0
SMTMGR_SET1	32'hxxxx_xx98			Static memory timing register Set1
SMTMGR_SET2	32'hxxxx_xx9C			Static memory timing register Set2
FLASH_TRPDR	32'hxxxx_xxA0	R/W	32 bits	FLASH memory tRPD timing register
SMCTLR	32'hxxxx_xxA4	R/W	32 bits	Static memory control register
Component Identification and Configuration Registers				
COMP_TYPE	32'hxxxx_xxFC	R	32 bits	Component type identification register
COMP_VERSION	32'hxxxx_xxF8	R	32 bits	Component version identification register
COMP_PARAMS_1	32'hxxxx_xxF4	R	32 bits	Parameter configuration register
COMP_PARAMS_2	32'hxxxx_xxF0	R	32 bits	Parameter configuration register

Note: During register access (when hreg_sel is active), the memory controller uses only address bits haddr[7:0]; other address bits are “don't care.” The register base address depends on how the AHB address decoder maps hsel_reg. For example, if the AHB decoder maps to hsel_reg a 16K area from 32'h4560_2000 to 32'h4560_2fff, then the system address – 32'h4560_2000, 32'h4560_2100, and so on (32'h4560_2x00) – is aliased to the DW_memctl register 0 (SCONR).

8.2 Register and Field Descriptions

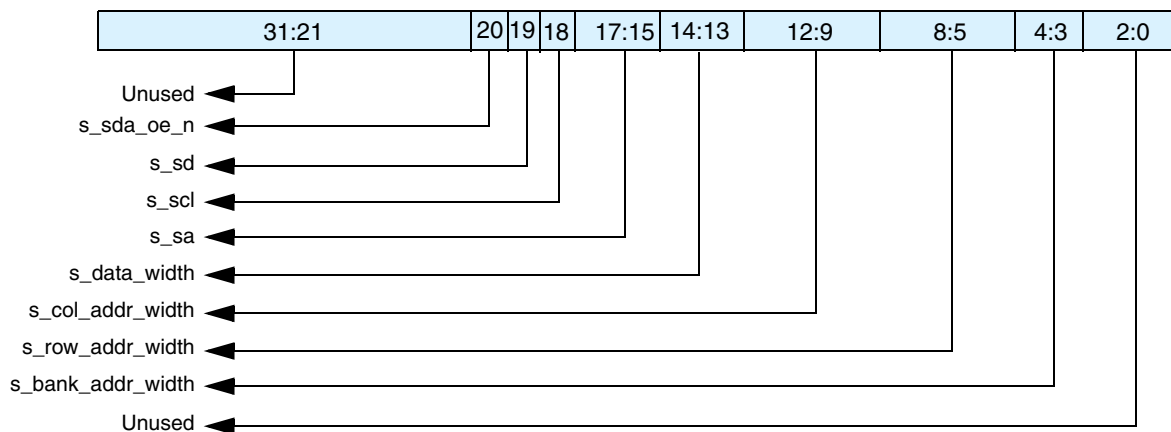
The following sections contain the memory diagrams and field descriptions for the DW_memctl registers.

8.2.1 Configuration Register

The values programmed for different parameters in this register should match the corresponding compile-time parameters that have already been specified. The programmed values should always be less than or equal to the compile-time parameter values. For example, if the S_ROW_ADDR_WIDTH compile-time parameter for the row address width was chosen as 11, then you cannot program it for more than 11 rows.

8.2.1.1 SCONR

- **Name:** SDRAM Config Register
- **Size:** 32 bits
- **Address:** 32'hxxx_x00
- **Read/Write access:** read/write



Bits	Name	R/W	Description
31:21	Unused		
20	s_sda_oe_n	R/W	Output enable for bi-directional data pin for I2C serial presence detect (SPD) logic 1 – corresponds to data written into bit 19 0 – programs for data reads Default Value: 1
19	s_sd	R/W	Bi-directional data for serial presence detect (SPD) logic; data written into bit goes in as data for SPD. During reads to this register, bit represents data read back from the SPD logic Default Value: 1
18	s_scl	R/W	Clock for serial presence detect logic Default Value: 1
17:15	s_sa	R/W	Serial presence detect address bits Default Value: 0

Bits	Name	R/W	Description
14:13	s_data_width	R/W	Specifies SDRAM data width in bits; becomes read-only if HARD_WIRE_SDRAM_PARAMETERS is set to 1. For DDR-SDRAM, bits represent: 00 – 8 bits 01 – 16 bits 10 – 32 bits 11 – 64 Bits For SDR-SDRAM, bits represent: 00 – 16 bits 01 – 32 bits 10 – 64 bits 11 – 128 bits Default Value: S_DATA_WIDTH encoded to 2 bits
12:9	s_col_addr_width	R/W	Number of address bits for column address; becomes read-only if HARD_WIRE_SDRAM_PARAMETERS is set to 1. 15 – reserved 7-14 – correspond to 8-15 bits 0-6 – reserved Value should be less than or equal to MAX_S_ROW_ADDR_WIDTH compile-time parameter. Default Value: S_COL_ADDR_WIDTH-1
8:5	s_row_addr_width	R/W	Number of address bits for row address; becomes read-only if HARD_WIRE_SDRAM_PARAMETERS is set to 1. 10-15 – correspond to 11-16 bits 0-10 – reserved Value should be less than or equal to MAX_S_ROW_ADDR_WIDTH compile-time parameter. Default Value: S_ROW_ADDR_WIDTH-1
4:3	s_bank_addr_width	R/W	Number of bank address bits; becomes read-only if HARD_WIRE_SDRAM_PARAMETERS is set to 1. Values of 0-3 correspond to 1-4 bits, and therefore select 2-16 banks Value should be less than or equal to MAX_S_BANK_ADDR_WIDTH compile-time parameter. Default Value: S_BANK_ADDR_WIDTH-1
2:0	Unused		

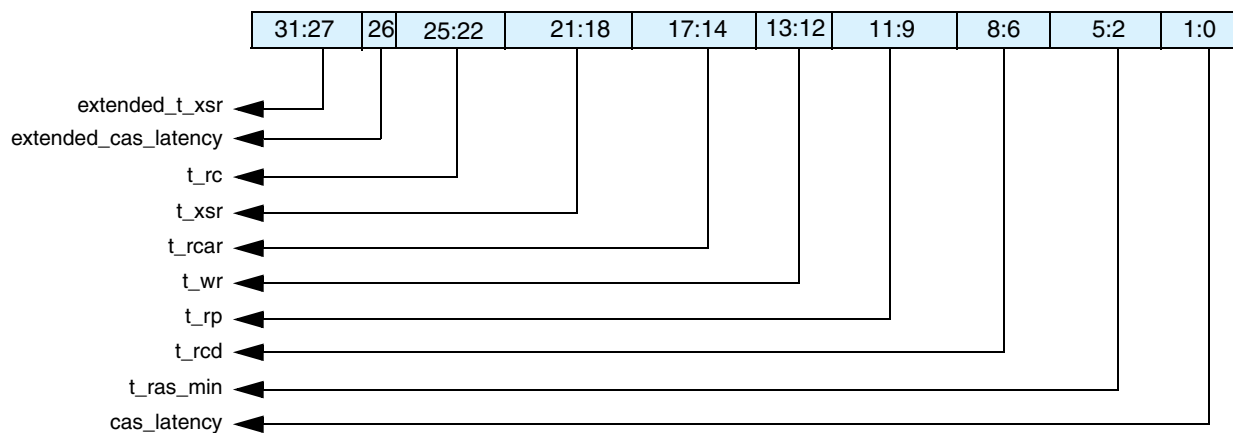
8.2.2 SDRAM Timing Registers

The STMG0R and STMG1R registers hold the SDRAM timing parameters; compile-time parameters set default values of these registers.

The DW_memctl uses the CAS latency value during the initialization sequence in order to program the mode register of the SDRAM. The user can also specifically force DW_memctl to do a mode register update by programming the set_mode_reg bit (bit 9 of SCTL0R). If you want to change the value of CAS latency during normal operation, you should first program the STMG0R timing register, and then program bit 9 of SCTL0R to 1. DW_memctl will reset this bit once it has updated the mode register.

8.2.2.1 STMG0R

- **Name:** SDRAM Timing Register0
- **Size:** 32 bits
- **Address:** 32'hxxxx_xx04
- **Read/write access:** read/write, unless HARD_WIRE_SDRAM_PARAMETERS is set to 1; then read-only.



Bits	Name	R/W	Description
25:22	t_rc	R/W*	Active-to-active command period; values of 0-15 correspond to t_rc of 1-16 clocks. Default Value: $T_{RC} - 1$
31:27	extended_t_xsr	R/W*	Exit self-refresh to active or auto-refresh command time; minimum time controller should wait after taking SDRAM out of self-refresh mode before issuing any active or auto-refresh commands; values 0-511 correspond to t_xsr of 1-512 clocks Default Value: $T_{XSR} - 1$
21:18	t_xsr		
17:14	t_rcar	R/W*	Auto-refresh period; minimum time between two auto-refresh commands; values 0-15 correspond to t_rcar of 1-16 clocks. Default Value: $T_{RCAR} - 1$

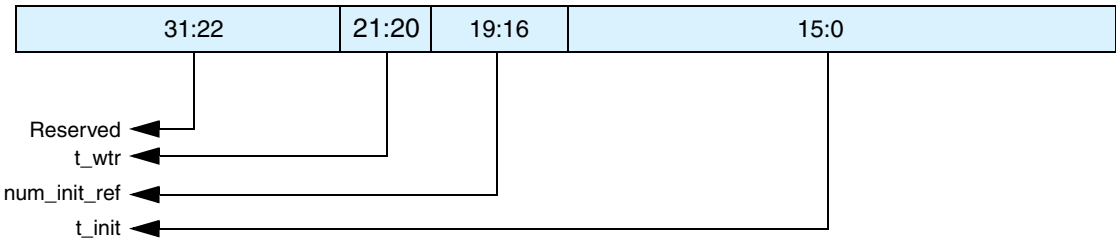
* Read-only if HARD_WIRE_SDRAM_PARAMETERS is set to 1

Bits	Name	R/W	Description
13:12	t_wr	R/W*	For writes, delay from last data in to next precharge command; values 0-3 correspond to t_wr of 1-4 clocks Default Value: <i>T_WR</i> - 1
11:9	t_rp	R/W*	Precharge period; values of 0-7 correspond to t_rp of 1-8 clocks Default Value: <i>T_RP</i> - 1
8:6	t_rcd	R/W*	Minimum delay between active and read/write commands; values 0-7 correspond to t_rcd values of 1-8 clocks Default Value: <i>T_RCD</i> - 1
5:2	t_ras_min	R/W*	Minimum delay between active and precharge commands; values of 0-15 correspond to T_RAS_MIN of 1-16 clocks Default Value: <i>T_RAS_MIN</i> - 1
26	extended_cas_latency	R/W*	Delay in clock cycles between read command and availability of first data
1:0	cas_latency		0 – 1 clock 1 – 2 clocks 2 – 3 clocks 3 – 4 clocks 4 – 1.5 clocks (DDR only) 5 – 2.5 clocks (DDR only) 6, 7 – reserved Default Value: <i>CAS_LATENCY</i> -1

* Read-only if HARD_WIRE_SDRAM_PARAMETERS is set to 1

8.2.2.2 **STMG1R**

- **Name:** SDRAM Timing Register1
- **Size:** 32 bits
- **Address:** 32'hxxxx_xx08
- **Read/write access:** read/write, unless HARD_WIRE_SDRAM_PARAMETERS is set to 1; then read-only



Bits	Name	R/W	Description
31:22	Reserved		
21:20	t_wtr	R/W*	Internal write-to-read delay for DDR-SDRAMs; values of 0-3 represent 1- 4 clocks delay Default Value: <i>T_WTR</i> - 1
19:16	num_init_ref	R/W*	Number of auto-refreshes during initialization; values 0-15 correspond to 1-16 auto-refreshes Default Value: <i>NUM_INIT_REF</i> - 1
15:0	t_init	R/W*	Number of clock cycles to hold SDRAM inputs stable after power up, before issuing any commands. Default Value: <i>T_INIT</i>

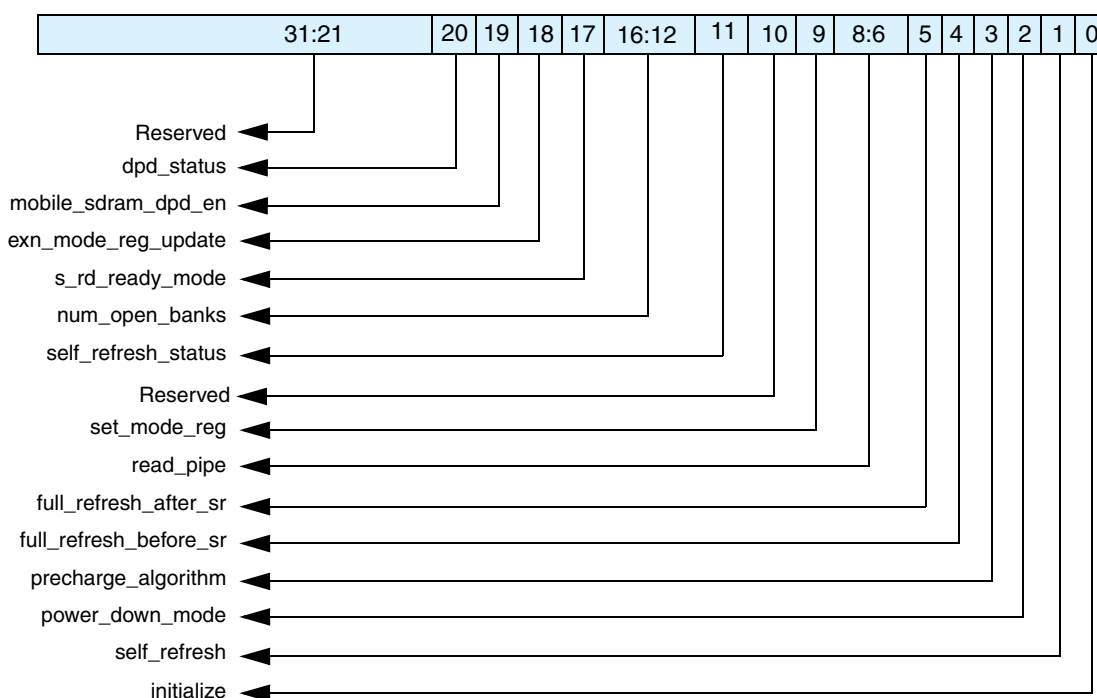
* Read-only if HARD_WIRE_SDRAM_PARAMETERS is set to 1

8.2.3 SDRAM Control Registers

You can program SDRAM control registers at any time after power-up. However, the SDRAM controller does not poll the registers until the SDRAM controller finishes current and pending SDRAM accesses in the write FIFO.

8.2.3.1 SCTLr

- **Name:** SDRAM Control Register
- **Size:** 32 bits
- **Address:** 32'hxxxx_xx0C
- **Read/write access:** read/write, unless HARD_WIRE_SDRAM_PARAMETERS is set to 1; then read-only



Bits	Name	R/W	Description
31:21	Reserved		Reserved – must be set to all 0s at all times
20	dpd_status	R	Status of deep power-down for mobile SDRAM. 0 – Mobile SDRAM not in deep power-down 1 – Mobile SDRAM in deep power-down Default Value: 0
** Read-only if HARD_WIRE_SDRAM_PARAMETERS is set to 1			

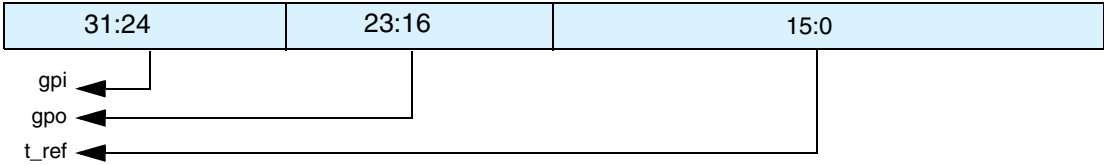
Bits	Name	R/W	Description
19	mobile_sdram_dpd_en	R/W	Mobile SDRAM deep power-down enable. Determines the type of SDRAM power-down sequence. Power-down mode is entered or exited by bit 2. Currently only supported for Mobile DDR-SDRAM devices. Should only be used with Mobile DDR-SDRAM devices that support deep power-down mode. 0 – standard power-down 1 – deep power-down Default Value: 0
18	exn_mode_reg_update	R/W	Commands controller to update extended-mode register; once mode register update is done, controller automatically clears bit Default Value: 0
17	s_rd_ready_mode	R/W**	SDRAM read-data-ready mode; set to 1, indicates SDRAM read data is sampled after s_rd_ready goes active Default Value: <i>S_READY_VALID</i>
16:12	num_open_banks	R/W**	Number of SDRAM internal banks to be open at any time; values of 0-15 correspond to 1-16 banks open. Default Value: <i>OPEN_BANKS-1</i>
11	self_refresh_status	R	Read only. When “1,” indicates SDRAM is in self-refresh mode. When “self_refresh” bit (bit 1 of SCTL) is set, it may take some time before SDRAM is put into self-refresh mode, depending on whether all rows or one row are refreshed before entering self-refresh mode defined by full_refresh_before_sr bit Before gating clock in self-refresh mode, ensure this bit is set Default Value: 0
10	Reserved		
9	set_mode_reg	R/W	Set to 1, forces controller to do update of SDRAM mode register; bit is cleared by controller once it has finished mode register update Default Value: 0
8:6	read_pipe	R/W	Indicates number of registers inserted in read data path for SDRAM in order to correctly latch data; values 0-7 indicate 0-7 registers Default Value: <i>READ_PIPE</i>
5	full_refresh_after_sr	R/W	Controls number of refreshes done by DW_memctl after SDRAM is taken out of self-refresh mode: 1 – Refresh all rows before entering self-refresh mode 0 – Refresh just 1 row before entering self-refresh mode Default Value: 0

** Read-only if HARD_WIRE_SDRAM_PARAMETERS is set to 1

Bits	Name	R/W	Description
4	full_refresh_before_sr	R/W	Controls number of refreshes done by DW_memctl before putting SDRAM into self-refresh mode: 1 – Refresh all rows before entering self-refresh mode 0 – Refresh just one row before entering self-refresh mode Default Value: 0
3	precharge_algorithm	R/W	Determines when row is precharged: 0 – Immediate precharge; row precharged at end of read/write operation 1 – Delayed precharge; row kept open after read/write operations Default Value: 1
2	power_down_mode	R/W	Forces DW_memctl to put SDRAM in power-down mode; bit 19 determines the type of power-down mode requested Default Value: 0
1	self_refresh	R/W	Forces DW_memctl to put SDRAM in self-refresh mode. Bit can be cleared by writing to this bit or by clear_sr_dp pin, generated by external power management unit Default Value: 0
0	initialize	R/W	Forces DW_memctl to initialize SDRAM; bit reset to 0 by DW_memctl once initialization sequence is complete Default Value: 1
** Read-only if HARD_WIRE_SDRAM_PARAMETERS is set to 1			

8.2.3.2 SREFR

- **Name:** SDRAM Refresh Interval Register
- **Size:** 32 bits
- **Address:** 32'hxxxx_xx10
- **Read/write access:** read/write



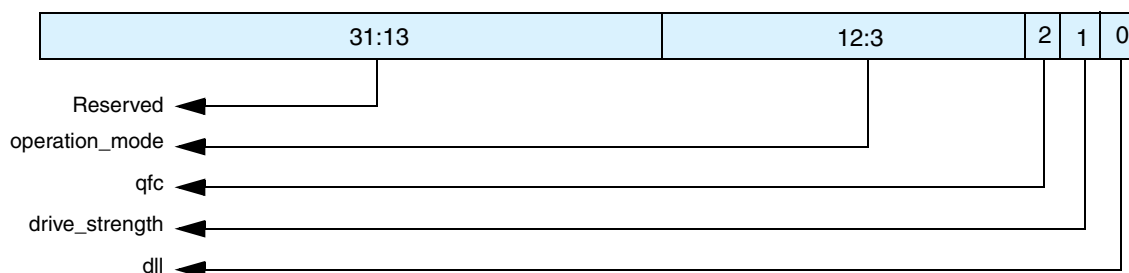
Bits	Name	R/W	Description
31:24	gpi	R	General purpose inputs; directly connected to gpi. Default Value: None
23:16	gpo	R/W	General purpose output signals; directly connected to gpo Default Value: 0
15:0	t_ref	R/W	Number of clock cycles between consecutive refresh cycles; for details on programming this register, refer to “Auto-Refresh” on page 63 Default Value: <i>T_REF</i>

8.2.3.3 EXN_MODE_REG

- **Name:** Extended Mode Register
- **Size:** 32 bits
- **Address:** 32'hxxxx_xxAC
- **Read/write access:** read/write, unless HARD_WIRE_SDRAM_PARAMETERS is set to 1; then read-only

This register holds the value for the Extended Mode register that is shared between a DDR-SDRAM, Mobile DDR-SDRAM, and Mobile SDR-SDRAM; default value is EXTENDED_MODE_REG.

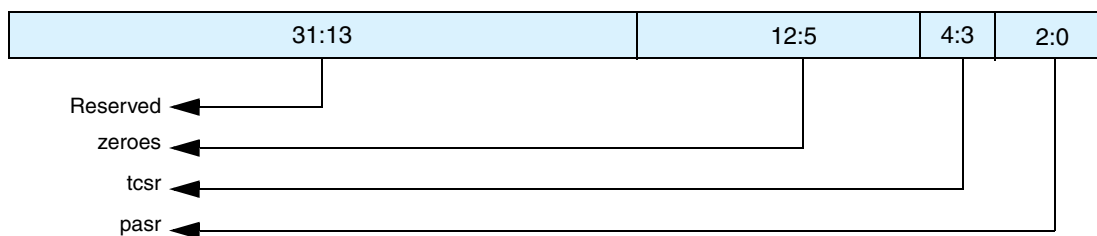
This figure illustrates the format of the register for DDR-SDRAM.



Bits	Name	R/W	Description
31:13	Reserved		
12:3	operation_mode	R/W*	0 – normal operation Others – reserved Default Value: See above register description
2	qfc	R/W*	0 – disable 1 – enable (optional) Default Value: See above register description
1	drive_strength	R/W*	0 – normal 1 – weak (optional) Default Value: See above register description
0	dll	R/W*	0 – DLL enable 1 – DLL disable NOTE: This value is ignored during DDR-SDRAM initialization and assumed to be 0 (DLL enabled), since DDR-SDRAM requires the DLL to be enabled during initialization. For more information about DDR-SDRAM initialization, refer to Figure 4-16 on page 68. Default Value: See above register description

* Read-only if HARD_WIRE_SDRAM_PARAMETERS is set to 1

This figure illustrates the format of the Extended Mode Register (EXN_MODE_REG) for Mobile DDR-SDRAMs and Mobile SDR-SDRAMs. The fields on those registers are device-dependent; some Mobile SDRAM devices might not support all of the field values.



Bits	Name	R/W	Description
31:13	Reserved		
12:7	zeroes	R/W	Always set to zero Default Value: 0
6:5	ds	R/W	Driver strength of SDRAM output drivers. 0 - Full 1 - 1/2 2 - 1/4 (optional) 3 - 1/8 (optional) Default Value: 0
4:3	tcsr	R/W	Maximum case temperature 0 – 70 1 – 45 2 – 15 3 – 85 Default Value: 0
2:0	pasr	R/W	Self refresh coverage 0 – 4 banks 1 – 2 banks 2 – 1 bank 5 – 1/2 bank 6 – 1/4 bank Others – reserved Default Value: 0

8.2.4 Address Decoder Registers

The DW_memctl can have a maximum of sixteen 32-bit registers for the base address value that corresponds to the chip selects. The maximum number of chip select registers depends on the value of the N_CS compile-time parameter and the AHB address width, H_ADDR_WIDTH.

Two 32-bit registers are dedicated for each chip select, as long as the AHB address width is greater than 32. If the AHB address width equals 32, then only one chip-select register corresponds to each chip select.

Default values for chip-select registers are specified as compile-time parameters, which enables you to later change a default value by programming it. The DW_memctl uses mask registers that specify the size of the memory connected to each chip select. The DW_memctl also supports aliasing and remapping, but these features are available only for chip select0 and chip select1.

8.2.5 Chip Select Registers (SCSLR_n)

Chip select registers are optional. The number of chip-select registers in the memory controller depends on the N_CS and H_ADDR_WIDTH compile-time parameters.

Each chip select can have up to two chip-select registers, depending on the width of the AHB address. For chip-select generation, only bits H_ADDR_WIDTH-1:16 are used for the chip-select registers, since the DW_memctl supports a minimum memory size of 64KB. If you choose to have only one chip select (N_CS=1), then there will be no chip-select registers; all memory accesses will be to chip select0.

The LSB bits of the chip-select registers are masked out, depending on the memory size controlled by the chip select. For example, if the memory size is 64KB, then bits 15:0 are not used for chip select generation; however, if the memory size is 2MB, then bits 20:0 of the chip select register are not used.

8.2.6 Address Aliasing and Remapping

The DW_memctl supports address aliasing and remapping, which can be done only on chip select0 and chip select1.

When aliasing is enabled, the chip select becomes active when the AHB address matches either the CHIP_SELECT_n_BASE_ADDRESS or the CHIP_SELECT_n_ALIAS_ADDRESS. When remapping is enabled, the chip select becomes active when the remap pin is "1" and the AHB address matches the CHIP_SELECT_n_REMAP_ADDRESS, or when the remap pin is "0" and the AHB address matches the CHIP_SELECT_n_BASE_ADDRESS.

ALIAS REGISTER0 (CSALIAS0_LOW) controls the aliasing for chipselect0. ALIAS REGISTER1 (CSALIAS1_LOW) controls the aliasing for chip select1.

You can selectively enable/disable the DW_memctl aliasing feature by setting the CHIP_SEL_n_ALIAS_ENABLE compile-time parameters. The CHIP_SEL0_ALIAS_ENABLE parameter controls aliasing for chipselect0, and CHIP_SEL1_ALIAS_ENABLE controls aliasing for chip select1. You can statically or dynamically control the values of the aliasing registers. By default, aliasing and remapping are disabled.

The CHIP_SELECT_n_ALIAS_ADDRESS and CHIP_SELECT_n_ALIAS_ADDRESS compile-time parameters specify the default values for the aliasing registers. You can dynamically change the values by programming the registers.

The remapping register CSREMAP0_LOW controls remapping for chip select0, while register CSREMAP1_LOW controls remapping for chip select1. As with the aliasing feature, you can selectively enable/disable the remapping feature by setting the `CHIP_SEL $n$ _REMAP_ENABLE` compile-time parameters (`CHIP_SEL0_REMAP_ENABLE` for chipselect0 and `CHIP_SEL1_REMAP_ENABLE` for chipselect1).

You can statically or dynamically control the values of the remap registers. The `CHIP_SELECT0_REMAP_ADDRESS` and `CHIP_SELECT1_REMAP_ADDRESS` compile-time parameters control the static timing values for these registers.

The remap input signal tells the DW_memctl when to switch to remap mode.

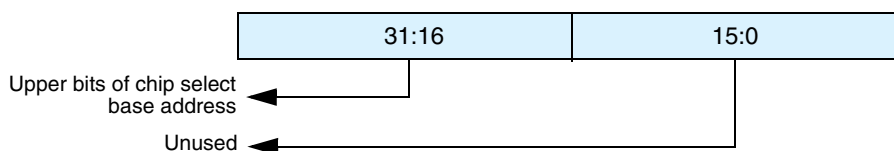
If aliasing and remapping are enabled and you do not intend to use them, then you can program the remap and alias registers to the same values as the corresponding chip select register values.

8.2.7 Address Register Details

The following table lists the relationship between `N_CS` and the number of chip select registers.

<code>N_CS</code>	Chip select registers available
1	0
$2 \leq n \leq 8$	n
	$2n$

The following illustrates the format of the 32 bits of the chip select register (SCSLRx0).

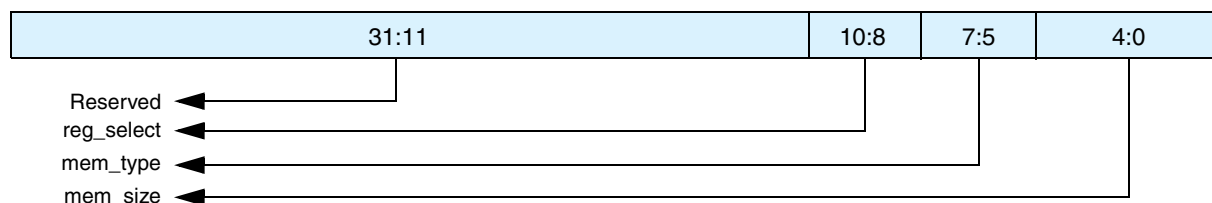


You should program the $SCSLR_n$ registers with the base addresses of the memory blocks you intend to use, which are listed below; these registers become read-only if $HARD_WIRE_CIPSELECT_PARAMETRS$ is set to 1.

Address	Name	Description
32'hxxx_x14	chip_select_register0_low – $SCSLR0_LOW$	Base address for chip select n ($s_sel_n[0]$)
32'hxxx_x18	chip_select_register1_low – $SCSLR1_LOW$	
32'hxxx_x1C	chip_select_register2_low – $SCSLR2_LOW$	
32'hxxx_x20	chip_select_register3_low – $SCSLR3_LOW$	
32'hxxx_x24	chip_select_register4_low – $SCSLR4_LOW$	
32'hxxx_x28	chip_select_register5_low – $SCSLR5_LOW$	
32'hxxx_x2C	chip_select_register6_low – $SCSLR6_LOW$	
32'hxxx_x30	chip_select_register7_low – $SCSLR7_LOW$	

8.2.7.1 $SMSKR_n$

- **Name:** Address Mask Registers
- **Size:** 32 bits
- **Address:** Refer to [Table 8-1](#)
- **Read/write access:** read/write, unless $HARD_WIRE_CIPSELECT_PARAMETRS$ is set to 1; then read-only



Bits	Name	R/W	Description
31:11	Reserved		
10:8	reg_select	R/W*	Register determines which timing parameters of memory connect to associated chip select; primarily used for specifying static memories 0 – register set 0 1 – register set 1 2 – register set 2 This is “don't care” if mem_type is SDRAM Default Value: REG_SELECT_n

* Read-only if $HARD_WIRE_CIPSELECT_PARAMETRS$ is set to 1

Bits	Name	R/W	Description																		
7:5	mem_type	R/W*	Type of memory connected to corresponding chip select: 0 – SDRAM 1 – SRAM 2 – FLASH Others – Reserved Default Value: <i>CHIP_SELECTn_MEM</i>																		
4:0	mem_size	R/W*	Size of memory connected to corresponding chip select; value of 0 specifies that no memory is connected to chip select: 0 – No memory is connected to the chip select <table><tr><td>1 – 64KB</td><td>7 – 4MB</td><td>13 – 256MB</td></tr><tr><td>2 – 128KB</td><td>8 – 8MB</td><td>14 – 512MB</td></tr><tr><td>3 – 256KB</td><td>9 – 16MB</td><td>15 – 1GB</td></tr><tr><td>4 – 512KB</td><td>10 – 32MB</td><td>16 – 2GB</td></tr><tr><td>5 – 1MB</td><td>11 – 64MB</td><td>17 – 4GB</td></tr><tr><td>6 – 2MB</td><td>12 – 128MB</td><td></td></tr></table> Default Value: <i>BLOCK_SIZE_n</i>	1 – 64KB	7 – 4MB	13 – 256MB	2 – 128KB	8 – 8MB	14 – 512MB	3 – 256KB	9 – 16MB	15 – 1GB	4 – 512KB	10 – 32MB	16 – 2GB	5 – 1MB	11 – 64MB	17 – 4GB	6 – 2MB	12 – 128MB	
1 – 64KB	7 – 4MB	13 – 256MB																			
2 – 128KB	8 – 8MB	14 – 512MB																			
3 – 256KB	9 – 16MB	15 – 1GB																			
4 – 512KB	10 – 32MB	16 – 2GB																			
5 – 1MB	11 – 64MB	17 – 4GB																			
6 – 2MB	12 – 128MB																				

* Read-only if HARD_WIRE_CIPSELECT_PARAMETERS is set to 1

The following table lists the mask registers and their addresses.

Name	Address	Description
Mask Register0 – SMSKR0	32'hxxxx_xx54	Mask register for chip select0; always active
Mask Register1 – SMSKR1	32'hxxxx_xx58	Mask register for chip select1; enabled only if N_CS > 1
Mask Register2 – SMSKR2	32'hxxxx_xx5C	Mask register for chip select2; enabled only if N_CS > 2
Mask Register3 – SMSKR3	32'hxxxx_xx60	Mask register for chip select3; enabled only if N_CS > 3
Mask Register4 – SMSKR4	32'hxxxx_xx64	Mask register for chip select4; enabled only if N_CS > 4
Mask Register5 – SMSKR5	32'hxxxx_xx68	Mask register for chip select5; enabled only if N_CS > 5
Mask Register6 – SMSKR6	32'hxxxx_xx6C	Mask register for chip select6; enabled only if N_CS > 6
Mask Register7 – SMSKR7	32'hxxxx_xx70	Mask register for chip select7; enabled only if N_CS > 7

8.2.8 Aliasing Registers (CSALIASn)

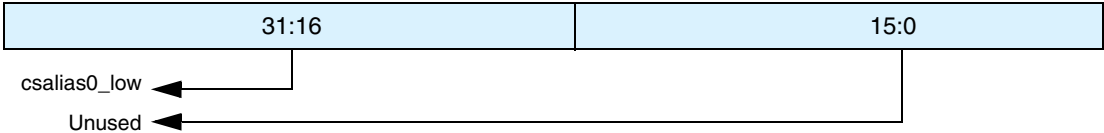
Chip select0 and chip select1 support aliasing. There are two aliasing registers per chip select, depending on the value of the AHB address width.

If the AHB address width is 32, then there is only one aliasing register per chip select; for example, chip select0 would have CSALIAS0_LOW.

8.2.8.1 CSALIAS0_LOW

- **Name:** ALIAS REGISTER0
- **Size:** Lower 32 bits
- **Address:** 32'hxxxx_xx74
- **Read/write access:** read/write, unless HARD_WIRE_CIPSELECT_PARAMETERS is set to 1; then read-only

The CSALIAS0_LOW register holds the aliasing address value for chip select0 when the H_ADDR_WIDTH is 32. This register is valid only if the compile-time parameter CHIP_SELn_ALIAS_ENABLE is 1. The compile-time parameter CHIP_SELECT0_ALIAS_ADDRESS specifies the default value for these registers.

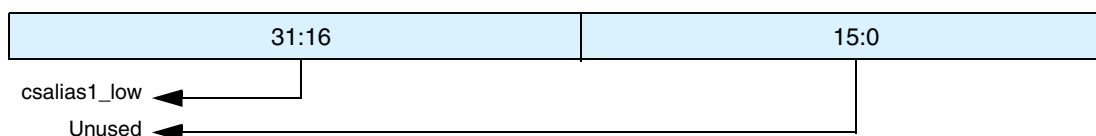


Bits	Name	R/W	Description
31:16	csalias0_low	R/W*	Lower aliasing register bits for chip select0; compared with corresponding AHB address to generate chip select0; number of bits compared depends on size of memory selected by chip select0 (specified in mask register): 64 KB – bits 31:16 compared 128 KB – bits 31:17 compared Default Value: CHIP_SELECT0_ALIAS_ADDRESS
15:0	Unused because DW_memctl supports a minimum memory size of 64KB		
* Read-only if HARD_WIRE_CIPSELECT_PARAMETERS is set to 1			

8.2.8.2 CSALIAS1_LOW

- **Name:** ALIAS REGISTER1
- **Size:** Lower 32 bits
- **Address:** 32'hxxxx_xx78
- **Read/write access:** read/write, unless HARD_WIRE_CIPSELECT_PARAMETERS is set to 1; then read-only

This register holds the aliasing address value for chip select1 when the H_ADDR_WIDTH is 32. This register is valid only when N_CS is >1 and the compile-time parameter CHIP_SEL1_ALIAS_ENABLE is 1. The compile-time parameter CHIP_SELECT1_ALIAS_ADDRESS specifies the default value for these registers.



Bits	Name	R/W	Description
31:16	csalias1_low	R/W*	Lower aliasing register bits for chip select1; compared with corresponding AHB address to generate chip select1; number of bits compared depends on size of memory selected by chip select1 (specified in mask register): 64KB – bits 31:16 compared 128 KB – bits 31:17 compared Default Value: CHIP_SELECT1_ALIAS_ADDRESS
15:0	Unused		because DW_memctl supports a minimum memory size of 64KB
* Read-only if HARD_WIRE_CIPSELECT_PARAMETERS is set to 1			

8.2.9 Remap Registers

Chip select0 and chip select1 can have up to two remapping registers per chip select, depending on the value of AHB address width; only chip select0 and chip select1 support remapping.

If the AHB address width is 32, then each chip select contains only one remapping register; for example, CSREMAP_LOW for chip select0.

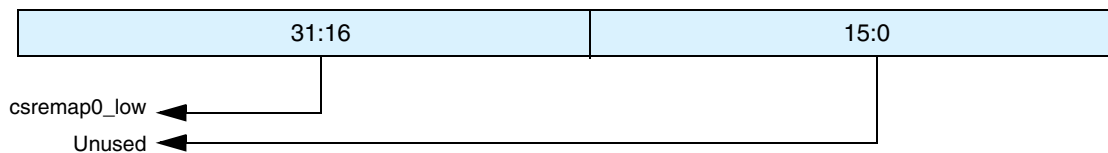
8.2.9.1 CSREMAP0_LOW

- **Name:** REMAP REGISTER0
- **Size:** Lower 32 bits
- **Address:** 32'hxxxx_xx84
- **Read/write access:** read/write, unless HARD_WIRE_CIPSELECT_PARAMETERS is set to 1; then read-only

This register holds the remap address value for chip select0 when the H_ADDR_WIDTH is 32. This register is valid only if the compile-time parameter CHIP_SEL_n_REMAP_ENABLE is 1.

The CHIP_SEL0_REMAP_ADDRESS compile-time parameter specifies the default value for these registers. When remapping is enabled for Chip select0, the chip select will be generated under the following conditions:

- When the remap input is 1 and the AHB address matches the remap address for Chip select0 (remap register0)
- When the remap input is 0 and the AHB address matches the base address for Chip select0 (chip select register0)



Bits	Name	R/W	Description
31:16	csremap0_low	R/W*	Represent lower remap register bits for chip select0; compared with corresponding AHB address to generate chip select0; number of compared bits depends on size of memory selected by chip select0 (specified in mask register): 64KB – bits 31:16 compared 128 KB – bits 31:17 compared Default Value: CHIP_SELECT0_REMAP_ADDRESS
15:0	Unused		Unused because DW_memctl supports a minimum memory size of 64KB

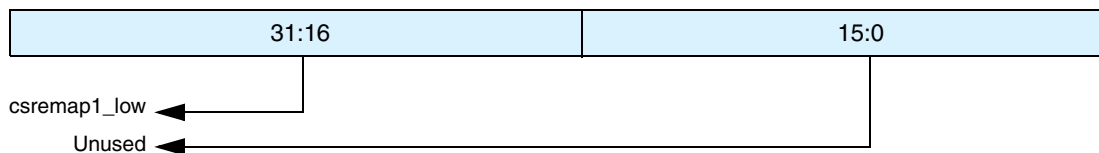
* Read-only if HARD_WIRE_CIPSELECT_PARAMETERS is set to 1

8.2.9.2 CSREMAP1_LOW

- **Name:** REMAP REGISTER1
- **Size:** Lower 32 bits
- **Address:** 32'hxxxx_xx88
- **Read/write access:** read/write, unless HARD_WIRE_CIPSELECT_PARAMETERS is set to 1; then read-only

This register holds the remap address value for chip select1 when the H_ADDR_WIDTH is 32. This register is valid only when N_CS is >1 and the compile-time parameter CHIP_SEL1_REMAP_ENABLE is 1. The CHIP_SEL1_REMAP_ADDRESS compile-time parameter specifies the default value for this register. When remapping is enabled for Chip select1, the chip select will be generated under the following conditions:

- When the remap input is 1 and the AHB address matches the remap address for Chip select1 (remap register1)
- When the remap input is 0 and the AHB address matches the base address for Chip select1 (chip select register1)

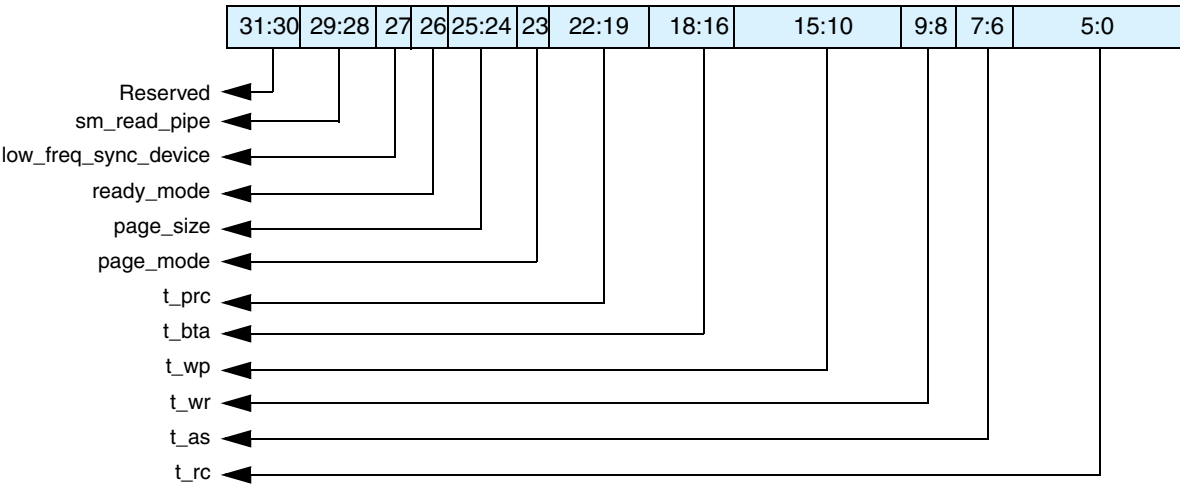


Bits	Name	R/W	Description
31:16	csremap1_low	R/W*	Lower remap register bits for chip select1; compared with corresponding AHB address to generate chip select1; number of compared bits depends on size of memory selected by chip select0 (specified in mask register): 64KB – bits 31:16 compared 128 KB – bits 31:17 compared Default Value: CHIP_SELECT1_REMAP_ADDRESS
15:0	Unused		Unused because DW_memctl supports a minimum memory size of 64KB
* Read-only if HARD_WIRE_CIPSELECT_PARAMETERS is set to 1			

8.2.9.3 SMTMGR_SETn

- **Name:** Static Memory Timing Register - Set*n*
- **Size:** 32 bits
- **Addresses:**
 - 32'hxxxx_xx94 - SMTMGR_SET0
 - 32'hxxxx_xx98 - SMTMGR_SET1
 - 32'hxxxx_xx9C - SMTMGR_SET2
- **Read/write access:** read/write, unless HARD_WIRE_STATIC_PARAMETERS is set to 1; then read-only

The following illustrates the format of the Static Timing Register Set*n* (SMTMGR_SET*n*), where “n” in this section refers to 0, 1, or 2; that is, SMTMGR_SET0, SMTMGR_SET1, or SMTMGR_SET2



The following table describes the bits in the Static Timing Register Set*n* (SMTMGR_SET*n*); “n” in the defaults can be 0, 1, or 2 and is dependent on which timing register set is used.

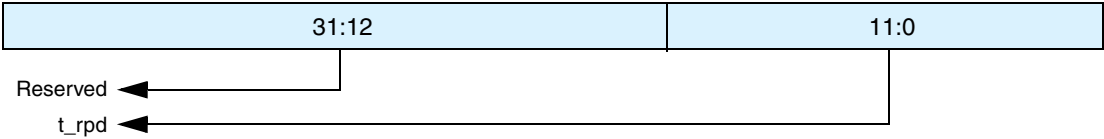
Bits	Name	R/W	Description
31:30	Reserved		
29:28	sm_read_pipe	R/W**	Number of registers inserted in the read data path for latching the data correctly, in the case of Static memory associated with register set0 Default Value: SM_READ_PIPE_SETn
** Read-only if HARD_WIRE_STATIC_PARAMETERS is set to 1			

Bits	Name	R/W	Description
27	low_freq_sync_device	R/W**	Valid if register set0 is used to control low-frequency synchronous device; instructs the memory controller to sample sm_clken before starting any Static memory operation Synchronous memory device could be same or sub-multiple of AMBA clock Default Value: <i>LOW_FREQ_DEVICE_SETn</i>
26	ready_mode	R/W**	Indicates if the static memory associated with register set <i>n</i> is a data-ready device (valid data indicated by a ready signal) Default Value: <i>READY_MODE_SETn</i>
25:24	page_size	R/W**	Page size: 0 – 4-word page 1 – 8-word page 2 – 16-word page 3 – 32-word page Default Value: <i>PAGE_SIZE_SETn</i>
23	page_mode	R/W**	Page-mode device: 0 – device does not support page mode 1 – device supports page mode Default Value: <i>PAGE_MODE_SETn</i>
22:19	t_prc	R/W**	Page mode read cycle time; values of 0-15 correspond to read cycle time of 1-16 clock cycles Default Value: <i>T_PRC_SETn-1</i>
18:16	t_bta	R/W**	Static memory idle cycles between “read to write”, or “write to read”, or “read to read when chip-select changes” for memory data bus turn around time; values of 0-7 correspond to 0-7 idle clock cycles. Default Value: <i>T_BTA_SETn</i>
15:10	t_wp	R/W**	Write pulse width; values of 0-63 correspond to write pulse width of 1-64 clock cycles Default Value: <i>T_WP_SETn-1</i>
9:8	t_wr	R/W**	Write address/data hold time; values of 0-3 correspond to write address/data hold time of 0-3 clock cycles Default Value: <i>T_WR_SETn</i>
7:6	t_as	R/W**	Write address setup time; values of 0-3 correspond to address setup time of 0-3 clock cycles; value of 0 is only valid in case of SSRAM Default Value: <i>T_AS_SETn</i>
5:0	t_rc	R/W**	Read cycle time; values of 0-63 correspond to read cycle time of 1-64 clock cycles Default Value: <i>T_RC_SETn-1</i>

** Read-only if HARD_WIRE_STATIC_PARAMETERS is set to 1

8.2.9.4 FLASH_TRPDR

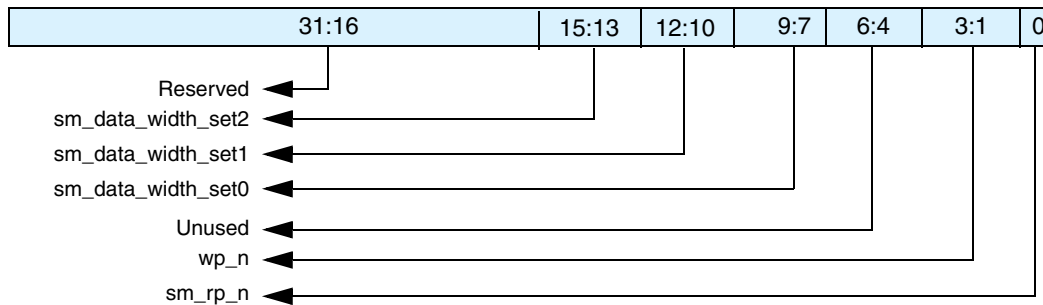
- ❖ **Name:** FLASH Timing Register
- ❖ **Size:** 32 bits
- ❖ **Address:** 32'hxxxx_xxA0
- ❖ **Read/write access:** read/write



Bits	Name	R/W	Description
31:12	Reserved		
11:0	t_rpd	R/W	FLASH reset/power-down high to read/write delay; values correspond to sm_rp_n high to read/write delay minus one Default Value: <i>T_RPD</i>

8.2.9.5 SMCTLR

- **Name:** Static Memory Control Register
- **Size:** 32 bits
- **Address:** 32'hxxxx_xxA4
- **Read/write access:** read/write, unless HARD_WIRE_STATIC_PARAMETERS is set to 1; then read-only



Bits	Name	R/W	Description
31:16	Reserved		
15:13	sm_data_width_set2 *	R/W**	Width of Static memory data bus controlled by Static memory register SET2: 000 – 16 bits 001 – 32 bits 010 – 64 bits 011 – 128 bits 100 – 8 bits When configured as both a Static memory controller and an SDRAM controller, Static memory data width must be less than or equal to SDRAM memory data width (see register field SCONR.s_data_width). Default Value: SM_DATA_WIDTH_SET2 encoded to 3 bits
12:10	sm_data_width_set1 *	R/W**	Width of Static memory data bus controlled by Static memory register SET1: 000 – 16 bits 001 – 32 bits 010 – 64 bits 011 – 128 bits 100 – 8 bits When configured as both a Static memory controller and an SDRAM controller, Static memory data width must be less than or equal to SDRAM memory data width (see register field SCONR.s_data_width). Default Value: SM_DATA_WIDTH_SET1 encoded to 3 bits

* Read-only if HARD_WIRE_STATIC_PARAMETERS is set to 1

** Read-only if HARD_WIRE_STATIC_PARAMETERS is set to 1

Bits	Name	R/W	Description
9:7	sm_data_width_set0	R/W**	Width of Static memory data bus controlled by Static memory register set 0: 000 – 16 bits 001 – 32 bits 010 – 64 bits 011 – 128 bits 100 – 8 bits When configured as both a Static memory controller and an SDRAM controller, Static memory data width must be less than or equal to SDRAM memory data width (see register field SCONR.s_data_width). Default Value: SM_DATA_WIDTH_SET0 encoded to 3 bits
6:4	Unused		
3:1	wp_n	R/W**	FLASH write-protection mode; writing 0 forces FLASH memory boot block to write protect; the three bits correspond to three register sets Default Value: 0
0	sm_rp_n	R/W**	FLASH reset/power-down mode; after reset, controller internally performs a power-down for FLASH and then sets this bit to 1 To force FLASH to power-down mode during normal operation: 0 – Forces FLASH to power-down mode 1 – Takes FLASH out of power-down mode Default Value: 1
* Read-only if HARD_WIRE_STATIC_PARAMETERS is set to 1			
** Read-only if HARD_WIRE_STATIC_PARAMETERS is set to 1			

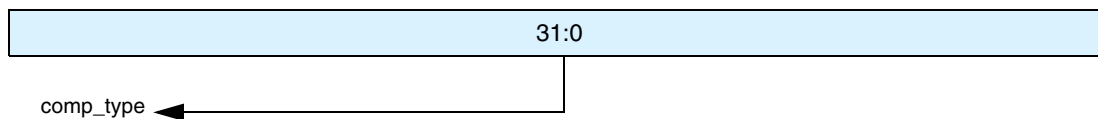
8.2.10 Component Identification Registers

Component identification registers allow an easy identification of the type and version of the component.

8.2.10.1 COMP_TYPE

- **Name:** Component Type Register
- **Size:** 32 bits
- **Address:** 32'hxxxx_xxFC
- **Read/write access:** read

This register holds the identification code for the DW_memctl component.

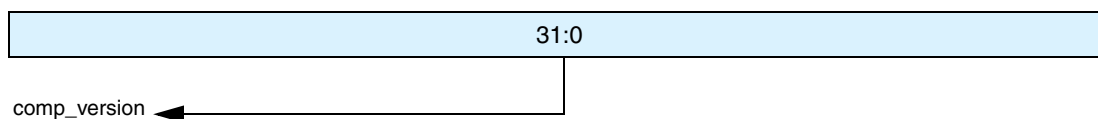


Bits	Name	R/W	Description
31:0	comp_type	R	Designware Component Type number = 0x44_57_11_10. This assigned unique hex value is constant and is derived from the two ASCII letters "DW" followed by a 16-bit unsigned number. Default Value: 0x4457_2110

8.2.10.2 COMP_VERSION

- **Name:** Component Version Register
- **Size:** 32 bits
- **Address:** 32'hxxxx_xxF8
- **Read/write access:** read

This register holds the version of the DW_memctl component.



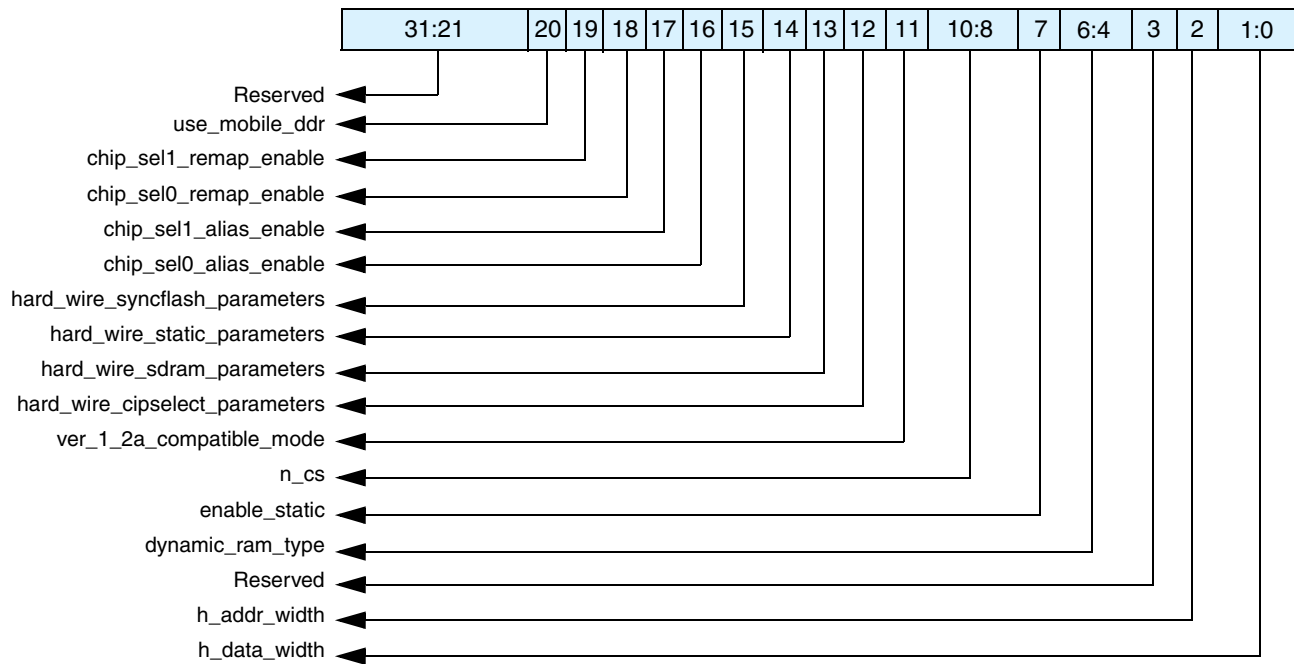
Bits	Name	R/W	Description
31:0	comp_version	R	Version of the component in ASCII code; three numbers are used followed by the "*" character; for example, version 2.78b is represented as 0x3237382a, which translates to character string "278*". Default Value: See the releases table in the Release Notes

8.2.11 Component Configuration Registers

Component configuration registers are constant read-only registers that contain encoded information about component parameter settings. The reset value for some of the bits in these registers depends on coreConsultant parameters.

8.2.11.1 COMP_PARAMS_1

- **Name:** Component Parameter Configuration Register
- **Size:** 32 bits
- **Address:** 32'hxxxx_xxF4
- **Read/write access:** read



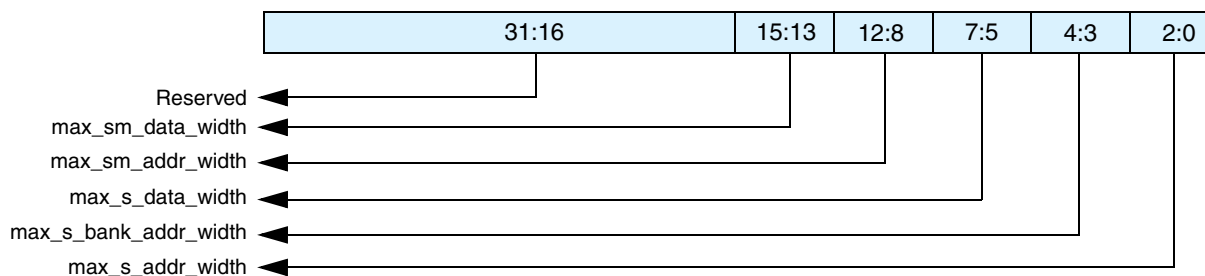
Bits	Name	R/W	Description
31:21	Reserved		
20	use_mobile_ddr	R	Fixed configuration setting Default Value: 0
19	chip_sel1_remap_enable	R	Chip select 1 remap logic enable ■ 0 – chip select 1 remap logic not enabled ■ 1 – chip select 1 remap logic enabled Default Value: <i>CHIP_SEL1_REMAP_ENABLE</i>

Bits	Name	R/W	Description
18	chip_sel0_ remap_enable	R	Chip select 0 remap logic enable 0 – chip select 0 remap logic not enabled 1 – chip select 0 remap logic enabled Default Value: <i>CHIP_SEL0_REMAP_ENABLE</i>
17	chip_sel1_ alias_enable	R	Chip select 1 alias logic enable 0 – chip select 1 alias logic not enabled 1 – chip select 1 alias logic enabled Default Value: <i>CHIP_SEL1_ALIAS_ENABLE</i>
16	chip_sel0_ alias_enable	R	Chip select 0 alias logic enable 0 – chip select 0 alias logic not enabled 1 – chip select 0 alias logic enabled Default Value: <i>CHIP_SEL0_ALIAS_ENABLE</i>
15	hard_wire_ syncflash_ parameters	R	Fixed configuration setting Default Value: 0
14	hard_wire_ static_ parameters	R	Static memory parameters hardwired 0 – static memory parameters not hardwired 1 – static memory parameters hardwired Default Value: <i>HARD_WIRE_STATIC_PARAMETERS</i>
13	hard_wire_ sdram_ parameters	R	SDRAM parameters hardwired 0 – SDRAM parameters not hardwired 1 – SDRAM parameters hardwired Default Value: <i>HARD_WIRE_SDRAM_PARAMETERS</i>
12	hard_wire_ cipselect_ parameters	R	Chip select parameters hardwired 0 – chip select parameters not hardwired 1 – chip select parameters hardwired Default Value: <i>HARD_WIRE_CIPSELECT_PARAMETERS</i>

Bits	Name	R/W	Description
11	ver_1_2a_compatible_mode	R	Fixed configuration setting Default Value: 0
10:8	n_cs	R	Total number of chip selects ■ 000 – 1 chip select ■ 001 – 2 chip selects ■ 010 – 3 chip selects ■ ... ■ 111 – 8 chip selects Default Value: 4
7	enable_static	R	Fixed configuration setting Default Value: 1
6:4	dynamic_ram_type	R	Fixed configuration setting Default Value: 000
3	Reserved		
2	h_addr_width	R	AHB address width ■ 0 – 32 bits ■ 1 – 64 bits Default Value: <i>H_ADDR_WIDTH</i>
1	h_data_width	R	AHB data width ■ 00 – 32 bits ■ 01 – 64 bits ■ 10 – 128 bits ■ 11 – reserved Default Value: <i>H_DATA_WIDTH</i>

8.2.11.2 COMP_PARAMS_2

- **Name:** Component Parameter Configuration Register
- **Size:** 32 bits
- **Address:** 32'hxxxx_xxF0
- **Read/write access:** read



Bits	Name	R/W	Description
31:16	Reserved		
15:13	max_sm_data_width	R	Maximum size of static memory data bus ■ 000 – 8 bits ■ 001 – 16 bits ■ 010 – 32 bits ■ 011 – 64 bits ■ 100 – 128 bits ■ 101-111 – Reserved Default Value: MAX_SM_DATA_WIDTH
12:8	max_sm_addr_width	R	Maximum size of static memory address bus ■ 00000 – 11 bits ■ 00001 – 12 bits ■ 00010 – 13 bits ■ ... ■ 10101 – 32 bits ■ 10110-11111 – Reserved Default Value: MAX_SM_ADDR_WIDTH

Bits	Name	R/W	Description
7:5	max_s_data_width	R	Maximum size of SDRAM memory data bus 000 – 8 bits 001 – 16 bits 010 – 32 bits 011 – 64 bits 100 – 128 bits 101-111 – Reserved Default Value: <i>MAX_S_DATA_WIDTH</i>
4:3	max_s_bank_addr_width	R	Maximum width of SDRAM bank address 00 – 1 01 – 2 10 – 3 11 – 4 Default Value: <i>MAX_S_BANK_ADDR_WIDTH</i>
2:0	max_s_addr_width	R	Maximum width of SDRAM address 000 – 11 001 – 12 010 – 13 ... 101 – 16 110-111 – Reserved Default Value: <i>MAX_S_ADDR_WIDTH</i>

9

Programming the DW_memctl

This chapter contains information on programming the DW_memctl.

9.1 Software Drivers

The DesignWare SDRAM and Static Memory Controller includes a Driver Kit for the DW_memctl component. This low-level Driver Kit allows you to easily program a DW_memctl component and integrate your code into a larger software system. The Driver Kit provides the following benefits to IP designers:

- Proven method of access to DW_memctl minimizing usage errors
- Rapid software development with minimum overhead
- Detailed knowledge of DW_memctl register bit fields not required
- Easy integration of DW_memctl into existing software system
- Programming at register level eliminated

You must purchase a source code license to use the DW_memctl Driver Kit; for information on source licenses, refer to [“Setting up Your Environment”](#) in the the *DesignWare SDRAM and Static Memory Controller Installation Guide*. However, you can access some Driver Kit files and documentation in \$DESIGNWARE_HOME/drivers/DW_memctl/latest. For more information about the Driver Kit, refer to the [DW_memctl Driver Kit User Guide](#). For more information about purchasing the source code license and obtaining a download of the Driver Kit, contact Synopsys at designware@synopsys.com for details.

9.2 DW_memctl Configuration

You can configure the DW_memctl to work as:

- Generic SDRAM controller, which supports SDR-SDRAM, Mobile SDR-SDRAM, Mobile DDR-SDRAM, or DDR-SDRAM (only one type of SDRAM in a controller)
- Generic Static memory controllers, which support SRAMs and FLASHes
- Both an SDRAM controller (SDR-SDRAM or Mobile SDR-SDRAM or Mobile DDR-SDRAM or DDR-SDRAM (only one type of SDRAM in a controller)) and a Static memory controller

The DW_memctl can provide an interface between each of the following memory devices and an AMBA AHB 2.0 bus:

- JEDEC-standard SDR-SDRAM
- JEDEC-standard Mobile SDR-SDRAM
- JEDEC-standard Mobile DDR-SDRAM (LPDDR)
- JEDEC-standard DDR-SDRAM
- Asynchronous SRAM, with or without page-mode
- Asynchronous FLASH, with or without page-mode
- Limited synchronous SRAM and FLASH interface support

These interfaces enable you to integrate the DW_memctl into embedded applications for system-on-a-chip (SOC) designs. The DW_memctl is an AHB slave in SOC designs and provides all the functionality for read/write transactions from multiple AHB masters to the interface memory device.

As an SDRAM controller, the DW_memctl can support up to sixteen row address bits, fifteen column address bits, and four bank address bits. As a Static memory controller, the DW_memctl can support up to 32 address bits.

The DW_memctl can support up to a maximum of eight different groups of external memory. Only one set of timing registers control SDRAMs, so different SDRAM groups must have the same timings. For Static memories, there are three different sets of timing registers.

Using the coreAssembler tool, you specify parameters based on your design specifications to describe the memory controller; coreAssembler then automatically configures, synthesizes, and verifies the components for your design environment. Additionally, if you want to use DesignWare Memory Models (DWMMs), coreAssembler generates preverified DWMMs and also automatically configures the DW_memctl core with the timing parameters for that particular DWMM.

9.2.1 DW_memctl Register Programming Sequence

If you plan to boot from a FLASH, ensure that the default compile-time parameters that you specified for the FLASH are correct. The chip-select base address, block size, and read access time, and so on, need to be correct in order for the memory controller to boot from FLASH after reset.

During compile time if you have already correctly set the configuration registers and you do not plan to change them, then you do not have to reprogram. Otherwise, program the DW_memctl registers using the following sequence:

```
- Address Decoder Registers
if (num_chip_selects >1)
{
-Program Chip Select Registers (SCSLRn_LOW) and Address \
Mask Registers (SMSKRn)
for (i=0;i<num_of_chip_selects;i=i+1)
{
Program Chip Select Register i (Address: REGISTER_BASE_ADDRESS \
+ h'14 + 4*i) .
Program Mask Register i (Address: REGISTER_BASE_ADDRESS + h'54 \
+ 4*i) .
}
}
```

```

-Program Chip Select Alias (CSALIASn_LOW) Register 0 and 1
if(Address Alias is to be enabled)
{
for (i=0;i<2_selects;i=i+1) {
Program Address Alias Register i (Address: \
REGISTER_BASE_ADDRESS + h'74 + 4*i)}
}

-Program Chip Select Remap (CSREMAPn_LOW) Register 0 and 1
if(Address Reamp is to be enabled)
{
for (i=0;i<2_selects;i=i+1) {
Program Address Remap Register i (Address: \
REGISTER_BASE_ADDRESS + h'84 + 4*i)}
}

- Dynamic Memory Controller Registers (SDR/Mobile/DDR-SDRAM)
if(SDRAM Controller is enabled)
{
-Program SDRAM Registers (SCONR,STMG0R,STMG1R,SREFR,SCTLR).

- SDRAM bank address , row/column address and data width (SCONR)
Program SDRAM Configuration Register (Address: \
REGISTER_BASE_ADDRESS + 0

- SDRAM timing parameters (STMG0R)
Program SDRAM Timing Register0 (Address: REGISTER_BASE_ADDRESS+4)

- The remaining SDRAM timing parameters (STMG1R)
Program SDRAM Timing Register1 (Address: REGISTER_BASE_ADDRESS+8)

- Refresh period (SREFR)
Program SDRAM Refresh Interval register (Address: \
REGISTER_BASE_ADDRESS + h'10)

- Mobile/DDR-SDRAM Extend Mode Register (EXN_MODE_REG)
if(Mobile/DDR-SDRAM is used)
{
Program Extended Mode Register (Address: REGISTER_BASE_ADDRESS \
+ h'AC)
}

- SDRAM control Register (SCTLR). Set the "initialize" bit for \
the controller to do auto
- SDRAM initialization sequence. Since setting the "initialize" \
bit starts the SDRAM
- initialization sequence, this register should be programmed last
Program SDRAM Control Register (Address REGISTER_BASE_ADDRESS+h'C)
}

-Static Memory registers for SRAM/FLASH
if(Static Memory Controller is enabled)
{

- Static Memory Timing Register (SMTMGR_SETn)
for (i=0;i<2;i=i+1)

```

```

{
Program Static Memory Timing Register Set i register (Address: \
REGISTER_BASE_ADDRESS + h'94 + 4*i);
}

- Flash memory trPD timing register (FLASH_TRPDR)
if(FLASH memory is used)
{
Program Flash timing Register (Address: REGISTER_BASE_ADDRESS \
+ h'A0)
}

- Static memory control register (SMCTLR)
Program Static memory control register (Address: \
REGISTER_BASE_ADDRESS + h'A4)
}

```

After you complete the DW_memctl configuration through the GUI, the power-on reset values of all the registers in your configuration are written for your reference to the workspace src/Poweron_Reset_Register_Values.txt file. A typical file will look like this:

```

Poweron_Reset_Register_Values.txt
-----
# Power-on Reset Register Values For Your Configuration
# The "initialize" bit-0 of SDRAM Control Register(SCTLR), on power on
# will be set to "1" and after auto initialization sequence completion
# will get reset to 0"
Relative Address 0x00: SCONR = 32'h1c3168
Relative Address 0x04: STMG0R = 32'h2426494
Relative Address 0x08: STMG1R = 32'h70008
Relative Address 0x0C: SCTLR = 32'h3089
Relative Address 0x10: SREFR = 32'h410
Relative Address 0x14: SCSLR0_LOW = 32'h80000000
Relative Address 0x18: SCSLR1_LOW = 32'h10000000
Relative Address 0x1C: SCSLR2_LOW = 32'h20000000
Relative Address 0x20: SCSLR3_LOW = 32'h30000000
Relative Address 0x24: SCSLR4_LOW = 32'h40000000
Relative Address 0x28: SCSLR5_LOW = 32'h50000000
Relative Address 0x2C: SCSLR6_LOW = 32'h60000000
Relative Address 0x30: SCSLR7_LOW = 32'h70000000
Relative Address 0x54: SMSKR0 = 32'h80c
Relative Address 0x58: SMSKR1 = 32'h809
Relative Address 0x5C: SMSKR2 = 32'h2a
Relative Address 0x60: SMSKR3 = 32'h44a
Relative Address 0x64: SMSKR4 = 32'h86a
Relative Address 0x68: SMSKR5 = 32'h400
Relative Address 0x6C: SMSKR6 = 32'h400
Relative Address 0x70: SMSKR7 = 32'h400
Relative Address 0x74: CSALIAS0_LOW = 32'h80000000
Relative Address 0x78: CSALIAS0_LOW = 32'h18000000
Relative Address 0x84: CSREMAP0_LOW = 32'h80000000
Relative Address 0x88: CSREMAP1_LOW = 32'h12000000
Relative Address 0x94: SMTMGR_SET0 = 32'h10441
Relative Address 0x98: SMTMGR_SET1 = 32'h7c4f5b
Relative Address 0x9C: SMTMGR_SET2 = 32'h1c4f5b
Relative Address 0xA0: FLASH_TRPDR = 32'hc8
Relative Address 0xA4: SMCTLR = 32'h2480
Relative Address 0xAC: EXN_MODE_REG = 32'h00000000

```

10

Verification

Synopsys supplies a complete functional verification environment for the DW_memctl. Alternatively, you can perform these verification activities directly from the UNIX command line by following the procedures in this chapter.

This chapter provides the information you need to understand how the verification environment works. With this knowledge, you can perform verification activities by executing simulation scripts yourself, instead of through the coreAssembler tool. Additionally, you can verify DW_memctl cores after they have been integrated into larger designs that use a VERA-based system testbench.

**Note**

The DW_memctl verification testbench is built with DesignWare AMBA Verification IP (VIP). Please make sure you have the supported version of the VIP components for this release, otherwise, you may experience some tool compatibility problems. For more information about supported tools in this release, refer to the [DesignWare SDRAM and Static Memory Controller Installation Guide](#).

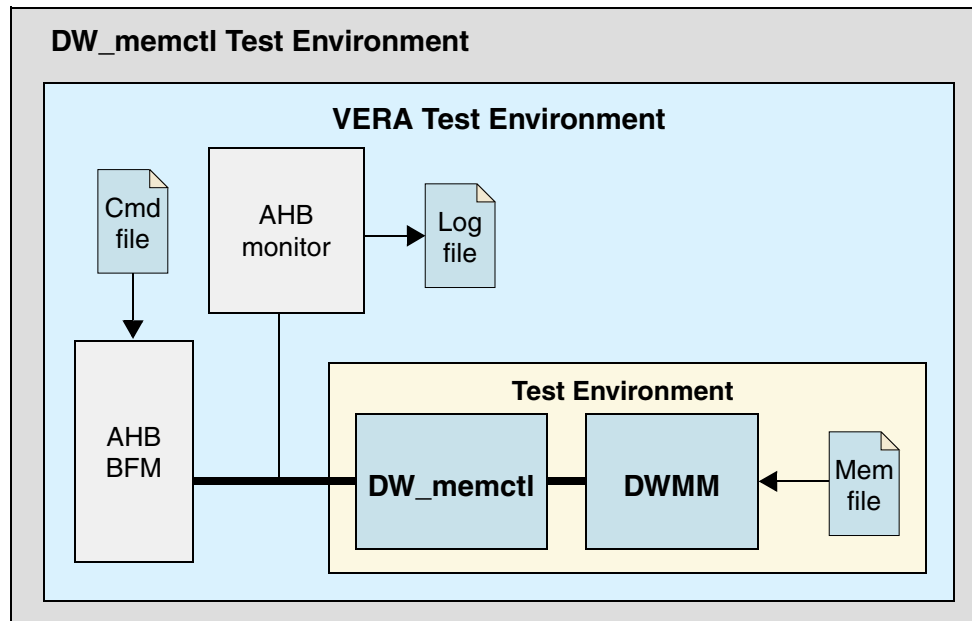
10.1 Verification Environment Overview

The DW_memctl core kit includes a verification environment that sets up and invokes your selected simulation tool to execute specified test algorithms. You can then analyze the results of the simulation.

10.1.1 DW_memctl Testbench

Figure 10-1 shows the structure of the DW_memctl verification environment.

Figure 10-1 DW_memctl Verification Environment



The files included in the DW_memctl verification environment are:

- DW_memctl testbench – Testbench that instantiates the DW_memctl (sim/testbench/DW_memctl_test_top.v), the device under test (DUT), and the vera_shell corresponding to it. Additionally, the default testbench has a bank with an SDR/DDR-SDRAM, SDRAM module, SRAM, FLASH, and wireless FLASH instantiated in it.



Note

It is recommended that you first install the core and verify it with the default configuration and settings before making modifications.

The core has a limited auto-testbench generation capability. You can pick a set of DWMMs supported by this core; a testbench will be automatically created for the selected models. Additionally, the default testbench has a bank with two SDR-SDRAMs, one SRAM, and two FLASHes instantiated in it.

- Input command files – Self-checking AHB write/read BFM files containing commands that control the behavior of the BFMs; provided for all supported memory types.
- Simulation output files – Log files generated by the AHB BFM, the AHB bus monitor, and the simulator output log file.

- Other files – Scripts that automatically link and compile the command files, then simulate your selected testcases. After each testcase is simulated, the output files are automatically compared with the expected data and a report is generated. Additionally, scripts create a log file of all test cases executed in batch test runs.

The verification environment has these major components:

- VERA test environment – Contains AHB BFM, AHB Monitor, and VERA command files
- Test environment – Contains DUT and DWMMs instantiated in it



Note

You can use the VERA BFM and monitors provided with the DW_memctl only for verification of the DW_memctl. For your system testbench environment, use the production AMBA VIP models from Synopsys:

<http://www.synopsys.com/products/designware>

A simulation requires one DesignWare license. Note that VERA and DWMMs interface to Verilog-compatible simulators through the PLI.

You should ensure that Synopsys, VERA, DWMMs, coreAssembler, and your simulator are installed; for installation information, refer to the [DesignWare SDRAM and Static Memory Controller Installation Guide](#).

As appropriate for your simulation environment, the following environment variables should be set properly in your .cshrc in addition to the path and license:

- SYNOPSYS - \$SYNOPSYS
- VERA - \$VERA_HOME
- VCS - \$VCS_HOME (or)
- Verilog-XL and NC-Verilog - \$CDS_INST_DIR

The following is an example .cshrc file; note that for “MTI,” there is no \$MTI_HOME requirement:

```
# Synopsys Setup
setenv SYNOPSYS /remote/release/2001.08-1
set path=( $SYNOPSYS/sparcOS5/syn/bin $path)

#vera setup
setenv VERA_HOME /remote/dw104/tools/vera-latest
set path = ( $VERA_HOME/bin $path )

#VCS setup
setenv VCS_HOME /remote/drg17/tools/VCS_6.0
setenv VCS_CC /opt/SUNWsprow/bin/cc
set path = ( ${VCS_HOME}/bin $path )

#XL and NC setup
setenv CDS_INST_DIR /remote/insync4/cadence/LDV3.2
set path = ($CDS_INST_DIR/tools/verilog/bin \
  $CDS_INST_DIR/tools/bin $path)
setenv LD_LIBRARY_PATH $CDS_INST_DIR/tools/lib:$LD_LIBRARY_PATH

#MTI setup
set path = ( /remote/insync4/MTI/5.5b/sunOS4/modeltech/sunos5 $path )
```

```
#license setup
setenv SNPSLMD_LICENSE_FILE 27000@pasta
setenv LM_LICENSE_FILE \
  /remote/insync2/cadenceIC4.42QSR1/share/license/license.dat: \
  /remote/dw104/tools/lib/flexlm/synopsys_snpslmd_keys.dat: \
  /remote/dw104/tools/lib/flexlm/license.dat: \
  /remote/dw104/tools/vss_v6.100_solaris/pgm/lmgr/license.dat
```

10.1.2 Testbench Structure

The DW_memctl top-level testbench file, located at *workspace/sim/testbench/DW_memctl_test_top.v(hdl)*, provides the top-level testbench module or entity DW_memctl_test_top that instantiates the following elements:

- DW_memctl – Device under test (DUT)
- vera_shell – Interface between the HDL testbench and the VERA AHB BFM and the AHB monitor
- Clock and reset generator – Clock source for hclk, the master clock signal for the core, which is provided to both the AHB and the SDRAM. The hresetn signal is driven from this module.
- Decoder for register (hsel_reg) and memory (hsel_mem) AHB device selects – Chip select signals that map the DW_memctl to any given AHB memory address space. The memory chip select decode is derived from the CHIP_SELECT base address and block size information; additional testbench configuration parameters are entered by the user during the coreAssembler configuration steps.
- Memory groups
- Waveform output – Conditionally generates waveforms at internal/external nodes of the DW_memctl.

The following is the file hierarchy:

```
DW_memctl_test_top.v
  DW_memctl_test.vshell
  DW_memctl.v
  mem_banks.v
    mem_banks_auto.v
```

These components are described in detail in the following sections.

10.1.3 DW_memctl

The top-level DW_memctl core module, in *workspace/src/DW_memctl.v(hdl)*, simulates directly as HDL code in your simulator.

Synopsys' coreAssembler automatically instantiates your configured version of the DW_memctl core as the DUT (device under test).



Note

If you use encrypted source for the DW_memctl and a non-Synopsys simulator or VCS version lower than 6.0, coreAssembler generates a GTECH simulation model for your selected DW_memctl core configuration and places it in *workspace/gtech/final/db*.

10.1.4 vera_shell

The vera_shell provides an HDL interface between the DW_memctl and the VERA verification system that instantiates the compiled VERA object – AHB BFM and AHB monitor.

For Verilog, the vera_shell is in *workspace/sim/vera/lib/DW_memctl.vshell*. The top-level HDL testbench, *workspace/sim/testbench/DW_memctl.test_top.v*, connects the vera_shell and an instance of the configured DW_memctl (from DW_memctl.v) together as HDL components for simulation in your simulator.

The simulation loads these VERA object files prior to a simulation:

- Top-level VERA program file – Calls AHB BFM and monitor; also contains links to test command files and time-out logic

- AHB BFM – Drives AHB interface to DW_memctl and logs activity in:

workspace/sim/log/test_xxx/ahb_bfm.log

- AHB bus monitor – Monitors AHB bus during simulation and logs activity in:

workspace/sim/log/test_xxx/ahb_mon.log

- Simulator log file – Captures simulation log activity in:

workspace/sim/log/test_xxx/test.log

**Note**

More information on these files is provided in the testmake script description in “run.scr” on page 214.

The AHB BFM and monitors are provided in encrypted VERA source files.

The only VERA files that are not precompiled are simulation test command files and higher-level tasks that use the AHB BFMs. During simulation, the BFM operates under the control of command files to generate stimulus on the AHB bus, so command files are compiled for each new simulation.

Simulation scripts automatically recompile the command files in *workspace/sim/log/test_xxx*. The scripts also include them inside the *sim/vera/lib/DW_memctl_test.vro* file, which contains the include statement for the symbolic link of the command file. The compile process is automatically executed by the supplied simulation scripts.

**Note**

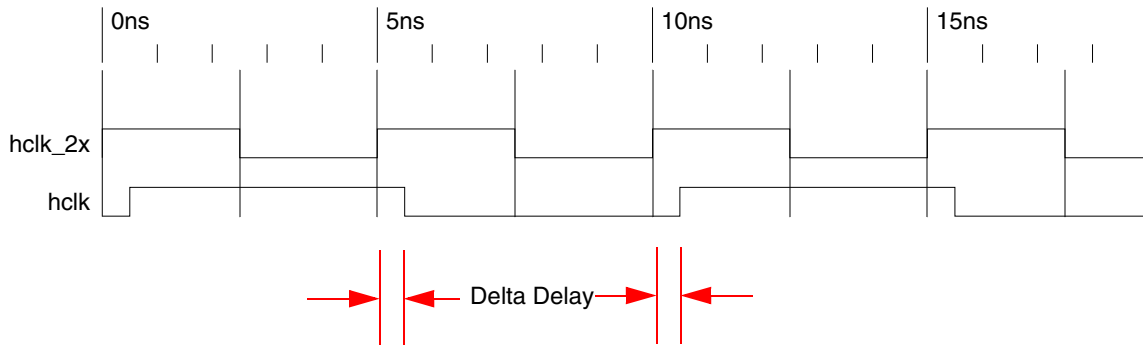
You cannot modify the AHB BFMs or AHB bus monitor because they are provided only as pre-compiled VERA object files.

10.1.5 Clock Generation

The DW_memctl testbench generates two clocks in the testbench: the hclk and hclk_2x. The hclk_2x is used only if the DDR-SDRAM or Mobile DDR-SDRAM is selected.

The hclk_2x clock is used only inside the DDR write-data generation module (DW_memctl_miu_ddrwr.v) and samples the data generated by the hclk clock. In order to avoid simulation race conditions and to meet the hold time, the hclk has a delta delay (#1) from the hclk_2x, as illustrated in Figure 10-2.

Figure 10-2 Delta Delay for hclk and hclk_2x



When you integrate the DW_memctl in your system, you must ensure that the hclk has delta delay from hclk_2x to avoid simulation race condition. Also ensure that the s_rd_data input to the DW_memctl meets the input hold time with regard to hclk.

10.1.6 Address Decoder

The AHB BFM uses an AHB address bus to drive the address to which the DW_memctl is connected. The hsel_reg and hsel_mem signals are the devices selected for the register and memory of the DW_memctl. In a system, these signals are driven from the centralized AHB address decoder.

The DW_memctl has registers mapped from 0x00 - 0xA7, which are selected when hsel_reg is active. For testing purposes in the DW_memctl_test_top.v, address 0x0000_1000 - 0x0000_13ff is mapped to hsel_reg. This base address can be changed during a coreAssembler configuration using the **Configure TestBench Parameters > Register Base Address** menu item. Although the DW_memctl has registers only in the 0x00 - 0xa7 range, a 1K-byte block is allotted to this decode, since the minimum recommended AHB address decoder block size is 1K-byte. All register tests (register *.cmd files) use this base address while testing the registers.

The hsel_mem decode signal is derived from the CHIP_SELECT_n_BASE_ADDRESS and BLOCK_SIZE_n parameters. Additionally, the SDRAM, SRAM, and FLASH base addresses can be changed for the testbench during a coreAssembler configuration using the **Configure Testbench Parameters > xxx Base Address** menu item. These configuration parameters come from the chip-select parameters, which you can overwrite. All memory tests use these base addresses while testing the SDRAM, SRAM, and FLASH.

If you have the same memory type in two banks, then only one bank is tested at a time. You need to change the testbench base address and rerun the tests in order to test the other banks of the same type of memories.

10.1.7 Memory Banks

DWMMs are instantiated in the `mem_banks.v` file, which has support logic to test write and read pipes. Additionally, the file converts the separate input/output data buses to a bidirectional data bus, which is the bus that is actually connected to the memories.

The memory banks and corresponding memory chips are instantiated in the `mem_banks_auto.v` file. The default file supplied with the `DW_memctl` has a bank of the following instantiated in it:

- SDR-SDRAM
- SDR-SDRAM
- SRAM
- FLASH

Each time you change your configuration, the `mem_banks_auto.v` file is recreated by the TCL/Perl scripts.

**Note**

These scripts are just a template generator. Since there are many different ways to interface memories, you should verify the auto-generated `mem_banks_auto.v` file before proceeding.

If you are not using DWMMs, or if you have your own memory model, you can modify `mem_banks_auto.v` as required. There is limited automatic testbench creation support when different DWMMs are selected; crosscheck the testbench/`mem_banks_auto.v` file for correctness.

For testing purposes, the FLASH files – `flash*.mem` – are provided in the testbench directory. These files are in Verilog `$readmemh` format and have repeated patterns of `0x00 - 0xff`.

If you choose different FLASH models during a coreAssembler configuration, then the Perl script `bin/TestBenchGen.pl` generates a memory file for each chip it instantiates, as `BankXUx.mem`. These memory files are passed to DWMMs through the `'defparam BankxUx.memoryfile = "rom/flashX.mem"'` statement inside the `mem_banks_auto.v` file; the `0x00 - 0xFF` patterns are needed for the FLASH test to pass. You can change the memory files as needed, but you should deselect the FLASH memory tests.

10.1.8 Waveform Output

Waveform output is available with only the Verilog version of the `DW_memctl`; you can enable this option in coreAssembler by selecting `VCD_ON`.

If you enable this option, simulations produce a VCD/VCDPLUS format file that you can display using a waveform viewer. When the VCS simulator is chosen, the dump file is in VCDPLUS format; for other simulators, the dump file is in VCD format. The VCD/VCDPLUS capture initial blocks are located at the end of the `sim/testbench/DW_memctl_test_top.v` file and can be modified, if necessary, in order to capture a different vector format supported by your simulator.

Every node that is internal or external to the configuration under test will have its waveform captured in the VCD/VCDPLUS file. The VCDPLUS is a more compact format than the VCD file. You can view VCD/VCDPLUS files by using the VCS graphical interface tool (`vcs -RPP`). Similarly, Verilog-XL has a waveform viewer called SimWave or SignalScan, which can be used to view VCD files.

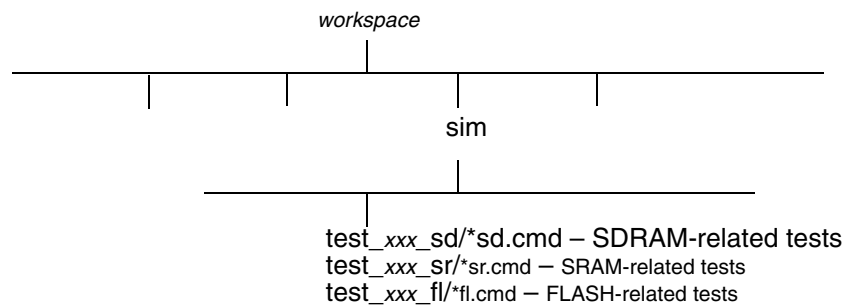
10.2 Simulation Command Files

The DW_memctl verification environment includes the command (.cmd) files required to drive the AHB BFM.

When you execute the Simulation Options activity, coreAssembler automatically selects command files that are suitable to your selected configuration of the core. For both default and non-default configurations, all appropriate tests are run – unless you deselect them – depending on whether SDRAM, SRAM, or FLASH memories are selected.

Figure 10-3 shows the locations of simulation command files in your DW_memctl workspace.

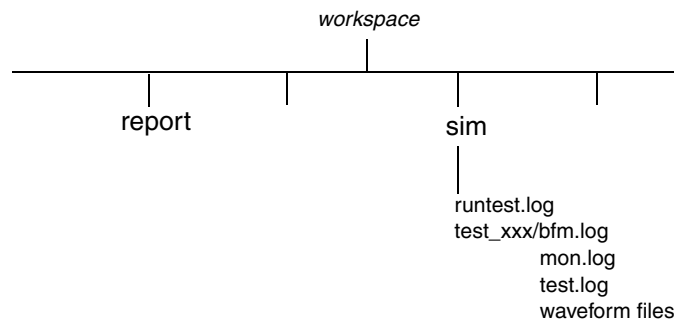
Figure 10-3 Simulation Command Files Directory



10.3 Simulation Output Files

Figure 10-4 shows the simulation output files that exist in your DW_memctl workspace at the end of a test case simulation. If the run.scr script was invoked, a new runtest.log was generated.

Figure 10-4 Simulation Output File Directories



Additionally for each command file in the testlist, test.log, mon.log, and bfm.log files are generated in the sim/log/testlist/cmd_file directory. In addition, the VCD/VCDPLUS dump file is also created if the VCD_ON parameter is enabled in the coreAssembler configuration.

When you simulate the testbench, either through coreAssembler or directly from the UNIX system prompt, the testbench writes the following files to your workspace/sim directory:

- sim/runtest.log – Shows log generated during the simulator executable generation (except for Verilog-XL)
- sim/test_XXX/bfm.log – Log file generated by AHB BFM

- `sim/test_XXX/mon.log` – Log file generated by AHB Monitor
- `sim/test_XXX/test.log` – File containing captured simulator output; contains INFO, WARNING, ERROR messages from DWMMs, BFM, the monitor, DW_memctl, or the checker
- `sim/test_XXX/waveform_files` – Optional vcdplus/vcd dump file

10.3.1 Using Log Files

Log files (.log) are text files that you can visually analyze. A simulation generates these primary log files:

- `bfm.log` – Output of AHB BFM
- `mon.log` – Output of application AHB bus monitor
- `test.log` – Simulator output log file

These are described in the sections that follow.

The `bfm.log` and `mon.log` files are useful for observing bus activity and matching it against the command files for the scenario.

The `test.log` contains the outputs from:

- Simulator
- AHB BFM
- AHB monitor
- DWMM messages
- DW_memctl checker messages

The decision on whether a test is passed or failed is done by looking for “ERROR,” “Error,” “Test Complete,” “Timed Out,” and “ILLEGAL” strings in the `test.log` and `runtest.log` files.

10.4 Simulation Scripts



Note

Before running the scripts described in this section, you *must* execute at least once the coreAssembler configuration and simulation activities. Prerequisite items generated by coreAssembler are listed in “[Verilog PLI Integration](#)” on page 215.

All simulation activities are controlled by a set of scripts, whether you indirectly execute them through coreAssembler or manually from the UNIX command line.

- If you use the coreAssembler GUI, you use the Simulation Options to select the tests that you want to run. When you click OK, commands are written into testlist scripts that create a simulator executable and execute the tests you selected; the testlist script is then executed by the top-level script, `run.scr`.
- If you run scripts from the UNIX command line, you can edit and run the top-level `run.scr` script in the same way that coreAssembler does, or you can simply execute the scripts that `runtest` calls.

10.4.1 run.scr

This is the top-level simulation script that coreAssembler generates to execute the compliance and corner case tests you selected from the list of supported tests for your chosen configuration. In coreAssembler, this script is executed when you click OK in the Simulation Options activity.

Use run.scr when you want to run multiple tests, such as for regression testing. Also, you can run this script – with the default core configuration and default tests enabled – to confirm that your simulation environment is set up correctly.

To execute this script from the command line, go to the *workspace/sim* directory and enter:

```
% run.scr
```

This script does the following:

1. Checks to see that no previous invocation of run.scr is currently executing.
2. Executes “sim/compile,” which creates a simulator executable if a non-Verilog-XL simulator is chosen; additionally recompiles sim/vera/src/get_params.vr.
3. Executes “runtest” for each of the selected tests:
 - a. Compiles the VERA test there
 - b. Invokes the simulation

For several simulators, the PLIs must be created before running the tests using the appropriate scripts:

- ❑ build_pli_mti.sh – MTI,
- ❑ xl_pli.sh – Verilog-XL,
- ❑ build_pli_nc.sh – NC-Verilog

You can use the Netscape browser on the *.../sim/runtest.log* file to see the results of these scripts.

4. Sends e-mail to inform you that all testcases in the testlist have executed, if this option is enabled.

The message includes the following for a coreAssembler simulation:

- ❑ Workspace directory
- ❑ Design name
- ❑ Status (for example, Completed)
- ❑ Path of the runtest.log file
- ❑ Contents of the runtest.log file

10.5 Verilog PLI Integration

Both VERA and DWMMs interface to Verilog simulators through Verilog PLIs. To integrate the PLIs, you add the PLI objects when you are generating the “simv” executable. For other simulators such as MTI, Verilog-XL, and NC-Verilog, veriusers.c is compiled first and passed to the simulator when creating the executable.

- For DWMMs, veriusers.c is in .../sim/pli/veriusers.c.
- For Vera, veriusers.c is in \$VERA_HOME/lib

For VCS – During a build, the PLI is passed as:

```
vcs -P $VERA_HOME/lib/vera_pli.tab \
$VERA_HOME/lib/libSysSciTask.a \
-P $LMC_HOME/sim/pli/src/slm_pli.tab \
$LMC_HOME/lib/sun4Solaris.lib/slm_pli.o \
```

For MTI:

- sim/build_pli_mti.sh – Compiles veriusers.c and creates ../sim/veriusers.so

Previously generated veriusers.so files can be used as a “-pli veriusers.so” option for the vsim command.

For XL:

- sim/xl_pli.sh – Execute vconfig utility delivered by Cadence; vconfig creates the cr_vlog script, which is executed to create the final Verilog binary:
 - vconfig -i build_pli_xl.key
 - modify cr_vlog into build_pli_xl.sh to fit your environment
 - execute build_pli_xl.sh generate vera_xl – executable for xl simulation has PLI built in

For NC:

- sim/build_pli_nc.sh – Compiles veriusers.c and creates ../sim/vera_ncelab and ../sim/vera_ncsim



For NC-Verilog, wherever you run the simulation, you must make a link to the libpli.so (or libpli.sl on hp), or you must set LD_LIBRARY_PATH (or SHLIB_PATH) which points to libpli.

The sim/build_pli_mti.sh or sim/build_pli_nc.sh is executed during a coreAssembler configuration. For Verilog-XL, during a coreAssembler configuration, vconfig is executed and generates a cr_vlog script, which is modified to build_pli_xl.sh. After that, build_pli_xl.sh is executed.

For more information on DWMMs, refer to the *DesignWare Memory Model User's Manual* for details. For more information on using DWMMs with VERA, refer to the *Simulation Configuration Guide for Synopsys Models*.

10.6 Verifying Default DW_memctl Configuration

This procedure is helpful if you are having problems using the verification environment with a non-default DW_memctl configuration and need to check your setup.

To verify the default configuration, perform the following steps:

1. To create a new workspace, use **File > New** in coreAssembler.
2. Apply the default values for the RTL and verification activities in coreAssembler. Choose Do Not Launch Simulation in the Execution Options item in order to write out the simulation command scripts without executing them.

3. On the UNIX command line, execute these commands:

```
% cd workspace/sim/vera
% make
% cd ..
% compile -xxx
% runtest -xxx
```

4. Open the *workspace/sim/results.log* file with a text editor and check the simulator report for each test in order to confirm that all tests were passed.

10.6.1 Verifying Non-Default DW_memctl Configurations



Note

Before starting this procedure, perform the full set of DW_memctl simulation activities at least once for your configuration. If you are not able to complete these activities successfully, perform the full set of DW_memctl simulation activities on the default configuration at least once to ensure that your verification environment is set up properly. For instructions, refer to [“Verilog PLI Integration”](#) on page 215.

You can execute any test selectable in the coreAssembler Simulation Options activity from the UNIX command line. To determine which tests are applicable to your non-default configuration, refer to the runtest.sh script, which contains the test(s) you selected from the full list in coreAssembler. This may be a subset of available tests, however.

10.6.1.1 Executing Individual Tests or Batch Jobs

To run just one test and automatically verify your simulation results, perform the following steps:

1. Go to your *workspace/sim* directory and edit the runtest.sh to include the tests you want to run.
2. Run the “run.scr” command.

10.7 DW_memctl Core VERA Files

The following lists information on DW_memctl core VERA files:

- *sim/vera/DW_vip_amba*, *DW_vip_foundation* – Contains AHB BFM and AHB monitors. The src directories under these directories contain all the encrypted source files, and the include directory has the .vri and .vrh include files.

- `sim/vera/src/DW_memctl_test.vr` – Contains main VERA program. The AHB BFM and the AHB monitors are instantiated in this program. You need a standard VERA license to modify and recompile this file. The command file task (`cmd_file_tests`) is called in this program. The AHB BFM and AHM monitor arguments are passed to the command file, so the user has visibility over both the BFM and the Monitor. The compiled vro file is located in `sim/vera/lib`.
- `sim/vera/lib/DW_memctl_test.vshell` – VERA-shell file that interfaces the `DW_memctl` to VERA.
- `sim/vera/src/new_tasks.vr` – Contains a few higher level tasks that use the AHB BFM tasks. The compiled version of this file is in `sim/vera/lib`. This file is recompiled at the user's location.
- `sim/vera/src/get_params.vr` – Passes parameters (such as `H_DATA_WIDTH`, `ENDIAN_MODE`, and so on) specified by the user during configuration to `DW_memctl_test.vr`. The compiled version of this file is in `sim/vera/lib`. This file is recompiled at the user's location.
- The `sim/vera/src/include` – Contains .vri and .vrh include files. Two main files are the VERA interface file for `DW_memctl.if.vrh` and `DW_memctl.bind.vr`, which is the binding file for the AHB BFM and AHM monitor.

Additionally during a coreAssembler configuration, all the `DW_memctl`-related parameters that form the `src/DW_memctl_params.v` are converted to VERA .vri format and placed by the `bin/ParamsParser.pl` script in `DW_memctl_params.vri`.

- `sim/DW_memctl_reg.vrl` – Lists all the VERA objects that are included during simulation.
- `sim/test_XXX` – Includes each of the tests and contains the corresponding command files.
 - `*sd.cmd` – tests for SDRAMs
 - `*sr.cmd` – tests for SRAMs
 - `*fl.cmd` – tests for FLASHes

This directory includes files that do the following tests:

- `simple_test_*.cmd` – Simple write/read tests.
- `basic_mem_*.cmd` – Do a single cycle of all types of AHB bursts.
- `burst_mem_*.cmd` – Do an exhaustive test of all AHB bursts and sizes.
- `busy_check*.cmd` – Tests AHB Busy cycles inserted in the middle of an AHB burst.
- `burst_term*.cmd` – Tests the AHB early burst termination.
- `register_*.cmd` – Tests the register write/reads.
- `write_read_mix*.cmd` – Tests back-to-back write/reads.
- `partial_word*.cmd` – Verify the AHB partial write/reads.
- `page_bank_sd.test` – Tests the page, bank hit/miss cycles.

10.8 Installing DWMMs Manually

If you want to install DesignWare memory model software or models outside coreAssembler, here is how to do it.

10.8.1 Downloading TSP

1. Go to:
<http://www.synopsys.com/dw/dwmm.php>
2. Click on the TSP Download link and do the download.
3. Click on Download sl_unzip for *platform* and do the download.

10.8.2 Downloading models

1. Go to:
<http://www.synopsys.com/dw/dwmm.php>
2. Type in the model name and click the search button.
3. Click on the model name and then click on "Add to download list."
4. Search all the models that need to be downloaded, and then add them to the download list.
5. After ensuring that all the models are in the download list, click on the "Proceed to download" button.
6. Specify the SolvNet ID and then download the models.

10.8.3 Installing DWMM software (TSP)

1. Run the following command:

```
% sl_unzip.platform dwmm_platform.zip
```
2. Do "setenv LMC_HOME *install_dir*".
3. Move to the cdrom directory and run the following command:

```
% sl_admin.csh
```
4. Click Open and select your platform from the dialog that appears; click Install.
5. Exit the Admin tool.

10.8.4 Installing Models

1. Issue these commands:

```
% sl_unzip.solaris mx_***.zip
```
2. Ensure that your LMC_HOME environment variable is set to the path that contains the DWMM TSP.
3. Move to the *image_name/platform* directory

4. Run `sl_admin.csh`.
5. Click Open and then click Install.

10.8.5 Using DesignWare Memory Models with VCS

1. Add the line ``include "mempro_pkg.v"` in your testbench.
(Check `DW_memctl_top.v` under *workspace/sim/testbench*.)
2. Generate Verilog wrapper for the model by running:

```
% $LMC_HOME/bin/vcs_sg -t -lc -m model_mx
```


For more details, refer to *workspace/sim/runtest.sh*.
3. Invoke vcs to compile your design; the following is an example:

```
% vcs \  
model_mx_bus.v \  
testbench.v \  
-lmc_swift.v \  
-P $LMC_HOME/sim/pli/src/slm_pli.tab \  
$LMC_HOME/lib/platform.lib/slm_pli.o \  
+incdir+$LMC_HOME/sim/pli/src
```

Check files `test.startsim` and `test.sim_command` in the *workspace/sim* directory.

11

Integrating DW_memctl into Your Design

This chapter provides the following information on instantiating and integrating the DW_memctl into a design.

11.1 Instantiating DW_memctl

Once you have configured, tested, and synthesized the core with the coreAssembler flow, you can integrate the output files from the coreKit into your design environment.

- The RTL source files – *workspace/src*

The top-level file is DW_memctl.v, an unencrypted Verilog file. For pin details and RTL hierarchy, refer to “[Architecture](#)” on page 33.

- DWMMs and simulation scripts

If you have chosen and configured the coreKit with the DWMMs you are planning to use, then the DWMM Verilog files are in *workspace/sim/testbench*. You can use the *sim/DW_memctl_test_top.v*, *sim/testbench/mem_bank.v*, and *sim/testbench/mem_banks_auto.v* files as reference to see how the memory controller, VERA-shell, and DWMMs are interfaced. To see how a simulation is invoked, refer to *sim/run.scr*, *compile*, *runtest*, and *runtest.sim_command*.

- VERA Files

For the verification environment, an AHB monitor and BFM are shipped with this core; the object and include files are in *workspace/sim/vera/DW_vip_amba* and *workspace/sim/vera/DW_vip_foundation*.

To see how the AHB BFM and monitors are integrated with the memory controller, look at the *sim/DW_memctl_reg.vrl* file. For more details, refer to “[Verification](#)” on page 205.

For your system testbench environment, use the production AMBA VIP models from Synopsys:

<http://www.synopsys.com/products/designware>

- Synthesis output files

If you run synthesis through the coreKit, the report and db files are in *workspace/syn/final* directory. You can view the synthesis scripts in the *syn/constrain/scripts* directory.

Depending on the configuration that you choose, you can instantiate the DW_memctl in the next higher-level design. [Example 11-1](#) shows a case where there are separate buses for the SDRAM and Static memories, as well as an EBI interface.

Example 11-1 SDR-SDRAM and Static Memory Controller – No Bus Sharing/EBI

```
DW_memctl dut (
    // Outputs

    .hrdata          ( hrdata ),
    .hready_resp     ( hready_resp ),
    .s_sel_n         ( s_sel_n[`N_CS-1:0] ),
    .s_ras_n         ( s_ras_n ),
    .s_cas_n         ( s_cas_n ),
    .s_cke           ( s_cke ),
    .s_bank_addr     ( s_bank_addr[`MAX_S_BANK_ADDR_WIDTH-1:0] ),
    .s_dqm           ( s_dqm[`MAX_S_DATA_WIDTH/8-1:0] ),
    .s_we_n         ( s_we_n ),
    .s_rd_start      ( s_rd_start ),
    .s_rd_pop        ( s_rd_pop ),
    .s_rd_end        ( s_rd_end ),
    .s_rd_dqs_mask   ( s_rd_dqs_mask ),
    .s_scl           ( s_scl ),
    .s_sda_in        ( s_sda_in ),
    .s_sda_out       ( s_sda_out ),
    .s_sda_oe_n      ( s_sda_oe_n ),
    .s_sa            ( s_sa ),
    .s_cas_latency   ( s_cas_latency ),
    .s_wr_data       ( s_wr_data[`MAX_S_DATA_WIDTH-1:0] ),
    .s_dout_valid    ( s_dout_valid ),
    .s_addr          ( s_addr[`MAX_S_ADDR_WIDTH-1:0] ),
    .sm_oe_n         ( sm_oe_n ),
    .sm_we_n         ( sm_we_n ),
    .sm_bs_n         ( sm_bs_n[`MAX_SM_DATA_WIDTH/8-1:0] ),
    .sm_rp_n         ( sm_rp_n ),
    .sm_wp_n         ( sm_wp_n ),
    .sm_adv_n        ( sm_adv_n ),
    .sm_wr_data      ( sm_wr_data[`MAX_SM_DATA_WIDTH-1:0] ),
    .sm_dout_valid   ( sm_dout_valid ),
    .sm_addr         ( sm_addr[`MAX_SM_ADDR_WIDTH-1:0] ),
    .gpo             ( gpo ),

    // Inputs

    .hclk            ( hclk ),
    .hresetn         ( hresetn ),
    .big_endian      ( big_endian ),
    .haddr           ( haddr[`H_ADDR_WIDTH-1:0] ),
    .hsel_mem        ( hsel_mem ),
    .hsel_reg        ( hsel_reg ),
    .hwrite          ( hwrite ),
    .htrans          ( htrans ),
    .hsize           ( hsize ),
```

```

.hburst      ( hburst ),
.hready      ( hready ),
.hresp       ( hresp ),
.hwdata      ( hwd_data[ `H_DATA_WIDTH-1:0 ] ),
.s_rd_ready  ( s_rd_ready ),
.power_down  ( power_down ),
.clear_sr_dp  ( clear_sr_dp ),
.s_rd_data   ( s_rd_data[ `S_RD_DATA_WIDTH-1:0 ] ),
.sm_ready    ( sm_ready ),
.sm_clken    ( sm_clken ),
.sm_power_down ( sm_power_down ),
.sm_rd_data  ( sm_rd_data[ `MAX_SM_DATA_WIDTH-1:0 ] ),
.gpi         ( gpi ),
.remap       ( remap )
);

```

11.2 DW_memctl Integration

Figure 11-1 illustrates a typical AMBA-based system environment using the DW_memctl.

Figure 11-1 DW_memctl Integrated in a Typical AMBA System

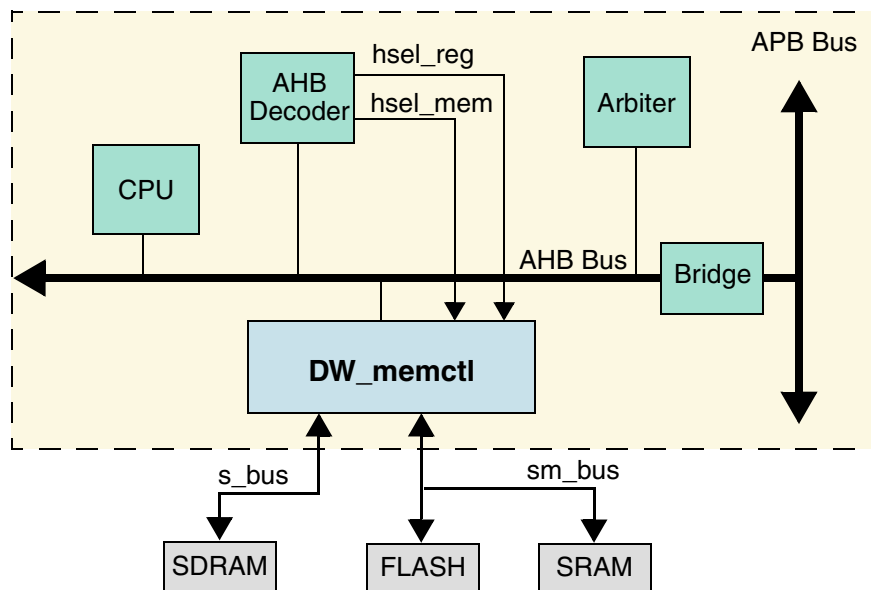
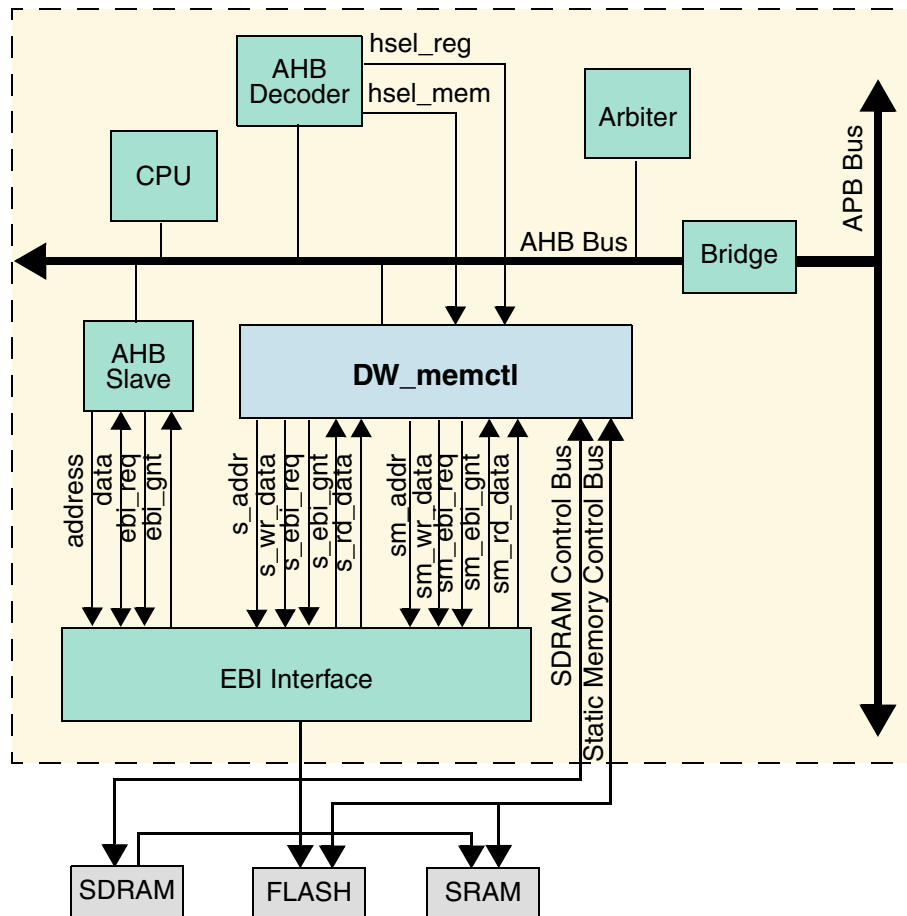


Figure 11-1 shows the DW_memctl instantiated with separate buses for Static memory and for SDRAM. The DW_memctl needs separate select signals – hsel_mem and hsel_reg – for the memory and control registers, which can be generated from the AHB decoder.

Figure 11-2 shows the DW_memctl sharing the external bus with another AHB slave through an External Bus Interface (EBI). For more details, refer to “External Bus Interface” on page 293.

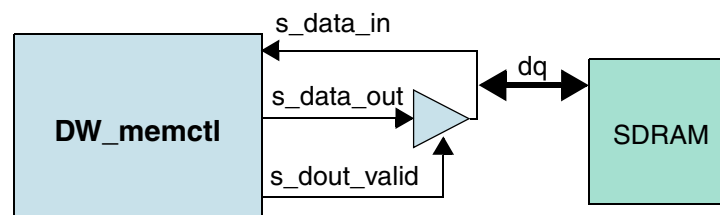
Figure 11-2 DW_memctl Sharing Pins with another AHB Slave through EBI



11.2.1 Bi-directional Data Buses

The DW_memctl provides only uni-directional signals. It provides output-enable signals for connecting uni-directional signals to bi-directional signals. Figure 11-3 shows the connection between the uni-directional SDRAM write and read data bus to the bi-directional memory bus.

Figure 11-3 Bi-directional Pin Connection



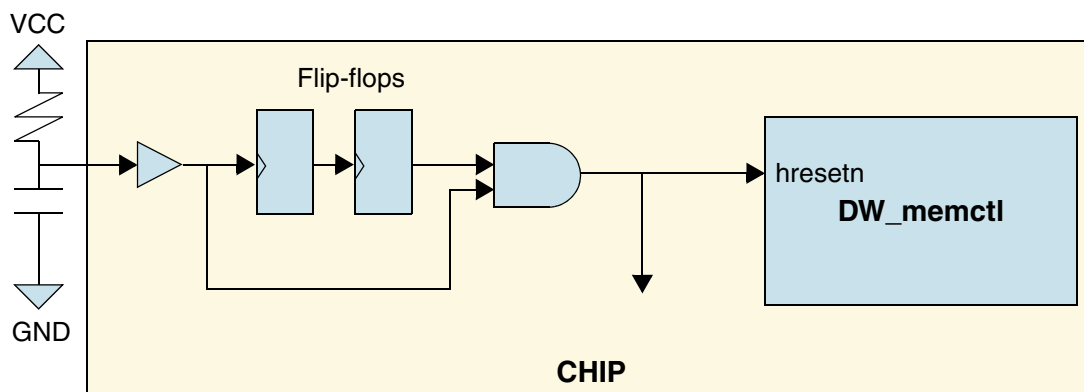
11.2.2 System Reset

In RTL, all the “clocked-always blocks” are coded for asynchronous reset, as shown in the following code; the reset is an active-low signal. In order to meet timing requirements and correct operation, the hresetn signal must be synchronously de-asserted with regard to the hclk clock and can be asynchronously asserted.

```
always @ (posedge hclk or negedge hresetn)
begin
    if (~hresetn)
    begin
        .....
    end
    else
    begin
        .....
    end
end
```

Figure 11-4 illustrates a typical system reset interface.

Figure 11-4 Typical System Reset Interface



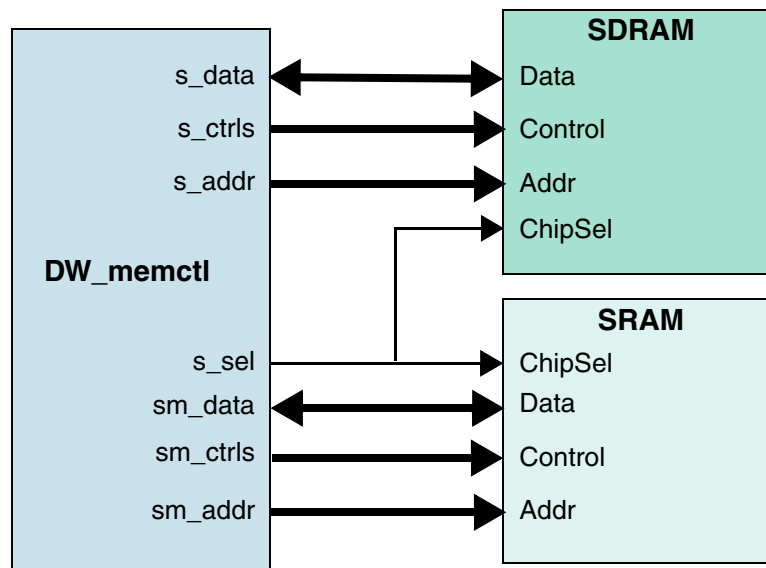
11.2.3 Data Connections

The following sections discuss various types of address and data connections.

11.2.3.1 Separate Address/Data Bus Connection

Figure 11-5 illustrates a connection where the SDRAM and Static memory address and data buses are separate.

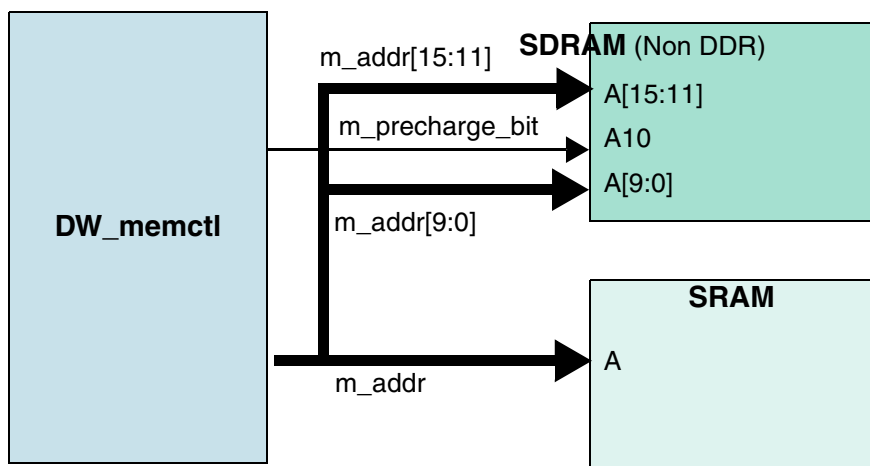
Figure 11-5 Separate Address/Data Bus Connection



11.2.3.2 Shared-Address Connection

Due to critical timing associated with DDR-SDRAMs, data-bus sharing between a DDR-SDRAM and Static memory is not allowed. Since DDR-SDRAMs need SST2 PADS, address bus sharing between a DDR-SDRAM and Static memory is not possible without an SST2-to-TTL/LVTTL translator outside the chip. [Figure 11-6](#) illustrates a connection where the addresses are shared between the SDRAM and the Static memory.

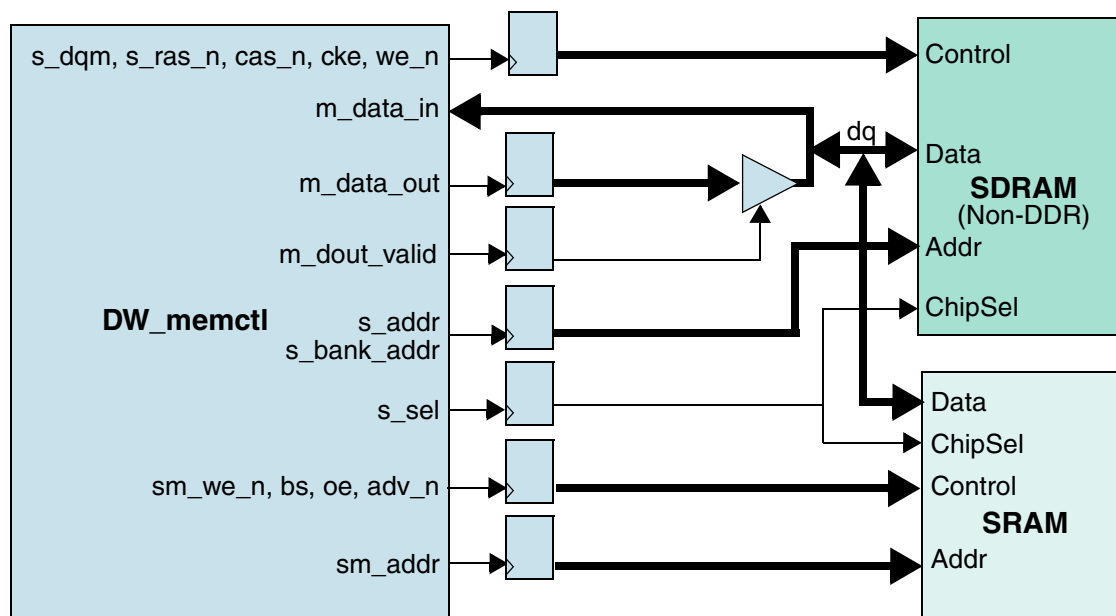
Figure 11-6 Shared-Address Connection



11.2.3.3 Shared-Data Connection - High Speed

Since `m_wr_data` has several MUXes after the flip-flops, it may not meet system timing. To solve this problem, `WRITE_PIPE`, `SM_WRITE_PIPE` should be set to 1, and all the address, control, and write-data outputs should be registered outside the core. [Figure 11-7](#) illustrates a high-speed connection where the data bus is shared.

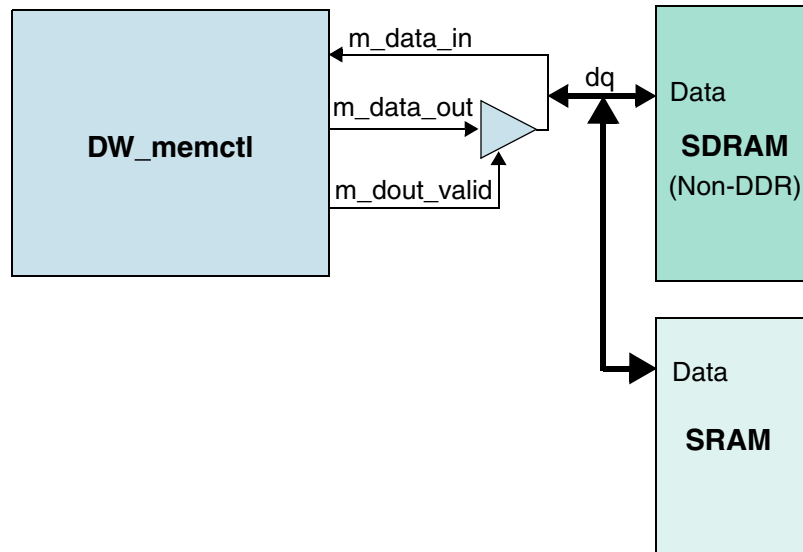
Figure 11-7 High-Speed Shared-Data Connection



11.2.3.4 Shared-Data Connection - Low Speed

Figure 11-8 illustrates a connection where the data bus is shared between the SDRAM and the Static memory.

Figure 11-8 Shared-Data Connection



In shared-data-bus mode, it is recommended that you add a write pipe to improve the write-data timing; the shared-data-bus mode without an external WRITE_PIPE is not directly supported in the GUI – that is, WRITE_PIPE=0 and SM_WRITE_PIPE=0. You can work around this by setting WRITE_PIPE/SM_WRITE_PIPE to 1 in the GUI, and later editing the src/DW_memctl_params.v file and changing the WRITE_PIPE and SM_WRITE_PIPE defines to 0. For more details, refer to [“Low-Performance – SDR-SDRAM, FLASH, On-Chip SSRAM”](#) on page 233.

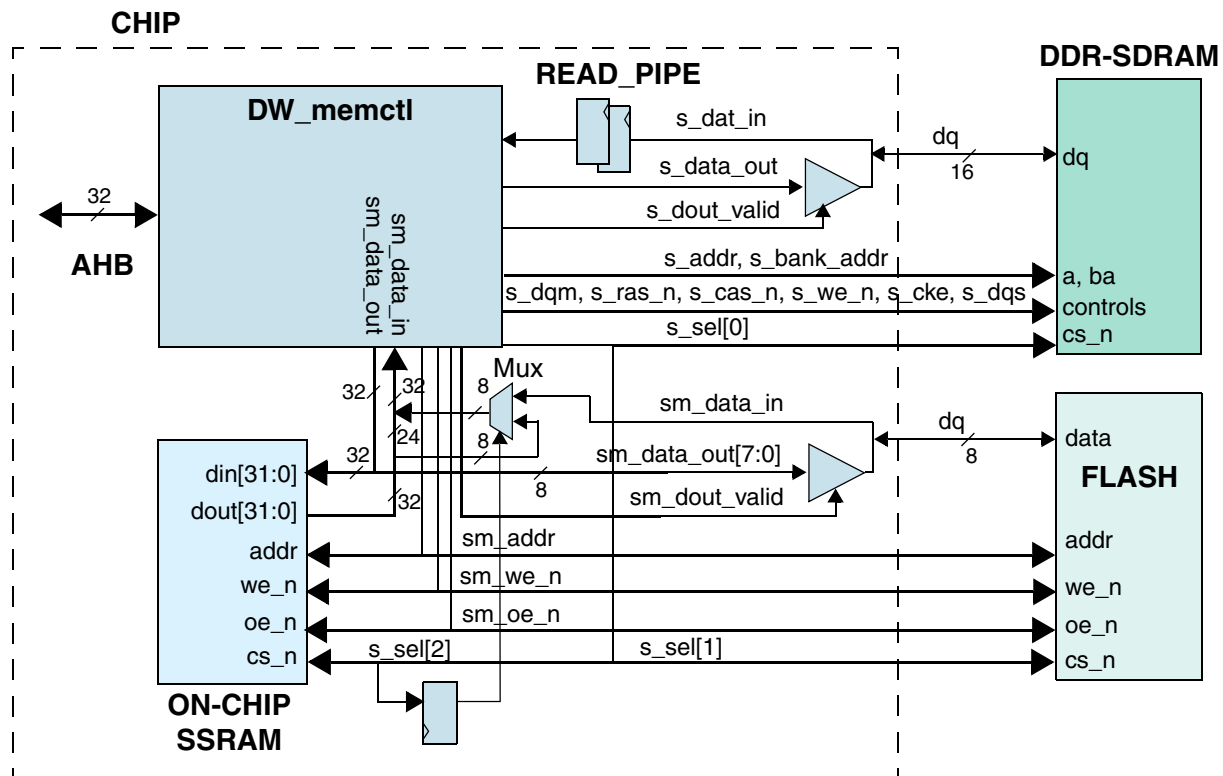
11.2.4 Typical Systems

The following sections discuss the performance of typical systems that contain different combinations of on-chip and off-chip situations involving SDRAMs and Static memories.

11.2.4.1 High-Performance – DDR-SDRAM, FLASH, On-Chip SSRAM

Figure 11-9 illustrates a typical high-performance 32-bit AHB system with on-chip SSRAM and off-chip DDR-SDRAM and FLASH memories. Parameters were chosen so that the SDRAM interface and external FLASH interface have separate address and data buses.

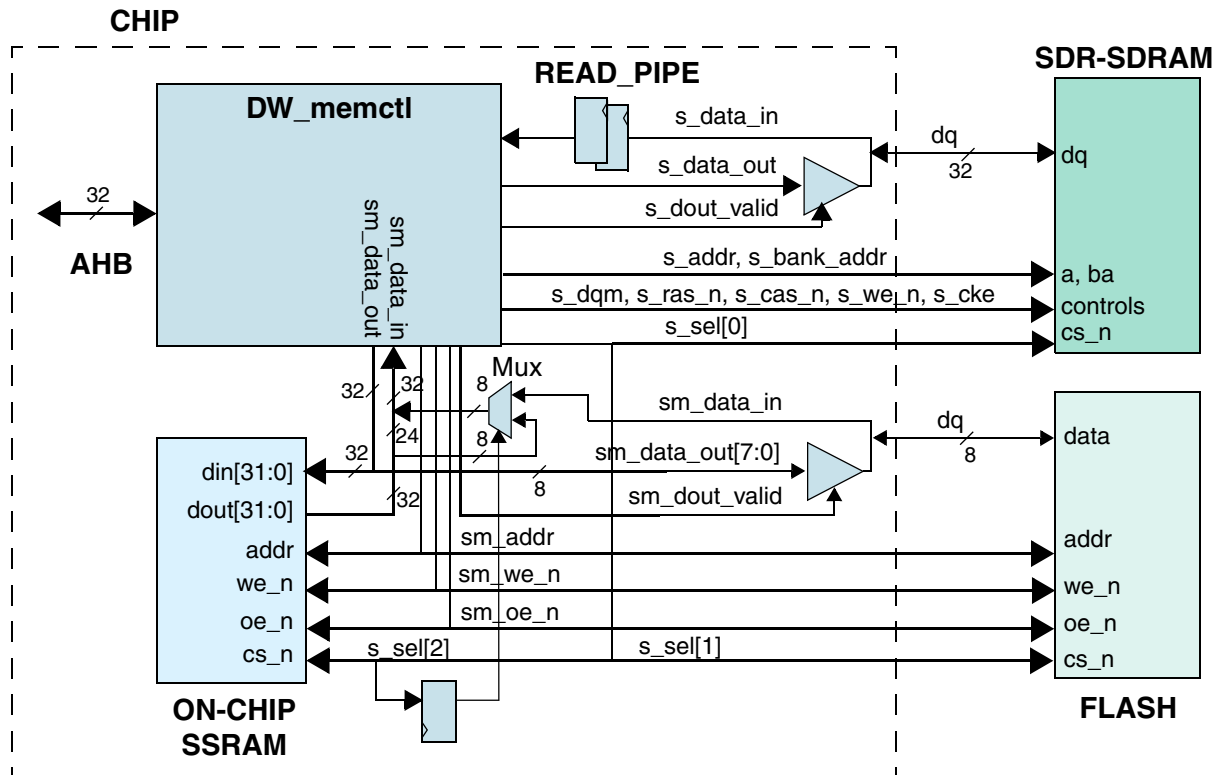
Figure 11-9 Typical High Performance System



11.2.4.2 High-Performance – SDR-SDRAM, FLASH, On-Chip SSRAM

Figure 11-10 illustrates a typical high-performance 32-bit AHB system with on-chip SSRAM and off-chip SDR-SDRAM and FLASH memories. Parameters were chosen so that the SDRAM interface and external FLASH interface have separate address and data buses.

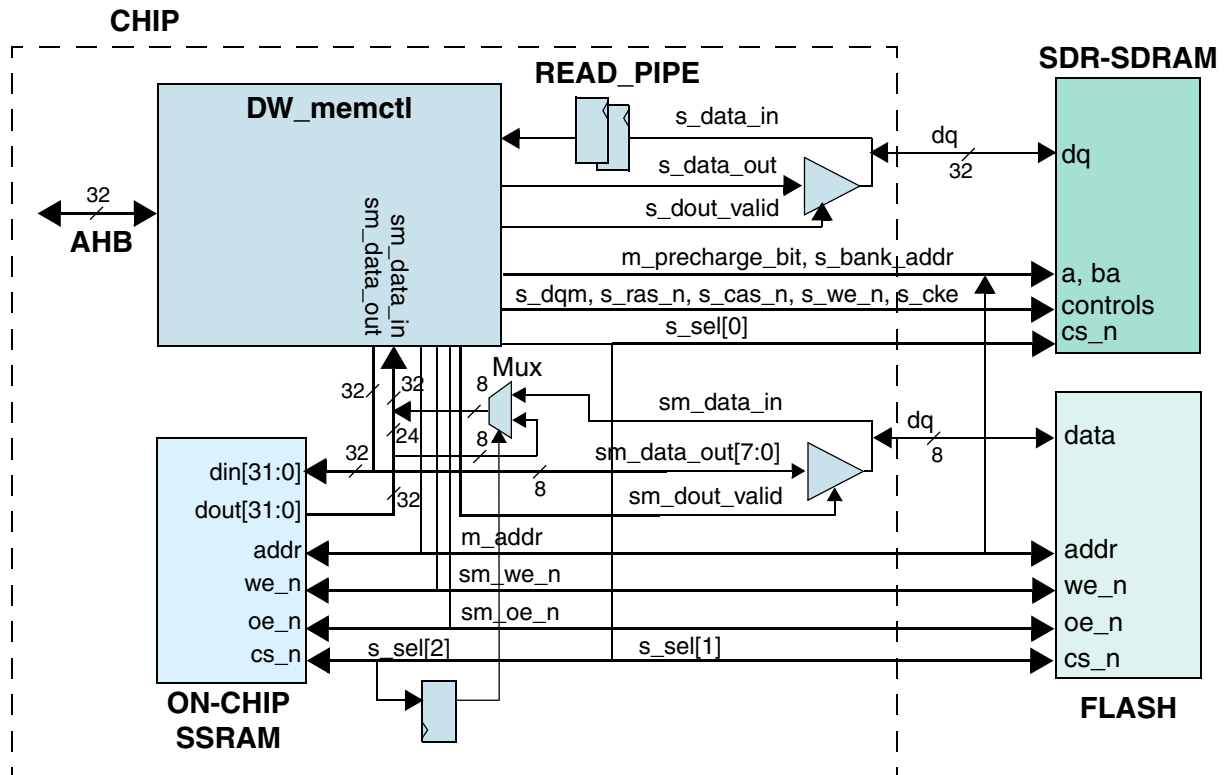
Figure 11-10 Typical High Performance System



11.2.4.3 Medium-Performance – SDR-SDRAM, FLASH, On-Chip SSRAM

Figure 11-11 illustrates a typical medium-performance 32-bit AHB system with on-chip SSRAM and off-chip SDR-SDRAM and FLASH memories. Parameters were chosen so that the SDRAM interface and the external FLASH interface share address buses, but have separate data buses.

Figure 11-11 Typical Medium Performance System

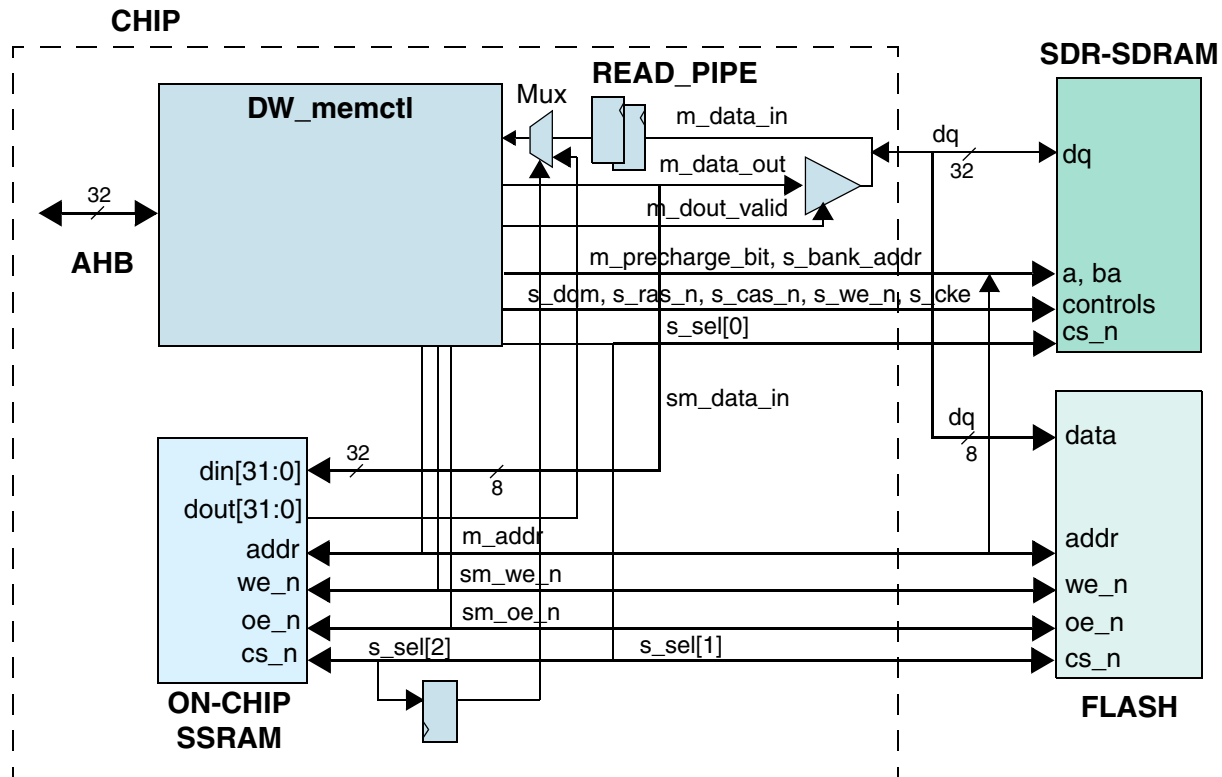


This system has fewer external pads compared to a system that has separate address buses, but this type of configuration does not affect the memory access performance. SDRAM refresh still occurs when a Static memory access happens; there is an additional MUX delay on the `m_addr` path, compared to the registered outputs `s_addr` and `sm_addr`. This situation has the potential to decrease the system frequency due to external signal routing if the SDRAM memories and the FLASH memory are located farther apart.

11.2.4.4 Low-Performance – SDR-SDRAM, FLASH, On-Chip SSRAM

Figure 11-12 illustrates a typical 32-bit AHB system with on-chip SSRAM and off-chip SDR-SDRAM and FLASH memories. Shared address and data buses are selected for the SDRAM interface and the external FLASH interface. This system has fewer external pads compared to the shared-address-only bus system. SDRAM refresh can occur when a Static memory access happens.

Figure 11-12 Typical Low-Performance System



In shared-data-bus mode, it is recommended that you add a write pipe to improve the write-data timing; the shared-data-bus mode without an external WRITE_PIPE – that is, WRITE_PIPE = 0 and SM_WRITE_PIPE = 0 – is not directly supported in the GUI. You can work around this by setting WRITE_PIPE/SM_WRITE_PIPE to “1” in the GUI, and later editing the src/DW_memctl_params.v file and changing the WRITE_PIPE and SM_WRITE_PIPE defines to “0.”

There are two paths to m_wr_data:

- One caused by the SDRAM memory path
- Another caused by the Static memory path. Since the Static memory interface supports 1:1 to 1:16 Memory:AHB data ratios, the path through Static memory is longer than the path through the SDRAM, which has only a 2:1 MUX after the flip-flops in order to support the 1:2 Memory:AHB data ratio.

Typically in a system, the SDRAM write data path is in the critical path, rather than the Static memory write data path. The Static memory timing path goes through DW_memctl/U_miu/U_smc/sm_write_data, and the SDRAM write data path goes through DW_memctl/U_miu/hiu_wr_data and a MUX. Note that the DW_memctl/U_miu/U_smc/sm_write_data also has a path through DW_memctl/U_miu/hiu_wr_data.

You must do two timing reports on the m_wr_data path to ensure that it meets your system timing requirement. Here is a sample report script:

```
report_timing -to m_wr_data
set_false_path -to m_wr_data -through {U_miu/U_smc/sm_write_data*}
report_timing -to m_wr_data
```

There is additional MUX delay on the m_addr path compared to the registered outputs, s_addr and sm_addr. Similarly, there is more delay on the m_data_out path compared to s_data_out. These delay differences have the potential to lower the system frequency due to external signal routing if SDRAM memories and the FLASH memory are located farther apart.

The timing inside the chip could be improved by adding a WRITE_PIPE.

11.2.5 Register Insertion

The following sections discuss different situations in which a register is inserted.

11.2.5.1 Register Insertion on Memory R/W Data Path

(READ_PIPE and WRITE_PIPE)

In a typical system, the clock seen by the memory is a delayed version of the DW_memctl clock input, since the memory clock input must go through the pads; this infers some delay. Other factors outside the chip can also add to this delay.

For the write data path, the same delay is applicable to the data and control signals generated by the DW_memctl, since they both have to traverse through the pads. However, the situation is different and more critical for the read data path.

The read data output from the memory are delayed by the memory access time from the clock edge. These data are further delayed on their return path from the memory to the DW_memctl through the pads.

After layout, the DW_memctl can be anywhere on a chip and not necessarily as close to the pins as the designer would like. This is especially true in a system that has a wide SDRAM data-bus interface – for example, 128 bits – where the SDRAM data pads are spread along two or three sides of a chip. Because of this, there can be unpredictable routing delays before the read data gets into the read buffer. You should be aware of these potential delays when using the DW_memctl in an application.

In the following discussions, these assumptions have been made:

- hclk – clock input to DW_memctl
- d_clk – actual clock input to SDRAM (delayed)
- DW_command – SDRAM command control signals at output of DW_memctl
- S_command – SDRAM command control signals as seen by SDRAM (delayed)
- S_data – read data output at SDRAM pins
- s_rd_data – read data from SDRAM at input pins of DW_memctl (delayed)
- DW_data_buf – read data after input pads.
- CAS Latency – assumed to be three

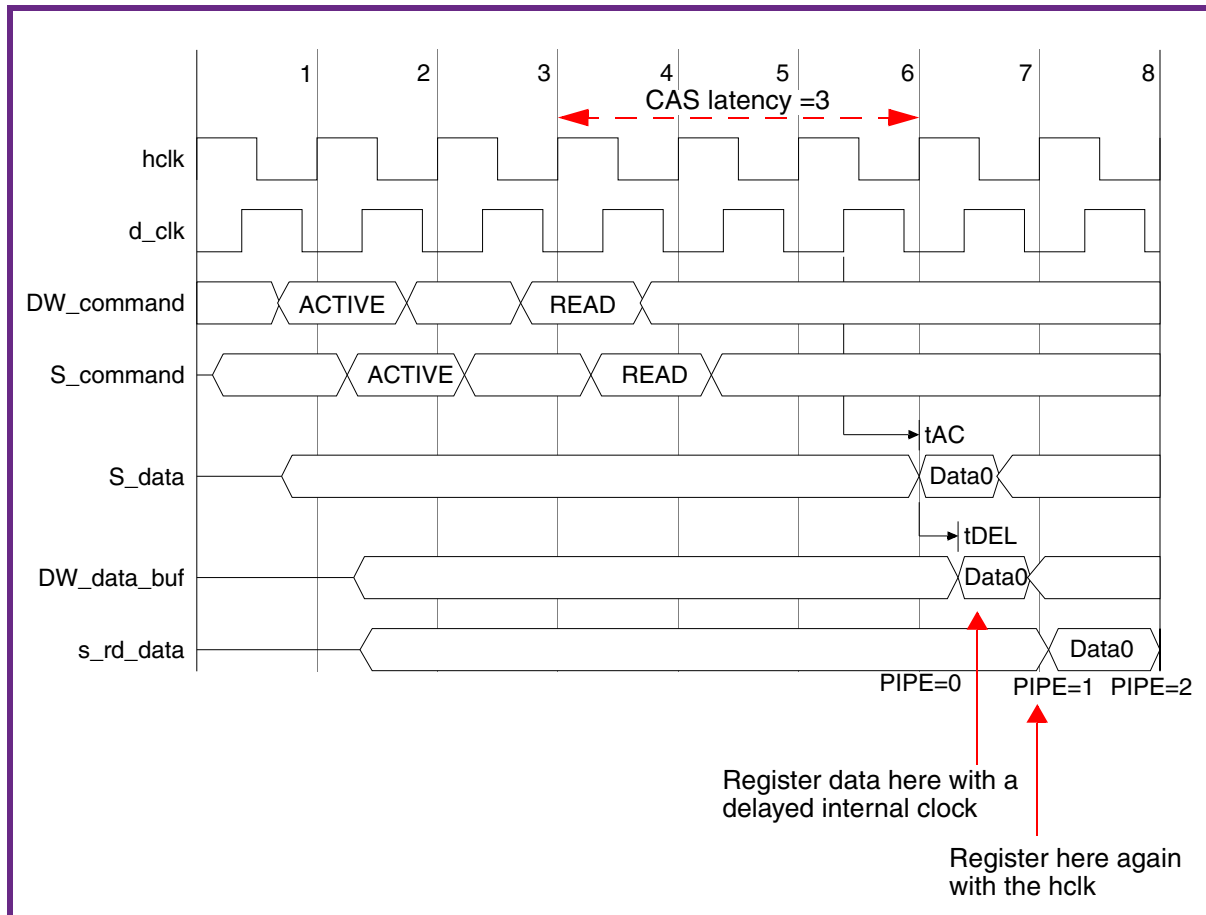
- tDEL – trace delay + pad delay for read data path
- tAC – SDRAM access time

11.2.5.2 Register Insertion on Memory Read Data Path

(READ_PIPE)

Figure 11-13 shows the timing diagram of a scenario in which it is necessary to put registers external to the DW_memctl in order to ensure that the read data are properly registered inside the DW_memctl.

Figure 11-13 Register Insertion Necessary on Read Data Bus



In this case, the clock seen by the memory (d_clk) has significant delay from the clock that the DW_memctl sees (hclk); the SDR-SDRAM access time is also slightly higher. Additionally, the SDR-SDRAM data are valid for only a short 3-4ns window, and they should be captured.

The read data are available on the SDR-SDRAM data bus at the rising edge of clock 6 of hclk. However, because of the further delay due to the pads (tDEL), the actual data are available only after the rising edge of clock 6 on hclk. In this case, the read data must be registered by a clock that is similar to the delayed clock (d_clk) outside the DW_memctl. Another register must also be added that captures the data on the hclk. Therefore, this example has a READ_PIPE of 2 with two registers in the read data path.

If you put registers on the read data path of the SDR-SDRAM, then the READ_PIPE compile-time parameter should get a value that denotes the number of flip-flops in the read data path outside the DW_memctl; in the case of a Static memory read data bus, the corresponding parameter is SM_READ_PIPE_SET0/1/2.



Note Asynchronous Static memories do not need READ_PIPES.

By extending the T_{RC} read access time, you can control when the read data will be available to the DW_memctl; there is no critical capture window. In the case of pipelined Static memories, the pipe is inside the memories. In order to simultaneously interface to different types of Static memories, the SM_READ_PIPE_SET n is separate for each set.

Figure 11-14 and Figure 11-15 illustrate register insertion on the read data path for different SDR-SDRAM data width ratios used by the DW_memctl. Probably the only time you might not insert registers on the SDR-SDRAM read data path is when the system is operating at a very low frequency, in the order of 25-33 MHz.

Figure 11-14 Register Insertion in SDR-SDRAM Read Data Path - Data Width Ratio = 1:1

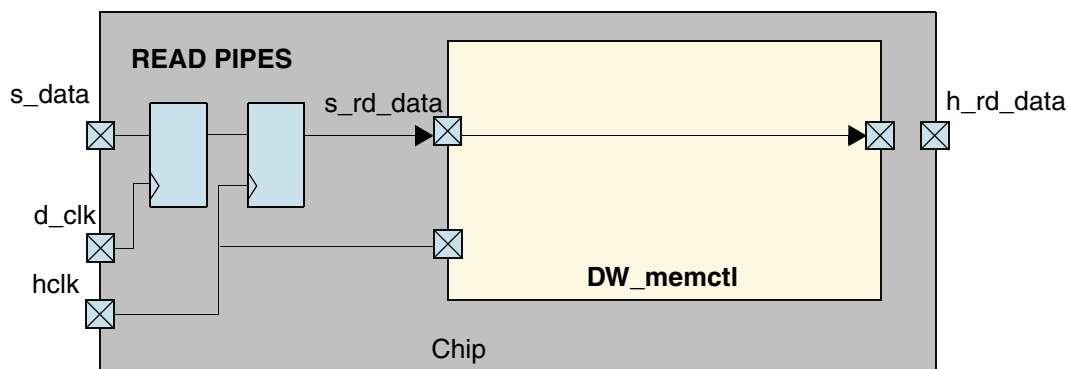


Figure 11-15 Register Insertion in SDR-SDRAM Read Data Path - Data Width Ratio = 2:1

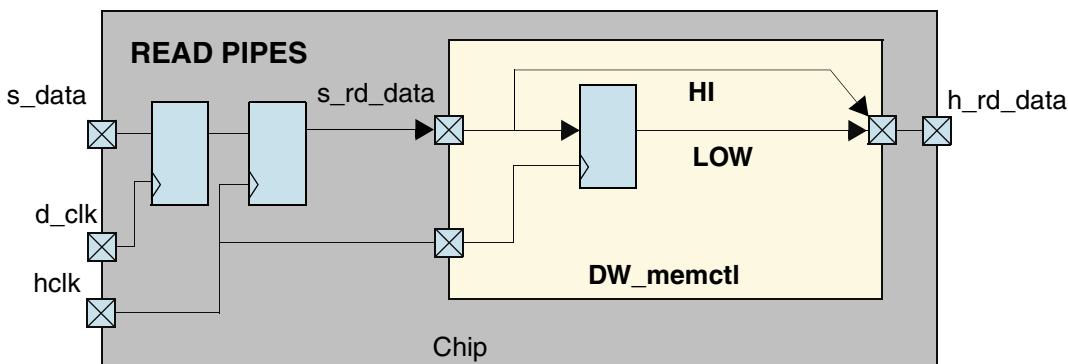


Figure 11-16 and Figure 11-17 illustrate register insertion on the read data path for the different DDR-SDRAM data width ratios used by the DW_memctl.

Figure 11-16 Register Insertion in DDR-SDRAM Read Data Path - Data Width Ratio = 2:1

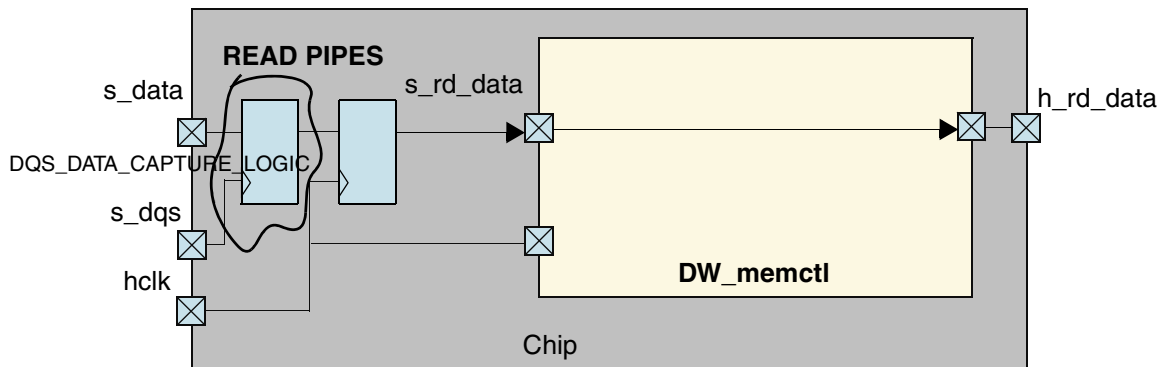


Figure 11-17 Register Insertion in DDR-SDRAM Read Data Path - Data Width Ratio = 4:1

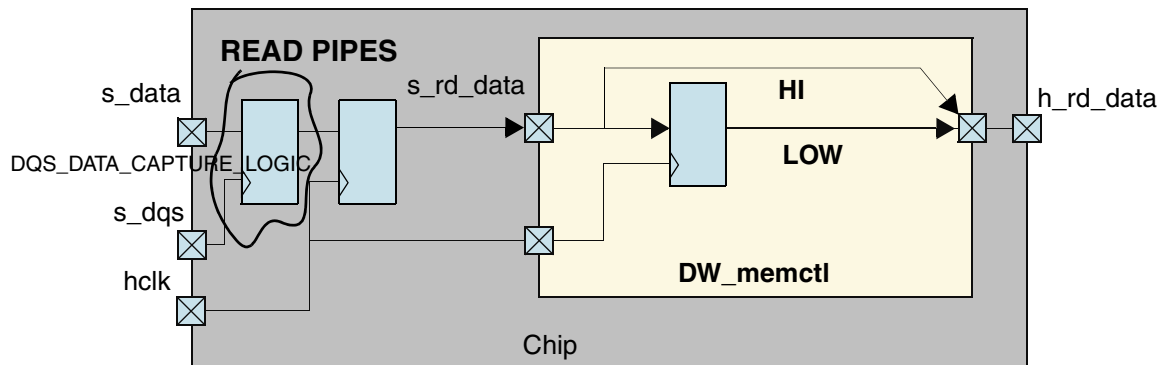
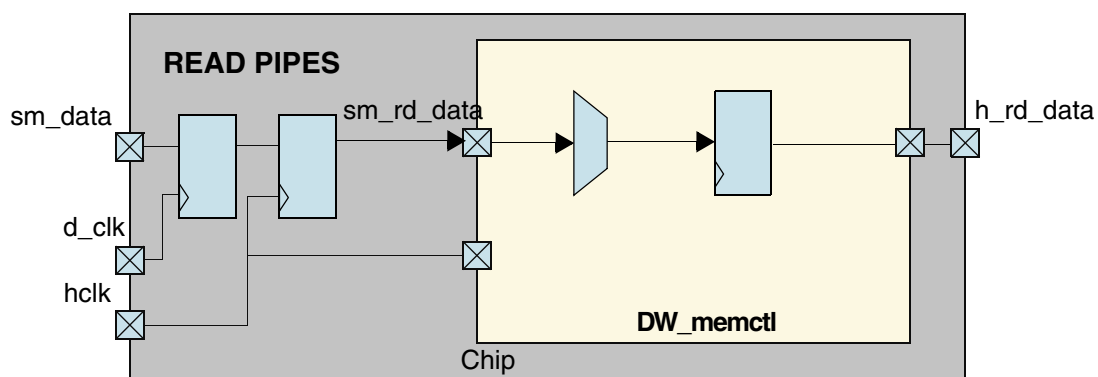


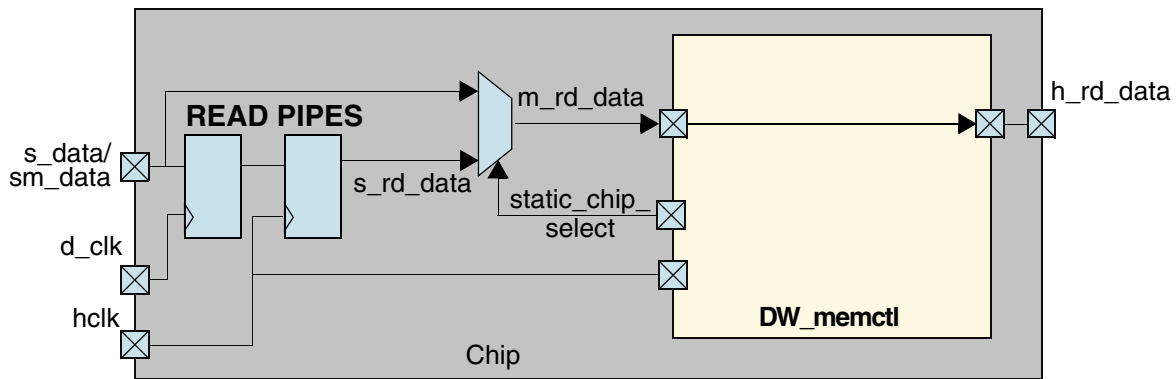
Figure 11-18 illustrates register insertion on the read data path for different Static memory data width ratios used by the DW_memctl.

Figure 11-18 Register Insertion in Static Memory Read Data Path



In shared-data-bus mode, you can choose different read pipes for SDRAM and Static memories; this is allowed in order to support pipes within a memory. Additionally, you can pipe the critical timing data coming from the SDRAMs and connect the non-critical timing Static memory data without READ_PIPEs, as shown in Figure 11-19.

Figure 11-19 Critical SDRAM and Non-Critical Static Memory Data Paths



The SM_READ_PIPEs can be different for different Static memory sets in order to support SSRAMs and SRAMs in the same controller. For example, a write-through SSRAM already has one READ_PIPE inside the memory, whereas an asynchronous FLASH memory does not have any READ_PIPE inside the memory.

11.2.5.3 Register Insertion on Memory Write Data, Address and Control Path

(WRITE_PIPE)

The write data output from the DW_memctl is the output of the write data FIFO (registered output). Since the memory controller supports programmable 1:1 and 2:1 AHB:SDRAM data-width ratios, there is also a 2:1 MUX after the FIFO (the synthesis constraint in 75% output delay on s_wr_data) in the case of an SDR-SDRAM. In the case of a DDR-SDRAM, the write data are clocked again by the 2x clock.

If you insert additional logic on the write data path outside the DW_memctl (for an external bus interface) or if the DW_memctl is too far from the pads, then you may need to register the write data path close to the pads. If you plan to register the write data path, then you must also register all the control signals – such as s_sel_n, s_ras_n, s_cas_n, s_we_n, s_cke, s_dqm, s_dout_valid, s_addr, s_bank_addr – so that the controller operates properly.

Figure 11-20 illustrates register insertion on a non-DDR-SDRAM write path. The combination logic is the optional extra logic you may need to put into the design for interfacing to a shared external bus interface (EBI).

Figure 11-20 SDR_SDRAM Register Insertion in Write Path

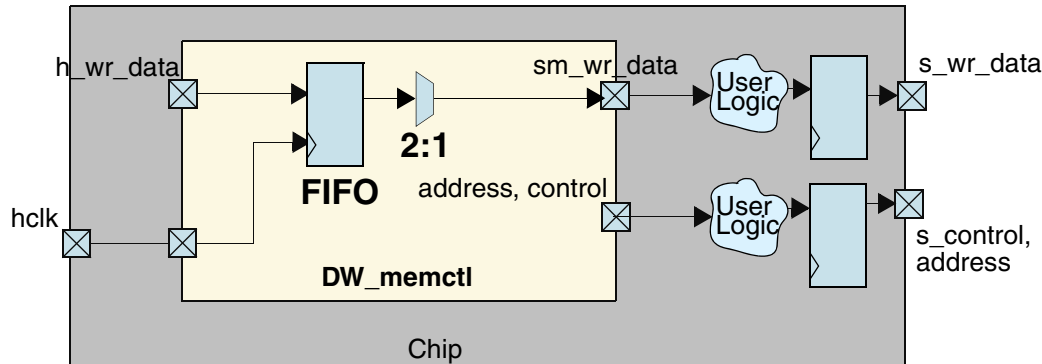
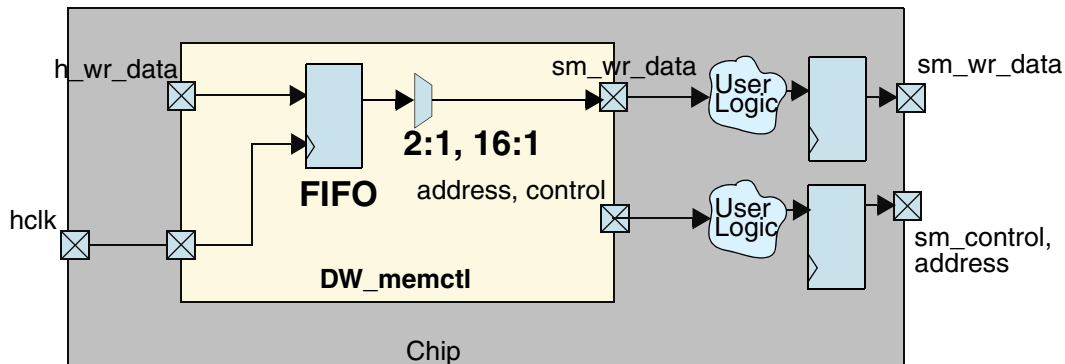


Figure 11-21 illustrates register insertion on the Static memory write path. The combination logic is the optional extra logic you may need to put into the design for interfacing to a shared external bus interface (EBI).

Figure 11-21 Static-Memory Register Insertion in Write Path

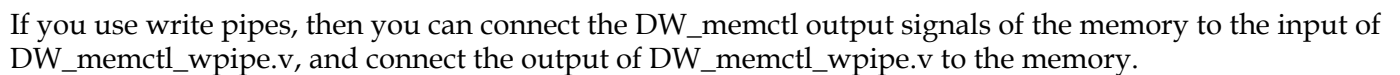


If you put registers on the write data path of memory outside the DW_memctl, you should set the WRITE_PIPE compile-time parameter to a value that denotes the number of flip-flops in the write data path outside the DW_memctl. In the case of a Static memory write data bus, the corresponding parameter is SM_WRITE_PIPE.

In the case of a Static memory, the MUX after the FIFO is larger because the memory controller supports from 1:1 to 16:1 ratios on the AHB:Static data width. If you add a register in the Static memory write data path, then you must also add registers on the Static memory control signals – such as s_sel_n, sm_oe_n, sm_we_n, sm_bs_n, sm_dout_valid, sm_addr.

In the case of an SDRAM, the memory controller adds CAS latency to the READ_PIPE and WRITE_PIPE parameters and uses that value to calculate when to expect the read data from the SDRAM. Similarly, in the case of Static memories, the SM_READ_PIPE_SETn, SM_WRITE_PIPE, and T_RC read_access time are added to find out when the controller can expect the read data. Since the read access is piped, you can still do a read access once on every T_RC clocks.

Figure 11-22 DDR_SDRAM Register Insertion in Write Path



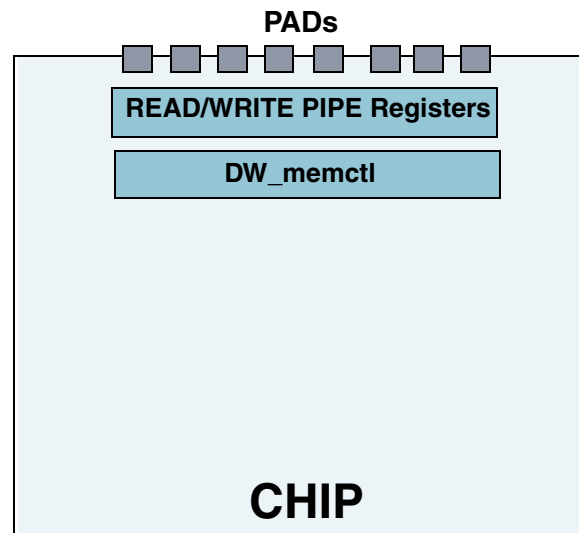
11.2.6 Design Considerations

The following sections discuss recommendations for creating a DW_memctl design.

11.2.6.1 Placement of DW_memctl in a Chip

It is recommended that you place the DW_memctl and the READ_PIPE and WRITE_PIPE registers close to the pads. This is crucial for timing-critical SDRAM signals; the interface to a FLASH may not have timing-critical signals.

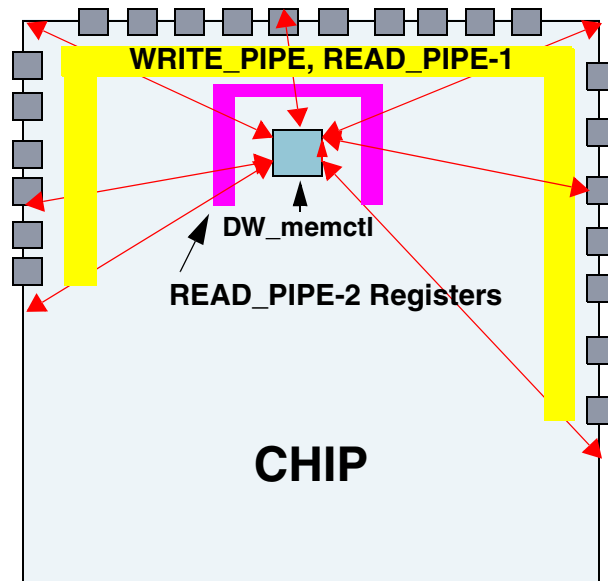
Figure 11-23 DW_memctl Placement Inside Chip



11.2.6.2 Wider Data Bus System Interface Challenges

The READ_PIPE and WRITE_PIPE registers are very useful in wider-data bus systems; that is, 128 bits or above. Since there are so many data pads, there is no optimal location to place the DW_memctl inside the chip, even though the memory controller is very small, as illustrated in Figure 11-24.

Figure 11-24 DW_memctl Placement Inside Chip



Since the pads span multiple sides of the chip, the write data that are driven from the memory controller to the pads will arrive at the pads at different times, which could be in the order of a few nanoseconds. Similarly, the read data from the pads to the DW_memctl will also have a large skew with respect to each other.

Routing and skew control outside the chip is a different problem to solve. Probably the best way to resolve this is to add a WRITE_PIPE flip-flop at the end of the write data close to the pads; similarly, add the first READ_PIPE flip-flop close to the pads and the second READ_PIPE flip-flop close to the DW_memctl.

11.2.7 Different System Clock Topologies

Figure 11-25, Figure 11-26, and Figure 11-27 illustrate a few ways that the DW_memctl and SDRAM clocks can be connected.

Figure 11-25 DW_memctl Clock and SDR-SDRAM Clock Connection-1

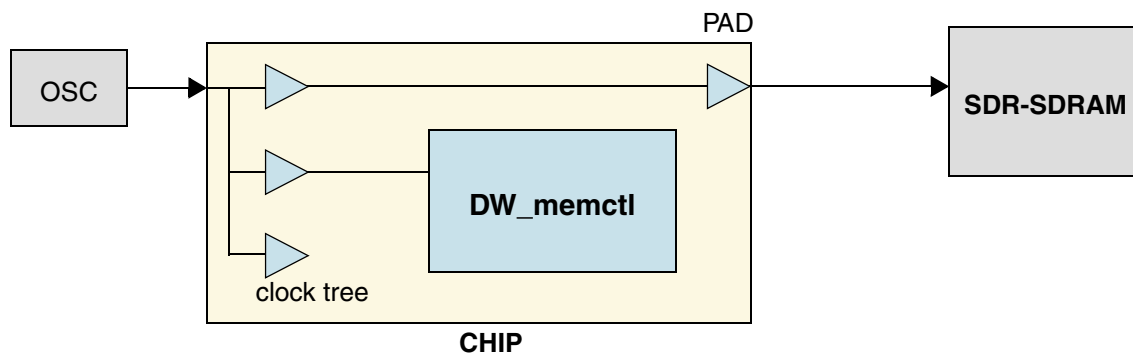


Figure 11-26 DW_memctl Clock and SDR-SDRAM Clock Connection-2

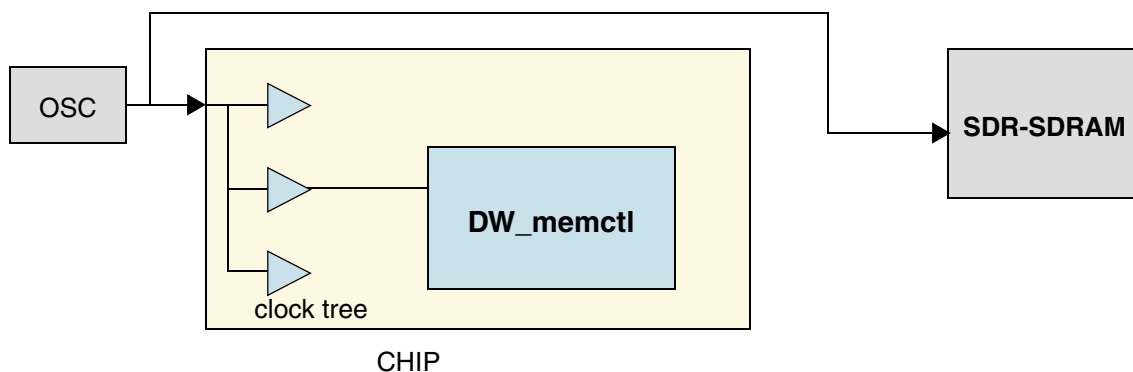


Figure 11-27 DW_memctl Clock and SDR-SDRAM Clock Connection-3

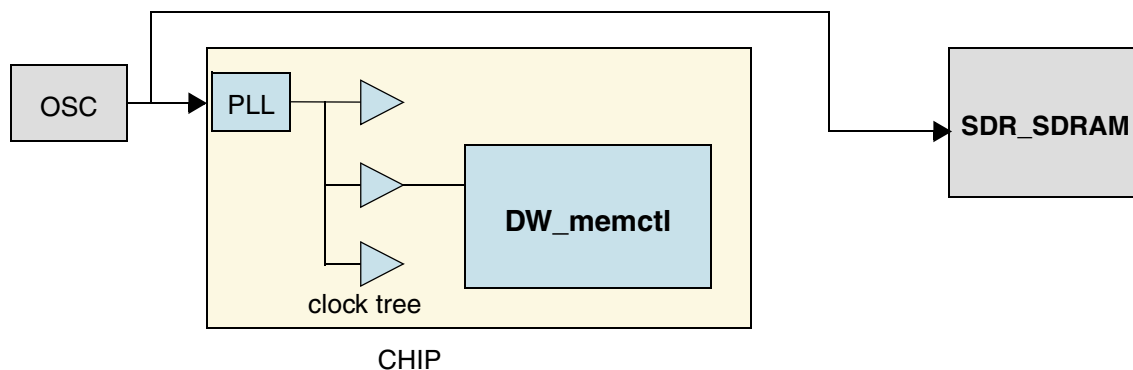
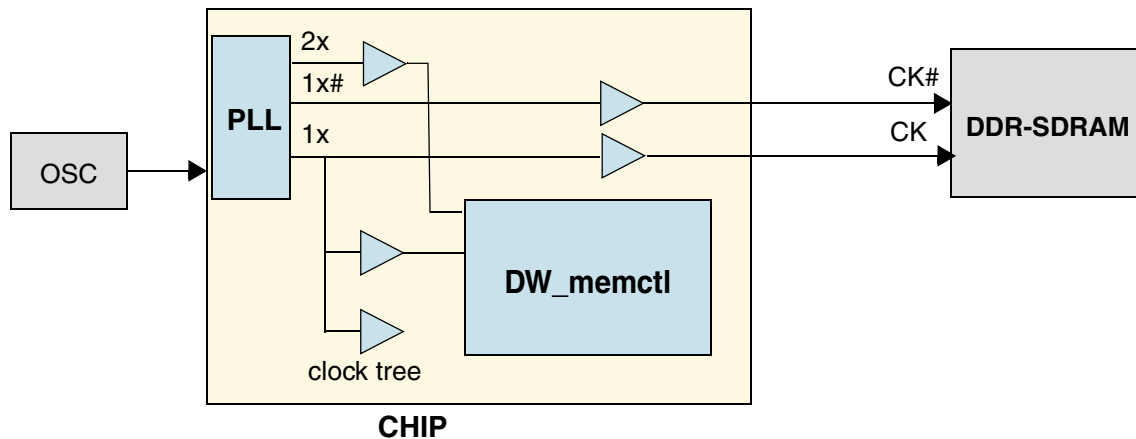


Figure 11-28 DW_memctl Clock and DDR-SDRAM Clock Connection

11.2.8 Typical Interfaces

The following sections discuss some common situations and alternate approaches in which memories are interfaced in a DW_memctl system.

11.2.8.1 Typical SDR-SDRAM/Mobile SDR-SDRAM Interface – Approach 1

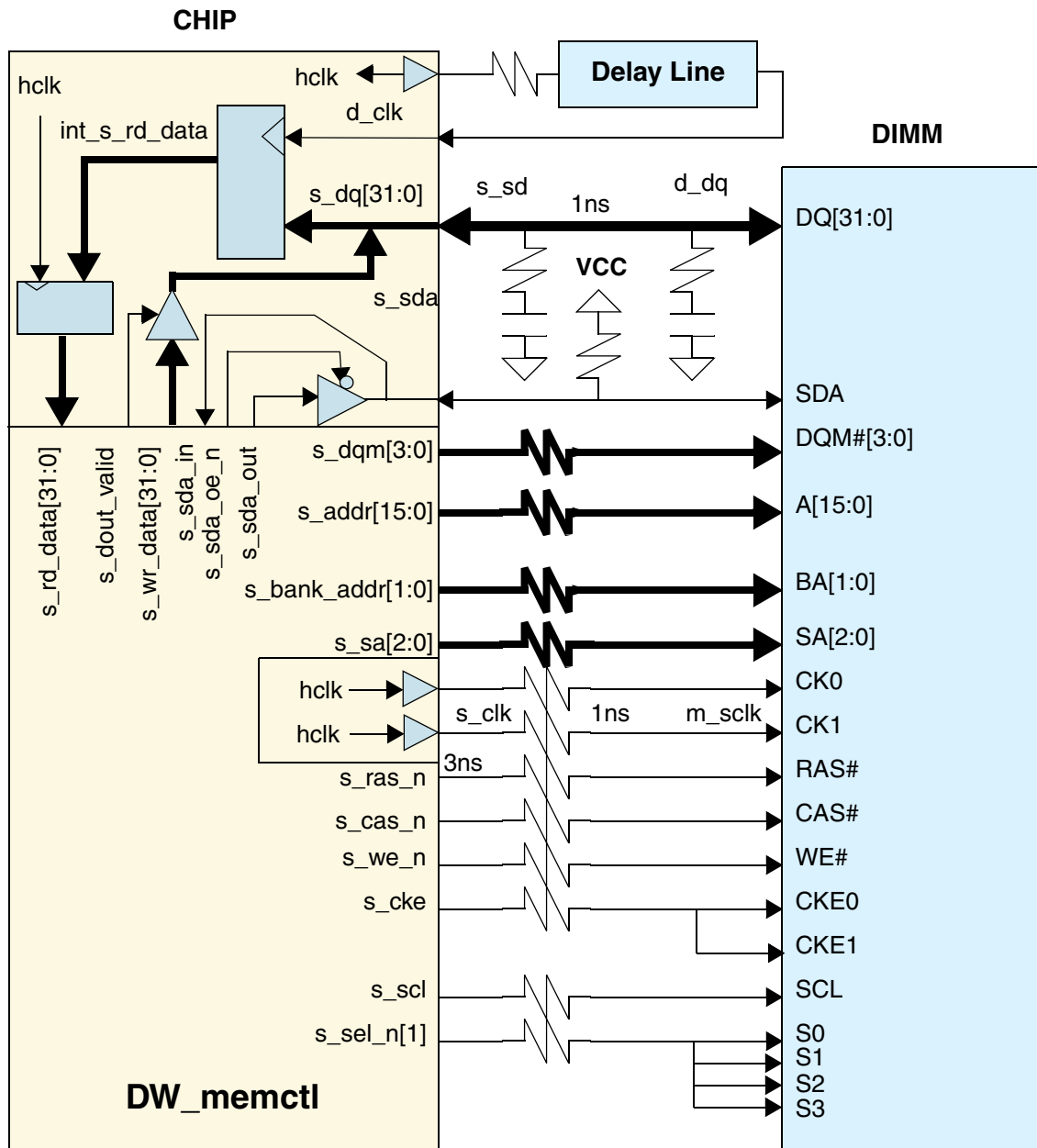
You should take great care when interfacing SDRAMs to the DW_memctl, since the SDRAM interface typically works at 133MHz. There are multiple ways to connect the clock of an SDRAM to the internal clock of the DW_memctl.

One method is to drive the internal chip clock through an output pad and connect it to the SDRAM chips.

Another method is to use a PLL to synchronize the internal clock connected to the DW_memctl and the clock to the SDRAMs. Since the clock also travels along with the other SDRAM signals from the DW_memctl to the SDRAM, this method helps to guarantee that the setup and hold times of the SDRAM input signal are met – assuming the same output pads, equal load, trace length on all the signals, and clock.

Figure 11-29 illustrates a typical SDRAM interface to the DW_memctl.

Figure 11-29 SDRAM Interface - READ_PIPE = 2; WRITE_PIPE = 0



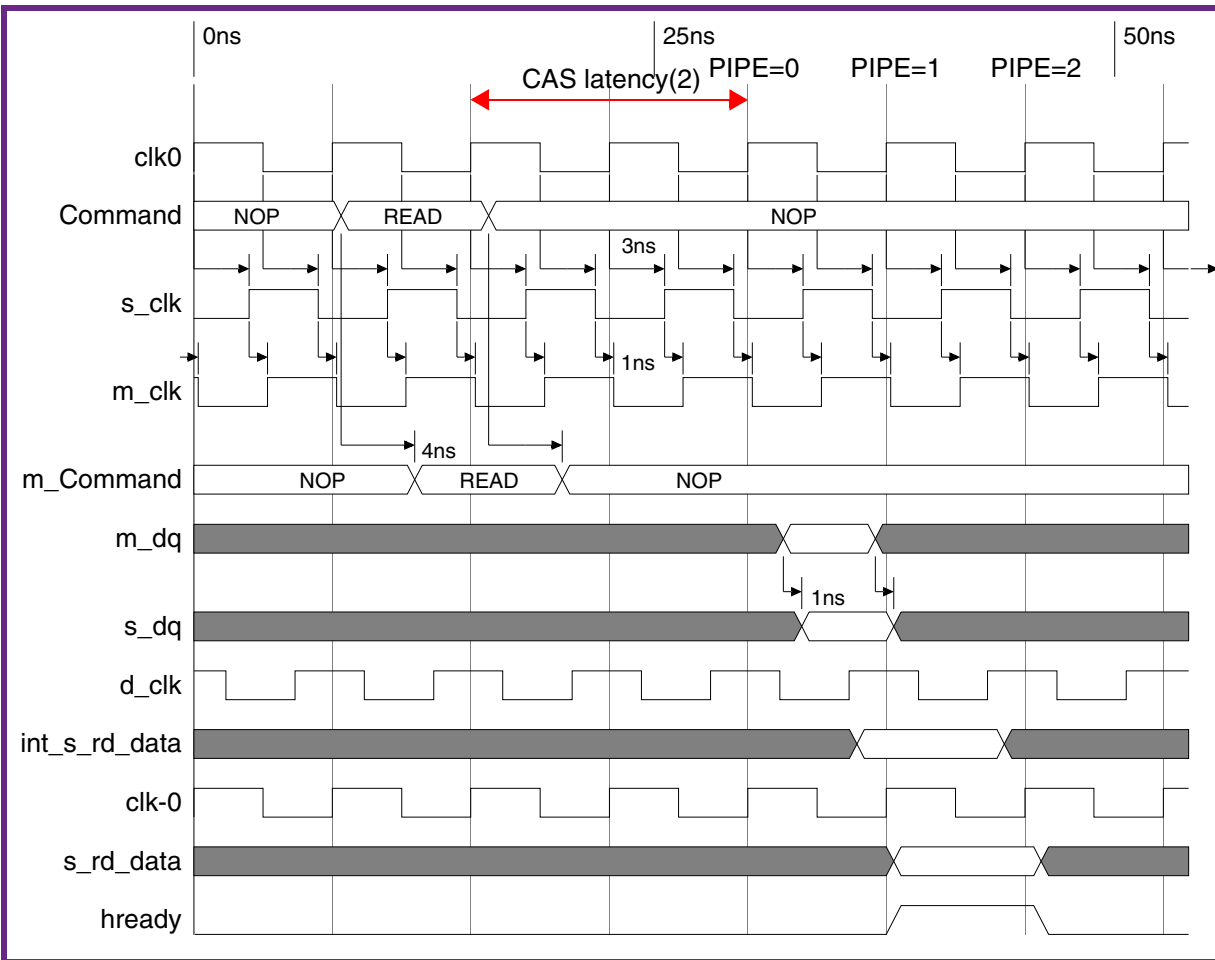
Another critical path in the SDRAM interface is the “read-data” capture. Usually, the SDRAM read data are valid only during a short window – typically 3ns to 4ns. The read data travels from the SDRAM to the input pad of the chip; these read data need to be latched inside the chip through a delayed version of an internal clock or an external “capture clock,” which can be generated outside the chip as shown in the example.

These input flip-flops should be placed close to the input pads. Depending on the operating speed of your device, the data from the first stage of the flip-flops need to be recaptured with the internal clock before passing the data to the DW_memctl.

The last stage of the flip-flops needs to be close to the DW_memctl block, since the s_rd_data are directly passed to AHB read-data.

The timing diagram in [Figure 11-30](#) illustrates the READ_PIPE with a value of 2.

Figure 11-30 SDRAM Timing Diagram – READ_PIPE = 2, CAS Latency = 2



Specifics of the illustration are:

- hclk – 7.5ns
- hclk – s_clk delay (clock output driver delay 3ns)
- s_clk – m_clk delay (transmission line delay 1ns)
- CAS latency – 2
- m_clk – m_dq (memory access time 6ns)
- m_dq – s_dq delay (transmission line delays 1ns)
- READ_PIPE – 2

This is a typical situation; you should calculate min/max values for both read and write paths in order to ensure that SDRAM setup/hold requirements are met, and also to ensure that the read data can be captured correctly. It is recommended that you run a SPICE analysis on this path to ensure that design constraints are met.

Whenever the DW_memctl block is far away from the pads, you may have to insert another read pipe in the read data path. Since the s_rd_data are driven directly into AHB hrddata, the DW_memctl needs s_rd_data with a maximum 10% input delay, which means the last read-pipe register needs to be close to the DW_memctl block.

When READ_PIPE = 0, it denotes an ideal system that needs to take only the SDRAM CAS latency into account.



READ_PIPE does not necessarily mean the number of registers in the read data path.

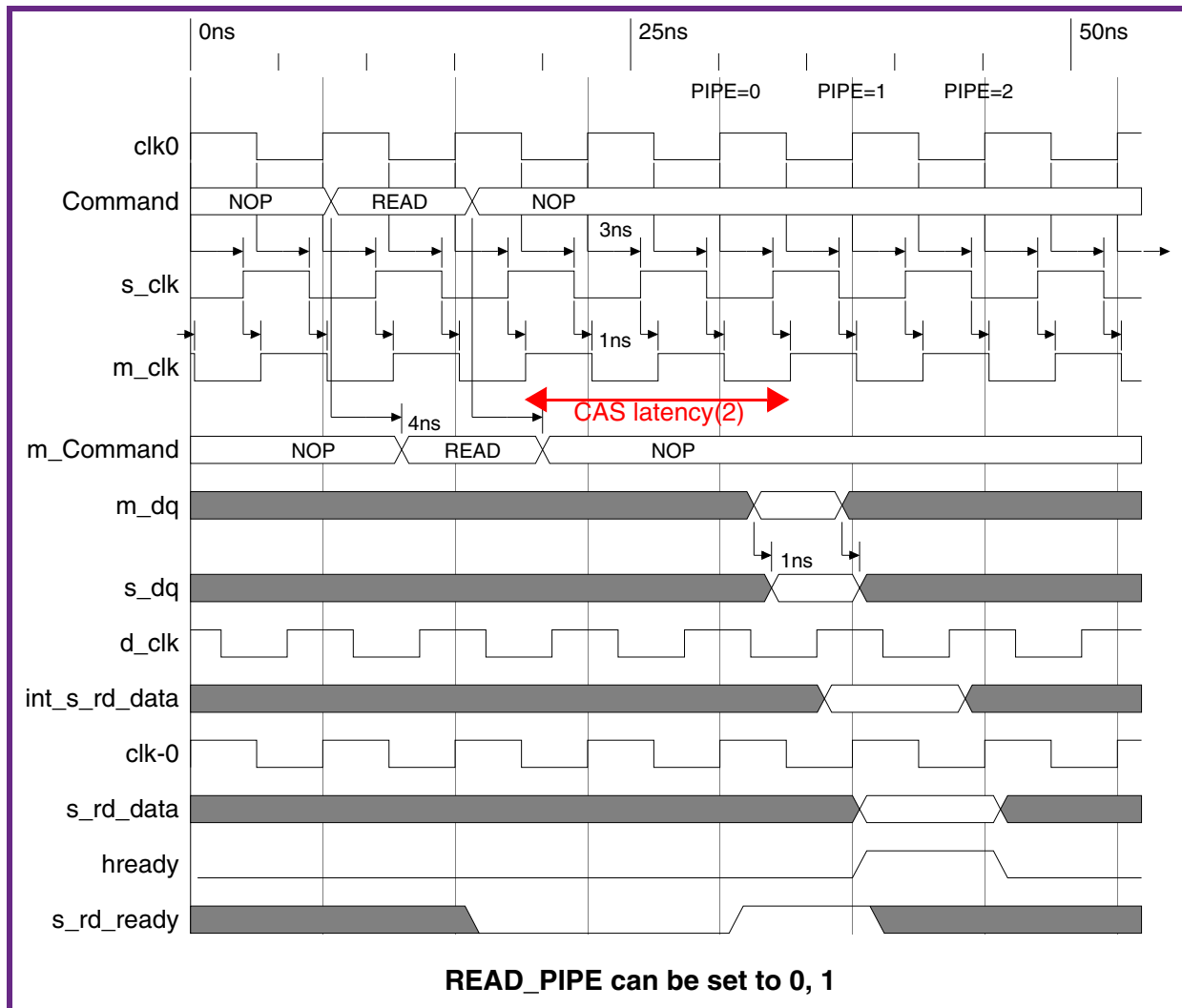
11.2.8.2 Typical SDR-SDRAM/Module SDR-SDRAM Interface – Approach 2

In some systems, you may not be able to accurately predict the skew between SDRAM read data and the internal hclk, but you may be able to generate a handshake signal, which indicates that the read data are ready.

You can select this mode by setting bit 17 of the SCTL register to 1. When you enable this handshake mode, the memory controller looks for a 1 on the s_rd_ready pin by counting the number of clocks equal to “CAS-latency + READ_PIPE - 1” after the SDRAM read command was issued. A 1 on s_rd_ready indicates that the read data are ready on the next clock. DW_memctl samples the s_rd_ready signal only once during a burst read; once DW_memctl detects a 1 on s_rd_ready, then the read data are expected every clock during a burst.

The timing diagram in [Figure 11-31](#) illustrates this SDRAM handshake method.

Figure 11-31 Timing Diagram Using SDRAM Handshake Method



11.2.8.3 Typical SDR-SDRAM/Module SDR-SDRAM Interface – Approach 3

In some systems, the relationship between the SDRAM read-data and the internal hclk may not be predictable. However, you can guarantee that the read-data will be ready in either one clock or definitely by the next hclk. You can use the following data sampling technique for these situations; however, keep in mind that this approach may add additional clocks in the read-data latency.

This approach uses a register file with a size of 2; the memory controller has sideband signals. The hclk is gated with the `s_rd_dqs_mask` sideband signal in order to generate a latching signal for the register file. The `s_rd_dqs_mask` becomes active in a time equal to $hclk/2$ before the expected read-data clock edge; `s_rd_dqs_mask` goes inactive in a time equal to $hclk/2$ after the last read-data clock edge. To match the clock and read-data trace delays and buffer delays, the `masked_clk` signal is routed outside to match the `clk+s_dq` trace.

Since two registers are used in this example, this design can handle up to a two-clock uncertainty between the data coming from the SDR-SDRAMs and the data being read by the controller. The first data are latched in register 1, the second data in register 2, and the third data in register 1 again. The sideband signals `s_rd_start`, `s_rd_pop`, and `s_rd_end` are used to generate the latch-enable and read-data-MUX control signals.

The timing diagram in [Figure 11-32](#) illustrates this method.

Figure 11-32 Timing Diagram with Read-Data Ready Guarantee

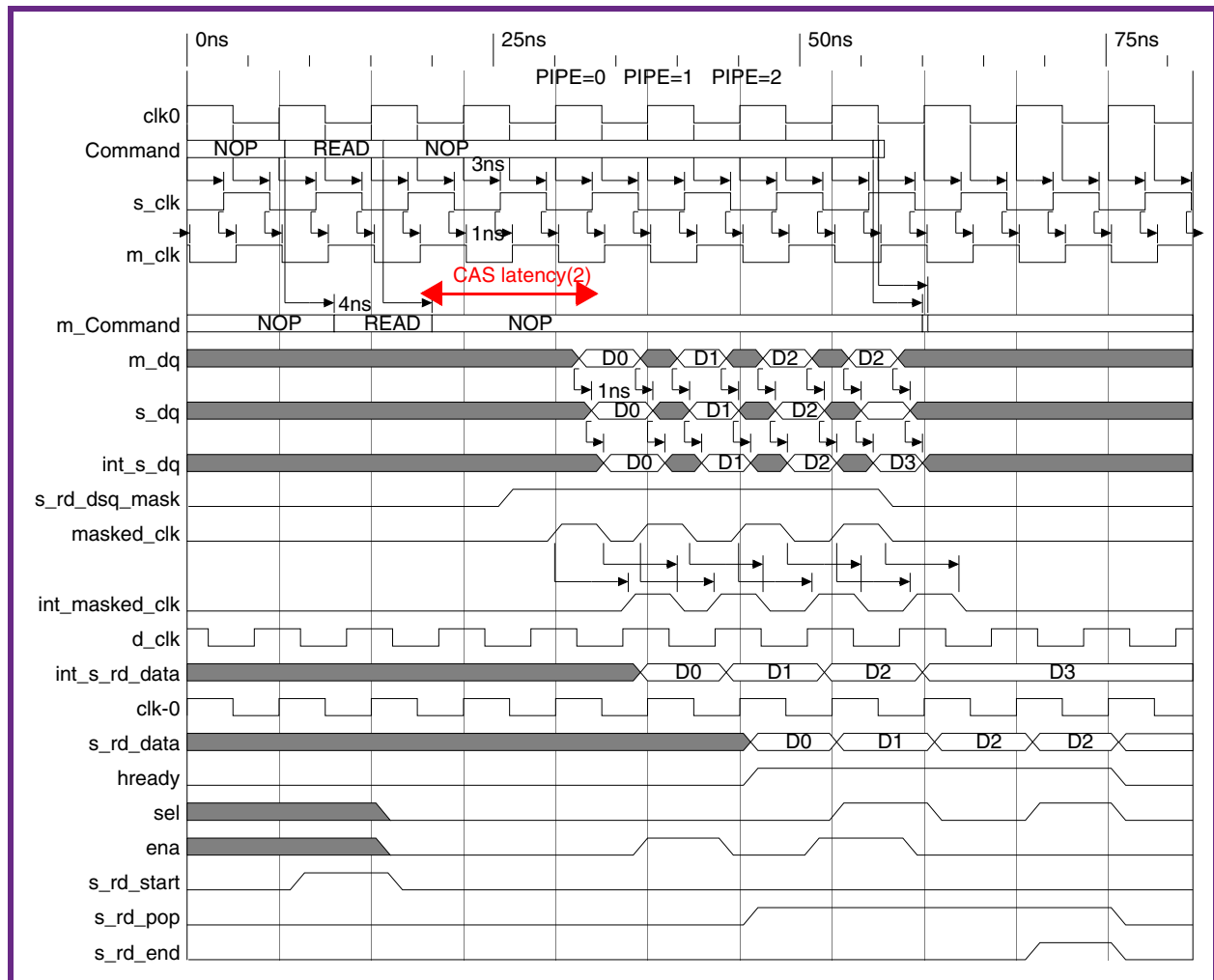
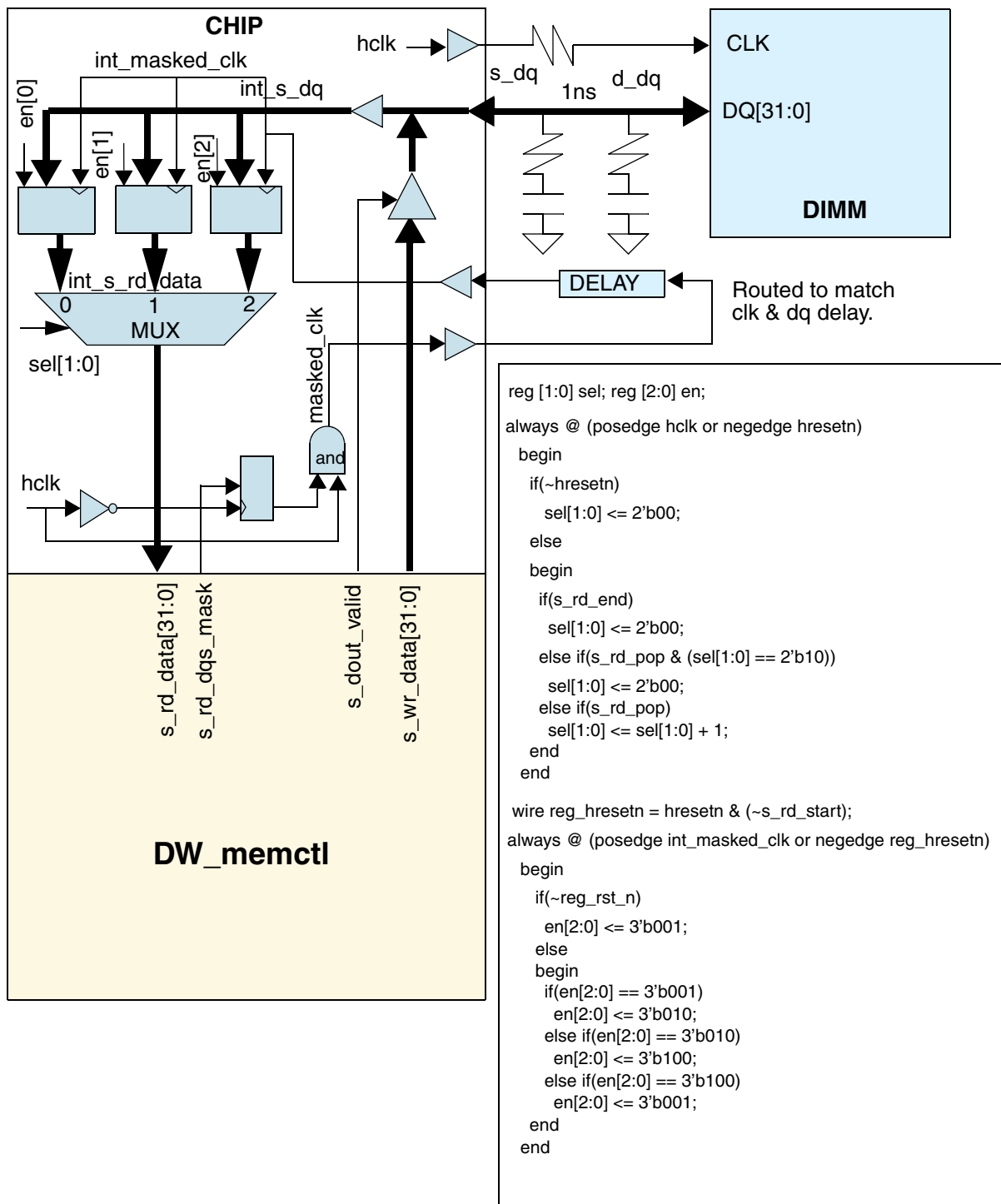


Figure 11-33 illustrates another method of interfacing an SDRAM to the DW_memctl.

Figure 11-33 SDRAM Interface - READ_PIPE = 3; WRITE_PIPE = 0



11.2.8.3.1 SDRAM Application Notes

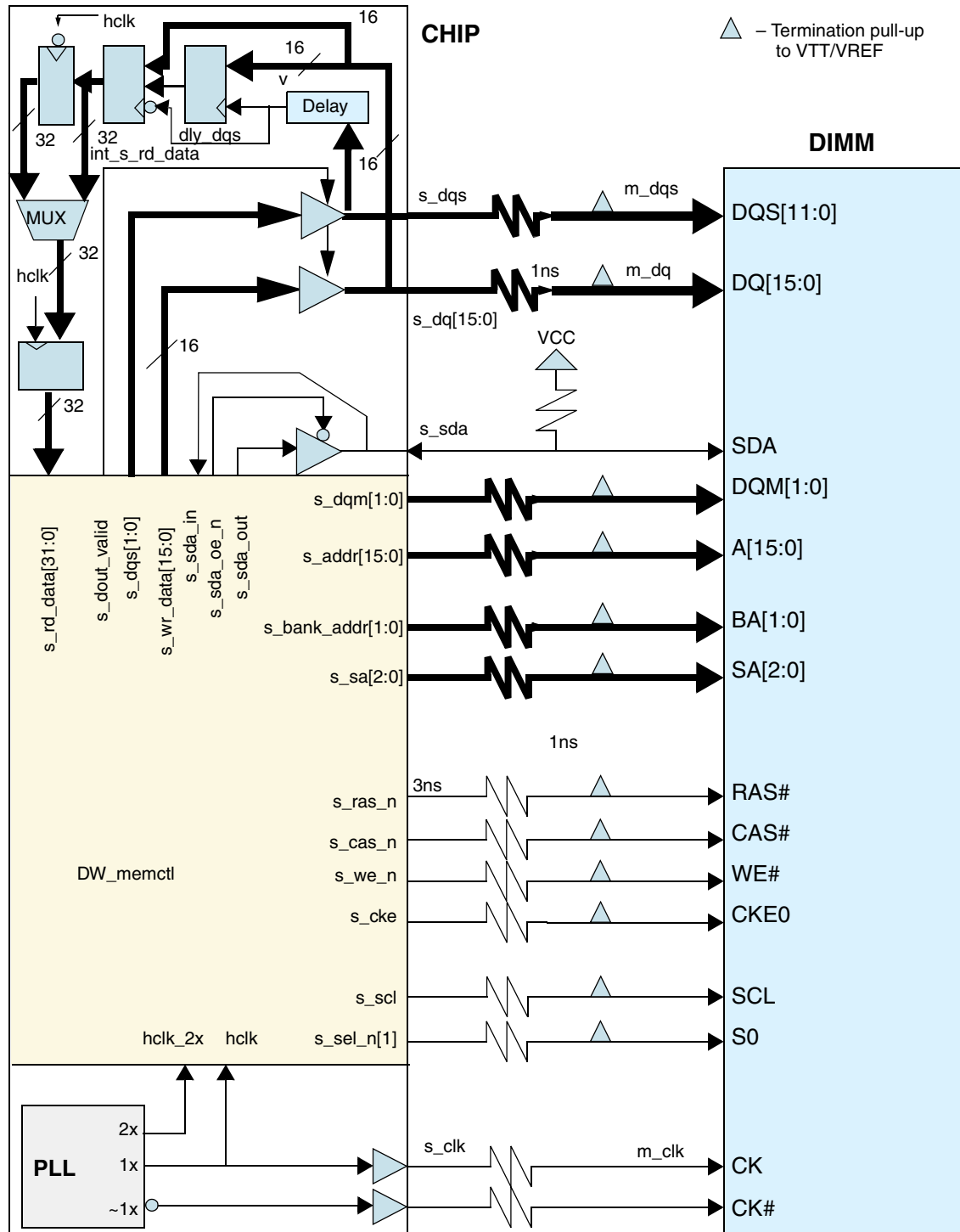
For more information on SDRAM issues, refer to the application notes on the Micron and Samsung web sites:

- http://www.micron.com/support/part_info/technotes
- <http://www.samsung.com/global/business/semiconductor/products/dram/DRAMApplicationNote.html>

11.2.8.4 Typical DDR-SDRAM Interface – Approach 1

Figure 11-34 illustrates a typical DDR-SDRAM interface to the DW_memctl controller.

Figure 11-34 DDR-SDRAM Interface - READ_PIPE = 2; WRITE_PIPE = 0



This example assumes that the skew between the dqs and hclk signals is predictable, and therefore the DDR-SDRAM read data captured by the delayed dqs can be re-captured by the system hclk. Since the DDR-SDRAM CAS latency can have a resolution of half a clock – that is, 1.5, 2, 2.5, and so on – an additional register and a MUX are shown in [Figure 11-34](#).

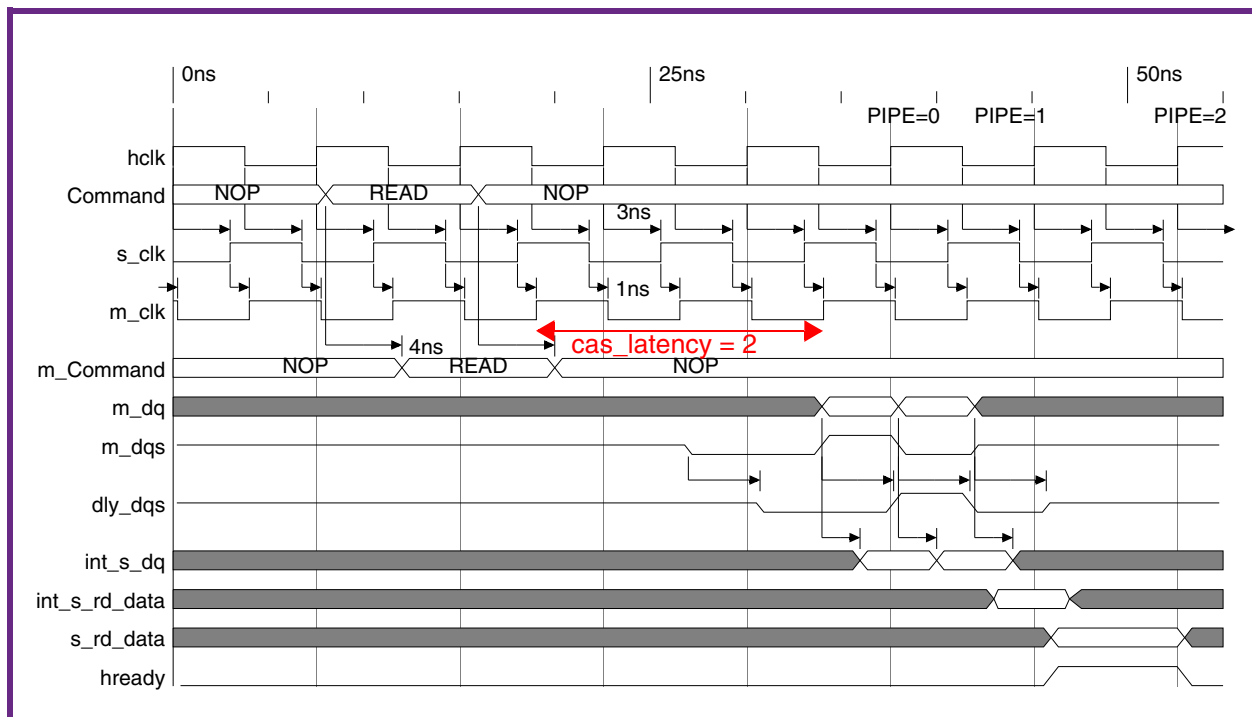
The MUX select signal can be controlled, depending upon whether the CAS latency has a half-clock or full-clock resolution. This predictable read-data latency method is recommended when the system clock is slow or when you have done careful layout and extensive SPICE analysis. This method always guarantees the lowest read-data latency.

**Note**

When compared with an SDR-SDRAM, the read data of a DDR-SDRAM are expected one clock later for the same CAS latency.

The timing diagram in [Figure 11-35](#) illustrates the READ_PIPE with a value of 2.

Figure 11-35 DDR-SDRAM Timing Diagram – READ_PIPE = 2, CAS Latency = 2



Specifics of the illustration are:

- hclk – 7.5ns
- hclk – s_clk delay (clock output driver delay 3ns)
- s_clk – m_clk delay (transmission line delay 1ns)
- CAS latency – 2
- m_clk – m_dq (memory access time +/-0.75ns)

- $m_dq - int_s_dq$ delay (transmission line delays 1ns+pad delay 1ns)
- $READ_PIPE - 2$

This is a typical calculation; one needs to do min/max calculations for both read and write paths to ensure that the SDRAM setup/hold is met, and also that the read data can be captured correctly. It is recommended that you run a SPICE analysis on this path to ensure the design constraints are met. For CAS latencies which have half a clock granularity, the $READ_PIPE$ is calculated after the CAS latency is rounded to the next rounded integer.

When $READ_PIPE = 0$, it denotes an ideal system that needs to take only the SDRAM CAS latency into account.



$READ_PIPE$ does not necessarily mean the number of registers in the read data path.

Whenever the DW_memctl block is far away from the pads, you may have to insert another read pipe in the read data path. Since the s_rd_data are driven directly into AHB hrddata, the DW_memctl needs s_rd_data with a maximum 10% input delay, which means the last read-pipe register needs to be close to the DW_memctl block.

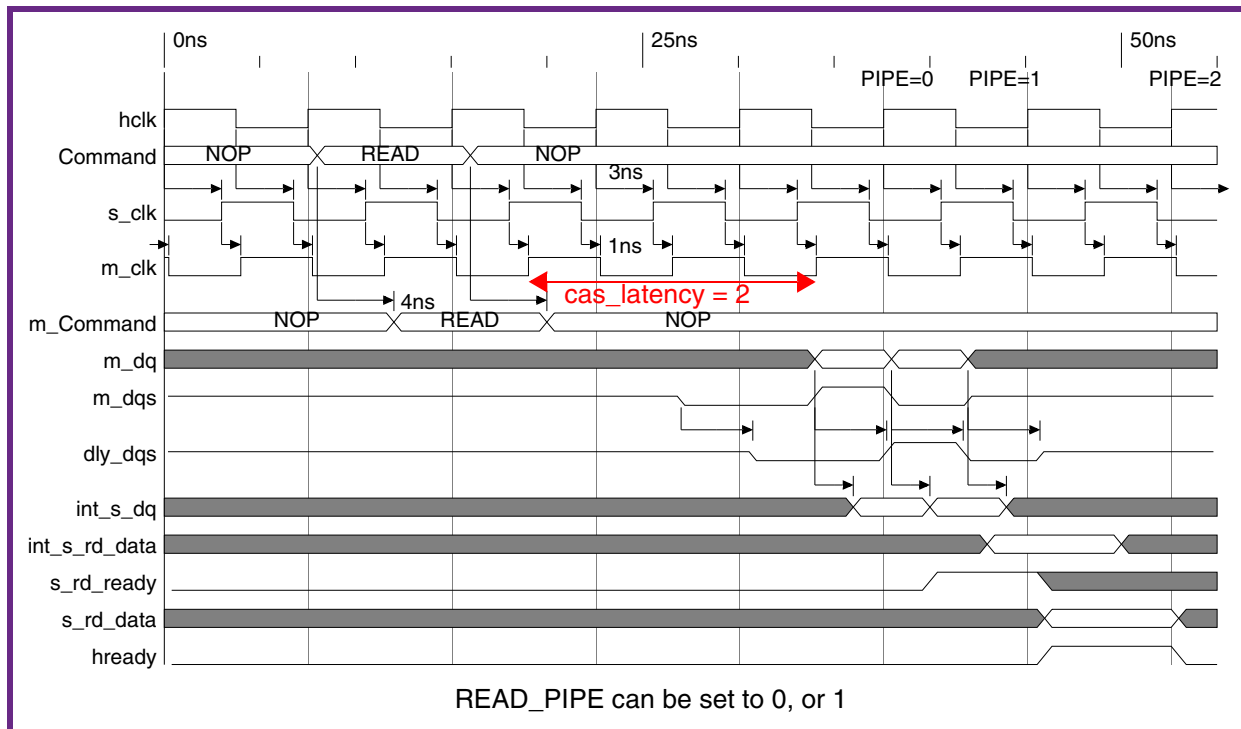
11.2.8.5 Typical DDR-SDRAM Interface – Approach 2

In some systems, you may not be able to accurately predict the skew between dqs and the internal $hclk$, but you may be able to generate a handshake signal, which indicates that the read data are ready.

You can select this mode by setting bit 17 of the SCTL register to 1. When you enable this handshake mode, the memory controller looks for a 1 on the s_rd_ready pin by counting the number of clocks equal to “CAS-latency+ $READ_PIPE$ ” after the SDRAM read command was issued. A 1 on s_rd_ready indicates that the read data are ready on the next clock. DW_memctl samples the s_rd_ready signal only once during a burst read; once DW_memctl detects a 1 on s_rd_ready , then the read data are expected every clock during a burst.

The timing diagram in [Figure 11-36](#) illustrates this DDR-SDRAM handshake method.

Figure 11-36 Timing Diagram Using DDR-SDRAM Handshake Method



11.2.8.6 Typical DDR-SDRAM Interface – Approach 3

In some systems, the relationship between the dqs and the internal hclk signals may not be predictable. However, you can guarantee that the read data will be ready in either one clock or definitely by the next hclk. You can use the following data sampling technique for these situations; however, keep in mind that this approach may add additional clocks in the read-data latency.

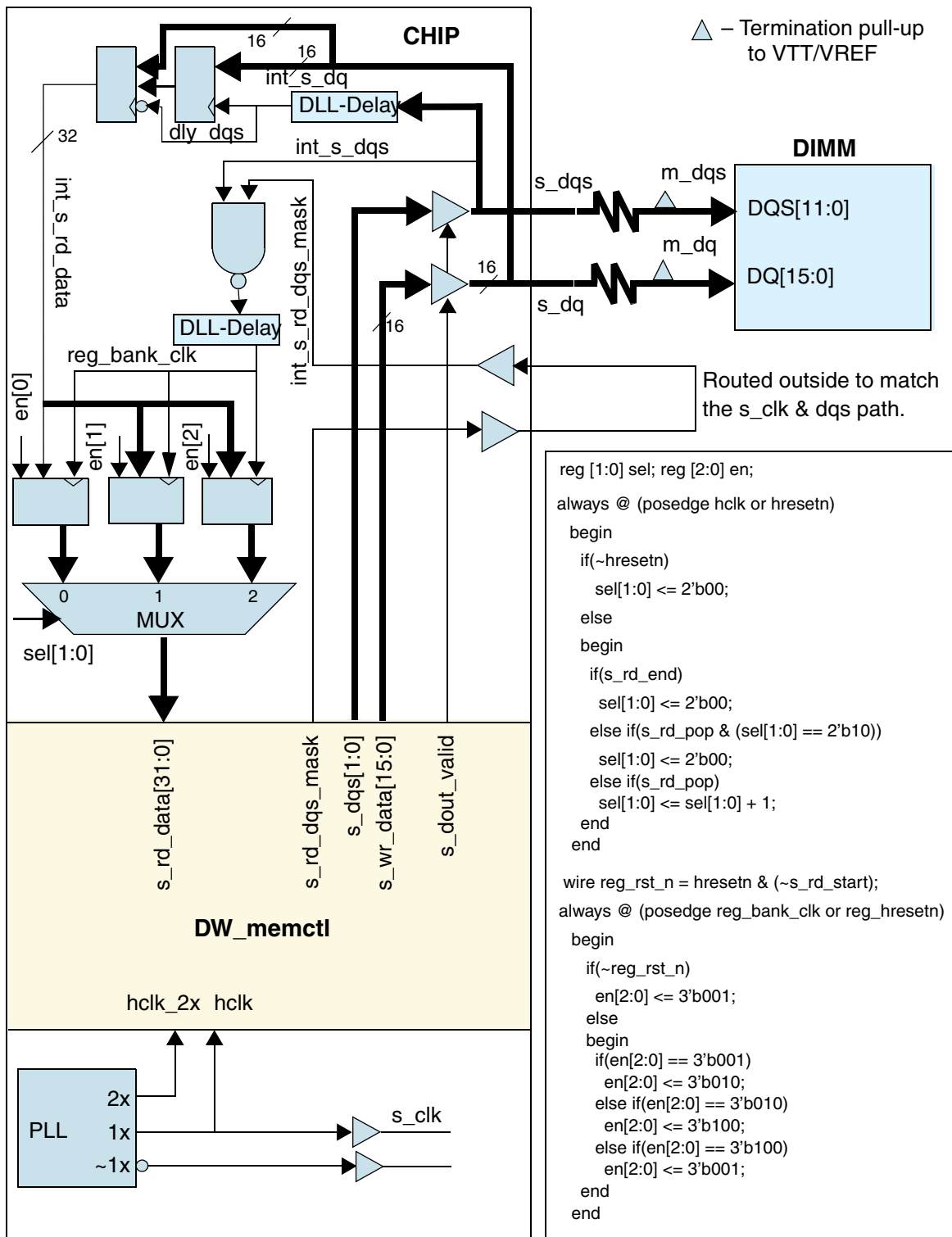
This approach uses a two-clock FIFO (or register file). The dqs signal cannot be directly used to clock the FIFO, since the dqs value is undefined before and after the conditions in this scenario and could pulse due to noise.

The memory controller has sideband signals in this approach. The dqs signal is gated with the s_rd_dqs_mask sideband signal in order to generate a latching signal for the register file. The s_rd_dqs_mask becomes active in a time equal to $\text{hclk_}2x/2$ before the first rising edge of dqs; s_rd_dqs_mask goes inactive in a time equal to $\text{hclk_}2x/2$ after the last falling edge of dqs. To match the clock and dqs trace delays and buffer delays, the s_rd_dqs_mask signal is routed outside to match $\text{clk} + \text{dqs}$ trace.

Since there are three registers in this example, this design can handle up to a three-clock uncertainty between the data coming from the DDR-SDRAMs and the data being read by the controller. The first data are latched in register 1, the second data in register 2, the third data in register 3, and the fourth data in register 1 again. The timing-critical read data from the DDR-SDRAM is first registered in an initial register, and data that is output from it is passed to the register file in order to avoid multiple loads on the timing-critical input signals. Since this register file is added after the first-stage data register, the write timing to this register file can be somewhat relaxed.

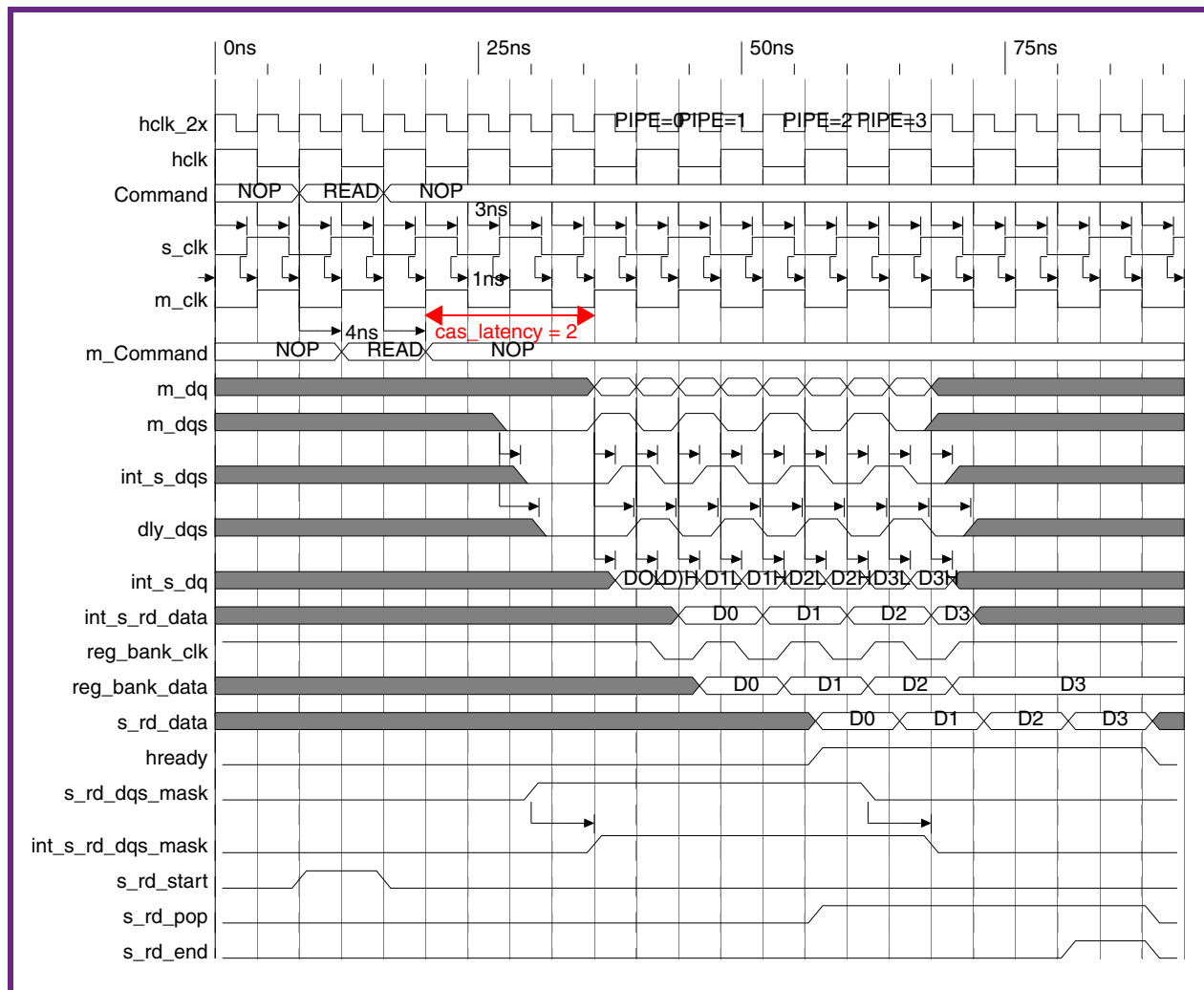
Figure 11-37 illustrates another interface between a DDR-SDRAM and the DW_memctl.

Figure 11-37 DDR-SDRAM Read data Interface - READ_PIPE = 3; WRITE_PIPE = 0



The `s_rd_start`, `s_rd_pop`, and `s_rd_end` sideband signals illustrated in Figure 11-38 are used to generate the latch-enable and read-data-MUX control signals.

Figure 11-38 Timing Diagram for Alternate DDR-SDRAM/DW_memctl Interface



11.2.8.6.1 DDR-SDRAM Application Notes

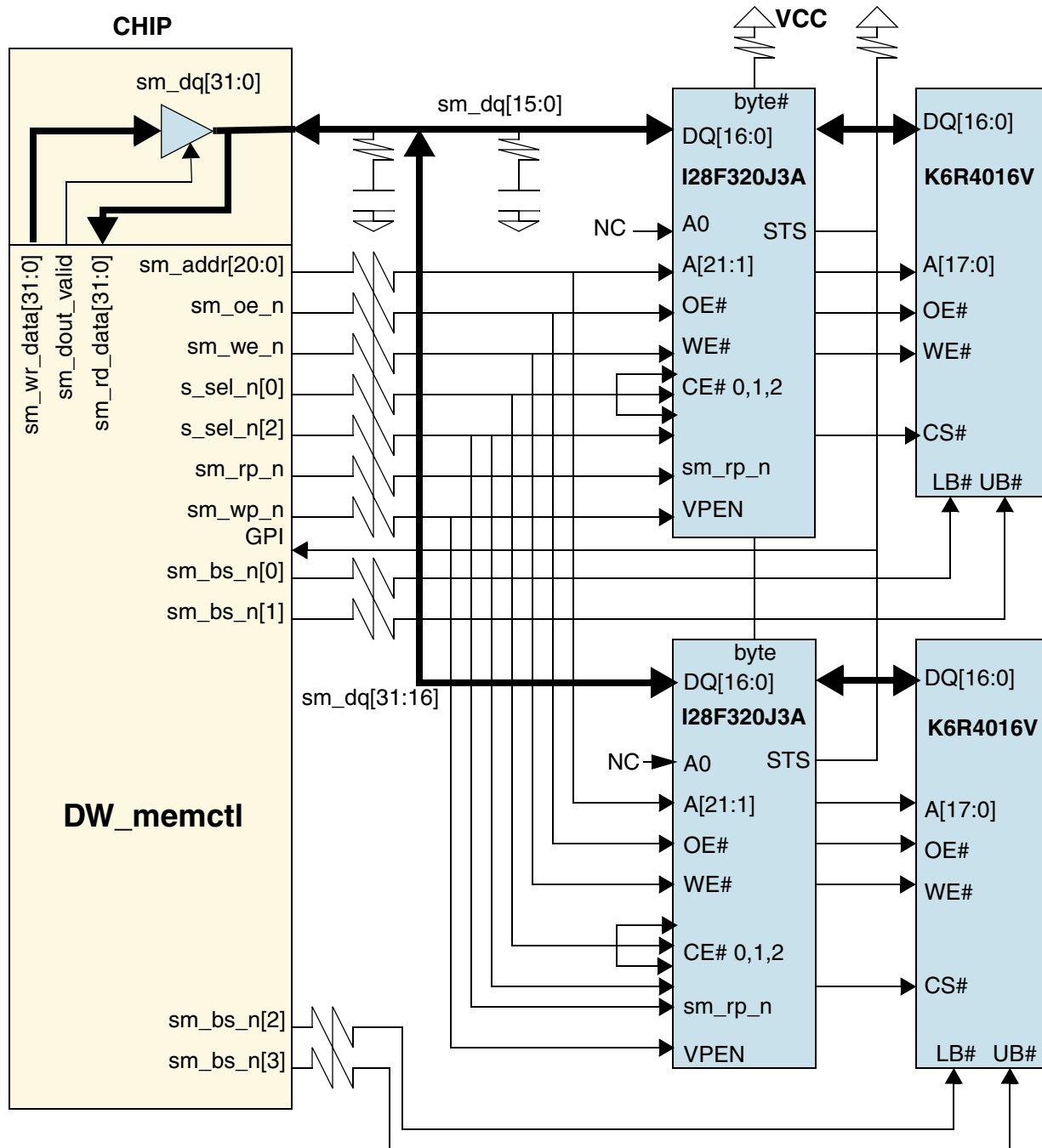
For more information on how to handle the DDR-SDRAM read data capture, refer to the application notes on the Micron and Samsung web sites:

- Key points for controller design, “Basic DDR-SDRAM Operations”
<http://www.samsung.com/global/business/semiconductor/products/dram/DRAMApplicationNote.html>
- General DDR SDRAM functionality, “Synchronous Timing for DDR SDRAM”
<http://www.micron.com/products/dram/ddr>
- “DDRSDRAM Presentations”
<http://www.micron.com/products/presentations.html>

11.2.8.7 Typical SRAM/FLASH Interface

Figure 11-39 illustrates typical SRAM and FLASH interfaces to the DW_memctl.

Figure 11-39 SRAM/FLASH Interface - SM_READ_PIPE_SETn = 0; SM_WRITE_PIPE = 0



When interfacing Static RAMs – SRAM/FLASH – the SM_READ_PIPE_SET n can be set to 0, provided that the T_RC read access time takes into account the trace/pad delays in the read data path. The T_RC parameter is set to equal the following:

clock-to-output delay of Static Memory address/control + output-pad delay +
trace delay + ram-access delay + trace delay + input-pad delay +
input-pad-to-DW_memctl-signals trace delay + 40% setup time

In performance-critical designs, you can add READ_PIPEs and WRITE_PIPEs in the read and write data paths, which improves performance during burst accesses. Note that, unlike the SDRAM read data path, the data in the Static Memory read data path is registered inside the core before passing the data to the AHB bus on the next cycle. This is unlike the SDRAM interface, which supports only 1:1 and 2:1 AHB:SDRAM data width ratio. The Static memory interface supports 1:1 to 8:1 AHB:Static ratio; sending out the data directly after multiplexing on to the AHB will not meet AHB requirements.

A

Application Notes

This appendix includes DW_memctl application notes.

A.1 Frequently Asked Questions

The following sections give answers to frequently asked questions about the DW_memctl.

A.1.1 General

Q. What is a DW_memctl?

A. The DW_memctl is a memory controller that you can configure and synthesize in order to control Synchronous DRAMs such as SDR-SDRAM, DDR-SDRAM and Mobile-SDRAM, as well as Static memories such as asynchronous SRAMs, FLASHes, ROMs, and synchronous SRAMs and FLASHes.

Q. What is the difference between SyncFlash and Synchronous Flash?

A. The SyncFlash is a special FLASH device from Micron, which has an SDR-SDRAM-like interface; like SDRAMs, it can support full-page burst. The Synchronous FLASH is the normal FLASH that has a synchronous-RAM-like interface and supports bursts of 4 or 8 words. A typical SyncFlash operates in the 60-166 Mhz range, and Synchronous FLASHes work around 50-60 Mhz. The burst throughput of SyncFlash is greater than the Synchronous FLASH. DW_memctl does not support SyncFlash memory. DW_memctl does have limited support for Synchronous FLASH memory.

Q. What kind of memory devices are supported?

A. The following memory devices are supported:

- ❑ JEDEC-standard SDR/DDR-SDRAMs and Mobile-SDRAMs
- ❑ Asynchronous SRAM
- ❑ Asynchronous page-mode FLASH
- ❑ Asynchronous page-mode ROM

- ❑ Limited synchronous SRAM
- ❑ Limited synchronous FLASH

Q. Are there any SDR/DDR-SDRAMs that can't be used with the DW_memctl?

A. The following SDR-SDRAMs from Elpida Memory, Inc. and Nanya Technology Corp. are not supported by the DW_memctl because they do not support burst termination:

hm5264805f	nt5sv16m16at
hm5264405f	nt5sv16m16atl
hm5264165f	nt5sv16m4dt
hm5259805b	nt5sv16m8ct
hm5259405b	nt5sv32m4ct
hm5259165b	nt5sv32m8at
hm5257805b	nt5sv4m16dt
hm5257405b	nt5sv64m4at
hm5257165b	nt5sv8m16ct
hm5225805b	nt5sv8m8dt
hm5225405b	
hm5225165b	

Version 1.2a (R1.2a) of the DW_memctl did not support some IBM and Infineon SDRAMs that did not support full-page burst.

Q. Can I interface Synchronous SRAMs to the DW_memctl?

A. Yes; refer to [“Interfacing to Synchronous SRAMs”](#) on page 112.

Q. Can I interface on-chip SDRAMs or SSRAMs to the DW_memctl?

A. Yes; refer to [“Interfacing DW_memctl with SDRAM SIMM/DIMM Modules”](#) on page 96 and [“On-Chip SDRAM BIST Solution”](#) on page 276.

Q. What kind of system-on-a-chip (SOC) bus is supported?

A. The DW_memctl provides an interface with an AMBA AHB 2.0 bus.

Q. What are the main features of the DW_memctl?

A. The following are features of the DW_memctl:

AMBA AHB Interface Features

- ❑ AMBA AHB bus-compatible
- ❑ Supports pin based little- or big-endian modes of operation
- ❑ Supports all types of AMBA bursts
- ❑ Supports AHB data widths of 32, 64, or 128 bits
- ❑ Supports AHB address width of 32 bits
- ❑ Supports narrow access on wide AMBA bus
- ❑ Supports busy and early terminations on AHB transactions
- ❑ Does not generate split, retry, or error responses on the AMBA bus
- ❑ Two-clock-cycle latency from AMBA bus hsel_mem assertion to issue of memory command, depending on optional registering of memory control and data signals
- ❑ Supports separate or shared memory address and/or data buses between SDRAM and Static memories
- ❑ Supports optional interface for multiplexing memory address and data pins with address and data pins of other external interface blocks

SDRAM Interface Features

- ❑ Glueless connection to all JEDEC-compliant SDRAM
- ❑ Supports up to 16 SDRAM address bits
- ❑ SDR-SDRAM and MOBILE-SDRAM memory data width that can be configured to 16, 32, 64, or 128, with 1:1 or 1:2 ratios with AHB data width. In the case of DDR-SDRAMs memory data width, can be configured to 8, 16, 32, or 64, with 1:2 or 1:4 ratios with the AHB data width.
- ❑ Programmable row and column address bit widths up to:
 - 15-bit column address
 - 16-bit row address
 - 4-bit bank address
- ❑ Supports 2K to 64K rows, 256 to 32K columns, and 2 to 16 banks
- ❑ Supports up to 8 chip selects, with a maximum of 4 GB of address space per chip select
- ❑ Compile-time and programmable configuration parameters
- ❑ SDRAM timing parameters – tRAS, tRCD, tRP, tWR, tWTR, tRCAR, tXSR, and tRC – can be programmed to values supported by different SDRAM vendors
- ❑ Supports auto-refresh with programmable refresh intervals
- ❑ Supports self-refresh
- ❑ Supports SDRAM power-down mode
- ❑ Programmable immediate precharge or delayed precharge modes

- ❑ Supports 1 to 16 (programmable) number of open banks for performance
- ❑ Supports SDRAM module serial presence detect

Static Memory Interface Features

- ❑ Supports asynchronous SRAMs, page-mode FLASHes and ROMs
- ❑ Supports up to three sets of timing registers
- ❑ Configurable address width of up to 32 bits
- ❑ Memory data width can be configured to 8, 16, 32, 64, or 128; can be 1:1 to 1:16 ratio with the AHB data width
- ❑ Supports external “READY” handshake pin to interface non-SRAM type device
- ❑ Limited synchronous SRAM and FLASH interface support
- ❑ Synchronous SRAM and FLASH frequency could be 1, 1/2, 1/3, 1/4, and so on of the AHB frequency

Q. How many external memory banks does the DW_memctl support?

A. The DW_memctl supports up to eight different external memory banks.

Q. How many sets of timing registers can the DW_memctl have?

A. The DW_memctl has only one set of timing registers for SDRAMs; therefore, different SDRAM banks must have the same timings. Thus, one DW_memctl can support only one device at a time in a design: an SDR-SDRAM or a DDR-SDRAM or a Mobile_SDRAM. (You can instantiate multiple DW_memctl memory controllers to support more than one type SDRAM at a time.) For Static memories, the DW_memctl can have up to three different sets of timing registers, which enables you to instantiate three different Static memories with one memory controller.

Q. What is the gate count of DW_memctl?

A. The gate count depends upon the configuration parameters. The gate counts for each memory controller with a default configuration are:

- ❑ Standalone SDRAM controller – approximately 25k gates
- ❑ Standalone Static memory controller – approximately 25k gates
- ❑ SDRAM controller plus Static memory controller – approximately 35k gates

Q. What is the DW_memctl maximum operating frequency?

A. The operating frequency depends on the technology library and also on the configuration; that is, shared bus, separate bus, and so on. The typical frequency range is between 133MHz and 166Mhz, using 0.25 μm to 0.18 μm , and around 200Mhz using 0.13 μm .

Q. What is the temperature range?

A. This is completely dependent on the silicon technology.

Q. *What are the I/O requirements (electrical)?*

A. The DW_memctl does not come with I/O pads. The user chooses the appropriate I/O pads from their technology library. The I/O electrical requirements are characteristic to the chosen technology library.

Q. *What are the product deliverables?*

A. Everything you need to integrate DW_memctl in your design, which includes:

- ❑ Synthesizable Verilog (encrypted)
- ❑ Complete verification environment that includes an automated test suite and testbench
- ❑ AMBA AHB BFM and DWMMs
- ❑ DesignWare-VERA – VERA-restricted compile feature
- ❑ coreConsultant – software tool for automatic installation, configuration, simulation and synthesis
- ❑ Documentation:
 - DesignWare DW_memctl Databook
 - [DesignWare SDRAM and Static Memory Controller Release Notes](#)

Q. *Do I need a VERA license to use the DW_memctl?*

A. No. DesignWare products include DesignWare-VERA, a VERA-restricted compile feature. When using the DW_memctl, you must install VERA, but you do not need a VERA license to do so. You need the VERA license only when you verify the DW_memctl in the VERA-based testbench.

Q. *What is coreConsultant?*

A. The coreConsultant tool is a separate Synopsys software tool that helps make design reuse a practical and reliable design solution by providing automatic configuration and high-quality synthesis of reusable cores in your environment. The coreConsultant tool provides the following services:

- ❑ Automatic installation and configuration of the core
- ❑ High-quality, automated synthesis of the core based on the application design context and target technology
- ❑ Ability to experiment with various core configurations

A coreConsultant license is included in the DW_memctl deliverables.

Q. *How do I find the version of the coreKit?*

A. The coreKit tar file name will have the version number in it. If the coreKit is already installed and configured, the top of the coreConsultant main window will show the coreKit version.

Q. *How do I get the DW_memctl?*

A. For information about installing this component or a related tool, refer to the [DesignWare SDRAM and Static Memory Controller Installation Guide](#).

Q. *Can I get the unencrypted (source) version?*

A. No. Only the encrypted version is available for standard downloads. However, you can contact your sales representative if you would like to purchase an un-encrypted source version.

Q. *Is DW_memctl silicon-proven?*

A. An FPGA demo board is built and tested as proof of silicon. More than 300 users have downloaded and used DW_memctl.

Q. *What are the supported libraries and technologies?*

A. DW_memctl is technology independent, so any library can be used, including FPGA.

Q. *Is there any ECC capability in DW_memctl?*

A. No, the DW_memctl currently has no ECC capability.

Q. *Does the DW_memctl have any delay compensation circuit?*

A. No.

Q. *How many DDR memories can one instance of the DW_memctl control?*

A. One memory controller can handle only one type of DDR-SDRAM, but can handle multiple banks of the same DDR-SDRAM type. In addition, it can handle three different types of Static memories.

Q. *In the configuration stage, I have set a value to the parameter "SDRAM INIT cycles." Before "SDRAM INIT cycles" are completed, can I access my memory?*

A. No. Until "SDRAM INIT cycles" are completed, you can't access *any* (SDRAM or static) of your memories. The writes will be queued in the FIFO, and reads will be put in wait-state during this time.

Q. *Can I change the DW_memctl lower register address, 00h - A7h, to some other address? Say 50h - F7. How can I change the register and memory base address?*

A. No, the lower register address needs to start at "00h." The DW_memctl uses only A [7:0] to access internal registers. Register addressing above this range is aliased. However, you can change the higher address. The higher-bit addressing depends upon how "hsl_reg" is decoded by the AHB system. The memory base address and block size are programmable and can be changed by reprogramming the DW_memctl base address and block size registers. However the memory base address should also be aligned with the block size. For example, a 256MB block base address cannot start at a 128MB address boundary; it needs to be aligned with 256MB address boundary.

Q. *If I have shared address ports between SDRAM and static memory, what will happen when the DW_memctl does "Precharge All" for SDRAM and read/write access to static memory concurrently?*

A. A10 is used to indicate "Precharge All" information to an SDRAM device, which is done before SDRAM refresh. Since the memory controller can simultaneously do both SDRAM refresh and Static memory access, there are two A10 address bits provided by the memory controller in "SHARED ADDRESS" mode; the m_precharge_bit and m_addr [10]. The m_addr bus should be connected to Static memories. Similarly, m_addr should be connected to the SDRAM devices, except for the

m_addr[10] pin. Instead of m_addr [10], m_precharge_bit should be connected to A10 of the SDRAM devices.

Some DDR devices use the A8 bit for precharge. The DW_memctl parameter A8_FOR_PRECHARGE should be set to 1 to indicate this.

Q. Where can I find more information about DW_memctl?

A. Visit <http://www.synopsys.com/designware> or send e-mail to support_center@synopsys.com.

Q. Whom should I contact if I need any technical support?

A. Contact your local field support center or send e-mail to support_center@synopsys.com. In the United States you can also call the Synopsys Support Center at 1-800-245-8005.

Q. How many AHB slave select signals does the DW_memctl have?

A. The DW_memctl has two select signals: one for memory access (hsl_mem) and one for register access (hsl_reg). The select signal for the memory is shared by all the memories connected to the DW_memctl.

Q. What is the difference between hready_resp and hready?

A. The hready_resp is an output version of the HREADY signal that is used by the DW_memctl to extend transfers and is fed through the bus multiplexer to the bus master performing the transfer. The hready is an input version of the HREADY signal, which comes from the output of the bus multiplexer. When the DW_memctl is accessed, then this signal is the same as the hready_resp. However, when another slave is being accessed, the hready signal indicates if that slave is inserting wait states.

Q. Does the DW_memctl support all kinds of burst operations on AMBA?

A. Yes. The DW_memctl supports all fixed-length bursts (SINGLE, WRAP4, INCR4, WRAP8, INCR8, WRAP16, and INCR16) as well as undefined-length bursts (INCR).

Q. Does the DW_memctl support transfer sizes narrower than the AMBA data bus width?

A. Yes. The minimum transfer size is 8 bits.

Q. Does the DW_memctl support the BUSY transfer type?

A. Yes. The DW_memctl supports the BUSY transfer type to ensure it is compatible with any bus master. The DW_memctl also supports a BUSY transfer occurring at the end of an undefined length burst (INCR).

Q. Does the DW_memctl ever generate the SPLIT and RETRY responses?

A. No. The DW_memctl gives only OKAY responses to the bus master.

Q. Can I use bursts to access registers?

A. You can use any burst type on AMBA to access the registers, including wrapping bursts (WRAP4, WRAP8, WRAP16).

- Q. Why does the DW_memctl take two internal cycles to access a register when the SDRAM data width is in 1:2 ratio with the AHB data width?
- A. The register access shares the same read/write data path as the memory. For example, if the AHB data width is 32 bits and the SDRAM data width is 16 bits, it takes two 16-bit read/write cycles internally.
- Q. If I configure the DW_memctl as only a Static memory controller and the data width ratio between Static memory and AHB is 1:2, how many cycles does the DW_memctl take to access a register?
- A. If the DW_memctl is configured as only a Static memory controller, it takes one cycle to access a register (reads take 2 clocks and writes take 3 clocks), regardless of the Static memory data width.
- Q. I'm using 64-bit AMBA data bus. Can I read or write two (32-bit) registers at one time by using HSIZE=64-bits?
- A. No. Registers have to be read or written by 32-bits or less, which means you cannot use HSIZE=64-bits for register access. However, you can access a portion of the register by using HSIZE=16-bits or HSIZE=8-bits.

A.1.2 SDRAM Controller

- Q. Can I enable auto-precharge mode?
- A. No. The DW_memctl always uses manual precharge, either immediate precharge or delayed precharge.
- Q. What happens after reset?
- A. The DW_memctl performs a power-on sequence of the SDR/DDR/Mobile-SDRAM after reset. holding the SDRAM in NOP condition for a minimum of T_INIT clock cycles before initializing the SDRAM. In the case of a DDR-SDRAM, the clock enable pin "s_cke" is driven low during "T_INIT" cycles. During the initialization, read and write accesses to the SDRAM are queued until the initialization is completed. In the case of Static memory FLASH devices, after reset for "T_RPD" cycles no access is made to the FLASH device and any access to Static Memory FLASH device is queued till "T_RPD" clocks. Refer to the ["DW_memctl as SDRAM Controller Functional Description"](#) on page 49.
- Q. What happens if an AMBA master tries to access an SDRAM that is in power-down mode?
- A. All transfer requests to the SDRAM in power-down mode are ignored and not sent to the SDRAM, although the AMBA master will receive OKAY responses associated with the transfer requests.

A.1.3 Static Memory Controller

- Q. Can I use an SRAM or FLASH with 8-bit data width?
- A. Yes. 8-bit data-width is supported for the DW_memctl. (The earlier 1.2a version of the DW_memctl had a 16-bit minimum data width requirement).

Q. What happens to accesses to the FLASH memories after reset?

- A. After reset, all requests to the FLASH will be queued until the T_RPD timer – which corresponds to the reset/power-down to read/write delay – for the FLASH expires.

A.1.4 Configuration

Q. What are the configurable parameters?

- A. The following are the configurable parameters used by the DW_memctl. The parameters with “*” can be reconfigured as well by setting registers at run time:

- ❑ AMBA data width
- ❑ AMBA address width
- ❑ Depth of address FIFO
- ❑ Depth of write data FIFO
- ❑ Parameter hard-wiring
- ❑ Instantiation of SDRAM controller
- ❑ Pipelining of address decoder outputs
- ❑ Delay in clock cycles on read data path of SDRAM outside DW_memctl*
- ❑ Delay in clock cycles on write data path of SDRAM outside DW_memctl*
- ❑ Width of SDRAM bank address*
- ❑ Width of SDRAM row address*
- ❑ Width of SDRAM data bus*
- ❑ Read data sampling mode of SDRAM*
- ❑ Instantiation of Static memory controller
- ❑ Delay in clock cycles on read data path of Static memory outside DW_memctl*
- ❑ Delay in clock cycles on write data path of Static memory outside DW_memctl*
- ❑ Width of Static memory address bus
- ❑ Width of Static memory data bus*
- ❑ Modes of Static Memory - synchronous/ready sampling*
- ❑ Page mode and size of the Static memory*
- ❑ Sharing of SDRAM and Static memory address and data pins
- ❑ Instantiation of external bus interface signals and logic
- ❑ Number of chip selects
- ❑ Base address for each chip select*
- ❑ Maximum block size of memory connected to each chip select*
- ❑ Memory type connected to each chip select*
- ❑ Timing parameters of memory connected to each chip select*

- ❑ Address remapping*
- ❑ Address aliasing*

A.2 Frequently Asked Debug Questions

Q. Can I configure the DW_memctl as a Static memory controller without an SDRAM controller?

A. Yes, you can configure the DW_memctl to work as:

- ❑ Standalone generic SDRAM (SDR/DDR/Mobile) controller
- ❑ Standalone generic Static memory controller, which supports asynchronous SRAMs, FLASHes, ROMs, and a limited number of synchronous SRAMs and FLASHes
- ❑ Both an SDRAM controller and a Static memory controller

Q. Under what conditions should I choose a deeper address FIFO than the default depth 4?

A. If your typical system behavior tends to initiate many short back-to-back write bursts, you may need deeper address FIFO to prevent unwanted AMBA wait cycles. Note that a wrapping burst (WRAP4, WRAP8, WRAP16) uses two FIFO entries.

Q. Under what conditions should I choose a deeper write data FIFO than the default depth 8?

A. You should select the depth of the write data FIFO based on typical AMBA burst type. For example, if sixteen beat bursts such as INCR16 or WRAP16 are typically used, select the depth 16. A deeper FIFO allows more pending write data and helps to prevent unwanted AMBA wait cycles.

Q. I want to sample SDRAM read data outside the DW_memctl. How does the DW_memctl support this?

A. Use the READ_PIPE parameter to denote the number of flip-flops in the read data path outside the DW_memctl. For more details, refer to [“Integrating DW_memctl into Your Design”](#) on page 221.

Q. How many reset signals are provided?

A. The DW_memctl uses a single asynchronous, active low reset, hresetn, that must be active for at least one clock. It can be asserted asynchronous to the clock, but it should be de-asserted synchronous to the clock.

Q. Are there any internal asynchronous resets or presets?

A. No.

Q. Are all the registers inside the memory controller initialized by reset?

A. No, all the register bits, except the DDR write data flip-flops that are delayed by the negative edge of hclk_2x (in DW_memctl_miu_ddrwr.v), are either set or reset by the asynchronous reset (hresetn).

Q. How many clock domains are in DW_memctl?

A. When used as a non-DDR SDRAM controller or Static Memory controller, only “posedge of hclk” is used in the design. When used as a DDR-SDRAM controller, most of the logic is clocked by the “posedge of hclk,” except a small portion of the design which controls the write-data path; it uses both the rising and falling edges of “hclk_2x.” This logic is isolated to one file, DW_memctl_miu_ddrwr.v, which is provided in un-encrypted format for user reference.

Q. *Are there any gated clocks?*

A. No, there is no gated clocks in the design.

Q. *Are there any embedded latches?*

A. There are no latches in the design.

Q. *Does it use any tri-state buses internally or for the DW_memctl I/O?*

A. There are no tri-states.

Q. *Is there any multicycle path in the design?*

A. No.

Q. *Does the memory controller guarantee the write/read cycle order?*

A. Yes, the write/read order is preserved.

Q. *Can I have separate clocks for the AHB and SDRAM interfaces?*

A. No, both the AHB and SDRAM clocks need to be the same.

Q. *Can I have separate clocks for the AHB and synchronous FLASH interfaces?*

A. Yes, but the synchronous FLASH clock needs to be the same or a sub-multiple of the AHB clock.

A.2.1 Testbench/Verification

Q. *Which Verilog simulators are supported?*

A. The DW_memctl supports:

- ❑ VCS – Synopsys, Inc.
- ❑ Verilog-XL and NC-Verilog – Cadence Design Systems, Inc.
- ❑ ModelSim – Mentor Graphics, Inc.

Q. *What is the verification environment for the DW_memctl?*

A. The complete verification environment for the DW_memctl includes a testbench, bus functional model for AMBA, and bus monitors for AMBA protocol checking.

Q. *Can I modify the AMBA bus functional model (BFM)?*

A. No. Synopsys compiles the BFM and distributes the associated object file.

Q. *Can I use the AMBA bus functional models shipped with the DW_memctl in my system testbench?*

A. No. The BFMs supplied with the core are intended for only the standalone testbench environment. You should use the Synopsys AMBA Verification IP models for your system testbench:

http://www.synopsys.com/products/designware/dw_amba.html

Q. *Can I create my own testcases?*

A. A detailed description on how to create custom testcases can be found in “Verification” on page 205.

Q. What is the DWMMtable file?

A. The DWMMtable file is the interface between the DW_memctl core and the DesignWare Memory Models (DWMMs). This file can be found under the DWMM directory of your configuration.

Q. When should I download a new DWMMtable file?

A. The DWMMtable file has timing and signal information for all memory models supported by the DW_memctl. coreAssembler uses the most current DWMMtable file, which is located in the *workspace/components/i_memctl/DWMM* directory. If you think you have a supported model but it is not included in the DWMMtable file, then you can look at the memory models online at:

<http://www.synopsys.com/dw/dwmm.php>

If you need to request an updated DWMMtable, you need to open a Case through Solvnet:

<https://solvnet.synopsys.com>

Q. What are the advantages of VERA?

A. The advantages are:

- ❑ Reduced verification time with automated testbench creation and analysis
- ❑ Ability to create modular, reusable testbenches with VERA HVL – the high-level language optimized for verification
- ❑ Use of the same testbench for VHDL and Verilog HDL designs
- ❑ Ability to perform thorough coverage analysis of even difficult corner cases
- ❑ Increased simulation efficiency with closed-loop reactive tests
- ❑ Increased simulation throughput with distributed processing capability

The following answer frequently-asked debug questions about the DW_memctl.

Q. Why are multiple chip selects active simultaneously?

A. There could be an overlap of address space between banks. You should ensure that the base address of the banks are different and also that the block sizes are correct so that they do not cause address spaces to overlap between banks.

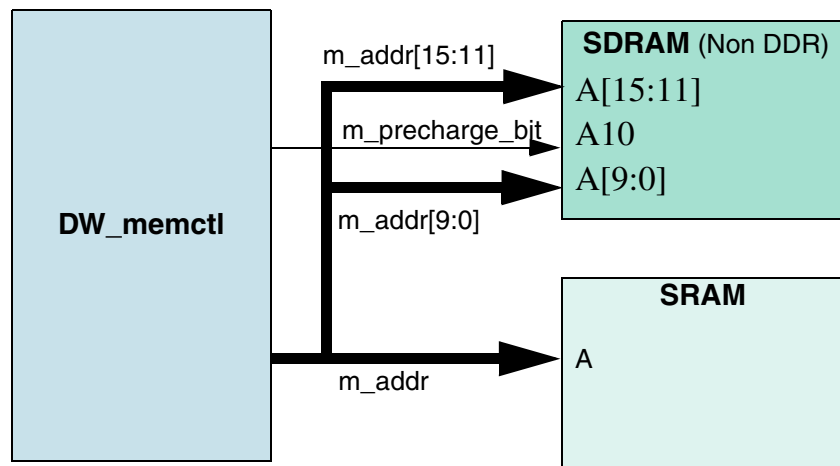
Q. What is the use of the m_precharge_bit pin?

A. The DW_memctl supports simultaneous SDRAM refresh and Static memory access. The A10 address of the SDRAM is used during the SDRAM refresh for precharge. In order to support simultaneous SDRAM refresh and Static memory access, when “ENABLE_BUS_SHARING = 1” (that is, address and data bus sharing between SDRAM and Static memory is enabled) along with the shared m_addr address bus, an additional m_precharge_bit pin is also provided.

Some DDR devices use the A8 bit for precharge. You should set the DW_memctl parameter A8_FOR_PRECHARGE to 1 to indicate this.

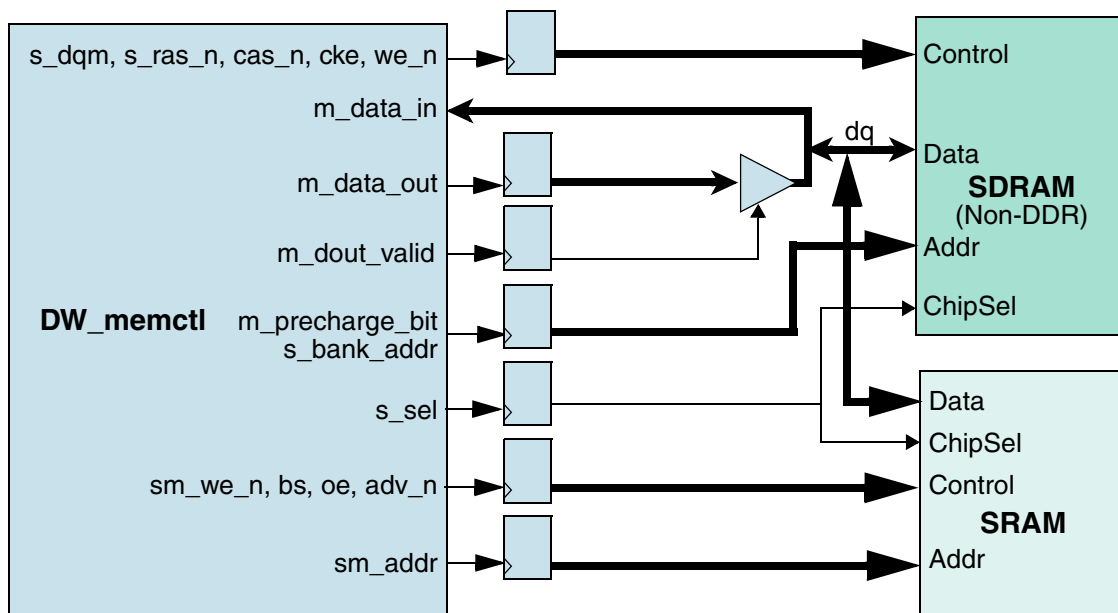
The address connection illustrated in [Figure A-1](#) is needed in this shared-address mode.

Figure A-1 Shared-Address Connection



In the case of high system clock frequencies, it is recommended that you add a WRITE_PIPE (WRITE_PIPE, SM_WRITE_PIPE = 1) in the shared address/data mode illustrated in [Figure A-2](#).

Figure A-2 WRITE_PIPE in High-Speed Shared-Address Mode



- Q. All registers seem to be programmed correctly. I am doing a memory access to FLASH with the "hsel_mem" signal set correctly, but why I am not seeing any memory access happening?
- A. For FLASH memories, there is a parameter called T_RPD (#RP high to read/write delay). The default value of this parameter is 200 clocks, unless you have modified it during configuration to meet your FLASH requirement. After reset goes high, all the FLASH write access is queued into the write FIFO, and read access is put into a wait state until the T_RPD expires. You should ensure that you have run the simulation longer than the number of T_RPD cycles.

SDRAMs also have a similar parameter called T_INIT. After reset goes high, for the number of clocks equal to T_INIT, a NOP will be issued to the SDRAMs, after which the SDRAMs will be initialized. Until that happens, all the writes are queued into the write FIFO and read access is put into a wait state.

- Q. *I am seeing correct read data from SDRAM memory and also correct "hrdata" on the AHB bus, but the "hready" and "hrdata" generated by the DW_memctl are not aligned on the same clock.*
- A. The DW_memctl memory controller has a parameter called READ_PIPE (SM_READ_PIPE for Static memories). The read data from the SDRAM is normally valid only during a very narrow window of 3ns to 4ns. In a system, this read data is normally captured with a delayed version of the internal clock, since internal clock and the clock to the SDRAM may not be phase-aligned. The captured data need to be re-captured with the internal clock before passing the data to the memory controller.

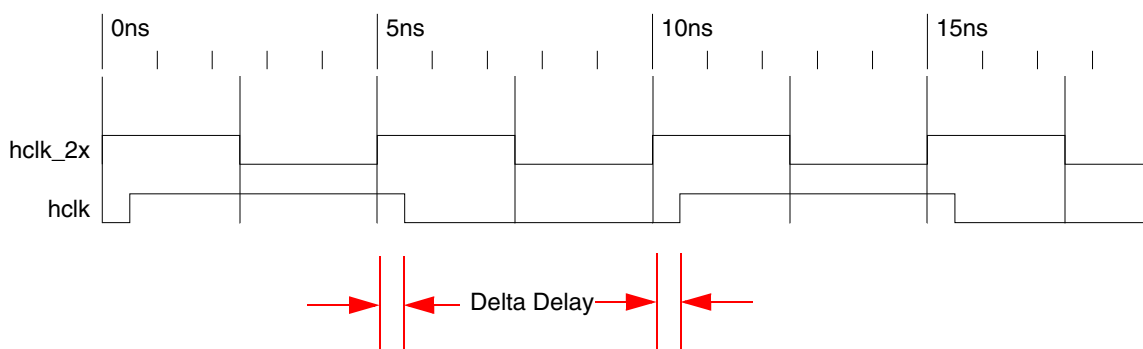
The memory controller expects these registers to be implemented outside the core, and the user sets the READ_PIPE parameter, depending upon the register pipes that are added outside the core. This parameter represents the additional delays in the read data path over the expected cas-latency. You should ensure that the READ_PIPE parameter is correctly set. For more details, refer to ["Integrating DW_memctl into Your Design"](#) on page 221.

- Q. *Why is even my basic DW_memctl AHB response cycle not correct?*
- A. Ensure that the hready_in signal is connected correctly in your AHB system. In an AHB system, all AHB slave hready_out signals are MUXed out to generate an hready_in signal that needs to be connected back to all the slaves, including the memory controller. If the DW_memctl memory controller is the only slave in your AHB system, you need to connect the DW_memctl's hready back to hready_in.
- Q. *When I read in the memory controller source files to Design Compiler, why am I getting "Undefined macro 'ifdef at or near token.....' error?"*
- A. The DW_memctl Verilog RTL has a Verilog "ifdef." Ensure that you set "hdlin_enable_vpp = true" before you read the source files.
- Q. *Is there any particular order in which I need to read the Verilog source files into the Design Compiler?*
- A. The DW_memctl has constants and parameters declared in "define" format in the DW_memctl_constants.v and DW_memctl_params.v files, respectively. Ensure that these two files are in the top of the list. Also ensure that "presto" is disabled; that is, hdlin_enable_presto = 0.
- Q. *I re-programmed the DW_memctl cas_latency value correctly in the SDRAM Timing Register0 (STMG0R), but why is the SDRAM output read timing still wrong?*
- A. After reset the DW_memctl automatically programs the mode-registers correctly depending upon the synthesized default values. Later when you re-program the cas_latency value, the mode register programming is not done automatically. You need to issue the "set mode register" command by setting the "set_mode_reg" bit of the SDRAM Control Register (SCTLR). For more details, refer to [page 175](#).
- Q. *During register accesses (hsel_reg is active), does the DW_memctl look at address bits other than haddr[7:0]?*
- A. No, the DW_memctl uses only haddr[7:0] to select internal registers; the registers will be aliased above this address range.

- Q. I understand SSRAMs have only limited support in the DW_memctl and also not supported in the standalone testbench. Is there an example I can use to verify my SSRAM interface?
- A. An example SSRAM testbench file is in `sim/testbench/mem_banks_ssram.v`. You can modify this with your model and copy the file into `sim/testbench/mem_banks.v`. For more details on the SSRAM interface, refer to [“Interfacing to Synchronous SRAMs”](#) on page 112.
- Q. I did some writes to the DW_memctl registers and then read them back. Why is the read data value the power-on reset value instead of the value I programmed?
- A. You should ensure that the `s_data_width` bits [14:13] of the SDRAM Config Register (SCONR) is set correctly. If `s_data_width` is invalid – for example, AHB data width is 32, but the `s_data_width` is set to 128 – the memory controller will ignore the register writes and return a dummy ready.
- Q. I have connected FLASH, SRAM, and SDRAM memories to three chip-selects of the DW_memctl. They are mapped to non-contiguous locations in my system memory. The AHB address decoder I have can map only one contiguous memory per address select. How do I interface the DW_memctl in my AHB?
- A. You can logically treat the DW_memctl as three AHB slaves. The DW_memctl’s `hsel_mem` can be generated by ORing three chip-select outputs from your AHB address decoder.
- Q. Are there any special testbench clock requirements?
- A. The DW_memctl testbench generates two clocks in the testbench: the `hclk` and `hclk_2x`. The `hclk_2x` is used only if the DDR-SDRAM is selected. The `hclk_2x` clock is used only inside the DDR write-data generation module (DW_memctl_miu_ddrwr.v) and samples the data generated by the “`hclk`” clock. In order to avoid simulation race conditions and to meet the hold time, the `hclk` has a delta delay (#1) from the `hclk_2x`, as illustrated in [Figure A-3](#).

When you integrate the DW_memctl in your system, you must ensure that the `hclk` has delta delay from `hclk_2x` to avoid simulation race condition. Also ensure that the `s_rd_data` input to the DW_memctl meets the input hold time with regard to `hclk`.

Figure A-3 Delta Delay for hclk and hclk_2x



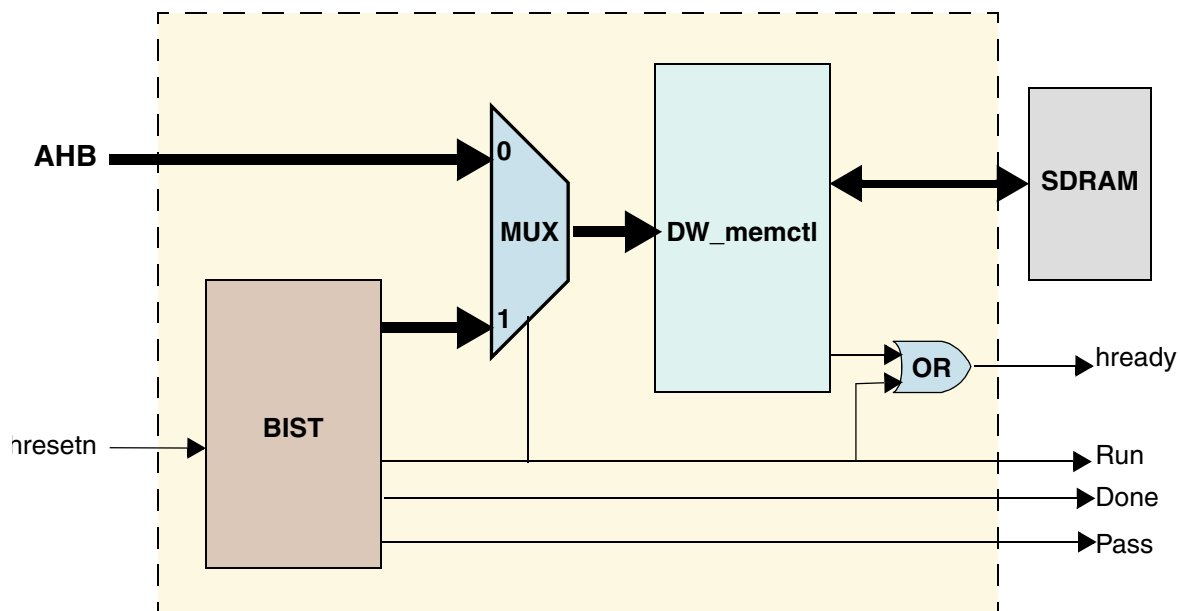
A.3 On-Chip SDRAM BIST Solution

Although the DesignWare DW_memctl is designed for interfacing to external SDRAMs, you can still interface on-chip SDRAMs to it. This application note talks about how to implement a Built-In Self Test (BIST) for the SDRAM logic. If you interface on-chip SRAMs to the DW_memctl, you can use similar logic to test it, or you can use the standard Static Ram BIST approach.

This application note talks about only SDRAM BIST.

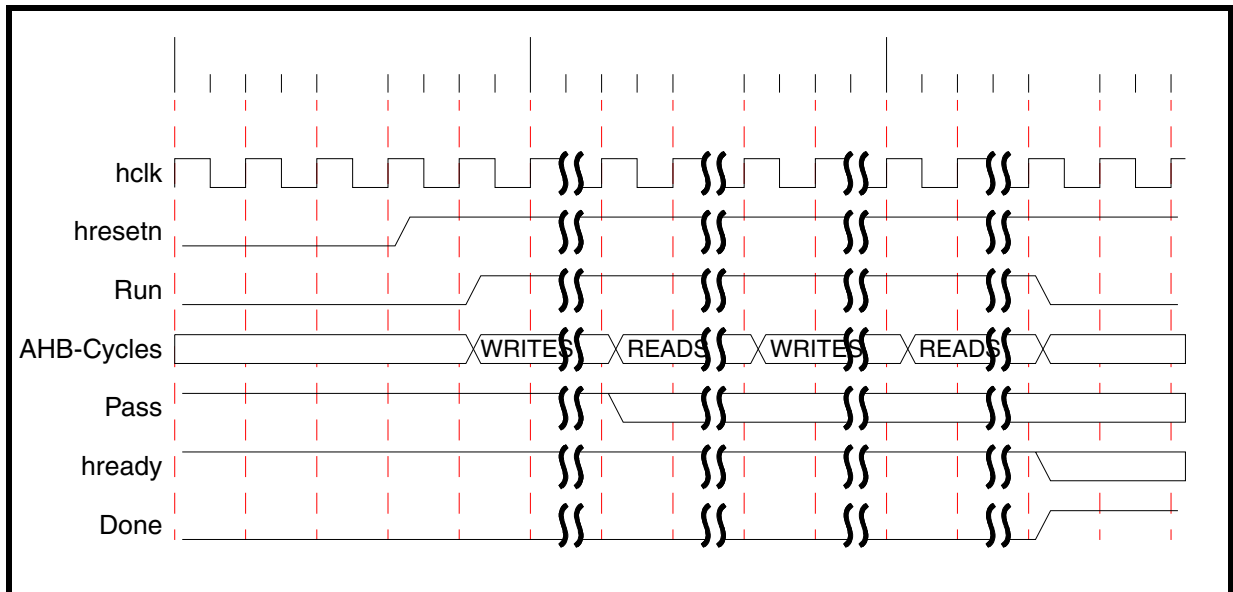
The BIST is done by generating AHB cycles onto the AHB interface of the SDRAM. One advantage of this method is that it not only tests the SDRAMs, but it also tests the DW_memctl. This BIST logic adds one 2:1 MUX delay on all AHB input paths to the DW_memctl, and an additional OR gate delay on the hready output, as illustrated in [Figure A-4](#).

Figure A-4 BIST Wrapper Block Diagram



The BIST state machine is a two-pass state machine. During the first pass, the BIST writes, reads back, and compares pseudo-random data patterns generated by a Linear Feedback Shift Register (LFSR), as illustrated in Figure A-5. During the second pass, the data is inverted in order to test for any stuck at “0” or “1” fault.

Figure A-5 BIST State Machine as Two-Pass State Machine



This test expects that all the power-on reset value parameters in the DW_memctl_params.v file are set correctly; these include:

- Refresh rate
- SDRAM specifications, such as row/column/bank timing
- Port sizes
- Chip select address
- Size

Additionally, you must change the “START_ADDR” and “END_ADDR” defined inside the code, depending on where your SDRAM chip select is mapped.

A.3.1 Sample SDRAM BIST Wrapper Code

```
//=====
//
//      This confidential and proprietary software may be used only
//      as authorized by a licensing agreement from Synopsys, Inc.
//
//      (C) Copyright 2001 - 2002, Synopsys, Inc.
//      ALL RIGHTS RESERVED
//
//      The entire notice above must be reproduced on all authorized copies
//      and any such reproduction must be pursuant to a licensing agreement
//      from Synopsys, Inc.
//
// File      : DW_memctl_bist_wrapper.v
```

```
// Author   : XYZ
// Date     : 06/03/02
// Version  : 2.0
// Abstract : An Example BIST wrapper for DW memctl on-chip SDRAM testing.
//           This module tests the memory with pseudo random (LFSR) patterns.
//           Uses Undefined Inc Burst length of 64, 64 was chosen so that
//           128-bit AHB's won't cross 1K byte boundary. This example has no
//           Static memory or EBI interface.
//
// Modification History:
// Date       By           Version  Change Description
//=====
// 12/02/01   SA           1.0      Initial Release
// 06/03/02   SA           2.0      R2.0 Update
//=====

`timescale 100ps/10ps
module DW_memctl_bist_wrapper (
  /*AUTOARG*/
  // Outputs
  hready_resp, hresp, hrdata, s_ras_n, s_cas_n, s_cke, s_wr_data, s_addr,
  s_bank_addr, s_dout_valid, s_sel_n, s_dqm, s_we_n, s_rd_start,
  s_rd_pop, s_rd_end, s_rd_dqs_mask, s_cas_latency, s_sa, s_scl,
  s_sda_out, s_sda_oe_n, gpo, Run, Done, Pass,
  // Inputs
  hclk, hresetn, haddr, hsel_mem, hsel_reg, hwrite, htrans, hsize,
  hburst, hready, hwdata, s_rd_data, s_rd_ready, s_sda_in, remap, power_down,
  clear_sr_dp, big_endian, gpi
);

  input          hclk;           // System Clock
  input          hresetn;        // System Reset
  input  [`H_ADDR_WIDTH-1:0]    haddr;      // AHB Address Bus
  input          hsel_mem;       // AHB Select - Memory
  input          hsel_reg;       // AHB Select - Register
  input          hwrite;         // AHB Transfer Direction
  input  [1:0]      htrans;       // AHB Transfer Type
  input  [2:0]      hsize;        // AHB Transfer Size
  input  [2:0]      hburst;       // AHB Burst Type
  input          hready;         // AHB Transfer Done -In
  input  [`H_DATA_WIDTH-1:0]    hwdata;     // AHB Write Data
  input  [`MAX_S_DATA_WIDTH-1:0] s_rd_data;  // SDRAM read data
  input          s_rd_ready;     // Data ready signal
  input          s_sda_in;       // Serial Presence Data In
  input          remap;          // Address remap control in
  input          power_down;     // External power down in
  input          clear_sr_dp;    // clear the self_ref_rp bit
  input          big_endian;     // Endiansess control
  input  [7:0]      gpi;         // General purpose inputs

  output         hready_resp;    // AHB Transfer Done - Out
  output  [1:0]    hresp;        // AHB Transfer Response
  output  [`H_DATA_WIDTH-1:0]    hrdata;     // AHB Read Data
  output         s_ras_n;        // SDRAM row addr. select
  output         s_cas_n;        // SDRAM column addr. sel
  output         s_cke;          // SDRAM clock enable
  output  [`MAX_S_DATA_WIDTH-1:0] s_wr_data;  // SDRAM write data
  output  [`MAX_S_ADDR_WIDTH-1:0] s_addr;    // SDRAM address
```

```

output  [`MAX_S_BANK_ADDR_WIDTH-1:0] s_bank_addr; // SDRAM bank address
output  [`MAX_S_DATA_WIDTH/8-1:0]   s_dout_valid; // SDRAM chip select
output  [`N_CS-1:0]                 s_sel_n;      // SDRAM chip select
output  [`MAX_S_DATA_WIDTH/8-1:0]   s_dqm;        // SDRAM data mask
output                                     s_we_n;   // SDRAM write enable
output                                     s_rd_start; // Read burst start
output                                     s_rd_pop;  // Data pop signal for data
output                                     // capture
output                                     s_rd_end;   // Read burst end
output                                     s_rd_dqs_mask; // Read dqs mask
output  [2:0]                         s_cas_latency; // SDRAM cas latency
output  [2:0]                         s_sa;         // Serial Presence Address
output                                     s_scl;      // Serial Presence Clock
output                                     s_sda_out;  // Serial Presence Data Out
output                                     s_sda_oe_n; // Serial Presence Data Ena
output  [7:0]                         gpo;         // General purpose outputs

output                                     Run;        // BIST Running (1)
output                                     Done;       // BIST Completed (1)
output                                     Pass;       // BIST Passed (1)

parameter LAST_ADDR_BIT = (`H_DATA_WIDTH == 32)? 2 :
                           (`H_DATA_WIDTH == 64)? 3 : 4;
parameter ADDR_INC_VAL  = (`H_DATA_WIDTH == 32)? 4 :
                           (`H_DATA_WIDTH == 64)? 8 : 16;

// Define the start and end address according to your system
`define START_ADDR 32'h0000_0000
`define END_ADDR   32'h007f_ffff
//`define END_ADDR 32'h0000_01ff // For testing

// Reg/wire declarations

wire          IncAddr;
wire          LastAddr;
wire          LoadData;
wire          ChangeData;
wire          CheckData;

wire          INThsel_reg;
wire          INThsel_mem;
wire  [2:0]    INThsize;
wire  [2:0]    INThburst;
wire  [1:0]    INThresp;
wire  [1:0]    INHthtrans;
wire          INThready_resp;
wire          INThready;
wire          INThwrite;
wire          INTremap;
wire          INTpower_down;
wire  [`H_ADDR_WIDTH-1:0] INThaddr;
wire  [`H_DATA_WIDTH-1:0] INThwdata;
wire  [`H_ADDR_WIDTH-1:0] EndAddress; // Memory Last Address
wire  [2:0]    BISThsize;
wire  [2:0]    BISThburst;
wire  [1:0]    BISThtrans;
wire  [`H_DATA_WIDTH-1:0] BISThwdata;

```

```

reg                                BISThwrite;
reg  [`H_ADDR_WIDTH-1:0] BISThaddr;
reg  [`H_DATA_WIDTH-1:0] BISTdata;    // LFSR Data

reg                                Done;
reg                                Run;
reg                                Pass;
reg                                DoneInt;
reg                                Toggle;
reg                                d_Toggle;
reg                                d_hresetn;
reg                                d_BISThwrite;

// AHB Cycle Generation

assign BISThburst  = 3'b001;

assign BISThsize   = (`H_DATA_WIDTH == 32)? 3'b010 :
                    (`H_DATA_WIDTH == 64)? 3'b011 : 3'b100;

assign BISThtrans  = (~d_hresetn | DoneInt)?                2'b00 :
                    ((`H_DATA_WIDTH==32) & (BISThaddr[7:2] == 6'h0))? 2'b10 :
                    ((`H_DATA_WIDTH==64) & (BISThaddr[8:3] == 6'h0))? 2'b10 :
                    ((`H_DATA_WIDTH==128) & (BISThaddr[9:4] == 6'h0))? 2'b10 :
                                                            2'b11 ;

assign BISThwdata  = (d_Toggle)? ~BISTdata : BISTdata;

assign EndAddress  = `END_ADDR;
assign LastAddr    = (BISThaddr[`H_ADDR_WIDTH-1:LAST_ADDR_BIT] ==
                    EndAddress[`H_ADDR_WIDTH-1:LAST_ADDR_BIT]);
assign CheckData   = ~d_BISThwrite & INThready_resp & Run;
assign ChangeData  = (BISThaddr[31:0] > `START_ADDR) & INThready_resp & Run;
assign LoadData    = (BISThaddr == `START_ADDR) & INThready_resp;
assign IncAddr     = INThready_resp & d_hresetn & Run;

always @ (posedge hclk or negedge hresetn)
begin
    if(~hresetn)
        begin
            d_hresetn      <= 1'b0;
            BISThaddr      <= `START_ADDR;
            BISTdata       <= 32'h1234_5678; // Non-Zero seed
            BISThwrite     <= 1'b1;
            d_BISThwrite   <= 1'b1;
            Pass           <= 1'b1;
            Done           <= 1'b0;
            Run            <= 1'b0;
            DoneInt        <= 1'b0;
            Toggle         <= 1'b0;
            d_Toggle       <= 1'b0;
        end
    else
        begin
            d_hresetn <= hresetn;

            if(LastAddr & INThready_resp)
                BISThaddr <= `START_ADDR;
        end
    end
end

```



```

else if(IncAddr)
    BISThaddr    <= BISThaddr + ADDR_INC_VAL;

if(LoadData)
    BISTdata <= 32'h1234_5678;    // Non-Zero seed
else if(ChangeData == 1'b1)
    BISTdata[31:0] <= {BISTdata[`H_ADDR_WIDTH-2:0],    // LSFR
        ((`H_DATA_WIDTH == 32)? (BISTdata[31] ^ BISTdata[21] ^
            BISTdata[1] ^ BISTdata[0]) :
        (`H_DATA_WIDTH == 64)? (BISTdata[63] ^ BISTdata[62] ^
            BISTdata[60] ^ BISTdata[59]) :
        (BISTdata[127] ^ BISTdata[125] ^
            BISTdata[100] ^ BISTdata[98]))};

if(INThready_resp)
begin
    d_BISThwrite    <= BISThwrite;
    d_Toggle        <= Toggle;
end

if(LastAddr & INThready_resp)
    BISThwrite <= ~BISThwrite;

if(CheckData & (hrdata != BISThwdata) & ~Done)
    Pass <= 1'b0;
else if(~Pass)
    Pass <= 1'b0;

if(LastAddr & ~BISThwrite & INThready_resp)
    Toggle <= ~Toggle;

if(LastAddr & ~BISThwrite & INThready_resp & Toggle)
    DoneInt <= 1'b1;
else if(Done)
    DoneInt <= 1'b1;

if(DoneInt & INThready_resp)
    Done <= 1'b1;

if(~d_hresetn & hresetn)
    Run <= 1'b1;
else if(DoneInt & INThready_resp)
    Run <= 1'b0;
else if(~Run)
    Run <= 1'b0;

end
end

// BIST Muxes

assign INThsel_reg    = Run? 1'b0      : hsel_reg;
assign INThsel_mem    = Run? 1'b1      : hsel_reg;
assign INThaddr       = Run? BISThaddr : haddr;
assign INThwdata       = Run? BISThwdata : hwdata;
assign INThwrite       = Run? BISThwrite : hwrite;
assign INThtrans       = Run? BISThtrans : htrans;

```

```

assign INThsize      = Run? BISThsize  : hsize;
assign INThburst     = Run? BISThburst : hburst;
assign INThready     = Run? INThready_resp : hready;
assign INTremap      = Run? 1'b0       : remap;
assign INTpower_down = Run? 1'b0       : power_down;

assign hready_resp   = Run | INThready_resp;

// Memory Controller Instantiation
DW_memctl U_memctl
(*AUTOINST*)
// Outputs
.hready_resp      (INThready_resp),
.hresp            (hresp[1:0]),
.hrddata          (hrdata[`H_DATA_WIDTH-1:0]),
.s_ras_n          (s_ras_n),
.s_cas_n          (s_cas_n),
.s_cke            (s_cke),
.s_wr_data        (s_wr_data[`MAX_S_DATA_WIDTH-1:0]),
.s_addr (s_addr[`MAX_S_ADDR_WIDTH-1:0]),
.s_bank_addr      (s_bank_addr[`MAX_S_BANK_ADDR_WIDTH-1:0]),
.s_dout_valid     (s_dout_valid),
.s_sel_n          (s_sel_n[`N_CS-1:0]),
.s_dqm            (s_dqm[1:0]),
.s_we_n          (s_we_n),
.s_rd_start       (s_rd_start),
.s_rd_pop         (s_rd_pop),
.s_rd_end         (s_rd_end),
.s_rd_dqs_mask    (s_rd_dqs_mask),
.s_cas_latency    (s_cas_latency),
.s_sa             (s_sa),
.s_scl            (s_scl),
.s_sda_out        (s_sda_out),
.s_sda_oe_n       (s_sda_oe_n),
.gpo              (gpo),

// Inputs
.hclk             (hclk),
.hresetn          (hresetn),
.haddr            (INThaddr[`H_ADDR_WIDTH-1:0]),
.hsel_mem         (INThsel_mem),
.hsel_reg         (INThsel_reg),
.hwrite           (INThwrite),
.htrans           (INHtrans[1:0]),
.hsize            (INThsize[2:0]),
.hburst           (INHburst[2:0]),
.hready           (INThready),
.hwdata           (INThwdata[`H_DATA_WIDTH-1:0]),
.remap            (INTremap),
.power_down       (INTpower_down),
.clear_sr_dp      (clear_sr_dp),
.big_endian       (big_endian),
.s_rd_data        (s_rd_data[`MAX_S_DATA_WIDTH-1:0]),
.s_rd_ready       (s_rd_ready),

```

```

        .s_sda_in          (s_sda_in) ,
        .gpi               (gpi)
    );

endmodule

```

A.4 Synthesizing DW_memctl Using dc_script

It is highly recommended that you go through the coreConsultant GUI flow for an error-free and optimal synthesis; this application note provides an example top-down compile dc_script flow. In any event, you must configure the core through coreConsultant, which will write out the correct DW_memctl.v, DW_memctl_params.v, and other source files in the *workspace/src* directory and configure them according to your parameter selections.

Since the typical gate count of the memory controller is roughly equivalent to between 40-50K NAND gates, a simple top-down compile would be sufficient and optimal.

The steps involved in this synthesis process are:

1. Set the core-specific parameters DYNAMIC_RAM_TYPE, ENABLE_STATIC, ENABLE_ADDRBUS_SHARING, ENABLE_DATA/BUS_SHARING, and EBI_INTERFACE, depending on your configuration; top-level pins change according to these parameters.
2. Set the library-specific constraints, such as target library, operating conditions, wireload model, and so on.
3. Read in, analyze, and elaborate the input source files.
4. Set the input/output drive/load conditions.
5. Set the input/output timing constraints.
6. Compile the design.
7. Insert scan and report/scan coverage, if selected.
8. Report synthesis results and save the .db file.

The main script is named DW_memctl_dc.scr, which includes the library.scr file that contains library-specific information. The example of this script, which starts on [page 284](#), assumes that these two scripts are located in the *workspace/src* directory. The output files DW_memctl.db, DW_memctl.vgate, DW_memctl_syn.rep, and the DW_memctl.vdb file, are written out in the same directory. The library.scr file assumes that the library files are located in the *../tech_lib* directory.

The DW_memctl_dc.scr example shows 60% input delay on all AHB input signals, which is typical in a normal AHB system. The hsel_reg and hsel_mem signals are set at 70% input delay, since they have an additional AHB decoder delay on them. The AHB outputs are set at 70-75% output delay. There is a 20-25% account for clock-to-Q delay of a flip-flop, and an additional one or two multiplexers in the path.

The SDRAM outputs are set at 75% output delay, which leaves 25% for the clock-to-Q of a flip-flop. SDR/Mobile-SDRAM write data output has a 2:1 on its path after the flip-flops. All DDR-SDRAM outputs come out of flip-flops. The DDR-SDRAM write data and data mask are clocked out on the falling edge of hclk_2x. The DDR-SDRAM data-strobe signals are clocked out on the rising edge of hclk_2x. The rest of the DDR-SDRAM outputs are clocked by hclk.

The Static memory outputs have 60% output delay. Since the Static interface supports from 1:1 to 1:16 Memory:AHB bus ratios, they have larger multiplexers on their path. Because the SDRAM read data is passed directly onto the AHB data read bus, it has 10% input setup time, and it is expected to come directly from a flip-flop. Since the Static memory read data is registered before being passed on to the AHB read data bus, it has a relaxed input delay of 60%. In shared mode, the shared mode outputs have 60% output delay, and the read data (m_rd_data) has a 10% input delay requirement; it is passed directly on to the AHB data read bus during SDRAM access.

Most SDRAM devices have a larger hold time requirement on their inputs – somewhere in the order of 0.8ns to 2.0ns. Depending upon the skew between the DW_memctl clock and the clock connected to the SDRAM device, you may have to change the minimum output constraints to negative numbers. For example:

```
set_output_delay -0.8 -min -clock hclk {all_outputs() }
```

A.4.1 Sample Synthesis Scripts

A.4.1.1 DW_memctl_dc.scr

```
/*-----
// Author   : S.A.
// File      : DW_memctl_dc.scr
// Version   : 1.0
// Abstract  : quick and easy design compiler script. Please change the values
//              as required. To invoke synthesis type:
//              compile tsmc_18u
//
// History:
// =====
// 02/06/00 s.a. 1.0      Initial Release
// =====*/

sh date

top_design_name = DW_memctl

/* ----- */
/* Core specific parameters      */
/* Change them as needed        */
/* ----- */

DYNAMIC_RAM_TYPE      = 4
ENABLE_STATIC         = 1
ENABLE_ADDRBUS_SHARING = 0
ENABLE_DATABUS_SHARING = 0
EBI_INTERFACE         = 0

/* Main tool specific parameters */
/* Change them as needed        */
/* ----- */
power_clock_gating = 0 /* 1 - gating; 0 - no-gating, need power compiler */
scan_insertion     = 1 /* 1 - scan; 0 - no-scan, need dft compiler */
use_dft_tetramax   = 0 /* 0 - test-compiler; 1 - use dft-compiler,tetramax */
use_ultra          = 1 /* 1 - use dc-ultra; 0 - don't use dc-ultra */
flatten           = 0 /* 1 - flatten; 0 - no flatten */
fix_hold          = 0 /* 1 - fix hold time; 0 - don't fix hold time */
dw_prefer_mc_inside = 0 /* 1 - enable dw_prefer_mc_inside; 0 - disable */
```

```

hdlin_enable_presto = 0 /* 1 - enable presto; 0 - disable presto */

/* Clock specification */
/* ----- */
/* Specified in the technology file */

/* create work directory */
/* ----- */
/*tech_name = tsmc_18u*/
get_unix_variable TECH
tech_name = dc_shell_status

if (DYNAMIC_RAM_TYPE == 0) {
    ram_type = "_sdc"
} else if (DYNAMIC_RAM_TYPE == 1) {
    ram_type = "_dsc"
} else {
    ram_type = "_sync"
}

report_name= "../report/" + top_design_name + "_" + tech_name + ram_type +
"_syn_rep.html"
db_name = "../db/" + top_design_name + "_" + tech_name + ram_type
lib_name = "../lib/" + tech_name + ram_type + "_lib"

sh rm -rf lib_name
sh mkdir lib_name
define_design_lib work -path lib_name

/* library information */
/* ----- */
include "." + tech_name + ".scr"

/* Tool selections */
/* ----- */
if (use_ultra == 1) {
    set_ultra_optimization true -force
}

if (dw_prefer_mc_inside == 1) {
    dw_prefer_mc_inside = "true"
}

/* Analyze and elaborate */
/* ----- */

hdlin_enable_vpp = true
module_list = { \
    DW_memctl_miu_addrdec \
    DW_memctl_miu_sdc \
    DW_memctl_miu_cr \
    DW_memctl_miu_refctl \
    DW_memctl_miu_smc \
    DW_memctl_miu_smcr \
    DW_memctl_miu_dmc \
    DW_memctl_miu_ddrwr \
    DW_memctl_miu_dsdc \
    DW_memctl_miu_sfc \

```

```

    DW_memctl_miu \
    DW_memctl_fifo \
    DW_memctl_hiu_afifo \
    DW_memctl_hiu_dfifo \
    DW_memctl_hiu_acore \
    DW_memctl_hiu_dcore \
    DW_memctl_hiu_rbuf \
    DW_memctl_hiu_ctl \
    DW_memctl_hiu \
    DW_memctl \
  }

foreach (module_name, module_list) {
  analyze -format verilog { \
    "../src/DW_memctl_params.v" \
    "../src/DW_memctl_constants.v" \
    "../src/" + module_name + ".v" }
}

if (power_clock_gating == 1) {
  set_clock_gating_style -sequential_cell latch -setup 0 -hold 0 \
    -control_point before -control_signal scan_enable
  elaborate -update -architecture "verilog" -gate_clock top_design_name
} else {
  elaborate -update -architecture "verilog" top_design_name
}

/* Number of scan Chains */
if (DYNAMIC_RAM_TYPE == 1) {
  numberScanChains = 3
} else {
  numberScanChains = 1
}

if (power_clock_gating == 1) {
  hookup_testports
}

/* Constraints */
/* ----- */
current_design top_design_name
link

/* clock */
create_clock -period clk_period hclk -wave {0 clk_high}
set_clock_uncertainty clk_uncertainty hclk
if (fix_hold == 1) {
  set_fix_hold hclk
}

/* ddr */
if (DYNAMIC_RAM_TYPE == 1) {
  create_clock -period clk2x_period hclk_2x -wave {0 clk2x_high}
  set_clock_uncertainty clk2x_uncertainty hclk_2x

  if (fix_hold == 1) {
    set_fix_hold hclk_2x
  }
}

```

```

}

/* input/output loads */
load_cell = slow_library_db + ":" + max_library_name + "/" + nand_name + "/" +
nand_input_pin
load = load_number * load_of(load_cell)
set_load load {all_outputs()}

set_driving_cell -cell nand_name -library slow_library_db + ":" + max_library_name \
-pin nand_output_pin {all_inputs()}
set_drive 0 {hclk, hresetn}
if(DYNAMIC_RAM_TYPE == 1) {
  set_drive 0 {hclk_2x}
}

/* Input Timings */

no_clk_input = {all_inputs() - find(port, hclk*)}
ddr_outputs = {s_dqm s_wr_data s_dqs s_rd_dqs_mask m_wr_data}
no_ddr_outputs = {all_outputs() - ddr_outputs}

/* Default */
set_input_delay clk_uncertainty -min -clock hclk {no_clk_input}
set_input_delay 0.50 * clk_period -max -clock hclk {no_clk_input}

/* Output Timings */
if(DYNAMIC_RAM_TYPE == 1) {
  set_output_delay 0 -min -clock hclk {no_ddr_outputs}
  set_output_delay 0.50 * clk_period -max -clock hclk {no_ddr_outputs}
} else {
  set_output_delay 0 -min -clock hclk {all_outputs()}
  set_output_delay 0.50 * clk_period -max -clock hclk {all_outputs()}
}

/* AHB */
set_input_delay 0.20 * clk_period -max -clock hclk {hresetn}
set_input_delay 0.70 * clk_period -max -clock hclk {hsel_mem}
set_input_delay 0.60 * clk_period -max -clock hclk {hsel_reg}
set_input_delay 0.70 * clk_period -max -clock hclk {hwdata}
set_input_delay 0.50 * clk_period -max -clock hclk {haddr hwrite htrans}
set_input_delay 0.50 * clk_period -max -clock hclk {hsize hburst}
set_input_delay 0.50 * clk_period -max -clock hclk {hready}

set_output_delay 0.70 * clk_period -max -clock hclk {hready_resp}
set_output_delay 0.75 * clk_period -max -clock hclk {hresp}
set_output_delay 0.70 * clk_period -max -clock hclk {hrdata}

/* Miscellaneous */
set_input_delay 0.50 * clk_period -max -clock hclk {s_sda_in}
set_input_delay 0.20 * clk_period -max -clock hclk {remap}
set_input_delay 0.50 * clk_period -max -clock hclk {gpi}
set_input_delay 0.05 * clk_period -max -clock hclk {big_endian}

set_output_delay 0.70 * clk_period -max -clock hclk {s_sda_out}
set_output_delay 0.70 * clk_period -max -clock hclk {s_sda_oe_n}
set_output_delay 0.75 * clk_period -max -clock hclk {s_scl s_sa}
set_output_delay 0.75 * clk_period -max -clock hclk {s_sel_n}
set_output_delay 0.75 * clk_period -max -clock hclk {gpo}

```

```

/* SDRAM */
if (DYNAMIC_RAM_TYPE != 6) {
    set_input_delay 0.20 * clk_period -max -clock hclk {power_down}
    set_input_delay 0.20 * clk_period -max -clock hclk {clear_sr_dp}
    set_input_delay 0.20 * clk_period -max -clock hclk {s_rd_ready}

    set_output_delay 0.75 * clk_period -max -clock hclk {s_ras_n s_cas_n s_we_n}
    set_output_delay 0.75 * clk_period -max -clock hclk {s_cke}
    set_output_delay 0.75 * clk_period -max -clock hclk {s_bank_addr}

    set_output_delay 0.50 * clk_period -max -clock hclk {s_rd_start}
    set_output_delay 0.50 * clk_period -max -clock hclk {s_rd_pop}
    set_output_delay 0.50 * clk_period -max -clock hclk {s_rd_end}
    set_output_delay 0.75 * clk_period -max -clock hclk {s_cas_latency}
}

if ((DYNAMIC_RAM_TYPE == 2) || (DYNAMIC_RAM_TYPE == 4) || (DYNAMIC_RAM_TYPE == 5)) {
    set_output_delay 0.50 * clk_period -max -clock hclk {s_rp_n}
}

if (DYNAMIC_RAM_TYPE == 1) {
    set_output_delay 0.70 * clk2x_period -max -clock hclk_2x -clock_fall {s_dqm}
    set_output_delay 0.70 * clk2x_period -max -clock hclk_2x -clock_fall {s_wr_data}
    set_output_delay 0.70 * clk2x_period -max -clock hclk_2x {s_dqs}
    set_output_delay 0.10 * clk2x_period -max -clock hclk_2x {m_wr_data} /*dummy*/
    set_output_delay 0.70 * clk2x_period -max -clock hclk {s_rd_dqs_mask}
    set_input_delay 0.10 * clk2x_period -max -clock hclk {test_mode}
    set_input_delay 0.10 * clk2x_period -max -clock hclk_2x {test_mode}
}

if ((DYNAMIC_RAM_TYPE != 1) && (DYNAMIC_RAM_TYPE != 6)) {
    set_output_delay 0.75 * clk_period -max -clock hclk {s_dqm}
    set_output_delay 0.75 * clk_period -max -clock hclk {s_wr_data}
    set_output_delay 0.75 * clk_period -max -clock hclk {s_rd_dqs_mask}
}

if ((DYNAMIC_RAM_TYPE != 6) && (ENABLE_ADDRBUS_SHARING == 0)) {
    set_output_delay 0.75 * clk_period -max -clock hclk {s_addr}
}

if ((DYNAMIC_RAM_TYPE != 6) && (ENABLE_DATABUS_SHARING == 0)) {
    set_input_delay 0.10 * clk_period -max -clock hclk {s_rd_data}
    set_output_delay 0.75 * clk_period -max -clock hclk {s_dout_valid}
}

if ((DYNAMIC_RAM_TYPE != 6) && (EBI_INTERFACE == 1)) {
    set_output_delay 0.30 * clk_period -max -clock hclk {s_ebi_req}
    set_input_delay 0.40 * clk_period -max -clock hclk {s_ebi_gnt}
}

/* STATIC */
if (ENABLE_STATIC == 1) {
    set_output_delay 0.50 * clk_period -max -clock hclk {sm_bs_n}
    set_output_delay 0.60 * clk_period -max -clock hclk {sm_wp_n}
    set_output_delay 0.60 * clk_period -max -clock hclk {sm_oe_n sm_we_n}
    set_output_delay 0.50 * clk_period -max -clock hclk {sm_adv_n}
    set_output_delay 0.50 * clk_period -max -clock hclk {sm_rp_n}
}

```



```

    set_input_delay 0.20 * clk_period -max -clock hclk {sm_clken}
    set_input_delay 0.50 * clk_period -max -clock hclk {sm_ready}
    set_input_delay 0.20 * clk_period -max -clock hclk {sm_power_down}
}

if ((ENABLE_STATIC == 1) && (ENABLE_ADDRBUS_SHARING == 0)) {
    set_output_delay 0.60 * clk_period -max -clock hclk {sm_addr}
}

if ((ENABLE_STATIC == 1) && (ENABLE_DATABUS_SHARING == 0)) {
    set_input_delay 0.60 * clk_period -max -clock hclk {sm_rd_data}
    set_output_delay 0.60 * clk_period -max -clock hclk {sm_dout_valid}
    set_output_delay 0.40 * clk_period -max -clock hclk {sm_wr_data}
}

if ((ENABLE_STATIC == 1) && (EBI_INTERFACE == 1)) {
    set_input_delay 0.30 * clk_period -max -clock hclk {sm_ebi_gnt}
    set_output_delay 0.40 * clk_period -max -clock hclk {sm_ebi_req}
}

/* SHARED */
if (ENABLE_ADDRBUS_SHARING == 1) {
    set_output_delay 0.70 * clk_period -max -clock hclk {m_addr}
    set_output_delay 0.75 * clk_period -max -clock hclk {m_precharge_bit}
}

if (ENABLE_DATABUS_SHARING == 1) {
    set_input_delay 0.10 * clk_period -max -clock hclk {m_rd_data}
    set_output_delay 0.60 * clk_period -max -clock hclk {m_dout_valid}
    set_output_delay 0.40 * clk_period -max -clock hclk {m_wr_data}
}

/* Operating condition and wireload */
set_operating_conditions -min min_op_condition -max max_op_condition \
    -min_library slow_library_db + ":" + max_library_name \
    -max_library fast_library_db + ":" + min_library_name

if (force_wireload) {
    set_wire_load_model -name wireload_model
    set_wire_load_mode top
} else {
    set_wire_load_mode top
}

check_design
set_fix_multiple_port_nets -feedthroughs -outputs
link

if (flatten == 1) {
    ungroup -all -flatten
}

set_max_area 0

if (power_clock_gating == 1) {
    propagate_constraints -gate_clock
}

```

```

set_scan_configuration -style multiplexed_flip_flop -methodology full_scan \
    -clock_mixing no_mix -chain_count numberScanChains \
    -dedicated_scan_ports true
set_critical_range 0.2 * clk_period top_design_name
uniquify
sh date

/* first pass compile */
/* ----- */
if (scan_insertion == 1) {
    compile -boundary_optimization -map_effort medium -scan
} else {
    compile -boundary_optimization -map_effort medium
}
sh date

/* html stuff */
echo "<html>" > report_name
echo "<body>" >> report_name
echo "<pre>" >> report_name
echo "<font size = 4>" >> report_name

/* second pass incremental compile */
/* ----- */
compile -incremental -map_effort high
if (flatten) {
    ungroup -all -flatten
}
sh date

if (scan_insertion == 1) {
    create_test_clock hclk -period 100 -waveform {55 45}
    if (DYNAMIC_RAM_TYPE == 1) {
        create_test_clock hclk_2x -period 100 -waveform {55 45}
    }

    if (DYNAMIC_RAM_TYPE == 1) {
        set_test_hold 1 test_mode
    }

    create_port -direction "in" {scan_en}
    set_scan_signal test_scan_enable -port "scan_en"

    if (numberScanChains > 1) {
        count = 0;
        while (count < numberScanChains) {
            create_port "scan_in[" + count + "]" -direction "in"
            create_port "scan_out[" + count + "]" -direction "out"
            count = count + 1;
        }
        create_bus scan_in[* scan_in
        create_bus scan_out[* scan_out

    } else {
        create_port scan_in -direction "in"
        create_port scan_out -direction "out"
    }
}

```

```

set_scan_signal test_scan_in -port find(port,"scan_in*")
set_scan_signal test_scan_out -port find(port,"scan_out*")

/* SCAN */

if (DYNAMIC_RAM_TYPE == 1) {
  set_input_delay 0.10 * clk2x_period -max -clock hclk_2x {scan_in[0]}
  set_input_delay 0.10 * clk2x_period -max -clock hclk_2x {scan_in[1]}
  set_input_delay 0.10 * clk2x_period -max -clock hclk_2x -clock_fall {scan_in[2]}
  set_output_delay 0.30 * clk2x_period -max -clock hclk_2x {scan_out[0]}
  set_output_delay 0.30 * clk2x_period -max -clock hclk_2x {scan_out[1]}
  set_output_delay 0.30 * clk2x_period -max -clock hclk_2x -clock_fall {scan_out[2]}
} else {
  set_input_delay 0.10 * clk_period -max -clock hclk {scan_in}
  set_output_delay 0.40 * clk_period -max -clock hclk {scan_out}
}
set_drive 0 {scan_en}
set_false_path -from {scan_en}

if(use_dft_tetramax == 1) {

  /* Hopefully this relates to TetraMAX that CLK and          */
  /* UnGatedCLK are really pretty much the same thing...      */
  test_stil_multiclock_capture_procedures = true

  /* Go ahead and set this up now too          */
  test_stil_netlist_format = verilog

  check_dft
  insert_dft -map_effort high
  check_dft >> report_name
  write_test_protocol -format stil -out db_name + ".spf"

} else {

  check_test
  insert_scan -map_effort high
  check_test >> report_name
  estimate_test_coverage >> report_name
  create_test_patterns -output db_name + ".vdb"
/*   report_test -coverage >> report_name */
/*   report_test -atpg_conflicts >> report_name */
/*   report_test -scan_path >> report_name */
}
}
sh date

/* save and report */
/* ----- */
write -f db -hier      -o db_name + ".db"
write -f verilog -hier -o db_name + ".vg"

report_area                >> report_name
report_timing -max_paths 40 >> report_name
report_timing -path end -to all_outputs() -nosplit >> report_name
report_timing -loops        >> report_name
all_registers -level_sensitive >> report_name

```

```

report_constraint -all_violators >> report_name
report_constraints >> report_name
report_timing -net -nworst 10 >> report_name

/* html stuff */
echo "</pre>" >> report_name
echo "</body>" >> report_name
echo "</html>" >> report_name

sh date
quit

library.scr

/* =====
// Author : xyz
// File : library.scr
// Version : 1.0
// Abstract : This is tsmc 0.18u library specific file, which gets included in
// the main synthesis script DW_memctl_dc.scr.
//
// History:
// =====
// 08/13/01 SA 1.0 Initial Release
// =====*/

separate_fast_lib = 1 /* If there is a fast library then 1, else 0 */

slow_library_db = "slow.db"
fast_library_db = "fast.db" /* If no fast lib then keep same as slow*/
target_library = slow_library_db
symbol_library = "core.sdb"
synthetic_library = {dw01.sldb, dw02.sldb, dw03.sldb, dw06.sldb}
link_library = "*" + target_library + synthetic_library
search_path = search_path + "../tech_lib"

min_op_condition = "fast"
max_op_condition = "slow"
min_library_name = "fast"
max_library_name = "slow"

force_wireload = 0 /* 1 - if specific wire load model need to be picked */
wireload_model = "" /* if so specify the name */

nand_name = "NAND2BX4"
nand_output_pin = "Y"
nand_input_pin = "AN"
load_number = 8 /* 8 times load of nand_input_pin */

/* min library */
read slow_library_db
if(separate_fast_lib == 1) {
    set_min_library slow_library_db -min_version fast_library_db
}

/* Library specific - don't use cells if any */

```

A.5 External Bus Interface

The External Bus Interface logic is used in an SoC design when you need to share DW_memctl external pins with other blocks. By reducing the pin count, you can reduce the packaging size of your SoC. You can use the EBI logic to share DW_memctl external pins under the following conditions:

- With other memory controllers or other blocks
- In two different instantiations of the DW_memctl within the same SoC

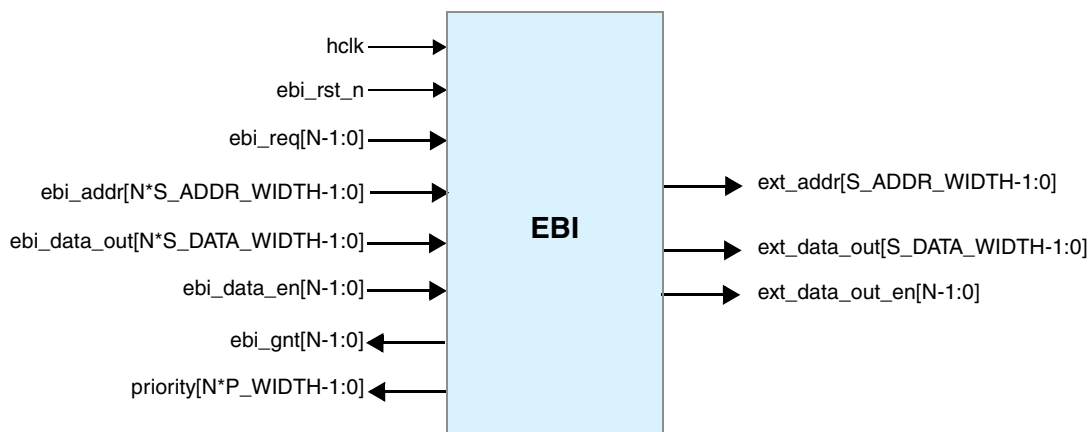
Only the external data and address pins are shared by the EBI. The control signals are not shared and should be connected directly to the output ports.

When `ENABLE_ADDRBUS_SHARING = 0`, the SDRAM refresh also goes through EBI arbitration. The external EBI arbiter should honor a DW_memctl EBI request before the refresh time to prevent SDRAM data loss. When `ENABLE_ADDRBUS_SHARING = 1`, the DW_memctl does not request the EBI during refresh. In the shared SDRAM and Static RAM address bus mode, since the `m_precharge_bit` is connected to SDRAM address bit 10, the refresh does not need to go through the EBI.

A.5.1 EBI Module Functionality

Figure A-6 illustrates the I/O interface of the EBI block.

Figure A-6 EBI Interface Diagram



The EBI block uses the two-tier DesignWare Foundation arbiter (DW_arbiter_2t) with a “fair-amoung-equal” arbitration scheme that arbitrates between the clients requesting the external interface; the clients can specify their priority through the priority input of the EBI block. A client requesting an external data transfer must keep its request active until the whole transaction is complete because the EBI block has no knowledge of when a transaction is complete.

The following parameters are assumed for the EBI block:

- `N` – Number of clients requesting
- `DATA_WIDTH` – External data bus width
- `ADDR_WIDTH` – External address bus width

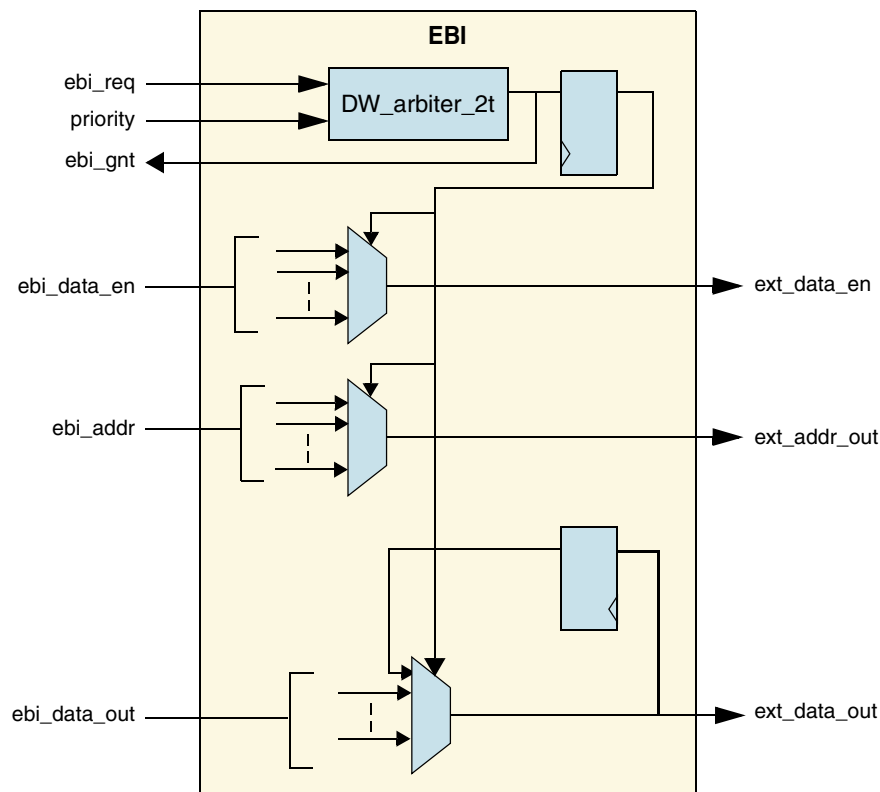
While the arbiter can provide same-clock-cycle arbitration – set through the arbiter parameter `output_mode` – you should use the one-clock arbitration if you want better timing results.

Other than the arbiter, the primary logic inside the EBI block contains MUXes that control the address, data, and enable-signal outputs. The select signal to these MUXes is the grant signal from the arbiter. The EBI module internally uses the last data output to drive the last defined value onto the external data during idle cycles (no client requesting).

The input read data from the shared pins should be separately connected to the client interface, since the EBI block does not takes care of this.

Figure A-7 illustrates the multiplexing logic inside the EBI.

Figure A-7 EBI Logic



A.5.2 EBI Interface Signals and Parameters

Table A-1 lists the EBI compile-time parameters.

Table A-1 EBI Compile-Time Parameters

Parameter	Default Value	Description
CTL_NUM	2	Number of EBI clients
P_WIDTH	2	Width of priority input
DATA_WIDTH	32	Width of external data bus
ADDR_WIDTH	32	Width of external address bus

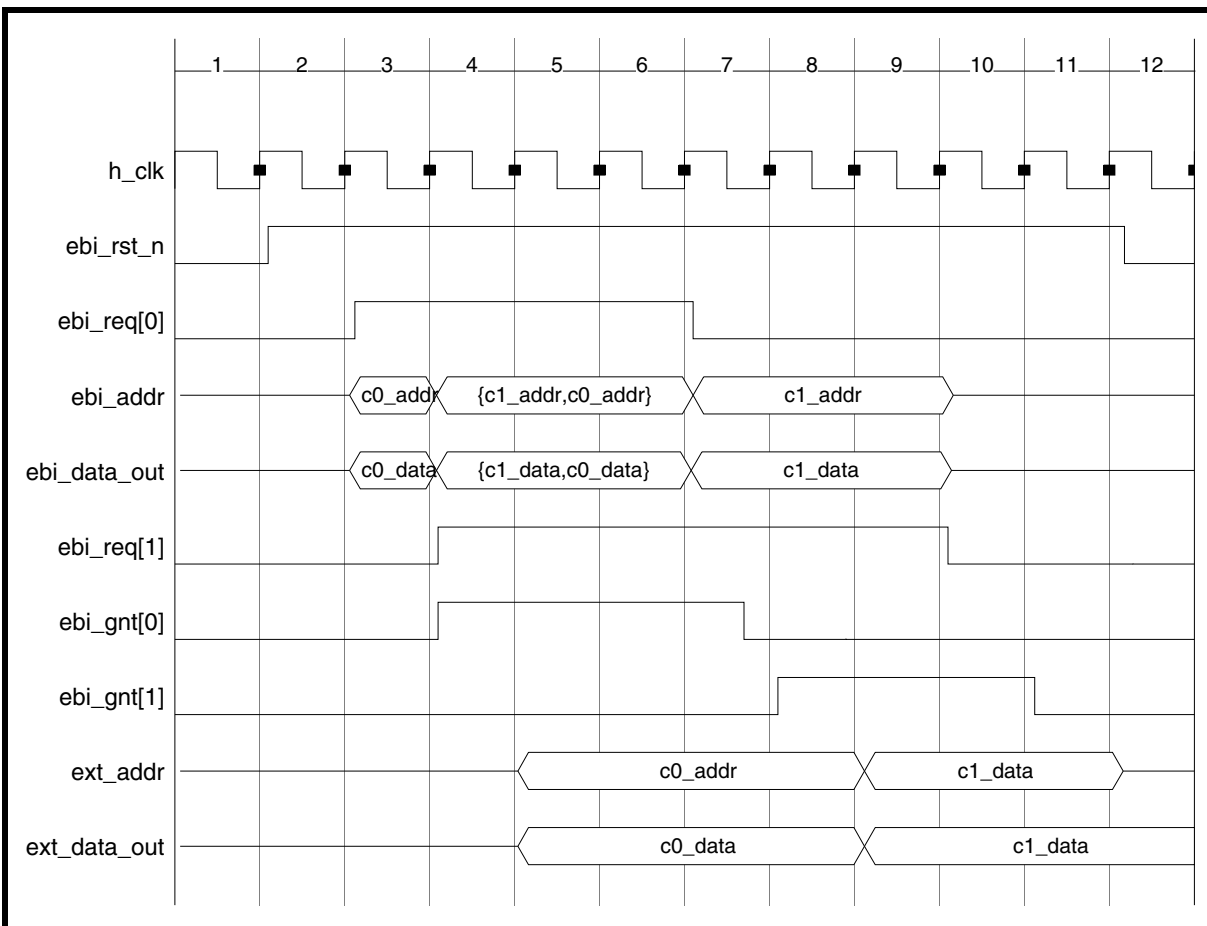
Table A-2 lists the EBI interface signals.

Table A-2 EBI Block Interface Signal Description

Signal Name	Polarity	Size	Description
h_clk	Input	1	Clock input to EBI block; should be same clock at which DW_memctl runs
ebi_rst_n	Input	1	Asynchronous reset signal
ebi_req	Input	CTL_NUM	EBI request from clients
ebi_addr	Input	CTL_NUM*ADDR_WIDTH	Concatenated EBI address inputs from clients
ebi_data_out	Input	CTL_NUM*DATA_WIDTH	Concatenated EBI data inputs from clients
priority	Input	CTL_NUM*P_WIDTH	Concatenated priority for clients
ebi_data_en	Input	CTL_NUM	Concatenated data enables from client
ebi_gnt	Output	CTL_NUM	Concatenated grant outputs to client
ext_addr	Output	ADDR_WIDTH	External address output
ebi_data_in	Output	DATA_WIDTH	Read data input to clients
ext_data_en	Output	1	Data output valid to tri-state buffer

Figure A-8 illustrates the “handshake” between the ebi_req and ebi_gnt signals where it takes one clock cycle for the arbitration. Clients must keep their requests active during the entire data transfer. The arbiter senses ebi_req[0] HIGH on the rising edge of clock 4; it asserts a grant in the next clock cycle.

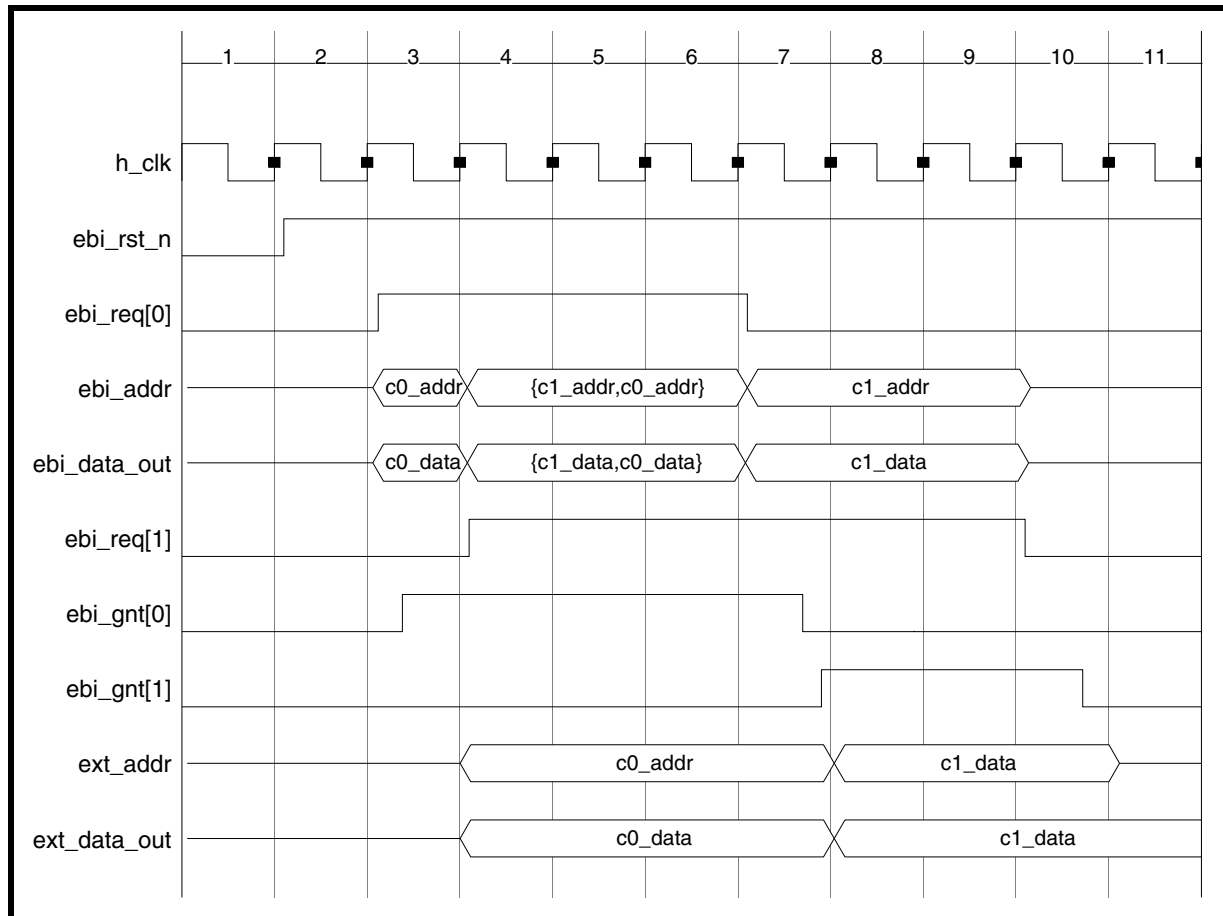
Figure A-8 ebi_req/ebi_gnt Handshaking (One Clock Arbitration)



The arbiter senses the ebi_req[1] on the rising edge of clock 5. However, since one request is already active, the grant for client1 is asserted only when client0 de-asserts its request.

Figure A-9 illustrates the handshake between the `ebi_req` and `ebi_gnt` signals where the arbitration occurs within the same clock cycle. Clients must keep their requests active during the entire data transfer. The arbiter senses `ebi_req[0]` HIGH in clock cycle 3 and immediately asserts a grant.

Figure A-9 ebi_req/ebi_gnt Handshaking (Same Clock Arbitration)



When the arbiter senses the `ebi_req[1]` in clock cycle 4, it has to wait because one request is already active; the grant for client1 is asserted within the same clock cycle that client0 de-asserts its request.

A.5.3 Interfacing DW_memctl with EBI

There are four signals in the `DW_memctl` that are dedicated to the EBI. The following signals are dedicated request and grant signals for the EBI interface of the SDRAM controller:

- `s_ebi_req`
- `s_ebi_gnt`

The following signals are dedicated request and grant signals for the EBI interface of the Static controller:

- `sm_ebi_req`
- `sm_ebi_gnt`

Users have the option of selecting the EBI interface at compile time. The compile-time parameter `EBI_INTERFACE` determines whether or not this interface exists. If this parameter is 0, then the `DW_memctl` does not support the EBI interface, and the EBI I/O pins are disabled.

Figure A-10 illustrates the `DW_memctl` as a Static and SDRAM memory controller sharing external pins with an SSRAM controller through the EBI block. Only those signals that are relevant to the EBI are illustrated. The control signals are not shared by the EBI and are therefore connected directly to the memory.

Figure A-10 DW_memctl Interface to EBI

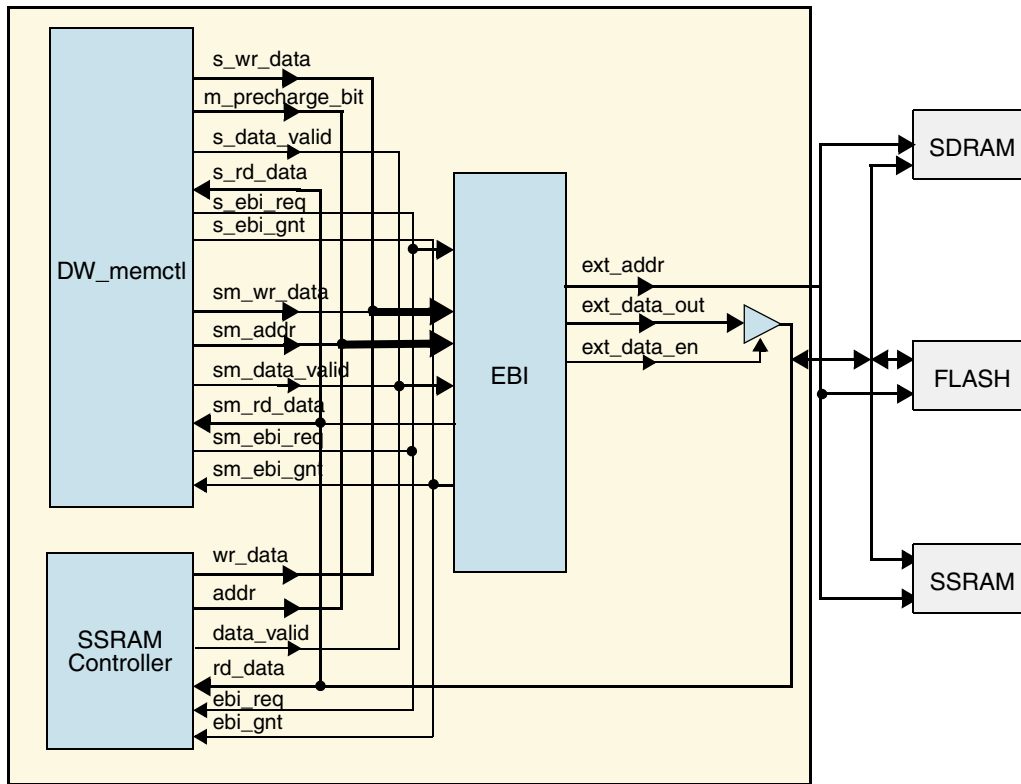
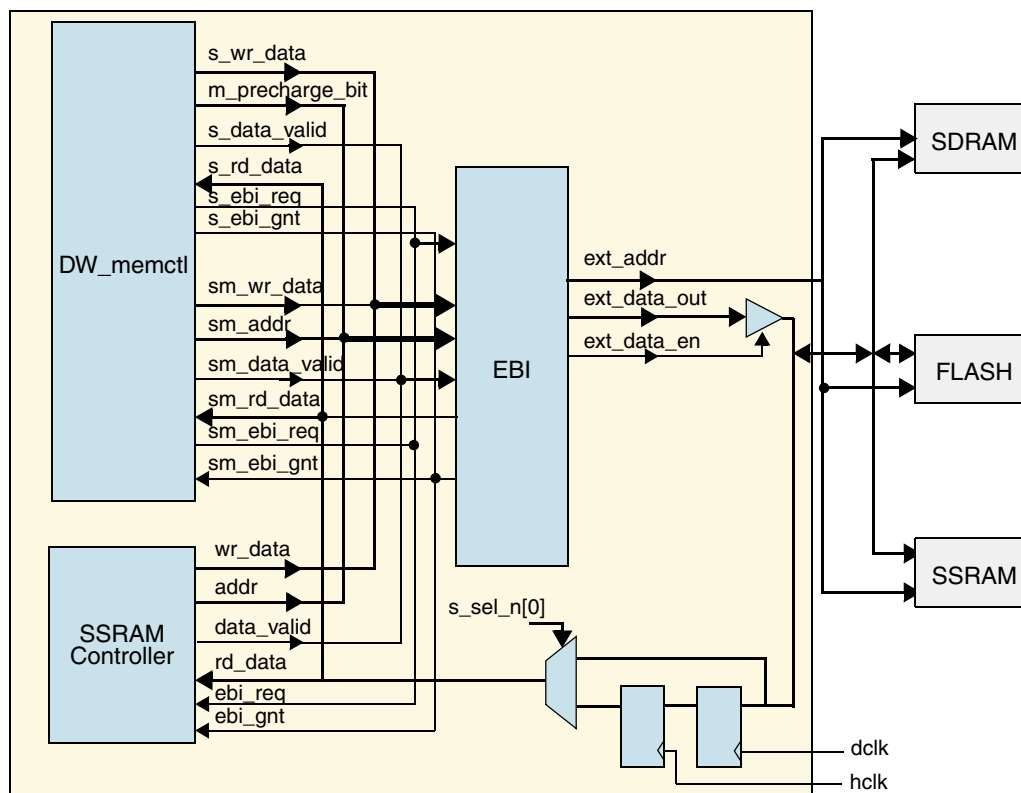


Figure A-11 DW memctl Interface to EBI, with Register Insertion in Read Data Path



In this particular case, a register is inserted for only the SDRAM read data path. It is assumed that chip select0, (s_sel_n[0]) generated by the DW_memctl, is connected to the SDRAM and that chip select1 (s_sel_n[1]) is connected to the FLASH. The select signals are used to multiplex the read data path.

Figure A-12 illustrates the handshake between the DW_memctl and the EBI block when there is an SDRAM read cycle. Once it receives the grant, the DW_memctl asserts the memory command in the next clock cycle. Notice that, because of a READ_PIPE value of 2, the ebi_req is active until the actual read data is received by the memory controller.

Figure A-12 DW_memctl/EBI Handshake – SDRAM Read with READ_PIPE Value 2

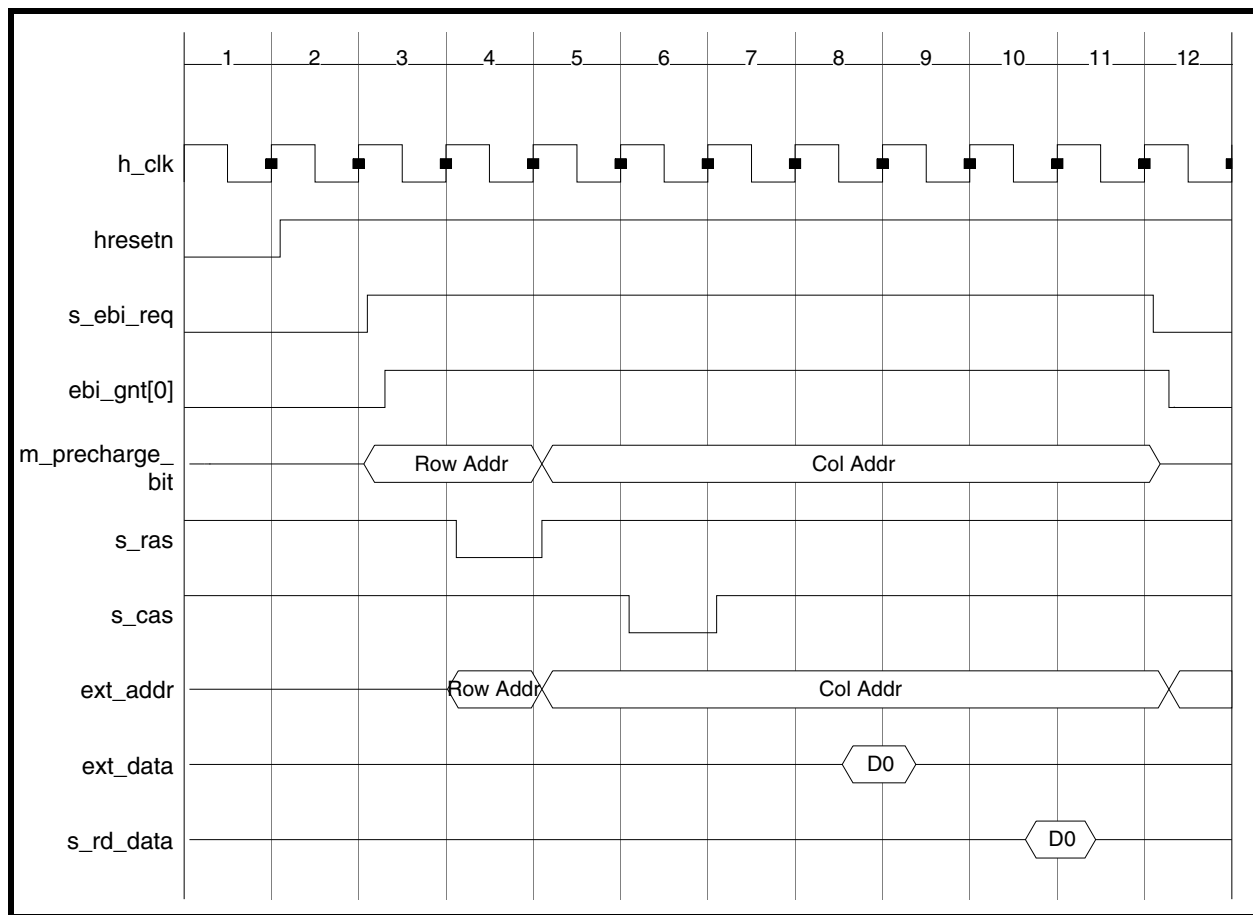


Figure A-13 illustrates the handshake between the DW_memctl and the EBI block when there is a FLASH read cycle. Notice in this case that the SM_READ_PIPE value is 0, so ext_data and sm_rd_data arrive almost at the same time, which results in the DW_memctl de-asserting ebi_req in the next clock.

Figure A-13 DW_memctl/EBI Handshake – FLASH Read with READ_PIPE Value 0

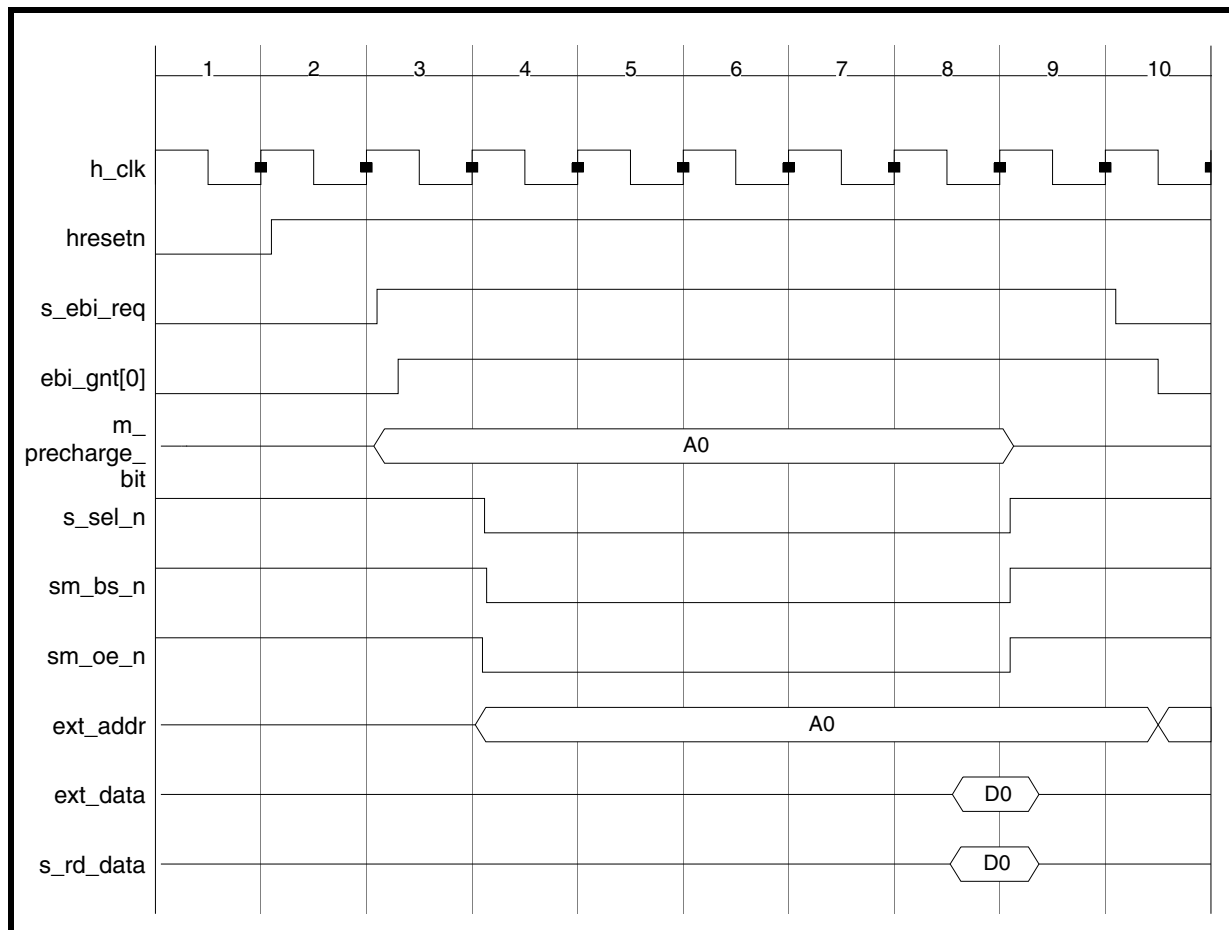
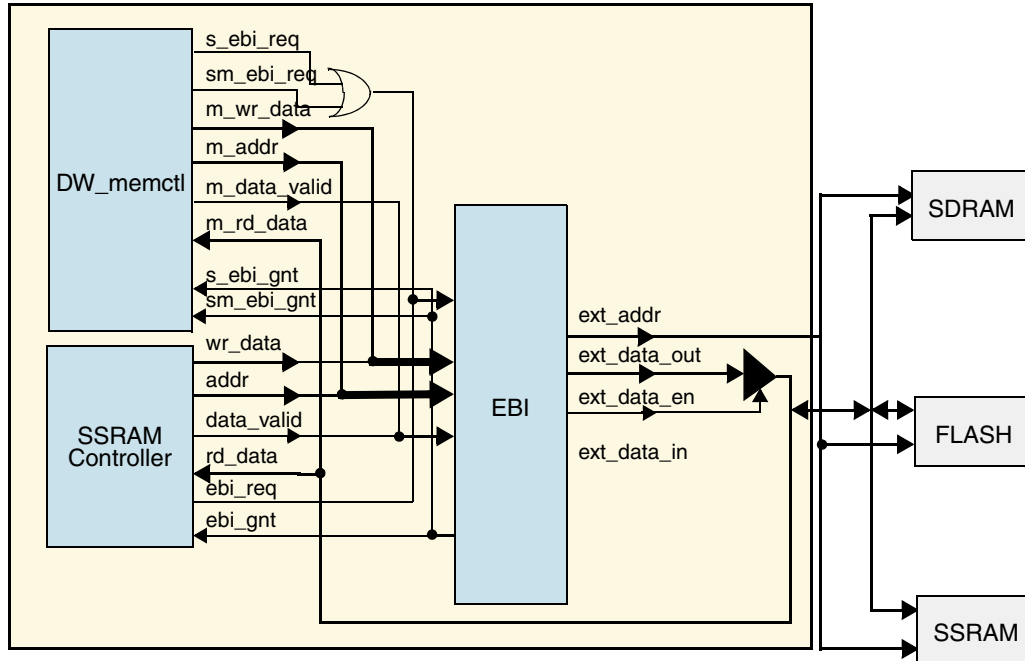


Figure A-14 illustrates the DW_memctl configured as an SDRAM controller and a Static memory controller, sharing external pins with an SSRAM controller through the EBI interface. In this situation, the internal pin-sharing mechanism of the DW_memctl is enabled.

Figure A-14 DW_memctl in Shared Mode, Interfacing to EBI



A.5.4 Verilog RTL for EBI Logic

The following is an example of Verilog RTL for the EBI:

```
module DW_memctl_ebi_rtl(
  // inputs
  h_clk,           // - clock input pin
  ebi_rst_n,       // - reset input pin (active low)
  ebi_req,         // - Memory controller request bus
  ebi_addr,        // - address bus (CTL_NUM*S_ADDR_WIDTH)
  ebi_data_out,    // - EBI data bus
  ext_data_in,     // - External data bus
  ebi_data_en,     // - Memory valid bits
  priority,        // - Priority for the clients

  // outputs
  ebi_gnt,         // - Memory controller grant bus
  ext_addr,        // - External address bus
  ext_data_out,    // - EBI data bus MUXed output
  ebi_data_in,     // - External data bus passed through
                  // to memory controllers
  ext_data_en      // - External memory valid bit

  // embedded script
  // synopsys dc_script_begin
  // suppress_errors = suppress_errors + {EQN-10}
  // save_suppress_state = suppress_errors - {EQN-10}

```

```

        // suppress_errors = {VHDL-2023 UID-95 EQN-19}
        // set_design_license {DesignWare-Foundation}
        // -limited {SynLib-Eval} -quiet
        // set_model_load 4 all_outputs()
        // set_model_drive 1 all_inputs()
        // set_max_area 0
        // set_attribute current_design "date" "09.05.2001"
        // -type string -quiet
        // set_attribute current_design "DesignWare" "TRUE"
        // -type boolean -quiet
        // set_attribute current_design "vendor" "Synopsys"
        // -type string -quiet
        // set_attribute current_design "abstract" "AMBA External
        // Bus Interface module" -type string -quiet
        // suppress_errors = save_suppress_state
        // remove_variable save_suppress_state
        // synopsys dc_script_end
    );

// Parameters
    parameter S_ADDR_WIDTH = 32;
    parameter S_DATA_WIDTH = 32;
    parameter CTL_NUM = 2;
    parameter P_WIDTH = 2;

// Calculate select range

`define r (CTL_NUM)
`define gnt_number ( ('r>8) ? 4 : (('r>4) ? 3 : (('r>2) ? 2 : 1)) )
`define idle 0
`define grant 1
`define wait 2
`define park_mode 0
`define park_index 0
`define output_mode 1
`define n (CTL_NUM)
`define width (P_WIDTH)

input h_clk;
input ebi_rst_n;
input [CTL_NUM-1:0] ebi_req;
input [CTL_NUM*S_ADDR_WIDTH-1:0] ebi_addr;
input [CTL_NUM*S_DATA_WIDTH-1:0] ebi_data_out;
input [S_DATA_WIDTH-1:0] ext_data_in;
input [CTL_NUM-1:0] ebi_data_en;
input [P_WIDTH*CTL_NUM-1:0] priority;

output [CTL_NUM-1:0] ebi_gnt;
output [S_ADDR_WIDTH-1:0] ext_addr;
output [S_DATA_WIDTH-1:0] ext_data_out;
output [S_DATA_WIDTH-1:0] ebi_data_in;
output ext_data_en;

wire [CTL_NUM-1:0] mask_int;
wire ['gnt_number-1:0] gnt_idx;

wire gnt_flag, parked_flag, locked_flag;

```

```

// Last sample of ext_data_in for Idle State.
reg [S_DATA_WIDTH-1:0] ebi_data_in_last;

// For combining feedback input from ext bus.
wire [((CTL_NUM+1)*S_DATA_WIDTH)-1:0] ebi_data_out_int;

// Register grant signals for mux control.
reg [CTL_NUM-1:0] ebi_gnt_reg;

// Function to multiplex the address, data, data_en signals
// split input for select
function [S_DATA_WIDTH-1:0] mux_data_out;
  input [CTL_NUM-1:0] sel_int;
  input [((CTL_NUM+1)*S_DATA_WIDTH)-1:0] mux_in;
  reg [S_DATA_WIDTH-1:0] mux_data_out_int;
  integer i, j;
  begin
    mux_data_out_int = {(S_DATA_WIDTH){1'b0}};
    for (j=0; j < S_DATA_WIDTH; j=j+1) begin
      mux_data_out_int[j] = mux_in[j+(CTL_NUM*S_DATA_WIDTH)];
    end // for
    for (i=CTL_NUM-1; i >= 0; i=i-1) begin
      if (sel_int[i] == 1'b1)
        for (j=S_DATA_WIDTH-1; j >= 0; j=j-1) begin
          mux_data_out_int[j] = mux_in[j+(i*S_DATA_WIDTH)];
        end // for
    end // for
    mux_data_out = mux_data_out_int;
  end // block
endfunction

function mux_data_en;
  input [CTL_NUM-1:0] sel_int;
  input [CTL_NUM-1:0] mux_in;
  reg mux_data_en_int;
  integer i;
  begin
    mux_data_en_int = 1'b1;
    for (i=CTL_NUM-1; i >= 0; i=i-1) begin
      if (sel_int[i] == 1'b1)
        mux_data_en_int = mux_in[i];
    end // for

    mux_data_en = mux_data_en_int;
  end // block
endfunction

function [S_DATA_WIDTH-1:0] mux_addr;
  input [CTL_NUM-1:0] sel_int;
  input [CTL_NUM*S_DATA_WIDTH-1:0] mux_in;
  integer i, j;
  reg [S_DATA_WIDTH-1:0] mux_addr_int;
  begin
    mux_addr_int = {S_DATA_WIDTH{1'b0}};
    for (i=CTL_NUM-1; i >= 0; i=i-1) begin
      if (sel_int[i] == 1'b1)
        for (j=S_DATA_WIDTH-1; j >= 0; j=j-1) begin
          mux_addr_int[j] = mux_in[j+(i*S_DATA_WIDTH)];
        end // for
    end // for
  end
endfunction

```



```

        end // for
    end // for
    mux_addr = mux_addr_int;
end // block
endfunction

assign mask_int = {CTL_NUM{1'b0}};

// pass signals through
assign ebi_data_in = ext_data_in;

// concat feedback data_in with data_out
assign ebi_data_out_int = {ebi_data_in_last, ebi_data_out};

// function calls for muxing
assign ext_data_out = mux_data_out(ebi_gnt_reg, ebi_data_out_int);
assign ext_data_en = mux_data_en(ebi_gnt_reg, ebi_data_en);
assign ext_addr = mux_addr(ebi_gnt_reg, ebi_addr);

DW_arbiter_2t #('n, 'width, 'park_mode, 'park_index, 'output_mode)
    U1 (.clk(h_clk), .rst_n(ebi_rst_n), .request(ebi_req), \
        .priority(priority), .lock(ebi_req), .mask(mask_int), \
        .parked(parked_flag), .granted(gnt_flag), .locked(locked_flag), \
        .grant(ebi_gnt), .grant_index(gnt_idx) );

//
// Sequential block
//
always @ (posedge h_clk or negedge ebi_rst_n)
    begin
        if (ebi_rst_n == 1'b0)
            begin
                ebi_data_in_last <= {(S_DATA_WIDTH){1'b0}};
                ebi_gnt_reg <= {CTL_NUM{1'b0}};
            end
        else
            begin
                ebi_data_in_last <= &ebi_req ? ext_data_in : ebi_data_in_last;
                ebi_gnt_reg <= ebi_gnt;
            end
    end
end // block

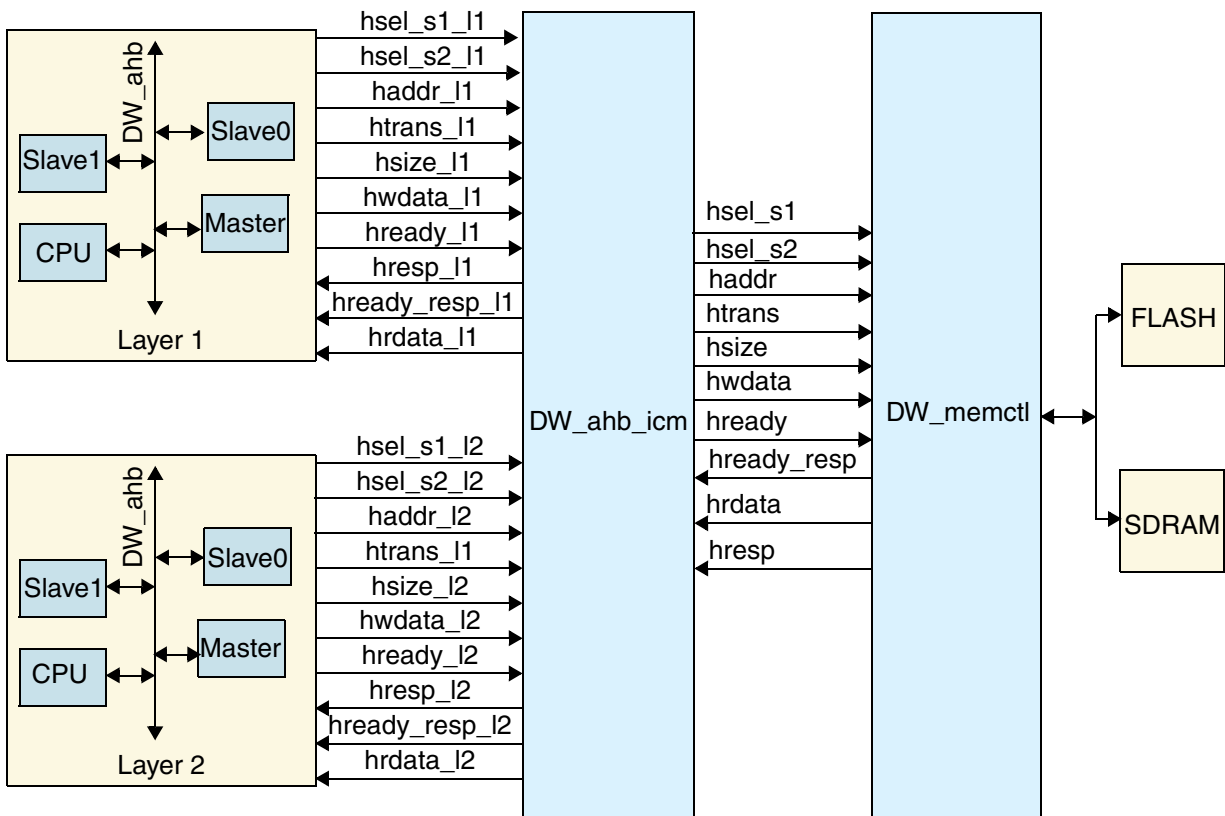
endmodule

```

A.6 DW_memctl in a Multi-Layer AMBA System

The DW_memctl can be used in a multi-layer AHB system using the DesignWare AHB multi-layer Interconnection Matrix (DW_ahb_icm), which comes as part of the DesignWare SDRAM and Static Memory Controller. [Figure A-15](#) illustrates how to connect the DW_memctl to the DW_ahb_icm in a multi-layer AHB environment.

Figure A-15 DW_memctl in Multi-Layer AHB Using DW_ahb_icm



In this example system, the two AHB layers (layer1 and layer2) share the DW_memctl through the DW_ahb_icm. Layer1 can access the DW_memctl when layer2 is not accessing it.

With this architecture, while the master in layer2 is accessing DW_memctl, the CPU in layer1 can access its local slave (Slave0).

When configuring the DW_ahb_icm for use with the DW_memctl, you can use the DW_ahb_icm parameters listed in [Table A-3](#).

Table A-3 DW_ahb_icm Parameter Usage with DW_memctl

DW_ahb_icm Parameter	Value for Use with DW_memctl	Description
ICM_HAS_MHSEL	1	Since DW_memctl has two select signals (hsel_reg and hsel_mem), this parameter needs to be set to 1.
ICM_NUM_HSEL	2	Specifies the number of select lines as 2,

Table A-3 DW_ahb_icm Parameter Usage with DW_memctl

DW_ahb_icm Parameter	Value for Use with DW_memctl	Description
ICM_HAS_PROT	0	DW_memctl does not require the protection control.
ICM_HAS_BURST	1	DW_memctl does require the burst control information.
ICM_HAS_LOCK	0	DW_memctl does not use the lock feature.
ICM_HAS_SPLIT	0	DW_memctl does not split transactions.

B

Glossary

active command queue	Command queue from which a model is currently taking commands; see also command queue.
activity	A set of functions in coreConsultant that step you through configuration, verification, and synthesis of a selected core.
AHB	Advanced High-performance Bus — high-performance system backbone bus. AHB supports the efficient connection of processors, on-chip memories and off-chip external memory interfaces (ARM Limited specification).
AMBA	Advanced Microcontroller Bus Architecture — a trademarked name by ARM Limited that defines an on-chip communication standard for high speed microcontrollers.
APB	Advanced Peripheral Bus — optimized for minimal power consumption and reduced interface complexity to support peripheral functions (ARM Limited specification).
APB bridge	DW_apb submodule that converts protocol between the AHB bus and APB bus.
application design	Overall chip-level design into which a subsystem or subsystems are integrated.
arbiter	AMBA bus submodule that arbitrates bus activity between masters and slaves.
BFM	Bus-Functional Model — A simulation model used for early hardware debug. A BFM simulates the bus cycles of a device and models device pins, as well as certain on-chip functions. See also Full-Functional Model.
big-endian	Data format in which most significant byte comes first; normal order of bytes in a word.

blocked command stream	A command stream that is blocked due to a blocking command issued to that stream; see also command stream, blocking command, and non-blocking command.
blocking command	A command that prevents a testbench from advancing to next testbench statement until this command executes in model. Blocking commands typically return data to the testbench from the model.
bus bridge	Logic that handles the interface and transactions between two bus standards, such as AHB and APB. See APB bridge.
command channel	Manages command streams. Models with multiple command channels execute command streams independently of each other to provide full-duplex mode function.
command stream	The communication channel between the testbench and the model.
component	A generic term that can refer to any synthesizable IP or verification IP in the DesignWare Library. In the context of synthesizable IP, this is a configurable block that can be instantiated as a single entity (VHDL) or module (Verilog) in a design.
configuration	The act of specifying parameters for a core prior to synthesis; can also be used in the context of VIP.
configuration intent	Range of values allowed for each parameter associated with a reusable core.
core	Any configurable block of synthesizable IP that can be instantiated as a single entity (VHDL) or module (Verilog) in a design. Core is the preferred term for a big piece of IIP. Anything that requires coreConsultant for configuration, as well as anything in the DesignWare Cores library, is a core.
core developer	Person or company who creates or packages a reusable core. All the cores in the DesignWare Library are developed by Synopsys.
core integrator	Person who uses coreConsultant or coreAssembler to incorporate reusable cores into a system-level design.
coreAssembler	Synopsys product that enables automatic connection of a group of cores into a subsystem. Generates RTL and gate-level views of the entire subsystem.
coreConsultant	A Synopsys product that lets you configure a core and generate the design views and synthesis views you need to integrate the core into your design. Can also synthesize the core and run the unit-level testbench supplied with the core.
coreKit	An unconfigured core and associated files, including the core itself, a specified synthesis methodology, interfaces definitions, and optional items such as verification environment files and core-specific documentation.
cycle command	A command that executes and causes HDL simulation time to advance.
decoder	Software or hardware subsystem that translates from and “encoded” format back to standard format.
design context	Aspects of a component or subsystem target environment that affect the synthesis of the component or subsystem.

design creation	The process of capturing a design as parameterized RTL.
Design View	A simulation model for a core generated by coreConsultant.
DesignWare Synthesizable Components	The Synopsys name for the collection of AMBA-compliant coreKits and verification models delivered with DesignWare and used with coreConsultant or coreAssembler to quickly build DesignWare Synthesizable Component designs.
DesignWare cores	A specific collection of synthesizable cores that are licensed individually. For more information, refer to www.synopsys.com/designware .
DesignWare Library	A collection of synthesizable IP and verification IP components that is authorized by a single DesignWare license. Products include SmartModels, VMT model suites, DesignWare Memory Models, Building Block IP, and the DesignWare Synthesizable Components.
dual role device	Device having the capabilities of function and host (limited).
endian	Ordering of bytes in a multi-byte word; see also little-endian and big-endian.
Full-Functional Mode	A simulation model that describes the complete range of device behavior, including code execution. See also BFM.
GPIO	General Purpose Input Output.
GTECH	A generic technology view used for RTL simulation of encrypted source code by non-Synopsys simulators.
hard IP	Non-synthesizable implementation IP.
HDL	Hardware Description Language – examples include Verilog and VHDL.
IIP	Implementation Intellectual Property — A generic term for synthesizable HDL and non-synthesizable “hard” IP in all of its forms (coreKit, component, core, MacroCell, and so on).
implementation view	The RTL for a core. You can simulate, synthesize, and implement this view of a core in a real chip.
instantiate	The act of placing a core or model into a design.
interface	Set of ports and parameters that defines a connection point to a component.
IP	Intellectual property — A term that encompasses simulation models and synthesizable blocks of HDL code.
little-endian	Data format in which the least-significant byte comes first.
MacroCell	Bigger IP blocks (6811, 8051, memory controller) available in the DesignWare Library and delivered with coreConsultant.
master	Device or model that initiates and controls another device or peripheral.
model	A Verification IP component or a Design View of a core.

monitor	A device or model that gathers performance statistics of a system.
non-blocking command	A testbench command that advances to the next testbench statement without waiting for the command to complete.
peripheral	Generally refers to a small core that has a bus connection, specifically an APB interface.
RTL	Register Transfer Level. A higher level of abstraction that implies a certain gate-level structure. Synthesis of RTL code yields a gate-level design.
SDRAM	Synchronous Dynamic Random Access Memory; high-speed DRAM adds a separate clock signal to control signals.
SDRAM controller	A memory controller with specific connections for SDRAMs.
slave	Device or model that is controlled by and responds to a master.
SoC	System on a chip.
soft IP	Any implementation IP that is configurable. Generally referred to as synthesizable IP.
static controller	Memory controller with specific connections for Static memories such as asynchronous SRAMs, Flash memory, and ROMs.
subsystem	In relation to coreAssembler, highest level of RTL that is automatically generated.
synthesis intent	Attributes that a core developer applies to a top-level design, ports, and core.
synthesizable IP	A type of Implementation IP that can be mapped to a target technology through synthesis. Sometimes referred to as Soft IP.
technology-independent	Design that allows the technology (that is, the library that implements the gate and via widths for gates) to be specified later during synthesis.
Testsuite Regression Environment (TRE)	A collection of files for stand-alone verification of the configured component. The files, tests, and functionality vary from component to component.
VIP	Verification Intellectual Property — A generic term for a simulation model in any form, including a Design View.
workspace	A network location that contains a personal copy of a component or subsystem. After you configure the component or subsystem (using coreConsultant or coreAssembler), the workspace contains the configured component/subsystem and generated views needed for integration of the component/subsystem at the top level.
wrap, wrapper	Code, usually VHDL or Verilog, that surrounds a design or model, allowing easier interfacing. Usually requires an extra, sometimes automated, step to create the wrapper.
zero-cycle command	A command that executes without HDL simulation time advancing.

Index

A

A8_FOR_PRECHARGE parameter [135](#)

Access

control register [42](#)

active command queue

definition [309](#)

activity

definition [309](#)

ADDR_FIFO_DEPTH parameter [129](#)

Address

aliasing [181](#)

register, details [182](#)

remapping [181](#)

Address decoder [210](#)

register [181](#)

Address FIFO [38](#), [39](#)

Address width [102](#)

Addresses

multiplexing

SDRAM [89](#)

Static memory [108](#)

AHB

definition [309](#)

AHB slave

DW_memctl as [36](#)

Aliasing, address [181](#)

AMBA

definition [309](#)

host interface unit (HIU) [16](#), [36](#), [37](#)

APB

definition [309](#)

APB bridge

definition [309](#)

application design

definition [309](#)

arbiter

definition [309](#)

Auto-refresh

command sequence, SDR/Mobile SDR-SDRAM
[64](#)

DDR-SDRM [83](#)

SDR-SDRAM [63](#)

B

Batch jobs, executing [216](#)

BFM

definition [309](#)

big_endian signal [154](#)

big-endian

definition [309](#)

Bits

number for generating chip selects [91](#)

Block diagrams

host interface unit [38](#)

BLOCK_SIZE parameter

details [143](#)

blocked command stream

definition [310](#)

blocking command

definition [310](#)

bus bridge

definition [310](#)

Buses

data, bi-direction [224](#)

C

CAS_LATENCY parameter

details [136](#)

Chip select

bits, number for generating [91](#)

decoding [91](#)

generation [91](#)

Chip selection

Static memory [102](#)

CHIP_SELECTn_ALIAS_ADDRESS parameter [181](#)

- details [144](#)
- CHIP_SELECTn_BASE_ADDRESS parameter
 - details [143](#)
- CHIP_SELECTn_MEM parameters [128](#)
- CHIP_SELECTn_REMAP_ADDRESS parameter [182](#)
 - details [144](#)
- CHIP_SELn_ALIAS_ENABLE parameter [144](#)
- CHIP_SELn_REMAP_ENABLE parameter [143](#)
- CHIP_SELx_ALIAS_ENABLE parameter
 - Parameters
 - CHIP_SELx_ALIAS_ENABLE [181](#), [185](#)
- CHIP_SELx_REMAP_ENABLE parameter [182](#), [187](#)
- clear_sr_dp signal [155](#)
- command channel
 - definition [310](#)
- command stream
 - definition [310](#)
- Commands
 - auto-refresh sequence, SDR/Mobile SDR-SDRAM [64](#)
 - auto-refresh, DDR-SDRM [83](#)
 - auto-refresh, SDR-SDRAM [63](#)
 - DDR-SDRM auto-refresh sequence [84](#)
 - DDR-SDRM self-refresh [86](#)
 - files, simulation [212](#)
 - power-down sequence, DDR-SDRM [87](#)
 - power-down sequence, SDR/Mobile SDR-SDRAM [66](#)
 - power-down, DDR-SDRM [87](#)
 - power-down, SDR/Mobile SDR-SDRAM [66](#)
 - power-on sequence, DDR-SDRM [68](#), [70](#)
 - power-on sequence, SDR/Mobile SDR-SDRAM [50](#)
 - self-refresh sequence, DDR-SDRM [86](#)
 - self-refresh sequence, SDR/Mobile SDR-SDRAM [65](#)
 - self-refresh, SDR/Mobile SDR-SDRAM [65](#)
- COMP_PARAMS_1 component configuration register [195](#)
- COMP_PARAMS_1 register [195](#)
- COMP_PARAMS_2 component configuration register [198](#)
- COMP_PARAMS_2 register [198](#)
- COMP_TYPE component identification register [194](#)
- COMP_TYPE register [194](#)
- COMP_VERSION component identification register [194](#)
- COMP_VERSION register [194](#)
- component
 - definition [310](#)
- Component configuration registers [195](#)
- Component identification registers [194](#)
- Configuration
 - DW_memctl, for Static memory [101](#)
- configuration
 - definition [310](#)
- configuration intent
 - definition [310](#)
- Configurations
 - verification, default [216](#)
 - verification, non-default [216](#)
- Control register
 - access [42](#)
- Control registers, summary [168](#)
- Controller
 - SDRAM, DW_memctl as [49](#)
 - Static memory, DW_memctl as [101](#)
- core
 - definition [310](#)
- core developer
 - definition [310](#)
- core integrator
 - definition [310](#)
- coreAssembler
 - definition [16](#), [310](#)
 - overview of usage flow [25](#)
- coreConsultant
 - definition [310](#)
 - overview of usage flow [22](#)
- coreKit
 - definition [310](#)
- CSALIASn aliasing registers [185](#)
- CSALIASn_LOW aliasing register [92](#), [181](#), [185](#), [186](#)
- CSALIASn_LOW register [92](#), [181](#), [185](#), [186](#)
- CSREMAPn_LOW register [187](#), [188](#)
- CSREMAPn_LOW remap register [187](#), [188](#)
- Customer Support [11](#)
- cycle command
 - definition [310](#)
- D**
- Data
 - width [101](#)

- Data buses
 - bi-directional [224](#)
- DDR-SDRM Auto-refresh
 - command sequence [84](#)
- Decoder
 - address [210](#)
- decoder
 - definition [310](#)
- Decoding
 - chip select [91](#)
- design context
 - definition [310](#)
- design creation
 - definition [311](#)
- Design flow
 - DW_memctl [35](#)
 - illustration [35](#)
- Design hierarchy [37](#)
- Design View
 - definition [311](#)
- Designs
 - integration [221](#), [223](#)
- DesignWare cores
 - definition [311](#)
- DesignWare Library
 - definition [311](#)
- DesignWare Synthesizable Components
 - definition [311](#)
- Diagrams
 - block, host interface unit [38](#)
 - host interface unit block [38](#)
 - timing, register read/write [42](#)
- Directories
 - simulations output file [212](#)
- dual role device
 - definition [311](#)
- DW_ahb
 - interfacing to [145](#)
 - parameters [125](#)
 - signal description [152](#)
- DW_memctl
 - as SDRAM controller [49](#)
 - as Static memory controller [101](#)
 - components [16](#)
 - configurations [33](#), [201](#)
 - configuring for Static memory [101](#)
 - design hierarchy [37](#)

- flow, automation of
 - Flow
 - automation of [34](#)
 - instantiation [221](#)
 - integration in designs [221](#), [223](#)
 - interfaces [33](#), [202](#)
 - overview [16](#)
 - slave, as [36](#)
 - testbench [206](#)
- DYNAMIC_RAM_TYPE parameters [127](#), [128](#)
- E**
 - EBI_INTERFACE parameter [130](#)
 - ENABLE_ADDRBUS_SHARING parameter [130](#)
 - ENABLE_DATABUS_SHARING parameter [130](#)
 - ENABLE_STATIC parameter [127](#)
 - endian
 - definition [311](#)
 - Environment
 - verification, command-line [205](#)
 - Environment, licenses [20](#)
 - Environment, verification [206](#)
 - EXN_MODE_REG register [179](#)
 - EXTENDED_MODE_REG parameter
 - details [134](#)
- F**
 - Features
 - SDRAM interface [18](#), [263](#)
 - static memory interface [19](#), [264](#)
 - FIFOs
 - address [38](#), [39](#)
 - write data [38](#), [40](#)
 - Files
 - directories, simulation output [212](#)
 - log [213](#)
 - simulation command [212](#)
 - simulation output [212](#)
 - VERA, core [216](#)
 - FLASH
 - interface with DW_memctl [111](#)
 - power-down control
 - Reset
 - control, FLASH [102](#)
 - reset control [102](#)
 - synchronous, timing parameters [119](#)
 - timing parameters [110](#)
 - FLASH timing register [102](#), [191](#)

FLASH_TRPDR register [102](#), [191](#)

FLASHes

- interfaced with DW_memctl [33](#), [202](#)
- interfaces [244](#)

Flow

- design, DW_memctl [35](#)

Full-Functional Mode

- definition [311](#)

Functions

- host interface unit [38](#)

G

gpi signal [155](#)

GPIO

- definition [311](#)

gpo signal [154](#)

GTECH

- definition [311](#)

H

H_ADDR_WIDTH parameter [129](#), [181](#), [185](#), [186](#), [187](#), [188](#)

H_DATA_WIDTH parameter [129](#)

haddr signal [154](#)

hard IP

- definition [311](#)

HARD_WIRE_CIPSELECT_PARAMETERS parameter [131](#)

HARD_WIRE_STATIC_PARAMETERS parameter [131](#)

hburst signal [153](#)

hclk signal [152](#)

hclk_2x signal [152](#)

HDL

- definition [311](#)

Hierarchy

- design [37](#)

HIGHER_ADDR_FOR_BANKS parameter [133](#)

HIU. See Host interface unit

Host interface unit (HIU) [16](#), [36](#), [37](#)

- block diagram [38](#)

- functions [38](#)

hrdata signal [154](#)

hready signal [153](#)

hready_resp signal [153](#)

hresetn signal [152](#)

hresp signal [153](#)

hsel_mem signal [152](#)

hsel_reg signal [152](#)

hsize signal [153](#)

htrans signal [153](#)

hwdata signal [154](#)

hwrite signal [153](#)

I

I/O interface between FLASH and DW_memctl [111](#)

I/O interface between SRAM and DW_memctl [109](#)

I/O signals, description of [152](#)

IIP

- definition [311](#)

implementation view

- definition [311](#)

Initialization

- power-on, DDR-SDRM [67](#), [69](#)

- power-on, SDR-SDRAM [49](#)

Input/output signals

- parameter controlled [146](#), [147](#), [148](#), [149](#), [150](#), [151](#)

instantiate

- definition [311](#)

Instantiation

- DW_memctl [221](#)

Integration

- DW_memctl in designs [221](#), [223](#)

- Verilog PLI [215](#)

interface

- definition [311](#)

Interfaces

- asynchronous FLASH, with
FLASHes

- asynchronous, interfaces with [110](#)

- asynchronous SRAMs, with
SRAMs

- asynchronous, interfaces with [109](#)

challenges [241](#), [242](#)

DW_memctl [33](#), [202](#)

FLASH [33](#), [202](#)

host interface unit (HIU)

I/O between FLASH and DW_memctl [111](#)

I/O between SRAM and DW_memctl [109](#)

memory interface unit (MIU)

off-chip SSRAMs [114](#)

on-chip SSRAMs [112](#)

parameter controlled [146](#), [147](#), [148](#), [149](#), [150](#), [151](#)

- SDRAM [33](#), [202](#), [244](#)
- SDRAM features [18](#), [263](#)
- SRAM [33](#), [202](#), [244](#)
- static memory features [19](#), [264](#)
- synchronous FLASH, with
 - FLASHes
 - synchronous, interfaces with [115](#)
- synchronous SRAMs, with
 - SRAMs
 - synchronous, interfaces with [112](#)
- typical [244](#)
- vera_shell [209](#)
- with SDRAM SIMM/DIMM modules [96](#)
- Interfacing
 - to DW_ahb [145](#)
- IP
 - definition [311](#)
- L**
- Latencies, read/write [52](#)
- Latency
 - read [102](#)
 - write [102](#)
- Licenses [20](#)
- little-endian
 - definition [311](#)
- Log files [213](#)
- LOW_FREQ_DEV_SETn parameter [142](#)
- M**
- m_addr signal [165](#)
- m_dout_valid signal [166](#)
- m_precharge_bit signal [165](#)
- m_rd_data signal [165](#)
- m_wr_data signal [165](#)
- MacroCell
 - definition [311](#)
- Mapping
 - memory, chip select registers [95](#)
- master
 - definition [311](#)
- MAX_S_ADDR_WIDTH parameter [133](#)
- MAX_S_BANK_ADDR_WIDTH parameter [133](#)
- MAX_S_DATA_WIDTH parameter [132](#)
- MAX_SM_ADDR_WIDTH parameter [102](#), [134](#)
- MAX_SM_DATA_WIDTH parameter [134](#)
- Memories
 - register insertion [234](#)
 - Static, chip selection [102](#)
- Memory
 - Static, configuring DW_memctl as [101](#)
- Memory banks [211](#)
- Memory interface unit (MIU) [16](#), [36](#), [46](#)
- Memory mapping
 - chip select registers [95](#)
- MIU. See Memory interface unit
- model
 - definition [311](#)
- monitor
 - definition [312](#)
- Multiplexing, address
 - SDRAM [89](#)
 - Static memory [108](#)
- N**
- N_CS parameters [127](#), [181](#)
- non-blocking command
 - definition [312](#)
- NUM_DQS parameter [133](#)
- NUM_INIT_REF parameter [138](#)
- O**
- Off-chip SSRAMs
 - interfacing to [114](#)
- On-chip SSRAMs
 - interfacing to [112](#)
- OPEN_BANKS parameter
 - details [136](#)
- Operations
 - read/write, DDR-SDRM [71](#)
- Output
 - waveform [211](#)
- Output files
 - GTECH [30](#)
 - RTL-level [29](#)
 - simulation [212](#)
 - Simulation model [30](#)
 - synthesis [30](#)
 - verification [30](#)
- P**
- PAGE_MODE_SETn parameter [141](#)
- PAGE_SIZE_SETn parameter [142](#)
- Parameters
 - A8_FOR_PRECHARGE [135](#)

- ADDR_FIFO_DEPTH 129
- BLOCK_SIZE_n
 - details 143
- CAS_LATENCY
 - details 136
- CHIP_SELECT_n_ALIAS_ADDRESS 181
 - details 144
- CHIP_SELECT_n_BASE_ADDRESS
 - details 143
- CHIP_SELECT_n_MEM 128
- CHIP_SELECT_n_REMAP_ADDRESS 182
 - details 144
- CHIP_SEL_n_ALIAS_ENABLE 144
- CHIP_SEL_n_REMAP_ENABLE 143
- CHIP_SEL_x_REMAP_ENABLE 182, 187
- DYNAMIC_RAM_TYPE 127, 128
- EBI_INTERFACE 130
- ENABLE_ADDRBUS_SHARING 130
- ENABLE_DATABUS_SHARING 130
- ENABLE_STATIC 127
- EXTENDED_MODE_REG
 - details 134
- FLASH timing, specifying 110
- FLASH, synchronous timing, specifying 119
- H_ADDR_WIDTH 129, 181, 185, 186, 187, 188
- H_DATA_WIDTH 129
- HARD_WIRE_CIPSELECT_PARAMETERS 131
- HARD_WIRE_STATIC_PARAMETERS 131
- HIGHER_ADDR_FOR_BANKS 133
- input/output signals controlled by 146, 147, 148, 149, 150, 151
- interfaces controlled by 146, 147, 148, 149, 150, 151
- LOW_FREQ_DEV_SET_n 142
- MAX_S_ADDR_WIDTH 133
- MAX_S_BANK_ADDR_WIDTH 133
- MAX_S_DATA_WIDTH 132
- MAX_SM_ADDR_WIDTH 102, 134
- MAX_SM_DATA_WIDTH 134
- N_CS 127, 181
- NUM_DQS 133
- NUM_INIT_REF 138
- OPEN_BANKS
 - details 136
- PAGE_MODE_SET_n 141
- PAGE_SIZE_SET_n 142
- PIPE_ADDRESS_DECODER 129
- READ_PIPE 132
- READY_MODE_SET_n 142
- S_BANK_ADDR_WIDTH
 - details 136
- S_COL_ADDR_WIDTH 135
 - details 135
- S_DATA_WIDTH
 - details 133
- S_ROW_ADDR_WIDTH
 - details 135
- SM_DATA_WIDTH 101
- SM_DATA_WIDTH_SET_n
 - details 143
- SM_READ_PIPE_SET_n 102
 - details 142
- SM_WRITE_PIPE 102, 134
- SRAM timing, specifying 109, 123
- T_AS_SET_n 139
- T_BTA_SET_n 141
- T_INIT 132
- T_PRC_SET_n 141
- T_RAS_MIN 136
- T_RC 137, 138
- T_RC_SET_n 139
- T_RCAR 137
- T_RCD 136
- T_REF 132
- T_RP 137
- T_RPD 134
- T_WP_SET_n 140
- T_WR 137
- T_WR_SET_n 140
- T_WTR 138
- T_XSR 137
- WRITE_FIFO_DEPTH 129
- WRITE_PIPE 132
- Parameters, of DW_ahb 125
- peripheral
 - definition 312
- PIPE_ADDRESS_DECODER parameter 129
- power_down signal 155
- Power-down
 - command sequence, DDR-SDRM 87
 - command sequence, SDR/Mobile SDR-SDRAM 66
 - control, FLASH 102
 - DDR-SDRM 87
 - SDR/Mobile SDR-SDRAM 66
- Power-on
 - command sequence, DDR-SDRM 68, 70
 - command sequence, SDR/Mobile SDR-SDRAM 50

Power-on initialization

DDR-SDRM 67, 69

Power-on initialization, SDR-SDRAM 49

R

Read latency 102

Read/write

latencies 52

Read/write operations

DDR-SDRM 71

Read/write operations, SDR/Mobile SDR-SDRAM
Operations

read/write, SDR/Mobile SDR-SDRAM 51

READ_PIPE parameter 132

READY_MODE_SETn parameter 142

Refresh

auto, DDR-SDRM 83

auto, SDR-SDRAM 63

self, DDR-SDRM 86

self, SDR/Mobile SDR-SDRAM 65

Registers

address decoder 181

address, details 182

aliasing 185

aliasing registers 92, 181, 185, 186

chip select 102, 181

chip select, memory mapping 95

COMP_PARAMS_1 195

COMP_PARAMS_2 198

COMP_TYPE 194

COMP_VERSION 194

Component configuration 195

component configuration registers 195, 198

Component identification 194

component Identification registers 194

control, access 42

control, summary 168

CSALIASn 185

CSALIASn_LOW 92, 181, 185, 186

CSREMAPn_LOW 187, 188

EXN_MODE_REG 179

FLASH timing registers 102, 191

FLASH_TRPDR 102, 191

insertion 234

mask

Mask registers 102, 128, 183

read/write timing diagrams 42

remap 187

remap registers 187, 188

SCSLRn 102, 181

SCTLR 86, 175

SDRAM control register 86, 175

SDRAM refresh interval register 178

SDRAM timing registers 43, 44, 45, 172, 174

SMCTLR 101, 102, 103, 192

SMTMGR_SETn 101, 109, 110, 119, 123, 189

SREFR 178

Static memory control registers 101, 102, 103, 192

Static memory read/write timing diagrams 103

Static memory timing registers 101, 109, 110, 119,
123, 189

STMGrR 43, 44, 45, 172, 174

Remap registers 187

remap signal 154

Remapping, address 181

Reset

system 225

RTL

definition 312

run.scr script 214

S

s_addr signal 159

s_bank_addr signal 159

S_BANK_ADDR_WIDTH parameter
details 136

s_cas_latency 161

s_cas_n signal 158

s_cke signal 158

S_COL_ADDR_WIDTH parameter 135
details 135

S_DATA_WIDTH parameter
details 133

s_dout_valid signal 159

s_dqm signal 160

s_dqs signal 160

s_ebi_gnt signal 162

s_ebi_req signal 162

s_ras_n signal 158

s_rd_data signal 159

s_rd_dqs_mask 161

s_rd_end signal 161

s_rd_pop signal 161

s_rd_ready signal 160

s_rd_start signal 161

- S_ROW_ADDR_WIDTH parameter
 - details [135](#)
- s_sa signal [157](#)
- s_scl signal [157](#)
- s_sda_in signal [158](#)
- s_sda_oe_n signal [158](#)
- s_sda_out signal [158](#)
- s_sel_n signal [160](#)
- s_we_n signal [160](#)
- s_wr_data signal [159](#)
- scan_mode signal [155](#)
- Scripts
 - run.scr [214](#)
 - simulation [213](#)
- SCSLRn chip select register [102](#), [181](#)
- SCTLR register [86](#), [175](#)
- SDRAM
 - controller, DW_memctl as [49](#)
 - definition [312](#)
 - SIMM/DIMM interfaces [96](#)
- SDRAM control register [86](#), [175](#)
- SDRAM controller
 - definition [312](#)
- SDRAM interface features [18](#), [263](#)
- SDRAM refresh interval register [178](#)
- SDRAM timing register [43](#), [44](#), [45](#), [172](#), [174](#)
- SDRAMs
 - controlled by DW_memctl [13](#)
 - interfaced with DW_memctl [33](#), [202](#)
 - interfaces [244](#)
- Select signal [36](#)
- Self-refresh
 - command sequence SDR/Mobile SDR-SDRAM [65](#)
 - command sequence, DDR-SDRM [86](#)
 - SDR/Mobile SDR-SDRAM [65](#)
- Self-refresh, DDR-SDRM [86](#)
- Signal description [152](#)
- Signals
 - big_endian [154](#)
 - clear_sr_dp [155](#)
 - gpi [155](#)
 - gpo [154](#)
 - haddr [154](#)
 - hburst [153](#)
 - hclk [152](#)
 - hclk_2x [152](#)
 - hrdata [154](#)
 - hready [153](#)
 - hready_resp [153](#)
 - hresetn [152](#)
 - hresp [153](#)
 - hsel_mem [152](#)
 - hsel_reg [152](#)
 - hsize [153](#)
 - htrans [153](#)
 - hwdata [154](#)
 - hwrite [153](#)
 - m_addr [165](#)
 - m_dout_valid [166](#)
 - m_precharge_bit [165](#)
 - m_rd_data [165](#)
 - m_wr_data [165](#)
 - power_down [155](#)
 - remap [154](#)
 - s_addr [159](#)
 - s_bank_addr [159](#)
 - s_cas_latency [161](#)
 - s_cas_n [158](#)
 - s_cke [158](#)
 - s_dout_valid [159](#)
 - s_dqm [160](#)
 - s_dqs [160](#)
 - s_ebi_gnt [162](#)
 - s_ebi_req [162](#)
 - s_ras_n [158](#)
 - s_rd_data [159](#)
 - s_rd_dqs_mask [161](#)
 - s_rd_end [161](#)
 - s_rd_pop [161](#)
 - s_rd_ready [160](#)
 - s_rd_start [161](#)
 - s_sa [157](#)
 - s_scl [157](#)
 - s_sda_in [158](#)
 - s_sda_oe_n [158](#)
 - s_sda_out [158](#)
 - s_sel_n [160](#)
 - s_we_n [160](#)
 - s_wr_data [159](#)
 - scan_mode [155](#)
 - select [36](#)
 - sm_addr [162](#)
 - sm_adv_n [164](#)
 - sm_bs_n [163](#)
 - sm_clken [164](#)
 - sm_dout_valid [163](#)

- sm_ebi_gnt [162](#)
- sm_ebi_req [162](#)
- sm_oe_n [162](#)
- sm_power_down [155](#)
- sm_rd_data [163](#)
- sm_ready [164](#)
- sm_rp_n [163](#)
- sm_we_n [162](#)
- sm_wp_n [163](#)
- sm_wr_data [163](#)
- SIMM/DIMM
 - SDRAM, interface with DW_memctl [96](#)
- Simulations
 - command files [212](#)
 - command-line [205](#)
 - output files [212](#)
 - scripts [213](#)
- slave
 - definition [312](#)
- sm_addr signal [162](#)
- sm_adv_nsignal [164](#)
- sm_bs_n signal [163](#)
- sm_clken signal [164](#)
- SM_DATA_WIDTH parameter [101](#)
- SM_DATA_WIDTH_SETn parameter
 - details [143](#)
- sm_dout_valid signal [163](#)
- sm_ebi_gnt signal [162](#)
- sm_ebi_req signal [162](#)
- sm_oe_n signal [162](#)
- sm_power_down signal [155](#)
- sm_rd_data signal [163](#)
- SM_READ_PIPE_SETn parameter [102](#)
 - details [142](#)
- sm_ready signal [164](#)
- sm_rp_n signal [163](#)
- sm_we_n signal [162](#)
- sm_wp_n signal [163](#)
- sm_wr_data signal [163](#)
- SM_WRITE_PIPE parameter [102](#), [134](#)
- SMCTLR register [101](#), [102](#), [103](#), [192](#)
- SMSKRn register [102](#), [128](#), [183](#)
- SMTMGR_SETn register [101](#), [109](#), [110](#), [119](#), [123](#), [189](#)
- SoC
 - definition [312](#)
- SoC Platform
 - AHB contained in [13](#)
 - APB, contained in [13](#)
 - defined [13](#)
- soft IP
 - definition [312](#)
- SRAM
 - interface with DW_memctl [109](#)
 - timing parameters [109](#), [123](#)
- SRAMs
 - interfaces [244](#)
- SRAMs interfaced with DW_memctl [33](#), [202](#)
- SREFR register [178](#)
- static controller
 - definition [312](#)
- Static memories
 - controlled by DW_memctl [13](#)
- Static memory
 - chip selection [102](#)
 - configuring DW_memctl [101](#)
 - controller, DW_memctl as [101](#)
- Static memory control register [101](#), [102](#), [103](#), [192](#)
- Static memory interface features [19](#), [264](#)
- Static memory timing register [101](#), [109](#), [110](#), [119](#), [123](#), [189](#)
- STMGRn register [43](#), [44](#), [45](#), [172](#), [174](#)
- subsystem
 - definition [312](#)
- synthesis intent
 - definition [312](#)
- synthesizable IP
 - definition [312](#)
- System reset [225](#)
- Systems
 - topologies [243](#)
- T**
 - T_AS_SETn parameter [139](#)
 - T_BTA_SETn parameter [141](#)
 - T_INIT parameter [132](#)
 - T_PRC_SETn parameter [141](#)
 - T_RAS_MIN parameter [136](#)
 - T_RC parameter [137](#), [138](#)
 - T_RC_SETn parameter [139](#)
 - T_RCAR parameter [137](#)
 - T_RCD parameter [136](#)

- T_REF parameter [132](#)
- T_RP parameter [137](#)
- T_RPD parameter [134](#)
- T_WP_SETn parameter [140](#)
- T_WR parameter [137](#)
- T_WR_SETn parameter [140](#)
- T_WTR parameter [138](#)
- T_XSR parameter [137](#)
- technology-independent
 - definition [312](#)
- Testbench
 - DW_memctl [206](#)
- Testbenches
 - structure [208](#)
- Tests
 - individual, executing [216](#)
- Testsuite Regression Environment (TRE)
 - definition [312](#)
- Timing
 - diagrams
 - read/write, Static memory [103](#)
 - FLASH parameters, specifying [110](#)
 - SRAM parameters, specifying [109](#), [123](#)
 - synchronous FLASH parameters, specifying [119](#)
- Timing diagrams
 - register read/write [42](#)
- Topologies
 - system [243](#)
- TRE
 - definition [312](#)
- U**
- USE_FOUNDATION [135](#)
- V**
- VERA
 - vera_shell interface [209](#)
- VERA compatibility [16](#)
- VERA core
 - files [216](#)
- vera_shell interface [209](#)
- Verification
 - default configuration [216](#)
 - environment [206](#)
 - environment, command-line [205](#)
 - non-default configuration [216](#)
- Verification environment
 - Environments
 - verification [16](#)
 - Verilog PLI integration [215](#)
 - VIP
 - definition [312](#)
- W**
- Waveform output [211](#)
- Width
 - address [102](#)
- workspace
 - definition [312](#)
- wrap
 - definition [312](#)
- wrapper
 - definition [312](#)
- Write Data FIFO [38](#), [40](#)
- Write data FIFO [38](#)
- Write latency [102](#)
- WRITE_FIFO_DEPTH parameter [129](#)
- WRITE_PIPE parameter [132](#)
- Z**
- zero-cycle command
 - definition [312](#)